TEOREME:
Oricat de puternice ar fi metodele de reprezentare, doar o multime numarabila poate fi reprezentata finit

- Din definitie clasa R este (propietati de inchidere, minima) unica.
- Pentru orice AFN exista un AFD echivalent.
- Clasa limbajelor acceptate de AF este inchisa in raport cu operatiile de: reuniune, concatenare, Kleene Star, complementare, intersectie (si diferenta)
- Un limbaj este regulat daca si numai daca este acceptat de un AF
- Propietati algoritmice:
- w ap Sigma*, M -> A.F., w ap L(M)
- M -> A.F., L(M) == Phsi ? (vid)
- M -> A.F., L(M) == Sigma*?
- M1, M2 -> A.F., L(M1) inclus in L(M2)?
- M1, M2 -> A.F., L(M1) == L(M2)?
- Lema de pompare: Fie L limbaj regulat infinit. Atunci exista sirurile x, y!=e, z a.i, xy^nz ap L oricare n ap N
- Alta forma: Fie L limbaj regulat, atunci exista n dependent de L si x, y!=e, z a.i. |xy| < n si xy^kz ap L oricare k >=0
- Un limbaj e regulat daca si numai daca e generat de o gramatica regulata
- Lema: Pentru GIC prin derivari stanga se ajunge la acelasi rezultat ca prin derivari oarecare
- Clasa LIC este clasa L(APD)
- Lema: Oricare LIC este acceptat de un APD
- Lema: Pentru orice limbaj acceptat de un APD pot construi o GIC care sa-l genereze
- LIC sunt inchise in raport cu: reuniune, concatenare, Kleene Star
- LIC sunt inchise in raport cu intersectia si diferenta cu LR
- Intersectia dintre un LIC si un LR este LIC
- Lema de pompare (LIC): Fie G GIC, atunci exista o constanta k dependenta de G a.i. oricare w ap L(G), |w| > k, w =uvxyz cu v!=e sau y!=e, uv^nxy^nz a L oricare n >= 0
- Limbajul a^nb^nc^n, n>=0 nu este LIC
- Limbajele LIC nu sunt inchise in raport cu intersectia si complementarea
- Propietati algoritmice:
- w ap Sigma*, G GIC, w ap L(G)?
- G GIC, L(G) == Multimea vida?
- Lema: M1 o M.T. cu banda nelimitata si M2 M.T.Standard atunci:
- daca M1 se opreste pentru w atunci si M2 se opreste
- daca M1 nu se opreste atunci nici M2 nu se opreste
- Orice functie calculabila, limbaj acceptat/decis pe o M.T, cu banda nelimitata este calculabila respectiv acceptat/decis pe o M.T.S.
- Lema: Daca M1 M.T. cu k benzi si M2 M.T.S atunci:
- Daca M1 se agata si M2 se agata
- Daca M1 nu se opreste atunci nici M2 nu se opreste
- Orice functie calculabila/ limbaj acceptat/decis de o M.T. cu k benzi este calculabila respectiv acceptat/decis de o M.T.S.

- Orice functie calculabila/ limbaj acceptat/decis de o M.T. cu mai multe capete de c/s respectiv banda bidimensionala este calculabila respectiv acceptat/decis de o M.T.S.
- Lema: Daca M1 M.T.N si M2 M.T.S:
- M1 se opreste pentru w atunci M2 se opreste pentru w
- M1 nu se opreste pentru w atunci nici M2 nu se opreste pentru w
- Orice functie calculabila/ limbaj acceptat/decis de o M.T.N. este calculabila respectiv acceptat/decis de o M.T.S.
- Orice Limbaj Turing Decidabil este Turing Acceptat
- Complementul unui Limbaj Turing Decidabil este Turing Decidabil
- Nu orice Limbaj Turing Acceptat este Turing Decidabil
- Complementul unui T.A. nu este neaparat T.A.
- Nu pot decide pentru o M.T. si un sir w daca M.T. se opreste pentru w
- Nu pot sa decid pentru o M.T. fixata si un sir w daca M se opreste pentru w
- Un Limbaj Turing este Decidabil daca si numai daca el si complementul sau sunt T.A.
- Un Limbaj e T.A. daca si numai daca e limbajul de iesire a unei M.T.
- Un limbaj este T.A. daca si numai daca este Turing Enumerabil/enumerat
- Un limbaj e T.A. daca si numai daca este generat de o gramatica (banuiesc ca d-aia 0)

Teorie Seminar 2 LFA

1. Automate finite

Un automat finit reprezintă un model teoretic al unei mașini de calcul, simplificat (restricționat la intrare finită, lipsa memoriei auxiliare, etc.).

Automatele finite sunt **acceptoare** de limbaje **regulate**.

Determinist = dă un răspuns **da**/**nu** la întrebarea "Aparține șirul limbajului?", stabilirea stării următoare fiind complet determinată de model.

2. Automate finite deterministe (A.F.D.) și nedeterministe (A.F.N.)

Definiție: Un A.F.D. este un tuplu M = (K, ∑, δ, s, F), unde:

a) **K** -> mulțimea stărilor ; **finită**
b) **∑** -> alfabet
c) δ -> funcție de tranziție ; δ : K x ∑ -> K
d) **s** -> starea inițială ; s ∈ K
e) **F** -> mulțimea stărilor finale ; F ⊆ K

Definiție: Un A.F.N. este un tuplu M = (K, ∑, Δ, s, F), unde:

a) **K** -> mulțimea stărilor ; **finită**
b) **∑** -> alfabet
c) **Δ** -> mulțimea de tranziții ; Δ ⊆ K x $\sum^*$ x K
d) **s** -> starea inițială ; s ∈ K
e) **F** -> mulțimea stărilor finale ; F ⊆ K

Configurație:

➔ Stare + simbol (șirul de parcurs)
➔ Element al $K \times \sum^*$

Trecerea între configurații ($\vdash_M$):

$(q, w) \vdash_M (q', w')$

AFD: $\vdash_M : K \times \sum^+ \rightarrow K \times \sum^*$

$\exists\ \sigma \in \sum$ astfel încât $w = \sigma w'$, $\delta(q, \sigma) = q'$

AFN: $\vdash_M : K \times \sum^* \rightarrow K \times \sum^*$

$\exists\ u \in \sum^*$ astfel încât $w = uw'$, $(q, u, q') \in \Delta$

Șir acceptat:

w este acceptat de M $\Leftrightarrow (s, w) \vdash^*_M (q, e)$, $q \in F$

Observație: $\vdash^*_M$ reprezintă aplicarea operatorului $\vdash_M$ de 0 sau mai multe ori.

Observație 2: În reprezentarea sub formă de graf, este echivalent cu a spune că există o cale de la s la q pe tranziții care compun șirul w.

Limbaj acceptat:

Definiție: Mulțimea tuturor șirurilor acceptate de automat.

3. Analiză comparativă A.F.D. – A.F.N.

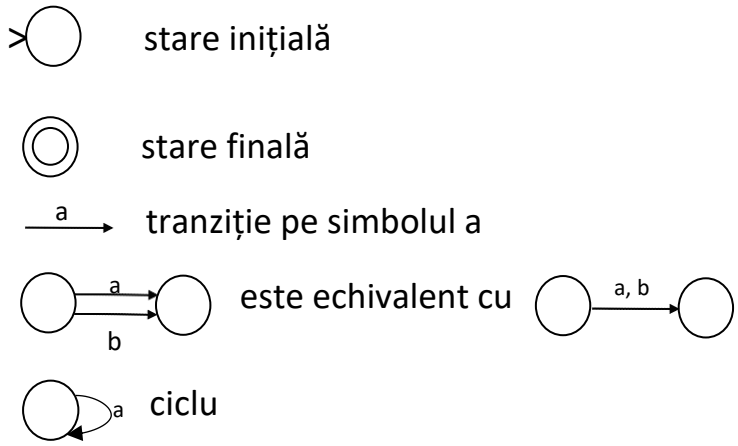
1) Un A.F.N. este un A.F.D. mai puțin restrictiv, în sensul că un A.F.N. are:
   a) Tranziții pe șirul vid
   b) Tranziții pe șiruri
   c) Absența unor tranziții
   d) Tranziții multiple pe același simbol/șir din aceeași stare

2) A.F.N. sunt echivalente cu A.F.D. (teorema de echivalență)
3) A.F.N. și A.F.D. acceptă limbaje regulate.

4) A.F.N. sunt ușor de proiectat dar greu de implementat, pe când la A.F.D. sunt viceversa.
5) **A.F.D. are definite tranziții din orice stare pe orice simbol din alfabet.**

4. Reprezentare

a) Grafuri



b) Tabel de tranziție

Stare	Simbol	Starea următoare
...	...	...

Teorie Seminar 3 LFA

1. Proprietăți ale limbajelor regulate

Este necesară existența unor tehnici de demonstrare a regularității unor limbaje pe baza reprezentării prin proprietăți, plecând de la caracteristici generale ale clasei limbajelor regulate.



L.R. ≡ L(A.F.) ≡ L(E.R)

2. Proprietăți de închidere ale limbajelor regulate

⇨ Închiderea unei mulțimi

Fie A o mulțime, x, y ∈ A și $\oplus$ operație aplicabilă pe x, y. A este **închisă în raport cu operația** $\oplus$ ⇔ x $\oplus$ y ∈ A.

⇨ Teoremă

L(A.F.) este închisă în raport cu: U, ·, *, $\bar{}$, ∩.

⇨ Demonstrație teoremă

I) Concatenare

E.R.: E1 · E2 = E

A.F.D.: Nu poate fi definită o funcție de concatenare.

A.F.N.: M = (K1 U K2, ∑1 U ∑2, Δ1 U Δ2 U {(f, e, s2) | f ∈ F1}, s1, F2)

II) Reuniunea

E.R.: E1 U E2 = E

A.F.D.: M = (K1 x K2, ∑ = min(∑1, ∑2), δ((p, q), a) = (δ1(p, a), δ2(q, a)) , (s1, s2), (F1 x K2) U (K1 \ F2))

A.F.N.: M = (K1 U K2 U { s }, ∑1 U ∑2, Δ1 U Δ2 U {(s, e, s1), (s, e, s2)}, s, F1 U F2)

III) Kleene-Star

E.R.: E = E1*

A.F.D.: nu avem tranziții pe e

A.F.N.: M = (K1 U { s }, ∑1, Δ1 U {(s, e, s1)} U {(f, e, s) | f ∈ F1}, s, F1 U { s })

IV) Complement

E.R.: -

A.F.D.: M = (K, ∑, δ, s, K \ F1)

A.F.N.: -

V) Diferența

L1 \ L2 = L1 ∩ $\overline{L2}$

3. Proprietăți algoritmice ale limbajelor regulate

➔ Determinarea algoritmică a limbajelor acceptate de A.F.
➔ Se poate demonstra algoritmic răspunsul la următoarele întrebări legate de L(A.F.):

I) w ∈ ∑*
M este A.F.
W ∈ L(M) ?

Trasând operațiile lui M în |w| pași din starea inițială pe tranzițiile cu simbolurile ce compun w verific că am ajuns în p ∈ F.

II) L(M) = Ø ?

Verific că nu există cale din starea inițială într-o stare finală.

III) L(M) = ∑* ?

Construiesc $\overline{M}$ și verific $L(\overline{M}) = \emptyset$.

IV) $L(M1) \subseteq L(M2)$?

Verific $L1 \setminus L2 = \emptyset \Leftrightarrow L(M1) \cap (\sum^* \setminus L(M2)) = \emptyset$.

V) $L(M1) = L(M2)$?

Verific $L(M1) \subseteq L(M2)$ și $L(M1) \supseteq L(M2)$.

Proprietăți ale limbajelor regulate

I. Proprietăți de periodicitate

→ șirurile din limbajele regulate infinite au o structură repetitivă din cauza unui ciclu în A.F. sau operatorului Kleene-Star într-o E.R.

II. Lema de pompare

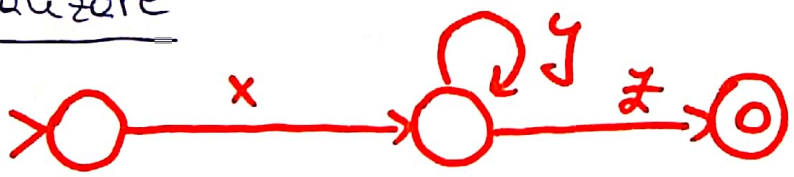
Dacă L este L.R. atunci $\exists\, n(L)$ o constantă naturală dependentă de limbaj, astfel încât $\forall\, w \in L$, $|w| \geq n(L)$, $\exists$ o scriere a lui w de forma $w = xyz$ cu proprietățile:

1) $|xy| < n$

2) $|y| > 0$

3) $xy^i z \in L$, $\forall i \in \mathbb{N}$

Vizualizare



Observații:

a) Proprietate a L.R. dar și a unor neregulate

b) **Implicația inversă este falsă!**

Scanned with CamScanner

Gramatici Independente de Context

I. Definiție:

O gramatică G este un tuplu (V, Σ, R, S), unde

- V → alfabetul gramaticii (terminali + neterminali)
- Σ → alfabetul limbajului (terminali)
- R → mulțimea regulilor
- S → simbolul de start

Obs:

$$R \subset (V \setminus \Sigma) \times V^*$$
$$\Leftrightarrow (A \to \gamma) \in R \Leftrightarrow A \in V \setminus \Sigma \text{ și } \gamma \in V^*$$

II. Derivarea $(\Rightarrow_G, \to)$

Fie $u, v \in V^*$

$u \Rightarrow_G v \Leftrightarrow \exists v', x, y \in V^*, A \in V \setminus \Sigma$, $(A \to v') \in R$ astfel încât $\begin{cases} u = xAy \\ v = xv'y \end{cases}$

Obs: Se înlocuiește partea stângă a regulii cu partea dreaptă.

III. Limbaj generat de gramatică

$$L(G) = \{w \in \Sigma^* \mid S \overset{*}{\underset{G}{\Rightarrow}} w\}$$

IV. Gramatică regulată

$G = (V, \Sigma, R, S)$ G.R. $\Leftrightarrow R \subseteq (V \setminus \Sigma) \times (\Sigma^* (V \setminus \Sigma) \cup \{e\})$

Obs: Toți neterminalii sunt fie în stânga, fie în dreapta.

V. Arbore de derivare

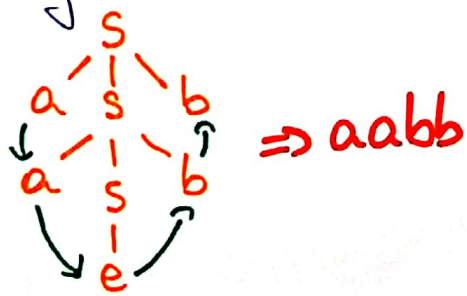
→ reprezentare grafică a derivărilor necesare pt. a obține un șir.

Obs: afișarea strict a frunzelor => șirul generat

ex: $S \to aSb \mid e$

Vrem să obținem $aabb$.

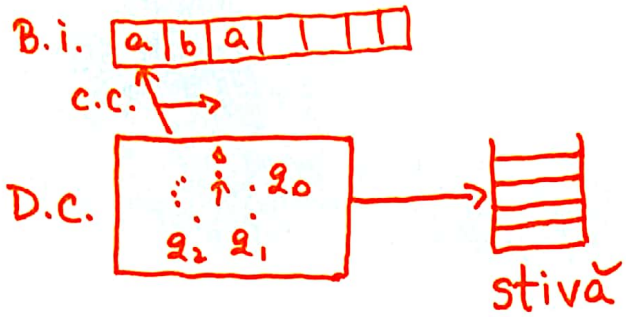
$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$



Scanned with CamScanner

Automate Push-Down

I. Problematică



→ automat extins cu o memorie auxiliară (stivă) ce poate reține anumite simboluri și poate fi manipulată prin operații push/pop

II. Noțiuni teoretice

Un APD este un tuplu $M = (K, \Sigma, \Gamma, \Delta, s, F)$ unde:

- K → mulțimea de stări (finită)
- Σ → alfabetul limbajului (de intrare)
- Γ → alfabetul stivei
- Δ → mulțimea de tranziții, $\Delta \subseteq (K \times \Sigma^* \times \Gamma^*) \times (K \times \Gamma^*)$ (relația de tranziție)
- s → starea inițială ($s \in K$)
- F → mulțimea stărilor finale, $F \subseteq K$

III. Configurația:

→ element din $K \times \Sigma^* \times \Gamma^*$ (stare, șir rămas de parcurs, stivă)

IV. Trecerea între configurații

$\vdash \in (K \times \Sigma^* \times \Gamma^*) \times (K \times \Sigma^* \times \Gamma^*)$

$\vdash^*$ = închiderea reflexivă și tranzitivă

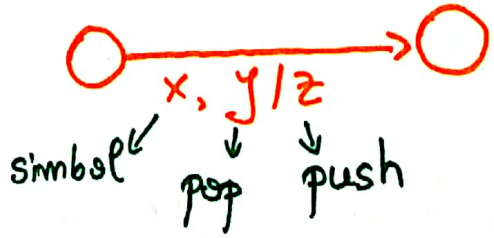
V. Șir acceptat

$w \in \Sigma^*$ acceptat $\Leftrightarrow (s, w, e) \vdash^* (f, e, e)$, $f \in F$
(acceptare cu stivă vidă)

VI. Limbaj acceptat

$L = \{$ mulțimea tuturor șirurilor acceptate $\}$

VII. Reprezentare grafică



Scanned with CamScanner

Proprietăți ale Limbajelor Independente de Context

I. Problematică

→ necesitatea unor tehnici de demonstrare a faptului că un limbaj este LIC pe baza proprietăților intrinseci ale clasei, plecând de la reprezentarea limbajului

II. Echivalența LIC și L(A.P.D.)

(T) Clasa $\mathcal{L}$(A.P.D.) este clasa LIC (în sensul că sunt generate de GIC)

Idee„:

GIC → APD

Construim un APD M care să simuleze regulile gramaticii

APD → GIC

Construim o GIC G care să simuleze tranzițiile APD

III. Proprietăți de închidere LIC

(T) Clasa LIC este închisă în raport cu operațiile $\cup, \cdot, ^*$.

III.1. Reuniunea

$G_1 = (V_1, \Sigma_1, R_1, S_1)$ $\quad M_1 = (K_1, \Sigma_1, \Gamma_1, \Delta_1, s_1, F_1)$

$G_2 = (V_2, \Sigma_2, R_2, S_2)$ $\quad M_2 = (K_2, \Sigma_2, \Gamma_2, \Delta_2, s_2, F_2)$

$G_\cup = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, R_1 \cup R_2 \cup \{S \to S_1 | S_2\}, S)$

$M_\cup = (K_1 \cup K_2 \cup \{s\}, \Sigma_1 \cup \Sigma_2, \Gamma_1 \cup \Gamma_2, \Delta_1 \cup \Delta_2 \cup \{((s,e,e),(s_1,e)), ((s,e,e),(s_2,e))\}, s, F_1 \cup F_2)$

Dacă avem neterminali comuni $(V_1 \setminus \Sigma_1) \cap (V_2 \setminus \Sigma_2) \neq \emptyset$ îi redenumim!

III.2. Concatenarea

$G_\cdot = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, R_1 \cup R_2 \cup \{S \to S_1 S_2\}, S)$

$M_\cdot = (K_1 \cup K_2, \Sigma_1 \cup \Sigma_2, \Gamma_1 \cup \Gamma_2, \Delta_1 \cup \Delta_2 \cup \{((f,e,e),(s_2,e)) | f \in F_1\}, s_1, F_2)$

III.3. Kleene-Star

$G_* = (V, \Sigma, R \cup \{S \to SS\}, S)$ $\quad M_* = (K \cup \{s'\}, \Sigma, \Delta \cup \{((s',e,e),(s,e))\} \cup \{((f,e,e),(s',e)) | f \in F\}, s', F \cup \{s'\})$

$M_* =$

(T) $LR \cap LIC = LIC$

Scanned with CamScanner

Proprietăți ale limbajelor regulate

I. Proprietăți de periodicitate

→ șirurile din limbajele regulate infinite au o structură repetitivă din cauza unui ciclu în A.F. sau operatorului Kleene-Star într-o E.R.

II. Lema de pompare

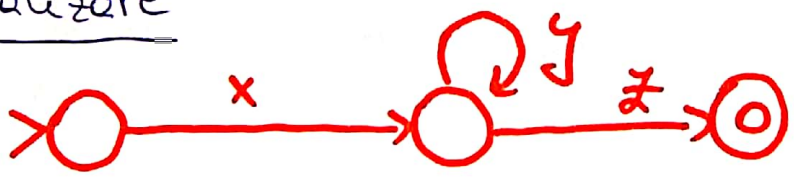
Dacă L este L.R. atunci $\exists\, n(L)$ o constantă naturală dependentă de limbaj, astfel încât $\forall\, w \in L$, $|w| \geq n(L)$, $\exists$ o scriere a lui w de forma $w = xyz$ cu proprietățile:

1) $|xy| < n$

2) $|y| > 0$

3) $xy^i z \in L$, $\forall i \in \mathbb{N}$

Vizualizare



Observații:

a) Proprietate a L.R. dar și a unor neregulate

b) **Implicația inversă este falsă!**

Scanned with CamScanner

Gramatici Independente de Context

I. Definiție:

O gramatică G este un tuplu (V, Σ, R, S), unde

- V → alfabetul gramaticii (terminali + neterminali)
- Σ → alfabetul limbajului (terminali)
- R → mulțimea regulilor
- S → simbolul de start

Obs:

$$R \subset (V \setminus \Sigma) \times V^*$$
$$\Leftrightarrow (A \to \gamma) \in R \Leftrightarrow A \in V \setminus \Sigma \text{ și } \gamma \in V^*$$

II. Derivarea $(\Rightarrow_G, \to)$

Fie $u, v \in V^*$

$u \Rightarrow_G v \Leftrightarrow \exists v', x, y \in V^*, A \in V \setminus \Sigma$, $(A \to v') \in R$ astfel încât $\begin{cases} u = xAy \\ v = xv'y \end{cases}$

Obs: Se înlocuiește partea stângă a regulii cu partea dreaptă.

III. Limbaj generat de gramatică

$$L(G) = \{w \in \Sigma^* \mid S \overset{*}{\underset{G}{\Rightarrow}} w\}$$

IV. Gramatică regulată

$G = (V, \Sigma, R, S)$ G.R. $\Leftrightarrow R \subseteq (V \setminus \Sigma) \times (\Sigma^* (V \setminus \Sigma) \cup \{e\})$

Obs: Toți neterminalii sunt fie în stânga, fie în dreapta.

V. Arbore de derivare

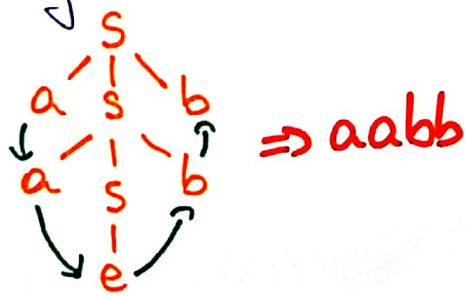
→ reprezentare grafică a derivărilor necesare pt. a obține un șir.

Obs: afișarea strict a frunzelor => șirul generat

ex: $S \to aSb \mid e$

Vrem să obținem $aabb$.

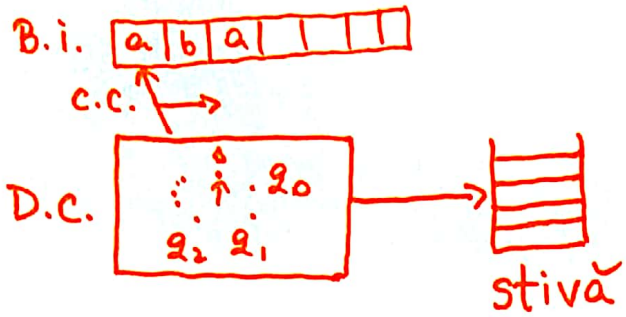
$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$



Scanned with CamScanner

Automate Push-Down

I. Problematică



→ automat extins cu o memorie auxiliară (stivă) ce poate reține anumite simboluri și poate fi manipulată prin operații push/pop

II. Noțiuni teoretice

Un APD este un tuplu $M = (K, \Sigma, \Gamma, \Delta, s, F)$ unde:

- K → mulțimea de stări (finită)
- Σ → alfabetul limbajului (de intrare)
- Γ → alfabetul stivei
- Δ → mulțimea de tranziții, $\Delta \subseteq (K \times \Sigma^* \times \Gamma^*) \times (K \times \Gamma^*)$ (relația de tranziție)
- s → starea inițială ($s \in K$)
- F → mulțimea stărilor finale, $F \subseteq K$

III. Configurația:

→ element din $K \times \Sigma^* \times \Gamma^*$ (stare, șir rămas de parcurs, stivă)

IV. Trecerea între configurații

$\vdash \in (K \times \Sigma^* \times \Gamma^*) \times (K \times \Sigma^* \times \Gamma^*)$

$\vdash^*$ = închiderea reflexivă și tranzitivă

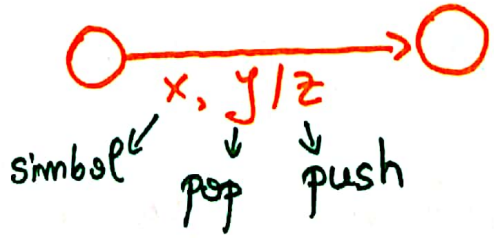
V. Șir acceptat

$w \in \Sigma^*$ acceptat $\Leftrightarrow (s, w, e) \vdash^* (f, e, e)$, $f \in F$
(acceptare cu stivă vidă)

VI. Limbaj acceptat

$L = \{$ mulțimea tuturor șirurilor acceptate $\}$

VII. Reprezentare grafică



Scanned with CamScanner

Proprietăți ale Limbajelor Independente de Context

I. Problematică

→ necesitatea unor tehnici de demonstrare a faptului că un limbaj este LIC pe baza proprietăților intrinseci ale clasei, plecând de la reprezentarea limbajului

II. Echivalența LIC și L(A.P.D.)

(T) Clasa $\mathcal{L}$(A.P.D.) este clasa LIC (în sensul că sunt generate de GIC)

Idee„:

GIC → APD

Construim un APD M care să simuleze regulile gramaticii

APD → GIC

Construim o GIC G care să simuleze tranzițiile APD

III. Proprietăți de închidere LIC

(T) Clasa LIC este închisă în raport cu operațiile $\cup, \cdot, ^*$.

III.1. Reuniunea

$G_1 = (V_1, \Sigma_1, R_1, S_1)$ $\quad M_1 = (K_1, \Sigma_1, \Gamma_1, \Delta_1, s_1, F_1)$

$G_2 = (V_2, \Sigma_2, R_2, S_2)$ $\quad M_2 = (K_2, \Sigma_2, \Gamma_2, \Delta_2, s_2, F_2)$

$G_\cup = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, R_1 \cup R_2 \cup \{S \to S_1 | S_2\}, S)$

$M_\cup = (K_1 \cup K_2 \cup \{s\}, \Sigma_1 \cup \Sigma_2, \Gamma_1 \cup \Gamma_2, \Delta_1 \cup \Delta_2 \cup \{((s,e,e),(s_1,e)), ((s,e,e),(s_2,e))\}, s, F_1 \cup F_2)$

Dacă avem neterminali comuni $(V_1 \setminus \Sigma_1) \cap (V_2 \setminus \Sigma_2) \neq \emptyset$ îi redenumim!

III.2. Concatenarea

$G_\cdot = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, R_1 \cup R_2 \cup \{S \to S_1 S_2\}, S)$

$M_\cdot = (K_1 \cup K_2, \Sigma_1 \cup \Sigma_2, \Gamma_1 \cup \Gamma_2, \Delta_1 \cup \Delta_2 \cup \{((f,e,e),(s_2,e)) | f \in F_1\}, s_1, F_2)$

III.3. Kleene-Star

$G_* = (V, \Sigma, R \cup \{S \to SS\}, S)$ $\quad M_* = (K \cup \{s'\}, \Sigma, \Delta \cup \{((s',e,e),(s,e))\} \cup \{((f,e,e),(s',e)) | f \in F\}, s', F \cup \{s'\})$

$M_* =$

(T) $LR \cap LIC = LIC$

Scanned with CamScanner

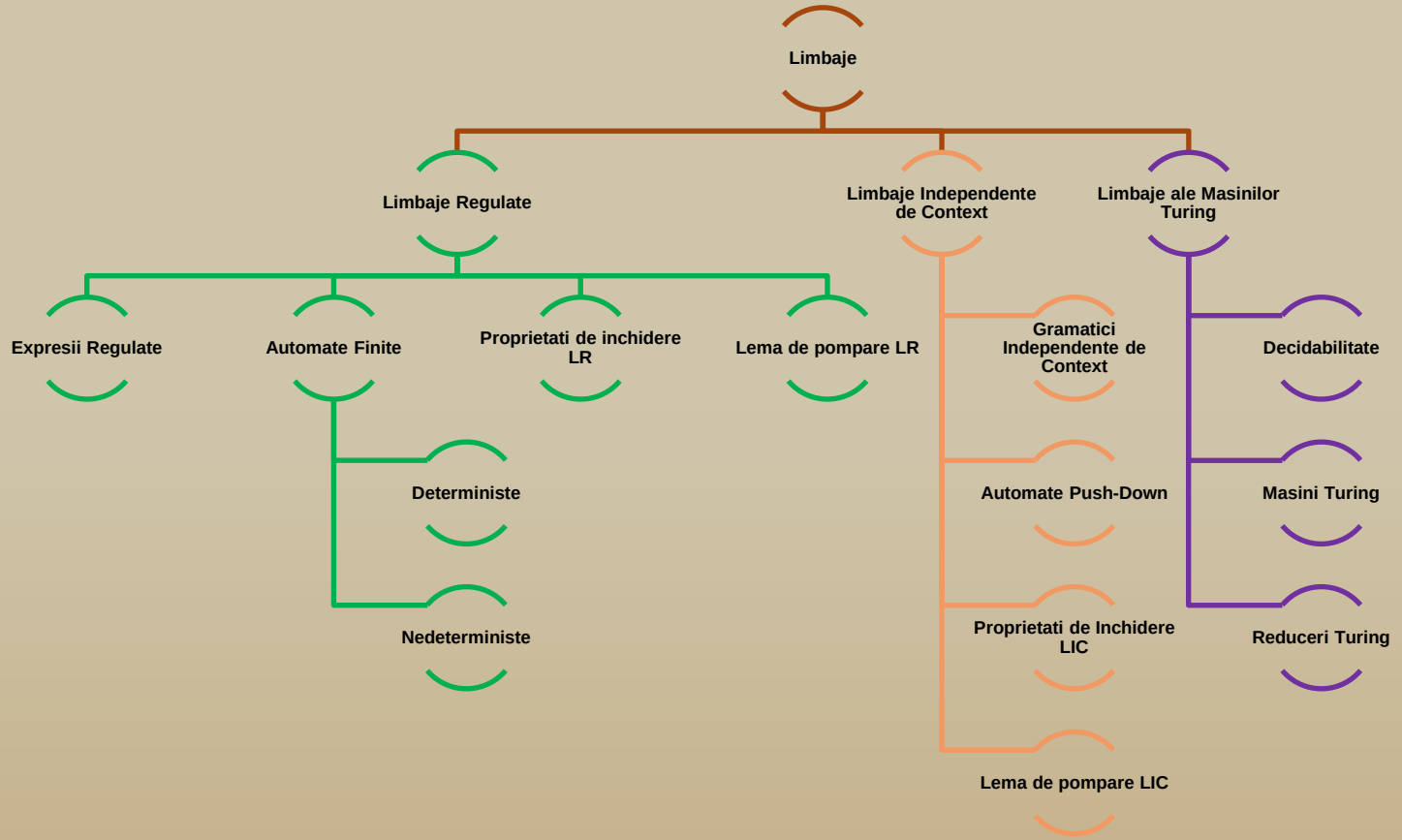
Seminar 1 – Automate Finite Deterministe

Alex Bogdan Andrei - LFA





Mind map LFA


Limbaje
Limbaje Regulate
Limbaje Independente de Context
Limbaje ale Masinilor Turing
Expresii Regulate
Automate Finite
Proprietati de inchidere LR
Lema de pompare LR
Gramatici Independente de Context
Decidabilitate
Deterministe
Automate Push-Down
Masini Turing
Nedeterministe
Proprietati de Inchidere LIC
Reduceri Turing
Lema de pompare LIC





Limbaj



Din ce este compus un limbaj?



Cum putem defini un alfabet?

Alfabet



Alfabetul Σ este o multime finita si nevida de caractere (simboli)

- $\Sigma = \{0, 1\} \rightarrow$ binar
- $\Sigma = \{a - z, A - Z, 0 - 9\} \rightarrow$ alfanumeric



Cum putem defini un cuvant?

Cuvant



Cuvantul w este o secventa finita de caractere din alfabetul Σ
Notam lungimea cuvantului cu $|w|$

- $\Sigma = \{0,1\} \rightarrow \mathrm{w} = 00110 \rightarrow |w| = 5$

Cuvant vid



Cuvantul w este o secventa finita de caractere din alfabetul Σ
Notam lungimea cuvantului cu $|w|$

- $\Sigma = \{0,1\} \rightarrow \mathrm{w} = 00110 \rightarrow |w| = 5$

Vom nota cuvantul vid cu e sau ϵ, acesta fiind cuvantul format prin "alegerea a 0 caractere din alfabet"

$$|e| = 0$$

Concatenare



Cuvantul w este o secventa finita de caractere din alfabetul Σ
Notam lungimea cuvantului cu $|w|$

- $\Sigma = \{0,1\} \rightarrow \mathrm{w} = 00110 \rightarrow |w| = 5$

Vom nota cuvantul vid cu e sau ϵ, acesta fiind cuvantul format prin "alegerea a 0 caractere din alfabet"

$$|e| = 0$$

Vom nota concatenarea cuvintelor cu

$$w = w_1 \cdot w_2 = w_1 w_2 \qquad w = we = ew$$

$$w^k = \underbrace{ww \ldots w}_{de\ k\ ori\ w}$$



Cum putem defini un limbaj?

Limbaj



Limbajul L este o multimer *finita* sau *infinita* de cuvinte p este alfabetul Σ

Limbaj vid



Limbajul L este o multimer *finita* sau *infinita* de cuvinte p este alfabetul Σ

Limbajul vid, notat $\emptyset$, este limbajul cu 0 cuvinte

$$(|\emptyset| = 0) \qquad \emptyset \neq \{e\}$$



Cum putem defini concatenarea?

$$L_1 \cdot L_2 = \{0, 01\} \cdot \{11,1\} = \{011, 01, 0111\}$$

Concatenare



$$L_1 \cdot L_2 = \{w_1 w_2 | w_1 \in L_1 \; si \; w_2 \in \; L_2\}$$

Vom defini concatenarea a 2 limbaje ca fiind multimea formata prin concatenarea tuturor cuvintelor din L_1 cu cele din L_2, avan d proprietatile:

$$L = L \cdot \{e\} = \{e\} \cdot L \qquad \emptyset = L \cdot \emptyset = \emptyset \cdot L$$

Concatenare



$$L_1 \cdot L_2 = \{w_1 w_2 | w_1 \in L_1 \; si \; w_2 \in \; L_2\}$$

Vom defini concatenarea a 2 limbaje ca fiind multimea formata prin concatenarea tuturor cuvintelor din L_1 cu cele din L_2, avan d proprietatile:

$$L = L \cdot \{e\} = \{e\} \cdot L \qquad \emptyset = L \cdot \emptyset = \emptyset \cdot L$$

$$L^k = \underbrace{LL \ldots L}_{de\ k\ ori\ L}$$

L^k = L concatenat cu el insusi de k ori.

L^0 = limbajul obtinut prin concatenarea cuvintelor din L de 0 ori

$L^0 = ?$

Concatenare



$$L_1 \cdot L_2 = \{w_1 w_2 | w_1 \in L_1 \; si \; w_2 \in \; L_2\}$$

Vom defini concatenarea a 2 limbaje ca fiind multimea formata prin concatenarea tuturor cuvintelor din L_1 cu cele din L_2, avan d proprietatile:

$$L = L \cdot \{e\} = \{e\} \cdot L \qquad \emptyset = L \cdot \emptyset = \emptyset \cdot L$$

$$L^k = \underbrace{LL \ldots L}_{de\ k\ ori\ L}$$

L^k = L concatenat cu el insusi de k ori.

L^0 = limbajul obtinut prin concatenarea cuvintelor din L de 0 ori

$L^0 = \{e\} \neq \{\}$

Kleene Star



$$L^* = \bigcup_{i \geq 0} L^i = L^0 \cup L^1 \cup L^2 \cup \cdots$$

Putem defini Kleene Star ca fiind reuniunea puterilor limbajului L, concatenarea lui L cu el insusi fiind realizata de

"0 sau mai multe ori"

$$L = \{a\} \Rightarrow L^* = \underbrace{\{e\}}_{L^0} \cup \underbrace{\{a\}}_{L^1} \cup \underbrace{\{aa\}}_{L^2} \cup \underbrace{\{aaa\}}_{L^3} \cup \cdots$$



AFD



Ce inseamna *Automat Finit Determinist*?

AFD informal



Automat = un mecanism de control ce oferă un răspuns în funcție de secvența de instrucțiuni pe care o urmează

Finit = numărul de stări este finit

Determinist = din fiecare stare pot ajunge în **fix** o altă stare unică consumând **fix** un caracter

AFD definitie



Un automat finit determinist (AFD) este un tuplu $M = (K, \Sigma, \delta, s_i, F)$, unde:

- **K** = mulțimea de stări
- $\boldsymbol{\Sigma}$ = alfabetul folosit
- $\boldsymbol{\delta : K \times \Sigma \rightarrow K}$= funcția de tranziție
- $\boldsymbol{s_i}$ = starea inițială
- **F** = mulțimea stărilor finale

Un automat finit determinist (AFD) este un tuplu $M = (K, \Sigma, \delta, s_i, F)$, unde:

- **K** = mulțimea de stări
- $\boldsymbol{\Sigma}$ = alfabetul folosit
- $\boldsymbol{\delta : K \times \Sigma \rightarrow K}$= funcția de tranziție
- $\boldsymbol{s_i}$ = starea inițială
- **F** = mulțimea stărilor finale

1. Cum putem defini o configuratie?
2. Cum putem defini o tranzitie?
3. Cand un cuvant este acceptat?
4. Ce este *multimea de cuvinte acceptate* a unui AFD?



AFD configuratie



O configurație reprezintă starea în care ne aflăm și cuvântul pe care îl mai avem de consumat.

$$(K \times \Sigma^*) = (q_i, w)$$

AFD tranzitie



O configurație reprezintă starea în care ne aflăm și cuvântul pe care îl mai avem de consumat.

$$(K \times \Sigma^*) = (q_i, w)$$

O tranziție reprezintă trecerea dintr-o configurație în alta prin consumarea unui caracter

$$(K \times \Sigma^*) \rightarrow (K \times \Sigma^*) = (q_i, \alpha w) \rightarrow (q_j, w), \qquad \alpha \in \Sigma$$

AFD acceptare



O configurație reprezintă starea în care ne aflăm și cuvântul pe care îl mai avem de consumat.

$$(K \times \Sigma^*) = (q_i, w)$$

O tranziție reprezintă trecerea dintr-o configurație în alta prin consumarea unui caracter

$$(K \times \Sigma^*) \rightarrow (K \times \Sigma^*) = (q_i, \alpha w) \rightarrow (q_j, w), \qquad \alpha \in \Sigma$$

Un AFD acceptă un cuvânt dacă

$$(s_i \times \Sigma^*) \vdash^* (F \times e) = (s_i, w) \rightarrow (f, e)$$

AFD
limbaj acceptat



O configurație reprezintă starea în care ne aflăm și cuvântul pe care îl mai avem de consumat.

$$(K \times \Sigma^*) = (q_i, w)$$

O tranziție reprezintă trecerea dintr-o configurație în alta prin consumarea unui caracter

$$(K \times \Sigma^*) \to (K \times \Sigma^*) = (q_i, \alpha w) \to (q_j, w), \qquad \alpha \in \Sigma$$

Un AFD acceptă un cuvânt dacă

$$(s_i \times \Sigma^*) \vdash^* (F \times e) = (s_i, w) \to (f, e)$$

Se numește Limbaj acceptat mulțimea cuvintelor ac ceptate de un AFD.

Sink State

Chiuveta



Un AFD va trebui sa consume mereu cuvantul primit ca input.

In cazul in care pattern-ul cerut nu a fost respectat, atunci vom merge intr-o stare care **nu este finala** si vom consuma restul cuvantului.

Exemplu AFD



Construiti un AFD care sa accepte string-urile in format email



Exercitii

$$\Sigma = \{0, 1\} \rightarrow L = \Sigma^*$$



$$\Sigma = \{0, 1\} \rightarrow L = \emptyset$$



$$\Sigma = \{a, b\} \rightarrow L = \{\mathrm{w} \in \Sigma^* \mid \#_{aba}(w) \geq 1\}$$



$$\Sigma = \{a, b\} \rightarrow L = \{\mathrm{w} \in \Sigma^* \mid w \; nu \; se \; termină \; cu \; baba\}$$



$$\Sigma = \{0,1\} \rightarrow L = \{\mathrm{w} \in \Sigma^* \mid \#_0(w) \vdots 3\}$$



$$\Sigma = \{0,1\} \rightarrow L = \{\mathrm{w} \in \Sigma^* \mid \#_{01}(w) = \#_{10}(w)\}$$





$$\Sigma = \{0,1\} \rightarrow L = \{\mathrm{w} \in \Sigma^* \mid w \; se \; termină \; cu \; 111\}$$



$$\Sigma = \{0, 1\} \rightarrow L = \{\mathrm{w} \in \Sigma^* \mid w \text{ începe cu } 111\}$$



$$\Sigma = \{1, 2, 3\} \rightarrow L = \{\mathrm{w} \in \Sigma^* \mid \#_1(w) \; este \; divizibil \; cu \; 3\}$$



$$\Sigma = \{a, b\} \rightarrow L = \{\mathrm{w} \in \Sigma^* \mid \#_a(w) + \#_b(w) = 3\}$$





Intrebari?

Oricand pe Facebook / Mail

Tema AFD

Având alfabetul Σ și limbajul L, desenați AFD pentru

1. (1p) $\Sigma = \{1,2,3\} \rightarrow L = \{w \in \Sigma^* \mid \#_1(w)\ este\ divizibil\ cu\ 3\}$ - număr minim de stări
2. (1p) $\Sigma = \{a,b\} \rightarrow L = \{w \in \Sigma^* \mid \#_a(w) + \#_b(w) = 3\}$ - număr minim de stări
3. (1p) $\Sigma = \{a,b\} \rightarrow L = \{w \in \Sigma^* \mid w\ \hat{i}ncepe\ \text{și}\ se\ termină\ cu\ aceeași\ literă\}$ - număr minim de stări
4. (1p) $\Sigma = \{a,b,c\} \rightarrow L = \{w \in \Sigma^* \mid w\ nu\ conține\ 2\ simboluri\ identice\ alăturate\}$

5. (1p) Dacă $M = (K, \Sigma, \delta, s, F)$ este un AFD și $\forall\, w \in \Sigma^*, \forall\, q \in K, (q,w) \vdash^* (s,e)$,
atunci putem afirma despre M că:
$L = \Sigma^*$ | $L = \emptyset$ | $w \in L \Rightarrow ww \in L$ | nu poate exista M conform enunțului (Justificați răspunsul)



Seminar 2
Expresii regulate

Alex Bogdan Andrei – LFA 336CB





Grila DFA

Grila AFD

Notati doar raspunsurile pe o foaie separata

1. Daca AFD-ul $M = (K, \Sigma, \delta, s_i, F)$, atunci:
   a) $|F| \geq 1$
   b) $|F| \geq 0$
   c) $|F| = 1$
   d) $|F| \leq 1000$
2. Care conditie este suficienta pentru ca AFD-ul $M = (K, \Sigma, \delta, s_i, F)$ sa accepte $\emptyset$?
   a) $\mathrm{s_i} \in F$
   b) $|\mathrm{F}| = 1$
   c) $|\mathrm{F}| = 0$
   d) δ sa fie o functie totala
3. Care conditie este necesara pentru ca AFD-ul $M = (K, \Sigma, \delta, s_i, F)$ sa accepte $\{e\}$?
   a) $\mathrm{s_i} \in F$
   b) $|\mathrm{F}| = 1$
   c) $|\mathrm{F}| = 0$
   d) δ sa fie o functie totala
4. Care este numarul maxim de stari necesar pentru un AFD care accepta limbajul $\{aa\}^*$?
   a) 0
   b) 1
   c) 2
   d) Nu avem un numar maxim de stari
5. Numarul minim de stari ale unui AFD care accepta limbajul $L = \{\mathrm{w} \in \Sigma^* \mid \#_1(w)\ este\ divizibil\ cu\ 3\}$ este:
   a) 0
   b) 1
   c) 2
   d) 4



Grila AFD

Notati doar raspunsurile pe o foaie separata

1. Daca AFD-ul $M = (K, \Sigma, \delta, s_i, F)$, atunci:
   a) $|F| \geq 1$
   b) $|F| \geq 0$
   c) $|F| = 1$
   d) $|F| \leq 1000$
2. Care conditie este suficienta pentru ca AFD-ul $M = (K, \Sigma, \delta, s_i, F)$ sa accepte $\emptyset$?
   a) $\mathrm{s_i} \in F$
   b) $|\mathrm{F}| = 1$
   c) $|\mathrm{F}| = 0$
   d) δ sa fie o functie totala
3. Care conditie este necesara pentru ca AFD-ul $M = (K, \Sigma, \delta, s_i, F)$ sa accepte $\{e\}$?
   a) $\mathrm{s_i} \in F$
   b) $|\mathrm{F}| = 1$
   c) $|\mathrm{F}| = 0$
   d) δ sa fie o functie totala
4. Care este numarul maxim de stari necesar pentru un AFD care accepta limbajul $\{aa\}^*$?
   a) 0
   b) 1
   c) 2
   d) Nu avem un numar maxim de stari
5. Numarul minim de stari ale unui AFD care accepta limbajul $L = \{\mathrm{w} \in \Sigma^* \mid \#_1(w)\ este\ divizibil\ cu\ 3\}$ este:
   a) 0
   b) 1
   c) 2
   d) 4



Grila AFD

Notati doar raspunsurile pe o foaie separata

1. Daca AFD-ul $M = (K, \Sigma, \delta, s_i, F)$, atunci:
   a) $|F| \geq 1$
   b) $|F| \geq 0$
   c) $|F| = 1$
   d) $|F| \leq 1000$
2. Care conditie este suficienta pentru ca AFD-ul $M = (K, \Sigma, \delta, s_i, F)$ sa accepte $\emptyset$?
   a) $s_i \in F$
   b) $|F| = 1$
   c) $|F| = 0$
   d) δ sa fie o functie totala
3. Care conditie este necesara pentru ca AFD-ul $M = (K, \Sigma, \delta, s_i, F)$ sa accepte $\{e\}$?
   a) $s_i \in F$
   b) $|F| = 1$
   c) $|F| = 0$
   d) δ sa fie o functie totala
4. Care este numarul maxim de stari necesar pentru un AFD care accepta limbajul $\{aa\}^*$?
   a) 0
   b) 1
   c) 2
   d) Nu avem un numar maxim de stari
5. Numarul minim de stari ale unui AFD care accepta limbajul $L = \{w \in \Sigma^* \mid \#_1(w) \; este \; divizibil \; cu \; 3\}$ este:
   a) 0
   b) 1
   c) 2
   d) 4



Grila AFD

Notati doar raspunsurile pe o foaie separata

1. Daca AFD-ul $M = (K, \Sigma, \delta, s_i, F)$, atunci:
   a) $|F| \geq 1$
   b) $|F| \geq 0$
   c) $|F| = 1$
   d) $|F| \leq 1000$
2. Care conditie este suficienta pentru ca AFD-ul $M = (K, \Sigma, \delta, s_i, F)$ sa accepte $\emptyset$?
   a) $\mathrm{s_i} \in F$
   b) $|\mathrm{F}| = 1$
   c) $|\mathrm{F}| = 0$
   d) δ sa fie o functie totala
3. Care conditie este necesara pentru ca AFD-ul $M = (K, \Sigma, \delta, s_i, F)$ sa accepte $\{e\}$?
   a) $\mathrm{s_i} \in F$
   b) $|\mathrm{F}| = 1$
   c) $|\mathrm{F}| = 0$
   d) δ sa fie o functie totala
4. Care este numarul maxim de stari necesar pentru un AFD care accepta limbajul $\{aa\}^*$?
   a) 0
   b) 1
   c) 2
   d) Nu avem un numar maxim de stari
5. Numarul minim de stari ale unui AFD care accepta limbajul $L = \{\mathrm{w} \in \Sigma^* \mid \#_1(w)\ este\ divizibil\ cu\ 3\}$ este:
   a) 0
   b) 1
   c) 2
   d) 4



Grila AFD

Notati doar raspunsurile pe o foaie separata

1. Daca AFD-ul $M = (K, \Sigma, \delta, s_i, F)$, atunci:
   a) $|F| \geq 1$
   b) $|F| \geq 0$
   c) $|F| = 1$
   d) $|F| \leq 1000$
2. Care conditie este suficienta pentru ca AFD-ul $M = (K, \Sigma, \delta, s_i, F)$ sa accepte $\emptyset$?
   a) $s_i \in F$
   b) $|F| = 1$
   c) $|F| = 0$
   d) δ sa fie o functie totala
3. Care conditie este necesara pentru ca AFD-ul $M = (K, \Sigma, \delta, s_i, F)$ sa accepte $\{e\}$?
   a) $s_i \in F$
   b) $|F| = 1$
   c) $|F| = 0$
   d) δ sa fie o functie totala
4. Care este numarul maxim de stari necesar pentru un AFD care accepta limbajul $\{aa\}^*$?
   a) 0
   b) 1
   c) 2
   d) Nu avem un numar maxim de stari
5. Numarul minim de stari ale unui AFD care accepta limbajul $L = \{w \in \Sigma^* \mid \#_1(w) \; este \; divizibil \; cu \; 3\}$ este:
   a) 0
   b) 1
   c) 2
   d) 4



Grila AFD

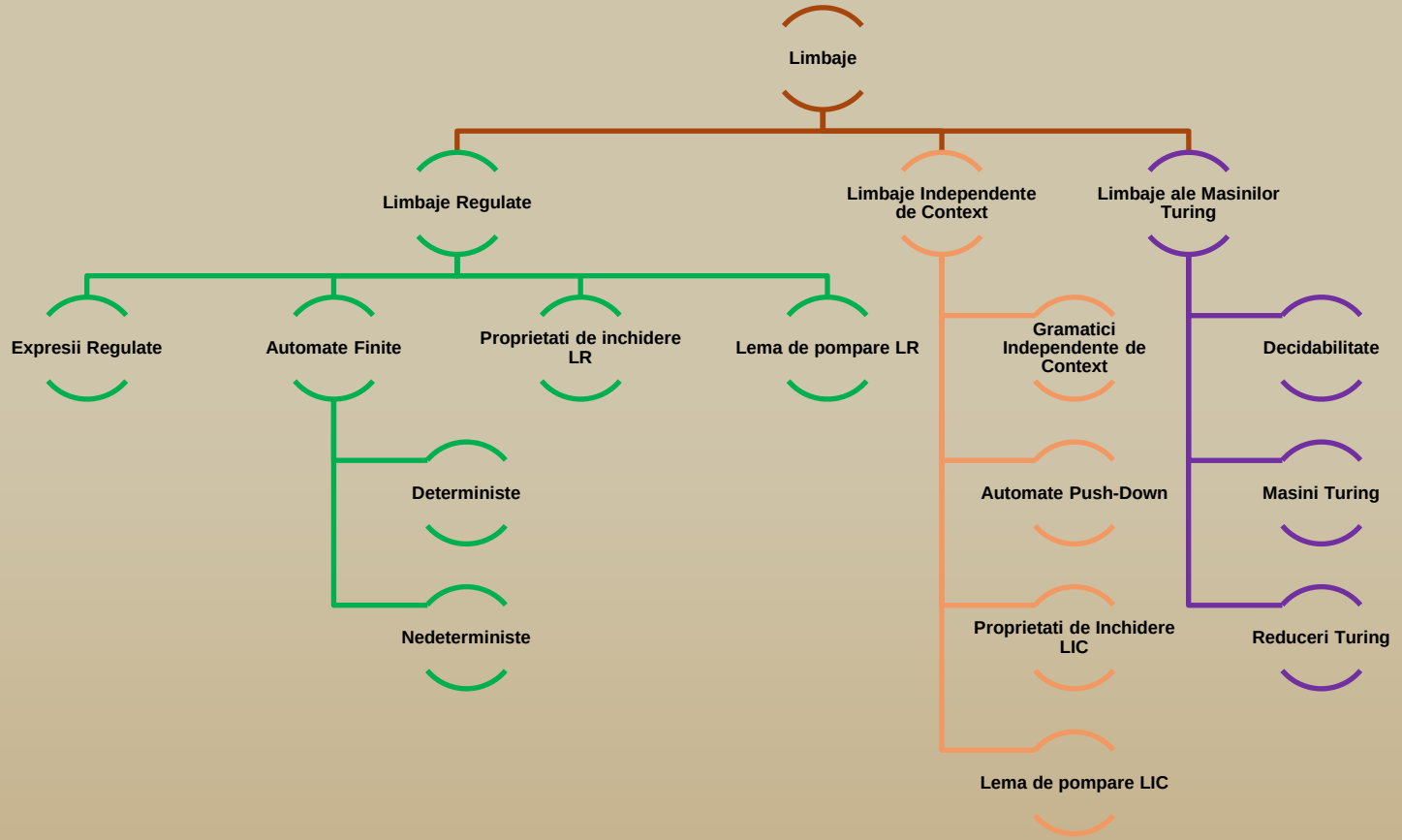
Notati doar raspunsurile pe o foaie separata

1. Daca AFD-ul $M = (K, \Sigma, \delta, s_i, F)$, atunci:
   a) $|F| \geq 1$
   b) $|F| \geq 0$
   c) $|F| = 1$
   d) $|F| \leq 1000$
2. Care conditie este suficienta pentru ca AFD-ul $M = (K, \Sigma, \delta, s_i, F)$ sa accepte $\emptyset$?
   a) $s_i \in F$
   b) $|F| = 1$
   c) $|F| = 0$
   d) δ sa fie o functie totala
3. Care conditie este necesara pentru ca AFD-ul $M = (K, \Sigma, \delta, s_i, F)$ sa accepte $\{e\}$?
   a) $s_i \in F$
   b) $|F| = 1$
   c) $|F| = 0$
   d) δ sa fie o functie totala
4. Care este numarul maxim de stari necesar pentru un AFD care accepta limbajul $\{aa\}^*$?
   a) 0
   b) 1
   c) 2
   d) Nu avem un numar maxim de stari
5. Numarul minim de stari ale unui AFD care accepta limbajul $L = \{w \in \Sigma^* \mid \#_1(w) \; este \; divizibil \; cu \; 3\}$ este:
   a) 0
   b) 1
   c) 2
   d) 3





Expresii Regulate


Limbaje
Limbaje Regulate
Limbaje Independente de Context
Limbaje ale Masinilor Turing
Expresii Regulate
Automate Finite
Proprietati de inchidere LR
Lema de pompare LR
Gramatici Independente de Context
Decidabilitate
Deterministe
Automate Push-Down
Masini Turing
Nedeterministe
Proprietati de Inchidere LIC
Reduceri Turing
Lema de pompare LIC





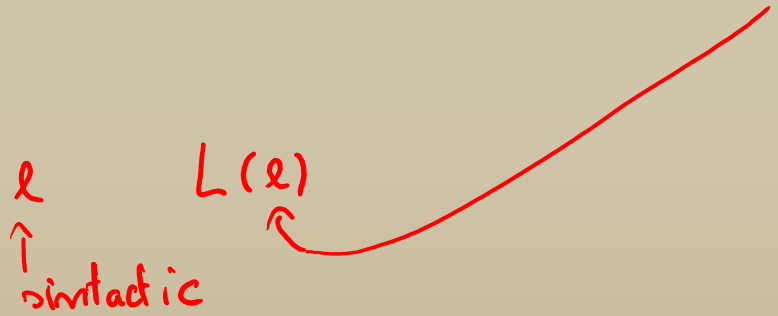
Cum putem defini o Expresie Regulata?

Expresie Regulata



O expresie regulata reprezinta un mod (o regula) de a descrie un limbaj formal

Expresia regulata este un cuvant (la nivel sintactic) a carui interpretare ne va oferi un limbaj formal (la nivel semantic)





Expresie Regulata ca TDA?

Expresie Regulata



Cazurile de baza (Expresii regulate simple)

- e (sirul vid)
- $\emptyset$ (expresie vida)
- $\alpha \in \Sigma$ (un singur caracter α)

Expresie Regulata



Cazurile de baza (Expresii regulate simple)

- e (sirul vid)
- $\emptyset$ (expresie vida)
- $\alpha \in \Sigma$ (un singur caracter α)

Cazuri recursive (formate din cazurile de baza)

- Concatenare: $r_1, r_2 \in ER \rightarrow r_1 \cdot r_2 \in ER$
- Reuniune: $r_1, r_2 \in ER \rightarrow r_1 \cup r_2 \in ER$
- Kleene Star: $r_1 \in ER \rightarrow (r_1)^* \in ER$

Expresie Regulata



Cazurile de baza (Expresii regulate simple)

- e (sirul vid)
- $\emptyset$ (expresie vida)
- $\alpha \in \Sigma$ (un singur caracter α)

Cazuri recursive (formate din cazurile de baza)

- Concatenare: $r_1, r_2 \in ER \rightarrow r_1 \cdot r_2 \in ER$
- Reuniune: $r_1, r_2 \in ER \rightarrow r_1 \cup r_2 \in ER$
- Kleene Star: $r_1 \in ER \rightarrow (r_1)^* \in ER$

Precedenta operatori

Reuniune < Concatenare < Kleene Star

$$(a(b^*)c) \cup e = ab^*c \cup e$$

Expresie Regulata Generalizata



Cazurile de baza (Expresii regulate simple)

- e (sirul vid)
- $\emptyset$ (expresie vida)
- $\alpha \in \Sigma$ (un singur caracter α)

Cazuri recursive (formate din cazurile de baza)

- Concatenare: $r_1, r_2 \in ER \rightarrow r_1 \cdot r_2 \in ER$
- Reuniune: $r_1, r_2 \in ER \rightarrow r_1 \cup r_2 \in ER$
- Kleene Star: $r_1 \in ER \rightarrow (r_1)^* \in ER$

Putem adauga alte 2 proprietati? $a^* \cap (aa)^* = (aa)^*$

- Intersectie: $r_1, r_2 \in ER \rightarrow r_1 \cap r_2 \in ER$
- Complement: $r_1 \in ER \rightarrow \overline{r_1} \in ER$ Σ^*

Expresie Regulata



$$r_1 \in ER \rightarrow (r_1)^* = \bigcup_{i \geq 0} r^i = r^0 \cup r^1 \cup r^2 \cup \cdots$$

Putem defini Kleene Star ca fiind expresia regulata obtinuta in urma concatenarii sale de “0 sau mai multe ori”

$$(a)^* = a^* = \underbrace{e}_{0 \text{ de } a} \cup a \cup aa \cup aaa \cup \cdots$$

Expresie Regulata



Avand alfabetul $\Sigma = \{\alpha_1, \alpha_2, \dots, \alpha_n\}$, vom nota expresia regulata

$$(\alpha_1 \cup \alpha_2 \cup \cdots \cup \alpha_n) = \Sigma$$

reprezentand “orice litera din alfabet”



Limbaj Regulat



Cum putem defini un Limbaj Regulat?

Limbaj Regulat



Plecand de la modul recursiv de definire a expresiilor regulate, vom avea urmatoarele proprietati:

Limbaj Regulat



Plecand de la modul recursiv de definire a expresiilor regulate, vom avea urmatoarele proprietati:

Cazuri de baza

$$\begin{cases} L(e) = \{e\} \\ L(\emptyset) = \{\,\} \\ L(\alpha \in \Sigma) = \{\alpha\} \end{cases}$$

Limbaj Regulat



Plecand de la modul recursiv de definire a expresiilor regulate, vom avea urmatoarele proprietati:

Cazuri de baza

$$\begin{cases} L(e) = \{e\} \\ L(\emptyset) = \{\,\} \\ L(\alpha \in \Sigma) = \{\alpha\} \end{cases}$$

Cazuri recursive

$$\forall r_1, r_2 \in ER \Rightarrow \begin{cases} L(r_1 \cdot r_2) = L(r_1) \cdot L(r_2) \\ L(r_1 \cup r_2) = L(r_1) \cup L(r_2) \\ L(r_1^*) = \big(L(r_1)\big)^* \end{cases}$$

Σ^*



Folosind notatia introdusa anterior

$$(\alpha_1 \cup \alpha_2 \cup \cdots \cup \alpha_n) = \Sigma$$

vom putea defini limbajul Σ^* ca fiind limbajul ce contine toate cuvintele posibile peste alfabetul Σ

Σ^*



Folosind notatia introdusa anterior

$$(\alpha_1 \cup \alpha_2 \cup \cdots \cup \alpha_n) = \Sigma$$

vom putea defini limbajul Σ^* ca fiind limbajul ce contine toate cuvintele posibile peste alfabetul Σ

$\Sigma^* \stackrel{\text{def}}{=}$ multimea tuturor string-urilor finite peste Σ

Astfel orice limbaj L este doar un subset al limbajului Σ^*

Expresie regulata pentru adrese de email



Expresie regulata pentru adrese IPv4

$N = 0 - 255$

$\Sigma = \{0, \ldots, 9, .\}$

$C = [0-9]$

$N = \overset{C}{[0-9]} \cup \overset{C}{[1-9][0-9]} \cup \overset{C}{1}\overset{C}{[0-9][0-9]} \cup$

$2\overset{C}{[0-4]}[0-9] \cup 25[0-5]$

$ip = N.N.N.N$





Exercitii

$$\Sigma = \{a, b\} \rightarrow L = \{aab, abb\}$$

aabUabb

a (a U b) b



$\Sigma = \{0, 1\} \rightarrow L = \{w \in \Sigma^* \mid w \text{ începe cu } 0\}$

Σ

$0\,(0 \cup 1)^*$

$0\,\Sigma^*$

l.r. $\cup\, \Sigma^* = \Sigma^*$

l.r. $\cdot\, \Sigma^* \neq \Sigma^*$



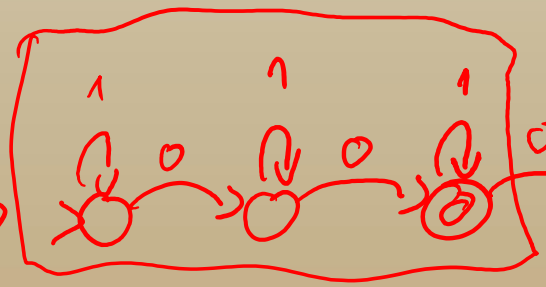
$$\Sigma = \{0,1\} \rightarrow L = \{w \in \Sigma^* \mid \#_0(w) = 2\}$$

$(00)^* \cup \Sigma^*$

$\Sigma^* 0 \Sigma^* 0 \Sigma^*$

$001^* \cup 1^*001^* \cup 1^*001^*$

$1^*01^*01^*$



$1^* = \varepsilon \cup 1 \cup 11 \cup \ldots$



$$\Sigma = \{a, b\} \rightarrow L = \{w \in \Sigma^* \mid |w| \,\%\, 2 = 0\}$$

$(\Sigma\ \Sigma)^*$ $(\Sigma \cup \Sigma)^* = \Sigma^*$

aa

ab

ba

bb



$$\Sigma = \{0,1\} \rightarrow L = \{w \in \Sigma^* \mid \#_0(w) \geq 1\}$$

$\Sigma^* 0 \Sigma^* =$
- $\Sigma^* 0$
- 0101 0 01110
- $0 \Sigma^*$



$\Sigma = \{0,1\} \rightarrow L = \{w \in \Sigma^* \mid \#_0(w)\ este\ par\}$

$(1^*(0^*1^*)^*1^*)^*$

$(00 \cup 1)^*$

111000011110100

001111000011100

$1^* \cup (1^*01^*01^*)^*$

10101 10111011



$\Sigma = \{a, b\} \rightarrow L = \{w \in \Sigma^* \mid w \text{ nu conține } ba\}$

a^*b^*

$(a^*b^*)^* = abab$

aaaaabbb



$$\Sigma = \{a, b, c\} \rightarrow L = \{\mathrm{w} \in \Sigma^* \mid \#_a(w) \geq 1 \text{ și } \#_c(w) \geq 1\}$$



$$\Sigma = \{a, b\} \rightarrow L = \left\{ \begin{matrix} w \in \Sigma^* \mid w \text{ incepe și se} \\ \text{termină cu aceeași literă} \end{matrix} \right\}$$

$$(a\Sigma^*a) \cup (b\Sigma^*b)$$



$$\Sigma = \{0,1\} \rightarrow L = \{w \in \Sigma^* \mid \#_0(w) \,\%\, 2 = 0 \text{ si } \#_1(w) \,\%\, 2 = 1\}$$

AFN $\longleftrightarrow$ E.R.



$\Sigma = \{0,1\} \rightarrow L = \{w \in \Sigma^* \mid |\#_1(w_1) - \#_0(w_1)| \leq 1, \forall\, w_1 \text{ prefix al lui } w\}$

$+ = \Sigma \cup \varepsilon$

$(01 \cup 10)^* \Sigma$

$\not\Sigma$

001





$\Sigma = \{0, 1\} \rightarrow L = \{w \in \Sigma^* \mid \exists w_1, w_2 \in \Sigma^* \text{ a.i. } w = w_1 w_2 w_1\}$

$L = \Sigma^*$

$w = w_1 \; w_2 \; w_1 \; , \; w_2 = \Sigma^* \rightarrow$ toate cuvintele

$w_1 = \varepsilon$, $w_1 = \varepsilon$



$$\Sigma = \{-, 0, 1, .., 9\} \rightarrow L = \mathbb{Z}$$

1012

$- \Sigma^*$

$- [1-9]^* \cup 0 \cup [1-9]^*$

$- [1-9][0-9]^* \cup 0 \cup [1-9][0-9]^*$



$\Sigma = \{a, b\} \rightarrow L = \{w \in \Sigma^* \mid$ orice a din w este precedat sau urmat de un b$\}$

$(ab^*)^* \rightarrow aba$

$\hookrightarrow aaaaa$

$(b^*ab^*)^*$

ab

ba

aba

$(b^*bab^*)^* \quad (b^*ab)^*$

$((ab) \cup (ba) \cup b \cup aba)^*$



$$\Sigma = \{a, b, c\} \rightarrow L = \{w \in \Sigma^* \mid w \text{ incepe cu } abc \text{ si se termina cu } cba\}$$



$$\Sigma = \{/, \$, a, .., z\} \rightarrow L = \left\{ \begin{matrix} w \in \Sigma^* \mid w \text{ este un comentariu C} \\ \text{folosind \$ în loc de } * \end{matrix} \right\}$$

/*

ceva $\rightarrow \Sigma^*$

*/



Cum construim o expresie regulată care să descrie numele de variabile din limbajul C?





Intrebari?

Oricand pe Facebook / Mail

Tema ER

1. (2p) $\Sigma = \{\%, /, a, .., z\} \rightarrow L = \left\{ \begin{matrix} w \in \Sigma^* \mid w \text{ este un block comment C} \\ \text{folosind \% în loc de } * \end{matrix} \right\}$
2. (1p) $\Sigma = \{a, b\} \rightarrow L = \{w = w1w2 \mid \exists\, w1, w2 \in \Sigma * \; astfel\; încât\; |w2| = |w1| \,\%\, 3\}$
3. (1p) $\Sigma = \{0,4\} \rightarrow L = \{w \in \Sigma^* \mid$ w nu conține ”404”$\}$
4. Fie expresia regulată $r = (aab \cup bbaba)^* baba$ și $L = L(r)$.
   1. (1p) Găsiți o expresie regulată pentru limbajul $L^r = \{w^r \mid w \in L\}$
   2. (2p) Definiți recursiv expresia regulată inversată $\mathbf{e^r}$ a unei expresii regulate e.
   3. (2p) Demonstrați că ”$\forall\, e \in ER, L = L(e) \Rightarrow L^r = L(e^r)$”.
5. (1p) Descrieți un algoritm (pas cu pas) prin care putem elimina simbolul Ø dintr-o expresie regulată, în cazul în care aceasta nu descrie limbajul vid.



Seminar 4

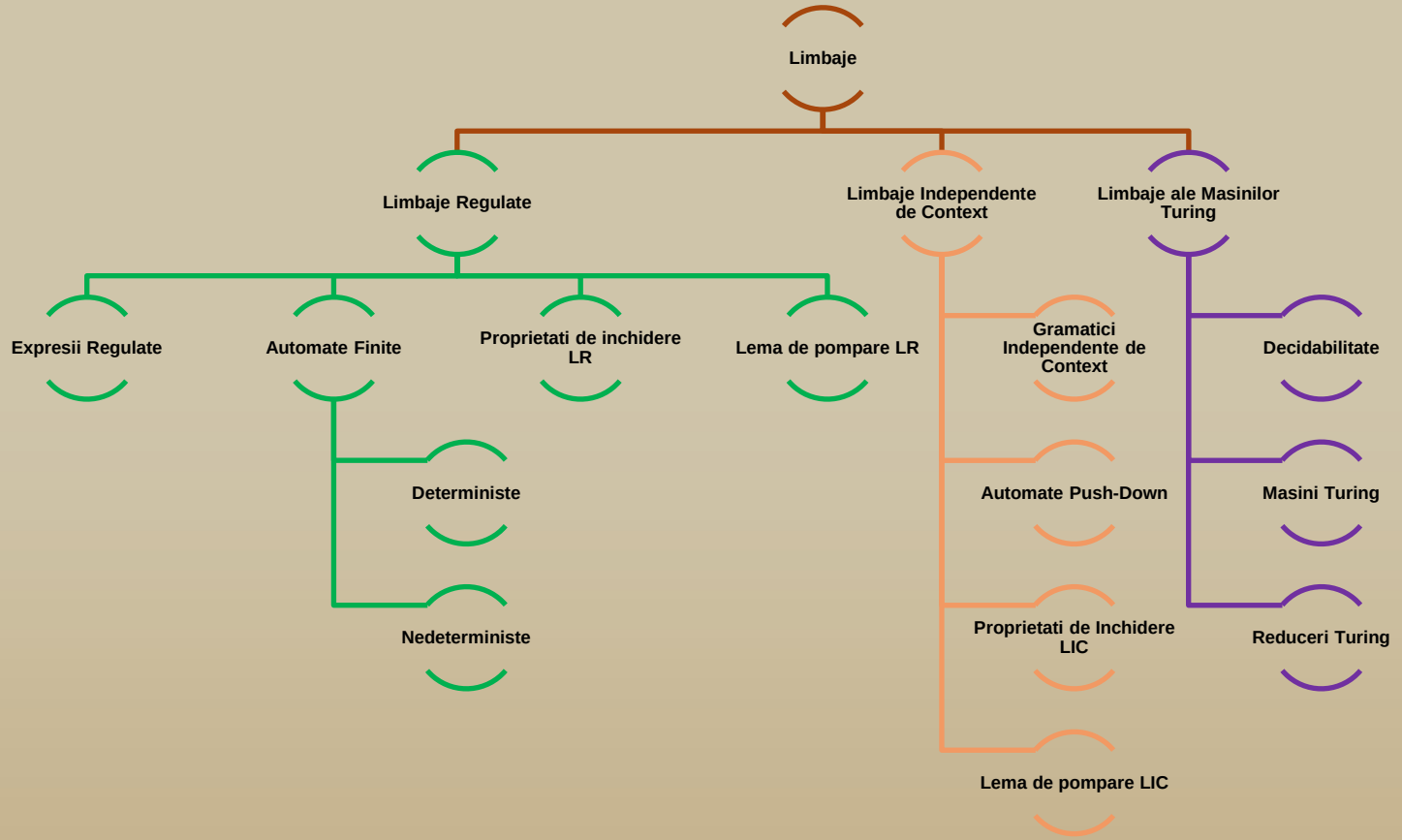
Proprietati de inchidere - LR

Alex Bogdan Andrei – LFA 336CB





Proprietati de inchidere


Limbaje
Limbaje Regulate
Limbaje Independente de Context
Limbaje ale Masinilor Turing
Expresii Regulate
Automate Finite
Proprietati de inchidere LR
Lema de pompare LR
Gramatici Independente de Context
Decidabilitate
Deterministe
Automate Push-Down
Masini Turing
Nedeterministe
Proprietati de Inchidere LIC
Reduceri Turing
Lema de pompare LIC





Ce este o “Proprietate de inchidere”?

Proprietati de inchidere



O mulțime A este închisă în raport cu un operator $\oplus$ dacă

$$\forall\, x, y \in A \Rightarrow x \oplus y \in A$$

Exemplu: Operatia de adunare pe multimea numerelor naturale



De ce am avea nevoie?



De ce am avea nevoie?

Cum definim un Limbaj Regulat?

Limbaj Regulat



Limbajele regulate sunt limbajele formale care:

Limbaj Regulat



Limbajele regulate sunt limbajele formale care:

1. Sunt generate de Expresii Regulate
2. Sunt acceptate de Automate Finite (Deterministe sau Nedeterministe)
3. Sunt formate prin Proprietati de inchidere din alte LR

Limbaj Neregulat



Limbajele regulate sunt limbajele formale care:

1. Sunt generate de Expresii Regulate
2. Sunt acceptate de Automate Finite (Deterministe sau Nedeterministe)
3. Sunt formate prin Proprietati de inchidere din alte LR

Limbajele neregulate:

1. Nu pot fi generate de Expresii Regulate
2. Nu pot fi acceptate de Automate Finite (Deterministe sau Nedeterministe)
3. Nu respecta Proprietati de inchidere
4. Nu respecta Lema de Pompare

Limbaj Neregulat



Limbajele regulate sunt limbajele formale care:

1. Sunt generate de Expresii Regulate
2. Sunt acceptate de Automate Finite (Deterministe sau Nedeterministe)
3. Sunt formate prin Proprietati de inchidere din alte LR

Limbajele neregulate:

1. ~~Nu pot fi generate de Expresii Regulate~~
2. ~~Nu pot fi acceptate de Automate Finite (Deterministe sau Nedeterministe)~~
3. Nu respecta Proprietati de inchidere
4. Nu respecta Lema de Pompare



Sub ce operatii sunt inchise LR?

Proprietati de inchidere



Clasa limbajelor regulate este inchisa in raport cu:

1. $\cup$ → reuniunea
2. $\cap$ → intersectia
3. $\bar{C}$ → complementul
4. $\setminus$ → diferenta
5. $\cdot$ → concatenarea
6. $*$ → Kleene Star
7. r → reverse
8. H → homomorfism

Demonstram prin ER, DFA / NFA sau alte Proprietati de Inchidere deja demonstrate.



Exercitii

HINT-uri Exercitii



1. Orice limbaj finit este un Limbaj Regulat.
2. Un limbaj rezultat in urma aplicarii unei operatii pe 2 limbaje arbitrare poate fi:
   a) Intotdeauna un Limbaj Regulat
   b) Intotdeauna un Limbaj Neregulat
   c) Uneori Regulat, unoeri Neregulat

Din acest motiv vom incepe mereu prin a cauta cate un exemplu pentru a valida c)

HINT-uri Exercitii



Limbaje Regulate

1. $\emptyset$ sau Σ^*
2. $L = L(a^*)$
3. $L = L(a^*b^*)$

Limbaje Neregulate

1. $L = \{a^n b^n \mid n > 0\}$
2. $L = \{w \mid w \text{ palindrom}\}$
3. $L = \{a^p \mid p \text{ prim}\}$
4. $L = \{a^{n^2} \mid n > 0\}$

Fie L_1 un limbaj regulat și L_2 un limbaj neregulat. Cum este $L = L_1 \cup L_2$?



Fie L_1 un limbaj regulat și L_2 un limbaj neregulat. Cum este $L = L_1 \cap L_2$?



Fie L_1 un limbaj regulat și L_2 un limbaj neregulat. Cum este $L = L_1 \cdot L_2$?



Fie L_1 un limbaj finit și L_2 un limbaj neregulat. Cum este $L = L_1 \cup L_2$?



Fie L_1 un limbaj finit și L_2 un limbaj neregulat. Cum este $L = L_1 \cap L_2$?



Fie L_1 un limbaj finit și L_2 un limbaj neregulat. Cum este $L = L_1 \cdot L_2$?



Fie L_1 un limbaj neregulat și L_2 un limbaj neregulat. Cum este $L = L_1 \cup L_2$?





Intrebari?

Oricand pe Facebook / Mail

Tema Closures – 4.11.2019

1. Fie L_1 un limbaj finit și L_2 un limbaj neregulat
   1) (1p) $L = L_1 \setminus L_2$
   2) (1p) $L = L_2 \setminus L_1$

2. Fie L_1 un limbaj regulat și L_2 un limbaj neregulat
   1) (1p) $L = L_1 \setminus L_2$
   2) (1p) $L = {L_2}^*$

3. Fie L_1 un limbaj neregulat și L_2 un limbaj neregulat
   1) (1p) $L = L_1 \cap L_2$
   2) (2p) $L = L_1 \cdot L_2$

4. Demonstrati ca Limbajele Regulate sunt inchise cu
   1) (2p) Reverse
   2) (3p) Homomorfismul



Seminar 5

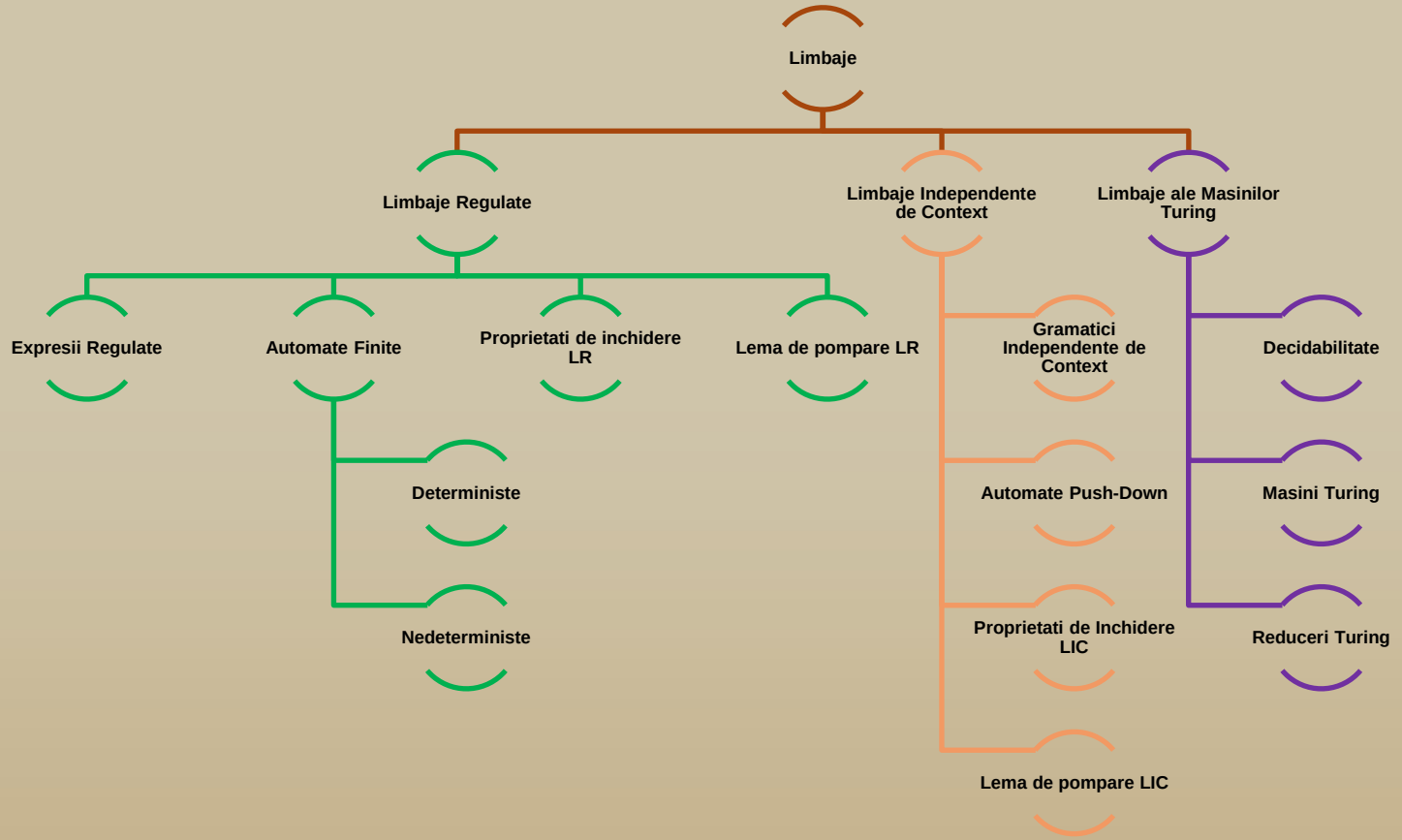
Lema de pompare - LR

Alex Bogdan Andrei – **LFA 336CB**





Lema de pompare


Limbaje
Limbaje Regulate
Limbaje Independente de Context
Limbaje ale Masinilor Turing
Expresii Regulate
Automate Finite
Proprietati de inchidere LR
Lema de pompare LR
Gramatici Independente de Context
Decidabilitate
Deterministe
Automate Push-Down
Masini Turing
Nedeterministe
Proprietati de Inchidere LIC
Reduceri Turing
Lema de pompare LIC



Lema de pompare



Daca $L \in LR$, atunci $\exists n \geq 1$ astfel incat $\forall w \in L$, cu $|w| > n$, poate fi rescris ca $w = xyz$ astfel incat:

- $y \neq \epsilon$
- $|xy| < n$
- $xy^iz \in L, \forall i \geq 0$

Lema de pompare



Daca $L \in LR$, atunci $\exists n \geq 1$ astfel incat $\forall w \in L$, cu $|w| > n$, poate fi rescris ca $w = xyz$ astfel incat:

- $y \neq \epsilon$
- $|xy| < n$
- $xy^iz \in L, \forall i \geq 0$

De la 1 la 10, cat de clar este?



Cum poate un DFA cu **k** stari sa accepte un cuvant de dimensiune **n**, $n > k$?

Lema de pompare



Daca un DFA cu k stari accepta un cuvant de lungime n $(n > k)$, atunci automatul contine un ciclu.

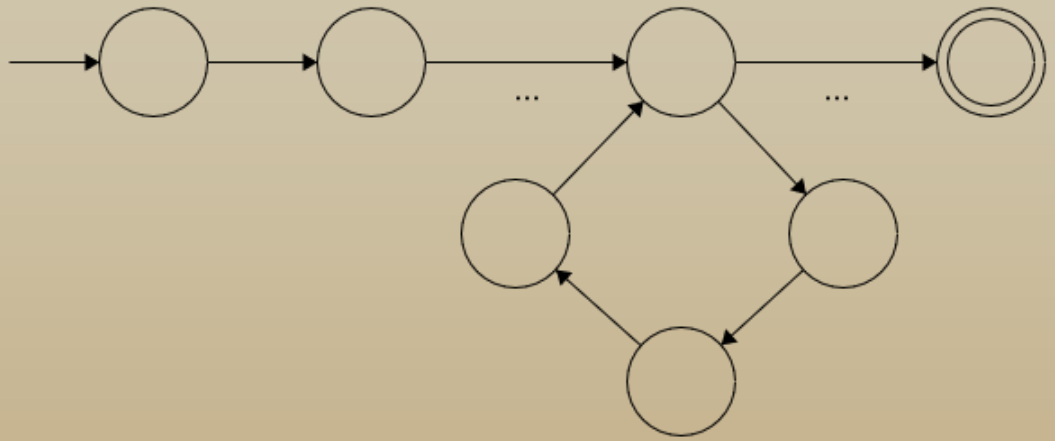
Ciclul poate fi prezent in orice stare.

Lema de pompare



Daca un DFA cu k stari accepta un cuvant de lungime n $(n > k)$, atunci automatul contine un ciclu.

Ciclul poate fi prezent in orice stare.

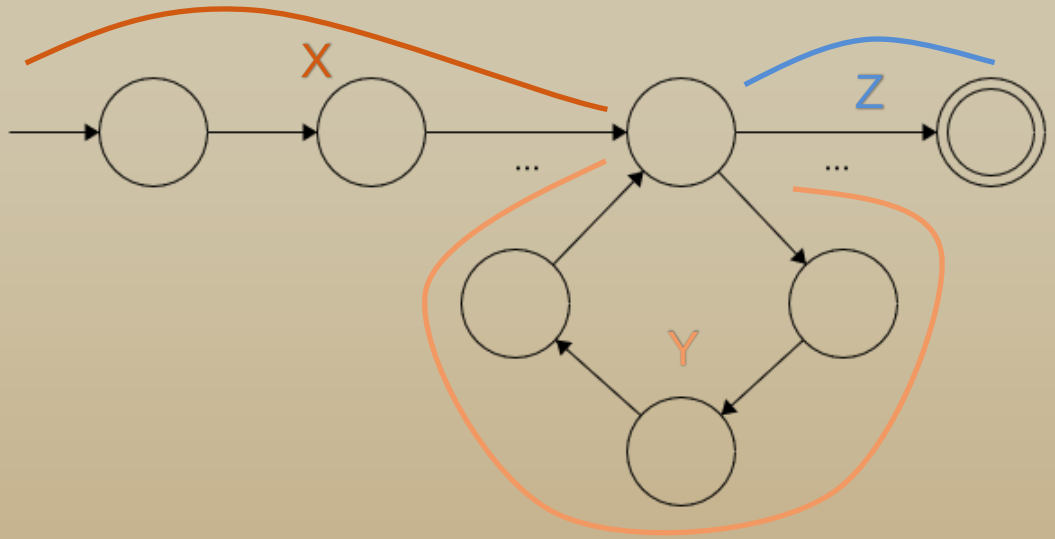


Lema de pompare



Daca un DFA cu k stari accepta un cuvant de lungime n $(n > k)$, atunci automatul contine un ciclu

Putem separa cuvantul acceptat w in 3 subsiruri **xyz**



Lema de pompare

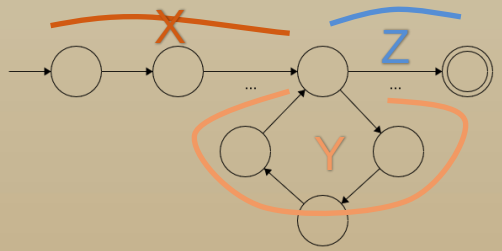


Definitie informala

Daca un DFA cu k stari accepta un cuvant de lungime n $(n > k)$, atunci automatul contine un ciclu

Putem separa cuvantul acceptat w in 3 subsiruri **xyz**

Ciclul poate fi pe orice stare.



Lema de pompare

$Daca\ L \in LR, atunci$
$\exists n \geq 1\ astfel\ incat\ \forall\ w \in L,$
$cu\ |w| > n, poate\ fi\ rescris$
$ca\ w = xyz\ astfel\ incat$:

- $y \neq \epsilon$
- $|xy| < n$
- $xy^i z \in L, \forall i \geq 0$

Lema de pompare

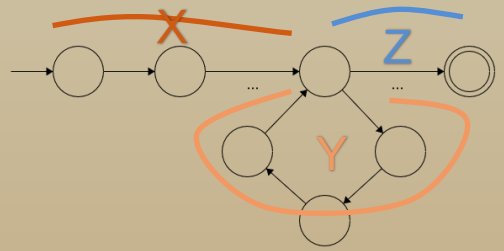


Definitie informala

Daca un DFA cu k stari accepta un cuvant de lungime n $(n > k)$, atunci automatul contine un ciclu

Putem separa cuvantul acceptat w in 3 subsiruri xyz

Ciclul poate fi pe orice stare.



Lema de pompare

$Daca\ L \in LR, atunci$
$\exists n \geq 1\ astfel\ incat\ \forall\ w \in L,$
$cu\ |w| > n, poate\ fi\ rescris$
$ca\ w = xyz\ astfel\ incat:$

- $y \neq \epsilon$
- $|xy| < n$
- $xy^i z \in L, \forall i \geq 0$

Lema de pompare

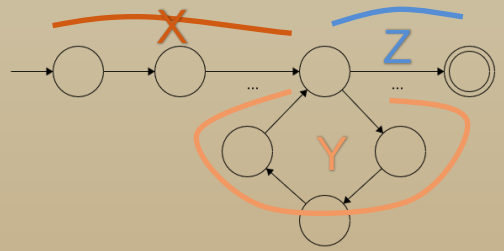


Definitie informala

Daca un DFA cu k stari accepta un cuvant de lungime n $(n > k)$, atunci automatul contine un ciclu

Putem separa cuvantul acceptat w in 3 subsiruri xyz

Ciclul poate fi pe orice stare.



Lema de pompare

$Daca\ L \in LR, atunci$
$\exists n \geq 1\ astfel\ incat\ \forall\ w \in L,$
$cu\ |w| > n, poate\ fi\ rescris$
$ca\ w = xyz\ astfel\ incat:$

- $y \neq \epsilon$
- $|xy| < n$
- $xy^i z \in L, \forall i \geq 0$

Lema de pompare

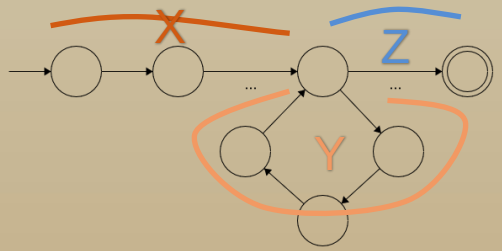


Definitie informala

Daca un DFA cu k stari accepta un cuvant de lungime n $(n > k)$, atunci automatul contine un ciclu

Putem separa cuvantul acceptat w in 3 subsiruri **xyz**

Ciclul poate fi pe orice stare.



Lema de pompare

$Daca\ L \in LR, atunci$
$\exists n \geq 1\ astfel\ incat\ \forall\ w \in L,$
$cu\ |w| > n, poate\ fi\ rescris$
$ca\ w = xyz\ astfel\ incat:$

- $y \neq \epsilon$
- $|xy| < n$
- $xy^i z \in L, \forall i \geq 0$

Lema de pompare

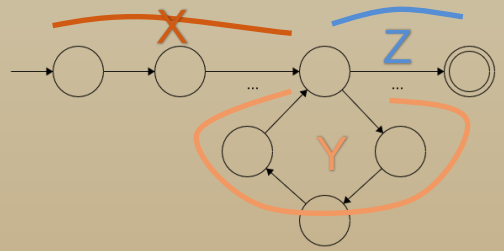


Definitie informala

Daca un DFA cu k stari accepta un cuvant de lungime n $(n > k)$, atunci automatul contine un ciclu

Putem separa cuvantul acceptat w in 3 subsiruri xyz

Ciclul poate fi pe orice stare.



Lema de pompare

$Daca\ L \in LR, atunci$
$\exists n \geq 1\ astfel\ incat\ \forall\ w \in L,$
$cu\ |w| > n, poate\ fi\ rescris$
$ca\ w = xyz\ astfel\ incat:$

- $y \neq \epsilon$
- $|xy| < n$
- $xy^i z \in L, \forall i \geq 0$

Lema de pompare

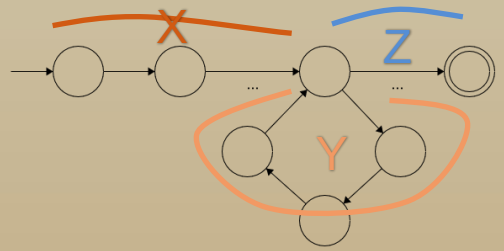


Definitie informala

Daca un DFA cu k stari accepta un cuvant de lungime n $(n > k)$, atunci automatul contine un ciclu

Putem separa cuvantul acceptat w in 3 subsiruri xyz

Ciclul poate fi pe orice stare.



Lema de pompare

$Daca\ L \in LR, atunci$
$\exists n \geq 1\ astfel\ incat\ \forall\ w \in L,$
$cu\ |w| > n, poate\ fi\ rescris$
$ca\ w = xyz\ astfel\ incat$:

- $y \neq \epsilon$
- $|xy| < n$
- $xy^i z \in L, \forall i \geq 0$

Lema de pompare

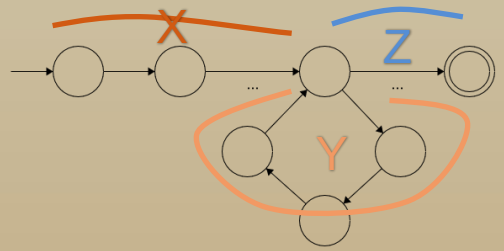


Definitie informala

Daca un DFA cu k stari accepta un cuvant de lungime n $(n > k)$, atunci automatul contine un ciclu

Putem separa cuvantul acceptat w in 3 subsiruri xyz

Ciclul poate fi pe orice stare.



Lema de pompare

$Daca\ L \in LR, atunci$
$\exists n \geq 1\ astfel\ incat\ \forall\ w \in L,$
$cu\ |w| > n, poate\ fi\ rescris$
$ca\ w = xyz\ astfel\ incat$:

- $y \neq \epsilon$
- $|xy| < n$
- $xy^i z \in L, \forall i \geq 0$



Cum folosim lema de pompare?

Aratati ca $L = \{a^n b^n \mid n > 0\}$ este un limbaj neregulat



Aratati ca $L = \{a^n b^n \mid n > 0\}$ este un limbaj neregulat

Negam lema de pompare

$Daca\ L \in LR, atunci$
$\exists n \geq 1\ astfel\ incat\ \forall\ w \in L,$
$cu\ |w| > n, poate\ fi\ rescris$
$ca\ w = xyz\ astfel\ incat:$

- $y \neq \epsilon$
- $|xy| < n$
- $xy^i z \in L, \forall i \geq 0$



Aratati ca $L = \{a^n b^n \mid n > 0\}$ este un limbaj neregulat

Presupunem ca $L \in LR$

Negam lema de pompare

$Daca\ L \in LR, atunci$
$\exists n \geq 1\ astfel\ incat\ \forall\ w \in L,$
$cu\ |w| > n, poate\ fi\ rescris$
$ca\ w = xyz\ astfel\ incat:$

- $y \neq \epsilon$
- $|xy| < n$
- $xy^i z \in L, \forall i \geq 0$



Aratati ca $L = \{a^n b^n \mid n > 0\}$ este un limbaj neregulat

Presupunem ca $L \in LR$

Fie **n** lungimea de pompare

Negam lema de pompare

$Daca\ L \in LR, atunci$
$\exists n \geq 1\ astfel\ incat\ \forall\ w \in L,$
$cu\ |w| > n, poate\ fi\ rescris$
$ca\ w = xyz\ astfel\ incat:$

- $y \neq \epsilon$
- $|xy| < n$
- $xy^i z \in L, \forall i \geq 0$



Aratati ca $L = \{a^n b^n \mid n > 0\}$ este un limbaj neregulat

Presupunem ca $L \in LR$

Fie **n** lungimea de pompare

Alegem cuvantul $w = 0^n 1^n$

Negam lema de pompare

$Daca\ L \in LR, atunci$
$\exists n \geq 1\ astfel\ incat\ \forall\ w \in L,$
$cu\ |w| > n, poate\ fi\ rescris$
$ca\ w = xyz\ astfel\ incat:$

- $y \neq \epsilon$
- $|xy| < n$
- $xy^i z \in L, \forall i \geq 0$



Aratati ca $L = \{a^n b^n \mid n > 0\}$ este un limbaj neregulat

Negam lema de pompare

$Daca\ L \in LR, atunci$
$\exists n \geq 1\ astfel\ incat\ \forall\ w \in L,$
$cu\ |w| > n, poate\ fi\ rescris$
$ca\ w = xyz\ astfel\ incat:$

- $y \neq \epsilon$
- $|xy| < n$
- $xy^i z \in L, \forall i \geq 0$

Presupunem ca $L \in LR$

Fie **n** lungimea de pompare

Alegem cuvantul $w = 0^n 1^n$, astfel incat w = xyz, cu:

- $x = 0^j \quad y = 0^k \quad z = 0^{n-k-j} 1^n$



Aratati ca $L = \{a^n b^n \mid n > 0\}$ este un limbaj neregulat

Negam lema de pompare

$Daca\ L \in LR, atunci$
$\exists n \geq 1\ astfel\ incat\ \forall\ w \in L,$
$cu\ |w| > n, poate\ fi\ rescris$
$ca\ w = xyz\ astfel\ incat:$

- $y \neq \epsilon$
- $|xy| < n$
- $xy^i z \in L, \forall i \geq 0$

Presupunem ca $L \in LR$

Fie **n** lungimea de pompare

Alegem cuvantul $w = 0^n 1^n$, astfel incat w = xyz, cu:

- $x = 0^j \quad y = 0^k \quad z = 0^{n-k-j} 1^n$
- $k \neq 0 \quad si \quad 0 < j + k < n$



Aratati ca $L = \{a^n b^n \mid n > 0\}$ este un limbaj neregulat

Negam lema de pompare

$Daca\ L \in LR, atunci$
$\exists n \geq 1\ astfel\ incat\ \forall\ w \in L,$
$cu\ |w| > n, poate\ fi\ rescris$
$ca\ w = xyz\ astfel\ incat$:

- $y \neq \epsilon$
- $|xy| < n$
- $xy^i z \in L, \forall i \geq 0$

Presupunem ca $L \in LR$

Fie **n** lungimea de pompare

Alegem cuvantul $w = 0^n 1^n$, astfel incat w = xyz, cu:

- $x = 0^j \quad y = 0^k \quad z = 0^{n-k-j} 1^n$
- $\mathrm{k} \neq 0 \quad \mathrm{si} \quad 0 < \mathrm{j} + \mathrm{k} < \mathrm{n}$

Alegem $w' = xy^2z$



Aratati ca $L = \{a^n b^n \mid n > 0\}$ este un limbaj neregulat

Negam lema de pompare

$Daca\ L \in LR, atunci$
$\exists n \geq 1\ astfel\ incat\ \forall\ w \in L,$
$cu\ |w| > n, poate\ fi\ rescris$
$ca\ w = xyz\ astfel\ incat$:

- $y \neq \epsilon$
- $|xy| < n$
- $xy^i z \in L, \forall i \geq 0$

Presupunem ca $L \in LR$

Fie **n** lungimea de pompare

Alegem cuvantul $w = 0^n 1^n$, astfel incat w = xyz, cu:

- $x = 0^j \quad y = 0^k \quad z = 0^{n-k-j} 1^n$
- $\mathrm{k} \neq 0 \quad \mathrm{si} \quad 0 < \mathrm{j} + \mathrm{k} < \mathrm{n}$

Alegem $w' = xy^2z$



Aratati ca $L = \{a^n b^n \mid n > 0\}$ este un limbaj neregulat

Negam lema de pompare

$Daca\ L \in LR, atunci$
$\exists n \geq 1\ astfel\ incat\ \forall\ w \in L,$
$cu\ |w| > n, poate\ fi\ rescris$
$ca\ w = xyz\ astfel\ incat:$

- $y \neq \epsilon$
- $|xy| < n$
- $xy^i z \in L, \forall i \geq 0$

Presupunem ca $L \in LR$

Fie **n** lungimea de pompare

Alegem cuvantul $w = 0^n 1^n$, astfel incat w = xyz, cu:

- $x = 0^j \quad y = 0^k \quad z = 0^{n-k-j} 1^n$
- $\mathrm{k} \neq 0 \quad \mathrm{si} \quad 0 < \mathrm{j} + \mathrm{k} < \mathrm{n}$

Alegem $w' = xy^2 z \Rightarrow w' = \left(0^j\right)\left(0^{2k}\right)\left(0^{n-k-j} 1^n\right) =$

$$= 0^{j+2k+n-k-j} 1^n = 0^{n+k} 1^n$$



Aratati ca $L = \{a^n b^n \mid n > 0\}$ este un limbaj neregulat

Negam lema de pompare

$Daca\ L \in LR, atunci$
$\exists n \geq 1\ astfel\ incat\ \forall\ w \in L,$
$cu\ |w| > n, poate\ fi\ rescris$
$ca\ w = xyz\ astfel\ incat:$

- $y \neq \epsilon$
- $|xy| < n$
- $xy^i z \in L, \forall i \geq 0$

Presupunem ca $L \in LR$

Fie **n** lungimea de pompare

Alegem cuvantul $w = 0^n 1^n$, astfel incat w = xyz, cu:

- $x = 0^j \quad y = 0^k \quad z = 0^{n-k-j} 1^n$
- $\mathrm{k} \neq 0 \quad \mathrm{si} \quad 0 < \mathrm{j} + \mathrm{k} < \mathrm{n}$

Alegem $w' = xy^2 z \Rightarrow w' = \left(0^j\right)\left(0^{2k}\right)\left(0^{n-k-j} 1^n\right) =$

$$= 0^{j+2k+n-k-j} 1^n = 0^{n+k} 1^n$$

Cum $k > 0 \Rightarrow n + k \neq n \Rightarrow w' \notin L \Rightarrow L \notin LR$





Exercitii

Aratati ca $L = \{a^n b a^{2n} \mid n > 0\}$ este un limbaj neregulat

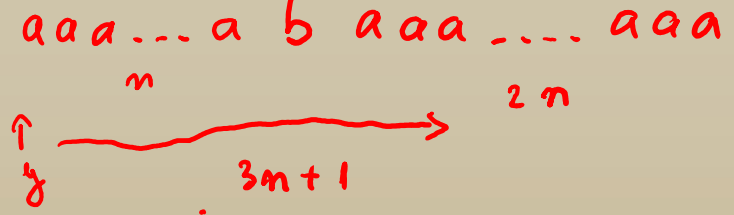
Negam lema de pompare

Daca $L \in LR$, *atunci*
$\exists n \geq 1$ *astfel incat* $\forall w \in L$,
cu $|w| > n$, *poate fi rescris*
ca $w = xyz$ *astfel incat*:

- $y \neq \epsilon$
- $|xy| < n$
- $xy^i z \in L, \forall i \geq 0$

$p = 3n + 1$

$|xy| < p$



$x = a^{+}$

$x = a^{+} b$

$x = a^{+} b a^{k}$



Aratati ca $L = \{a^{n^2} \mid n > 0\}$ este un limbaj neregulat

Negam lema de pompare

$Daca\ L \in LR, atunci$
$\exists n \geq 1\ astfel\ incat\ \forall\ w \in L,$
$cu\ |w| > n, poate\ fi\ rescris$
$ca\ w = xyz\ astfel\ incat:$

- $y \neq \epsilon$
- $|xy| < n$
- $xy^i z \in L, \forall i \geq 0$



Aratati ca $L = \{a^p \mid p \ numar \ prim\}$ este un limbaj neregulat

Negam lema de pompare

Daca $L \in LR$, *atunci*
$\exists n \geq 1$ *astfel incat* $\forall w \in L$,
cu $|w| > n$, *poate fi rescris*
ca $w = xyz$ *astfel incat*:

- $y \neq \epsilon$
- $|xy| < n$
- $xy^i z \in L, \forall i \geq 0$

k par

$w = a^p > n$

$0 < j + k < n$

$x = a^j \quad y = a^k \quad z = a^{p-k-j}$

$w' = x y^i z = a^j a^{k \cdot i} a^{p-k-j}$





Intrebari?

Oricand pe Facebook / Mail

Tema Lema de pompare (02.12.2019)

Folosind lema de pompare, demonstrați că următoarele limbaje nu sunt Limbaje Regulate:

1. (2.5p) $\Sigma = \{a, b\} \rightarrow L = \{a^i b^j a^k | \, k > i + j\}$
2. (2.5p) $\Sigma = \{a, b\} \rightarrow L = \{a^i b^j | \, j = i \; sau \; j = 2i\}$
3. (2.5p) $\Sigma = \{a, b\} \rightarrow L = \{a^i b^j | \, j \; este \; multiplu \; de \; i\}$
4. (2.5p) $\Sigma = \{a, b\} \rightarrow L = \{ww | \, w \in \Sigma^*\}$



Limbaje formale si automate

Note de curs

MAȘINA TURING

Mașina Turing reprezintă formalizarea ideii de mașină de calcul. În general, atunci când vorbim despre modelul abstract al unei mașini de calcul ne referim la un automat, un dispozitiv cu un număr finit de stări, care operează asupra unui șir de simboluri, pe baza unei mulțimi finite de reguli. Există diferite tipuri de automate, care pot fi clasificate, în funcție de complexitatea lor structurală și comportamentală, în automate finite, automate cu stivă, mașini Turing.

Mașinile Turing formează o clasă maximală și stabilă de automate în termenii operațiilor pe care le pot efectua. Ele au fost proiectate pentru a satisface simultan trei criterii:

- să respecte definiția unui automat;
- să fie ușor de descris, formalizat, și raționat;
- să fie cât mai generale, în termenii operațiilor efectuate.

1.1. Definirea mașinii Turing

O mașină Turing constă dintr-un dispozitiv de control finit și o bandă de intrare:

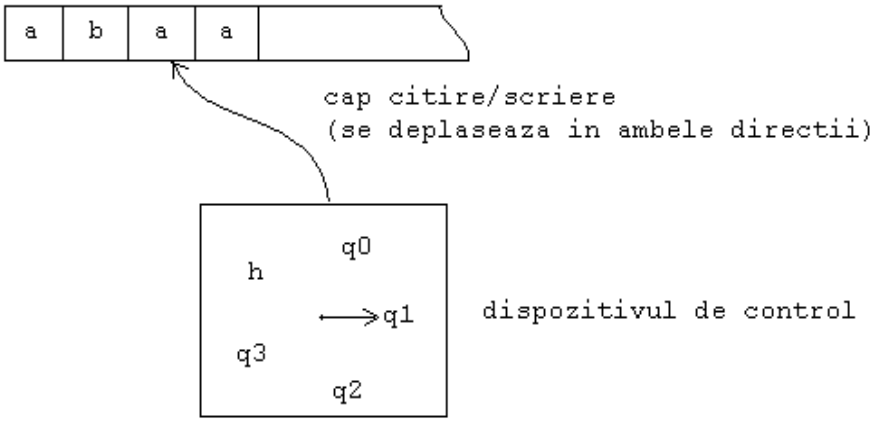


Figura 1-1

Comunicarea între cele două componente este făcută printr-un cap care poate să citească / scrie simboluri pe bandă sau să se deplaseze o poziție la stânga sau la dreapta. Dispozitivul de control acționează în pași discreți. La fiecare pas execută două acțiuni în funcție de starea curentă și de simbolul de pe bandă indicat de capul de citire / scriere, respectiv:

1. pune dispozitivul de control într-o nouă stare, și
2. fie
   a. scrie un simbol pe bandă în poziția curentă, înlocuindu-l pe cel existent
   b. mută capul de citire / scriere o poziție la stânga sau la dreapta

Banda de intrare este limitată la stânga, dar se extinde la dreapta indefinit. Considerăm că banda conține în pozițiile libere simbolul $\#$. Cum mașina poate muta capul doar câte o poziție, după un număr finit de operații numai un număr finit de poziții pot fi accesate. Dacă mașina încearcă să deplaseze capul la stânga capătului benzii, ea încetează să funcționeze.

Pentru că o mașină Turing poate scrie pe bandă, ea poate lăsa un răspuns la sfârșitul operațiilor, de aceea nu trebuie prevăzute stări finale (ca în cazul automatelor finite sau a celor cu stivă).

Există o stare specială de $halt$ (notată h) care este utilizată pentru a indica sfârșitul operațiilor, dar nu oferă informații asupra succesului sau insuccesului operațiilor.

Definiţia 1.1

O mașină Turing este un tuplu (K, Σ, δ, s), unde:

K este mulțimea finită de stări care nu conține starea h;

Σ este un alfabet care conține simbolul $\#$, dar nu conține simbolurile L, R;

$s \in K$ este starea inițială;

δ este funcția de tranziție, $\delta: K \times \Sigma \rightarrow (K \cup \{h\}) \times (\Sigma \cup \{L, R\})$.

Dacă $q \in K$, $a \in \Sigma$, și $\delta(q, a) = (p, b)$ atunci M, dacă este în starea q și scanează simbolul a, intră în starea p. Dacă $b \in \Sigma$ atunci scrie b în locul lui a. Dacă b este L sau R mută capul în direcția b. Cum δ este o

funcție, mașina M este deterministă și se va opri când intră în starea de *halt* sau încearcă să depășească capătul stânga al benzii de intrare.

Exemplu 1.1

Să construim mașina Turing care primind ca intrare un șir de simboluri *a*, cu capul poziționat pe primul *a* din dreapta, se deplasează spre stânga înlocuind *a* cu #. Mașina se oprește pe primul # care delimitează stânga șirul. Fie $M = (K, \Sigma, \delta, s)$, unde

$K = \{q_0, q_1\}$

$\Sigma = \{a, \#\}$

$s = q_0$

funcția de tranziție δ este definită prin tabelul următor:

q	σ	$\delta(q, \sigma)$
q_0	a	$(q_1, \#)$
q_0	#	$(h, \#)$
q_1	a	(q_0, a)
q_1	#	(q_0, L)

M pornește în starea inițială q_0, deplasează capul spre stânga schimbând simbolurile *a* cu # pe măsură ce înaintează, până când găsește un simbol # și se oprește.

Să presupunem că șirul de intrare este format din trei simboluri *a*, primul (din stânga) fiind precedat de #. Mașina pornește cu capul pe primul simbol *a* din dreapta, trece din starea q_0 în q_1 și invers de trei ori schimbând *a* cu # și deplasând capul spre stânga. Când ajunge în starea q_0 cu # pe bandă se oprește. Practic starea q_0 este utilizată pentru a înlocui simbolul *a* de pe bandă cu #, iar starea q_1 pentru a deplasa capul de citire o poziție la stânga. A treia linie din tabel descrie o tranziție care practic nu poate avea loc, dar a fost introdusă pentru că δ este o funcție totală pe domeniul $K \times \Sigma$, și prin urmare trebuie definită pe tot domeniul.

Exemplu 1.2

Să construim mașina Turing care primind un șir de simboluri a la intrare cu capul poziționat pe primul a din stânga șirului, se deplasează la dreapta pâna la simbolul #. Fie $M = (K, \Sigma, \delta, s)$, unde

$K = \{q_0\}$

$\Sigma = \{a, \#\}$

$s = q_0$

și δ este dată prin tabelul următor:

q	σ	$\delta(q, \sigma)$
q_0	a	(q_0, R)
q_0	#	$(h, \#)$

Mașina se deplasează la dreapta până când găsește un # și se oprește.

Pentru a specifica starea operațiilor unei mașini Turing, trebuie specificată starea mașinii, conținutul benzii și poziția capului. Conținutul benzii va fi specificat ca un șir, împarțit în trei părți: subșirul (eventual vid) aflat la stânga poziției curent scanate a benzii, simbolul din poziția curentă, și subșirul (eventual vid) aflat la dreapta poziției curente. Pentru a evita situația în care două tupluri să corespundă aceleași combinații a capului și conținutului benzii, impunem condiția ca ultimul șir să nu se termine cu #.

Definiția 1.2

Configurația unei mașini Turing $M = (K, \Sigma, \delta, s)$ este un element din

$(K \cup \{h\}) \times \Sigma^* \times \Sigma \times (\Sigma^* (\Sigma - \#) \cup \{e\})$.

O configurație care conține starea de $halt$ se numește *configurație de oprire*.

Trecerea mașinii Turing dintr-o configurație în alta este specificată prin definiția următoare.

Definiția 1.3

Fie $M = (K, \Sigma, \delta, s)$ o mașină Turing și fie (q_1, w_1, a_1, u_1) și (q_2, w_2, a_2, u_2) configurații ale lui M. Atunci

$(q_1, w_1, a_1, u_1) \vdash_M (q_2, w_2, a_2, u_2)$

dacă și numai dacă, pentru $b \in \Sigma \cup \{L, R\}$, $\delta(q_1, a_1) = (q_2, b)$ fie:

1. $b \in \Sigma$, $w_1 = w_2$, $u_1 = u_2$, și $a_2 = b$;

sau

2. $b = L$, $w_1 = w_2 a_2$ și fie
   (a) $u_2 = a_1 u_1$, dacă $a_1 \neq \#$ sau $u_1 \neq e$ sau
   (b) $u_2 = e$, dacă $a_1 = \#$ și $u_1 = e$;

sau

3. $b = R$, $w_2 = w_1 a_1$, și fie
   (a) $u_1 = a_2 u_2$, sau
   (b) $u_1 = u_2 = e$ și $a_2 = \#$

În cazul 1, M rescrie un simbol în poziția curentă. În cazul 2, M își deplasează capul o poziție la stânga; dacă se mută la stânga benzii neocupate, simbolul # din poziția curentă dispare din configurație. În cazul 3, M își deplasează capul o poziție la dreapta; dacă se mută pe banda neocupată un nou simbol # apare în configurație, ca fiind noul simbol scanat.

Pentru a simplifica notația vom descrie o configurație (q, w, a, u) prin $(q, w\underline{a}u)$, unde simbolul din poziția curentă este subliniat.

Exemplu 1.3

Fie șirurile $w, u \in \Sigma^*$, unde u nu se termină cu #, și fie $a, b \in \Sigma$.

Cazul 1. $\delta(q_1, a) = (q_2, b)$

$(q_1, w\underline{a}u) \vdash_M (q_2, w\underline{b}u)$

Cazul 2. $\delta(q_1, a) = (q_2, L)$

(a) $(q_1, wb\underline{a}u) \vdash_M (q_2, w\underline{b}au)$

(b) $(q_1, wb\underline{\#}) \vdash_M (q_2, w\underline{b})$

Cazul 3. $\delta(q_1, a) = (q_2, R)$

(a) $(q_1, w\underline{a}bu) \vdash_M (q_2, wa\underline{b}u)$

(b) $(q_1, w\underline{a}) \vdash_M (q_2, wa\underline{\#})$

Se observă că dacă $b = L$ și $w = e$, atunci (q_1, w_1, a_1, u_1) nu duce la nicio configurație, pentru că nu există $w_2 \in \Sigma^*$ și $a_2 \in \Sigma$ astfel încât $w_1 = w_2a_2$. O astfel de configurație este numită configurație de agățare.

Definiția 1.4

Pentru orice mașină Turing M, $\vdash_M^*$ este închiderea reflexivă și tranzitivă a relației $\vdash_M$; din configurația C_1 se ajunge în configurația C_2 în zero sau mai mulți pași dacă $C_1 \vdash_M^* C_2$. Un calcul efectuat de M este o secvență de configurații $C_1, C_2, ..., C_n$ astfel încât

$$C_0 \vdash_M C_1 \vdash_M C_2 \vdash_M ... \vdash_M C_n$$

Spunem că lungimea calculului este n sau că are n pași.

1.2. Utilizarea mașinii Turing

Pentru a putea prezenta puterea de calcul a mașinii Turing trebuie să stabilim câteva convenții de utilizare a mașinii. Considerăm că șirul de intrare este încadrat de câte un simbol # și este plasat la limita stânga a benzii de intrare; capul de citire / scriere este poziționat pe simbolul # care delimitează capătul dreapta al șirului; mașina începe operațiile din starea inițială.

Fie $M = (K, \Sigma, \delta, s)$ o mașină Turing și $w \in \Sigma^*$. M se oprește pe intrarea w, dacă și numai dacă pornind din configurația inițială $(s, \#w\underline{\#})$ în urma operațiilor efectuate asupra lui w, mașina ajunge într-o configurație de oprire (configurația care conține starea h).

Pentru început vom analiza calculabilitatea cu mașina Turing în termenii funcțiilor definite pe șiruri.

Definiția 1.5

Fie Σ_0 și Σ_1 alfabete care nu conțin simbolul $\#$. Fie $f : \Sigma_0 \to \Sigma_1$ o funcție. Mașina Turing $M = (K, \Sigma, \delta, s)$ calculează funcția f, dacă pentru $\Sigma_0, \Sigma_1 \subseteq \Sigma$, $\forall\ w \in \Sigma_0^*$, $f(w) = u$, atunci

$$(s, \#w\underline{\#}) \vdash_M^* (h, \#u\underline{\#})$$

Dacă o astfel de mașină Turing există, atunci f este o funcție calculabilă Turing.

Astfel, când M are pe banda de intrare șirul w conform convenției prezentate, M trebuie să se oprească cu șirul $f(w)$ pe bandă, încadrat de $\#$, cu capul la dreapta valorii calculate. Restul benzii trebuie să conțină doar $\#$. Nu se garantează comportamentul mașinii dacă nu se respectă convențiile prezentate.

O mașină Turing care calculează o funcție nu se agață pentru nicio intrare din domeniul funcției. Acest lucru este important pentru că putem presupune că o mașină care calculează o funcție nu încearcă să consulte simboluri aflate la stânga șirului de intrare.

De notat că nu se dau asigurări despre comportarea mașinii Turing dacă nu satisface specificația, de exemplu, dacă şirul de intrare conţine simboluri din $\Sigma - \Sigma_0$.

Exemplu 1.4

Fie $\Sigma_0 = \Sigma_1 = \{a, b\}$ şi fie $f : \Sigma_0^* \to \Sigma_1^*$ definită astfel:

$\forall\ w \in \Sigma_0^*$, $f(w) = \overline{w}$

unde $\overline{w}$este rezultatul înlocuirii fiecărei apariţii a simbolului a în w prin b şi invers.

Funcția f este calculată de o mașină Turing M care parcurge intrarea de la dreapta la stânga, interschimbând simbolurile a cu b, până când simbolul $\#$ care delimitează capătul stânga al șirului este găsit. Pentru a lăsa banda în forma standard, M trebuie să poziţioneze capul pe simbolul $\#$ care se află la limita dreapta a șirului.

Fie $M = (K, \Sigma, \delta, s)$ unde

$K = \{q_0, q_1, q_2\}$,

$\Sigma = \{a, b, \#\}$,

$s = q_0$

şi δ este definită prin următorul tabel:

q	σ	$\delta(q, \sigma)$
q_0	a	(q_1, L)
q_0	b	(q_1, L)
q_0	$\#$	(q_1, L)
q_1	a	(q_0, b)
q_1	b	(q_0, a)
q_1	$\#$	(q_2, R)
q_2	a	(q_2, R)
q_2	b	(q_2, R)
q_2	$\#$	$(h, \#)$

De exemplu:

$$
\begin{aligned}
(q_0, \#aab\#) &\vdash_M (q_1, \#aa\underline{b}) \\
&\vdash_M (q_0, \#aaa) \\
&\vdash_M (q_1, \#a\underline{a}a) \\
&\vdash_M (q_0, \#a\underline{b}a) \\
&\vdash_M (q_1, \#\underline{a}ba) \\
&\vdash_M (q_0, \#\underline{b}ba) \\
&\vdash_M (q_1, \underline{\#}bba) \\
&\vdash_M (q_2, \#\underline{b}ba) \\
&\vdash_M (q_2, \#b\underline{b}a) \\
&\vdash_M (q_2, \#bb\underline{a}) \\
&\vdash_M (q_2, \#bba\underline{\#}) \\
&\vdash_M (h, \#bba\underline{\#})
\end{aligned}
$$

De notat de asemenea că M calculează corect valoarea $f(e) = e$:

$$
\begin{aligned}
(q_0, \#\#) &\vdash_M (q_1, \underline{\#}\#) \\
&\vdash_M (q_2, \#\underline{\#}) \\
&\vdash_M (h, \#\underline{\#})
\end{aligned}
$$

Dacă M porneşte dintr-o configuraţie ca $(q_0, \#\#ab\underline{\#}\#ab)$ rezultatul obținut de mașină este irelevant pentru întrebarea dacă M calculează f.

Cum $(q_0, \#w\#) \vdash_M^* (h, \#\overline{w}\#)$, $\forall\ w \in \Sigma_0^*$, M calculează f şi f este calculabilă Turing.

Noţiunea de funcţie calculabilă Turing pe şiruri poate fi extinsă în mai multe moduri. Putem considera funcţii cu orice număr de argumente, inclusiv zero. Astfel, fie f o funcţie, $f : (\Sigma_0^*)^k \to \Sigma_1^*$, $k \geq 0$, o maşină Turing $M = (K, \Sigma, \delta, s)$, unde $\Sigma_0, \Sigma_1 \subseteq \Sigma$. $\forall\ w_1, \ldots, w_k \in \Sigma^k$ dacă $f(w_1, \ldots, w_k) = u$, atunci $(s, \#w_1\#w_2\#\ldots\#w_k\underline{\#}) \vdash_M^* (h, \#u\underline{\#})$. Spunem că M calculează f, respectiv f este calculabilă Turing.

O altă extindere posibilă este cea a funcţiilor calculabile Turing definite pe mulţimea numerelor naturale.

Fie I un simbol diferit de #. Un număr natural n poate fi reprezentat (în notaţie unară) prin şirul I^n (0 este reprezentat prin şirul vid).

O funcţie $f : N \to N$ este calculată de o mașină Turing M, dacă M calculează funcţia $f' : \{I\}^* \to \{I\}^*$, unde $f'(I^n) = I^{f(n)}$, $n \in N$. Adică o mașină Turing calculează o funcţie de la numere la numere, calculând funcţia corespunzătoare de la notaţii unare la notaţii unare. Mai general, o mașină Turing M calculează o funcţie $f : N^k \to N$, dacă, calculează funcţia $f' : (\{I\}^*)^k \to \{I\}^*$, unde $f'(I^{n1}, \ldots, I^{nk}) = I^{f(n1 \ldots nk)}$, $\forall\, n_1, \ldots, n_k \geq 0$.

O funcţie definită pe numere sau pe k – tupluri de numere este calculabilă Turing, dacă există o mașină Turing care o poate calcula.

Exemplu 1.5

Fie f funcţia succesor: $f(n) = n+1$, $n \in N$.

Proiectăm o mașină Turing M, care calculează f. Mașina scrie un simbol I în poziția curentă a benzii, mută capul o poziție la dreapta şi se oprește.

Formal, $M = (K, \Sigma, \delta, s)$ unde

$K = \{q_0\}$,

$\Sigma = \{I, \#\}$,

$s = q_0$

şi δ este specificată prin tabelul:

q	σ	$\delta(q, \sigma)$
q_0	I	(h, R)
q_0	$\#$	(q_0, I)

De exemplu,

$(q_0, \#I\underline{I}\#) \vdash_M (q_0, \#I\underline{I}I) \vdash_M (h, \#I\underline{I}\#)$

În cazul general,

$(q_0, \#I^n\#) \vdash_M^* (h, \#I^{n+1}\underline{\#})$.

De notat în particular cazul $n = 0$

$(q_0, \#\underline{\#}) \vdash_M^* (q_0, \#I) \vdash_M (h, \#\underline{I}\#)$

Astfel, M calculează funcţia $f' : \{I\}^* \to \{I\}^*$ astfel încât $f'(I^n) = I^{n+1}$, $\forall$ n $\in$ N, deci M calculează funcţia succesor f : N $\to$ N.

O altă consecinţă importantă a noţiunii de funcţie calculabilă Turing este aceea de limbaj decidabil Turing.

Să considerăm un alfabet Σ_0 care nu conţine simbolul #. Limbajul $L \subseteq \Sigma_0^*$ este decidabil Turing, dacă şi numai dacă funcţia $X_L : \Sigma_0^* \to \{\text{Y}, \text{N}\}$ este calculabilă Turing, unde $\forall\ w \in \Sigma_0^*$

$$X_L(\text{w}) = \begin{cases} \text{Y}, & w \in L \\ \text{N}, & w \notin L \end{cases}$$

Dacă X_L este calculat de o maşină Turing M, atunci spunem că M decide L sau că este o procedură de decizie pentru L.

Exemplu 1.6

Fie $\Sigma_0 = \{a\}$ şi fie $L = \{w \in \Sigma_0^* : |w| \text{ este pară}\}$.

Atunci următoarea maşină Turing $M = (K, \Sigma, \delta, s)$ este o procedură de decizie pentru L:

$K = \{q_0, \ldots, q_6\}$

$\Sigma = \{a, \text{Y}, \text{N}, \#\}$

$s = q_0$

şi δ este definită prin tabelul:

q	σ	$\delta(q, \sigma)$
q_0	#	(q_1, L)
q_1	a	$(q_2, \#)$
q_1	#	(q_4, R)
q_2	#	(q_3, L)
q_3	a	$(q_0, \#)$
q_3	#	(q_6, R)
q_4	#	$(q_5, \underline{Y})$
q_5	$\underline{Y}$	(h, R)
q_5	$\underline{N}$	(h, R)
q_6	#	$(q_5, \underline{N})$

Intrările omise sunt nerelevante şi pot fi definite arbitrar.

M şterge şirul de intrare de la dreapta la stânga, păstrând paritatea numărului de simboluri pe care le-a şters. Stările q_0, q_1, q_2, q_3 sunt utilizate în acest scop. La descoperirea simbolului # care delimitează capătul stânga al intrării, M trece din starea q_1 în q_4 sau din q_3 în q_6, scrie $\underline{Y}$ sau $\underline{N}$, mută capul o poziţie la dreapta şi se opreşte. Astfel:

$(q_0, \#a^n\#) \vdash_M^* (h, \# \underline{Y} \#)$, n este par;

$(q_0, \#a^n\#) \vdash_M^* (h, \# \underline{N} \#)$, n este impar;

Prin urmare M calculează X_L, deci L este decidabil Turing.

O altă modalitate de utilizare a maşinii Turing este ca acceptor de limbaj.

Fie Σ_0 un alfabet care nu conţine #. Spunem că o maşină Turing M acceptă şirul $w \in \Sigma_0^*$, dacă M se opreşte pentru intrarea w. Astfel M acceptă limbajul $L \subseteq \Sigma_0^*$, dacă şi numai dacă

$L = \{ w \in \Sigma_0^* : \text{M acceptă } w \}$.

Limbajul este acceptat Turing dacă există o anumită maşină Turing care să-l accepte. Vom vedea că orice limbaj decidabil Turing este acceptat Turing, dar nu şi invers.

Exemplu 1.7

Fie $\Sigma_0 = \{a, b\}$ şi fie $L = \{w \in \Sigma_0^*$: w conţine cel puţin un simbol $a\}$. Atunci L este acceptat de maşina Turing $M = (K, \Sigma, \delta, s)$, unde

$K = \{q_0\}$

$\Sigma = \{a, b, \#\}$

$s = q_0$

şi δ este definită prin tabelul:

q	σ	$\delta(q, \sigma)$
q_0	a	(h, a)
q_0	b	(q_0, L)
q_0	$\#$	(q_0, L)

Pornind din configuraţia $(q_0, \#w\#)$, $w \in \Sigma_0^*$, maşina se deplasează la stânga până când găseşte un simbol a şi se opreşte. Dacă nu găseşte niciun simbol a maşina se agaţă, prin urmare nu se opreşte. Astfel L este exact mulţimea şirurilor $w \in \Sigma_0^*$ cu proprietatea că M se opreşte pentru intrarea w, deci M acceptă L.

1.3. Compunerea maşinilor Turing

Scopul acestei secţii este de a prezenta o metodă pentru compunerea maşinilor Turing, astfel încât maşinile Turing pot fi văzute ca module sau subrutine ale altor maşini Turing.

Lema 1.1

Fie M o maşină Turing şi fie $(q_i, w_i\underline{a}_i u_i)$, $(i = 1, 2, 3)$ configuraţii ale lui M.
Atunci, dacă

$(q_1, w_1\underline{a}_1 u_1) \vdash_M^* (q_2, ww_2\underline{a}_2 u_2)$ pentru un şir w, şi

$(q_2, w_2\underline{a}_2 u_2) \vdash_M^* (q_3, w_3\underline{a}_3 u_3)$

atunci

$(q_1, w_1\underline{a}_1u_1) \vdash_M^* (q_3, ww_3\underline{a}_3u_3)$

Demonstrație. Această lemă, afirmă în esenţă, că dacă o maşină Turing poate realiza anumite operaţii fără a-şi muta capul la stânga unei anumite poziții, atunci va realiza aceste operaţii, indiferent dacă este cea mai din stânga poziție a benzii sau o poziție oarecare de pe bandă. În cazul din urmă, conţinutul părţii stânga a benzii rămâne neschimbat.

Cheia demonstraţiei este că dacă M nu se agaţă în timpul operaţiilor $(q_2, w_2\underline{a}_2u_2) \vdash_M^* (q_3, ww_3\underline{a}_3u_3)$, cum M nu are o modalitate directă de a detecta capătul stânga al benzii, nu trebuie să facă nicio încercare de a-şi muta capul la stânga primului simbol din $w_2a_2u_2$. Apoi, dacă porneşte în configuraţia $(q_2, ww_2\underline{a}_2u_2)$ va ajunge în configurația $(q_3, ww_3\underline{a}_3u_3)$ fără a pătrunde în primele $|w|$ pătrate benzii. ■

Această lemă este utilă pentru compunerea maşinilor Turing văzute ca subrutine. Să presupunem că o maşină Turing M_1 a fost proiectată pentru a se comporta într-un anumit mod, în particular, se ştie că M_1 nu se agaţă atunci când intrarea este conform convenţiilor prezentate. Atunci M_1 poate fi utilizată ca o componentă a unei alte maşini Turing. M_2 pregăteşte un anumit şir ca intrare pentru M_1, plasându-l la limita dreapta a porţiunii nevide a benzii, cedează controlul lui M_1 cu capul de citire / scriere poziționat la dreapta şirului. La sfârşit primeşte controlul de la M_1 când aceasta termină de calculat. Rezultatele operaţiilor parţiale efectuate de M_2 înainte de a ceda controlul lui M_1 vor fi păstrate în timpul operaţiilor lui M_1, cu condiţia ca aceste rezultate parţiale să fi fost salvate pe bandă, la stânga poziției de pe bandă unde intrarea lui M_1 începe. Este garantat că M_1 nu va invada rezultatele parţiale salvate de M_2, pentru că M_1 nu îşi va muta capul la stânga simbolului # care marchează capătul stânga al intrării sale.

Vom presupune că maşinile care sunt compuse nu au nicio stare în comun. Ca şi în cazul altor automate (automate finite, cu stiva) operaţiile unei maşini Turing nu depind de alegerea particulară a elementelor utilizate ca stări, ci doar de modul în care acestea sunt legate unele de altele prin funcţiile de tranziţie.

Putem acum să descriem construcţia unor maşini Turing simple şi modul în care pot fi compuse pentru a forma maşini mai puternice. Considerăm un alfabet Σ.

Maşini de bază. Sunt de două tipuri: *scriere*, *deplasare*.

1. Există $|\Sigma|$ *maşini scriere*, câte una pentru fiecare simbol din Σ. Fiecare maşină *scrie* un anumit simbol în poziţia curentă şi apoi se opreşte. Conţinutul anterior al poziției benzii este ignorat şi capul nu este mutat.

Formal, maşina Turing care scrie simbolul a este $w_a=(K, \Sigma, \delta, s)$ unde $K=\{q\}$, pentru o stare oarecare q, $s = q$ şi $\delta(q, b) = (h, a)$, $h \in \Sigma$. Pentru că aceste maşini sunt utilizate foarte des, le vom nota cu a (în loc de w_a).

2. Maşini *deplasare*, mută capul o poziţie la stânga sau la dreapta şi apoi se opresc. Fie q o anumită stare fixată. Fie $V_L=(\{q\}, \Sigma, \delta_L, q)$ unde $\delta_L(q, a)=(h, L)$, $\forall\ a \in \Sigma$. Fie $V_R=(\{q\}, \Sigma, \delta_R, q)$ unde $\delta_R(q, a) = (h, R)$, $\forall\ a \in \Sigma$. Le vom nota cu L, R.

Reguli de compunere a maşinilor Turing. Maşinile Turing vor fi compuse într-un mod sugestiv pentru structura unui automat finit (AF). Maşinile individuale sunt similare stărilor unui AF, şi vor fi conectate unele cu altele analog conectării stărilor unui AF. Conectarea unei maşini la alta nu este realizată până când prima maşină nu se opreşte; cealaltă maşină porneşte din starea ei iniţială cu banda şi poziţia capului aşa cum au fost lăsate de prima maşină.

Astfel, dacă M_1 şi M_2 sunt maşini Turing, maşina din figură

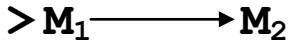


Figura 1-2

funcţionează astfel: porneşte din starea iniţială a lui M_1, funcţionează ca M_1 până când M_1 s-ar fi oprit; iniţiază M_2 şi funcţionează ca M_2.

Conexiunea dintre două maşini poate fi contingentă simbolului din poziţia curentă la oprirea primei maşini (aşa cum conexiunea dintre două stări ale unui automat finit poate depinde de următorul simbol citit). Astfel, dacă M_1, M_2, M_3 sunt maşini Turing, atunci maşina compusă începe prin a iniţia M_1; dacă M_1 se opreşte, este iniţiată M_2 sau M_3 în funcţie de conţinutul poziţiei curente în acel moment (a sau b).

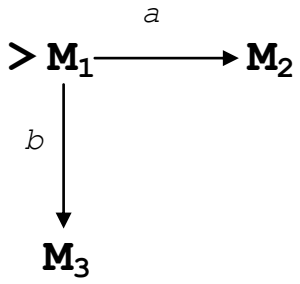


Figura 1-3

Definiţia 1.6

O schemă de maşină este un triplet $(\mathcal{m}, \eta, M_0)$, unde

$\mathcal{m}$ este o mulţime finită de maşini Turing cu un alfabet comun Σ şi mulţimi de stări disjuncte;

$M_0 \in \mathcal{m}$ este maşina iniţială;

$\eta : \mathcal{m} \times \Sigma \to \mathcal{m}$ este o funcţie.

O schemă $(\mathcal{m}, \eta, M_0)$ reprezintă o maşină Turing compusă din maşinile individuale $M \in \mathcal{m}$. Maşină compusă – notată $\overline{M}$, îşi începe operaţiile simulând pe cele ale maşinii iniţiale M_0. Când M_0 se opreşte, $\overline{M}$ începe operaţiile unei alte maşini din $\mathcal{m}$. În particular, dacă M_0 s-ar fi oprit în timp ce scana simbolul a, atunci:

- dacă $\eta(M_0, a)$ este nedefinit, $\overline{M}$ se opreşte
- dacă $\eta(M_0, a) = M \in \mathcal{m}$, atunci $\overline{M}$ continuă operaţiile din starea iniţială a lui M.

Definiţia 1.7

Fie $\mathcal{m} = \{M_0, ..., M_m\}$ $(m \geq 0)$, $i = 0 \ldots m$, $M_i = (K_i, \Sigma, \delta_i, s_i)$. Fie $q_0, ..., q_m$ stări noi care nu sunt în niciuna dintre mulţimile K_i. Atunci dacă $(\mathcal{m}, \eta, M_0)$ este o schemă de maşini Turing, ea reprezintă maşina Turing $M = (K, \Sigma, \delta, s)$, unde

$K = K_0 \cup \ldots \cup K_m \cup \{ q_0, \ldots, q_m \}$

$s = s_0$

și δ este definită astfel:

(a) dacă $0 \le i \le m$, $q \in K_i$, $a \in \Sigma$, $\delta_i(q, a) = (p, b)$, $p \ne h$ atunci $\delta(q, a) = \delta_i(q, a) = (p, b)$

(b) dacă $0 \le i \le m$, $a \in \Sigma$, $q \in K_i$, $a \in \Sigma$, $\delta_i(q, a) = (h, b)$ atunci $\delta(q, a) = (q_i, b)$

(c) dacă $0 \le i \le m$, $a \in \Sigma$, $\eta(M_i, a)$, nu este definit, atunci $\delta(q_i, a) = (h, a)$

(d) dacă $0 \le i \le m$, $a \in \Sigma$, $\eta(M_i, a) = M_j$, $\delta_j(s_j, a) = (p, b)$, atunci

$$\delta(q_i, a) = \begin{cases} (p, b), p \ne h \\ (q_j, b), p = h \end{cases}$$

Stările $q_0, \ldots, q_m$ sunt introduse ca puncte de oprire temporare între diferite mașini; q_i semnifică starea de *halt* a mașinii M_i. În definiția funcției δ, punctul (a) afirmă că fiecare mașină funcționează așa cum este de așteptat până când este gata să se oprească; conform punctului (b) mașina M_i intră în starea q_i în loc să se oprească; punctele (c) și (d) menționează că din starea q_i mașina compusă fie se oprește fie continuă primul pas al mașinii M_j în funcție de conținutul poziției curente. De notat că mulțimile de stări K_i sunt disjuncte ceea ce denotă că δ este o funcție.

Exemplu 1.8

Fie m mașina R definită anterior; $m = \{M_0\}$ unde $M_0 = R = \{(q), \Sigma, \delta_R, q\}$. Fie schema (m, η, M_0) unde

$$\eta(M_0, a) = \begin{cases} M_0, a \ne \# \\ \text{nedefinit}, a = \# \end{cases}$$

Atunci (m, η, M_0) reprezintă mașina $R_\#$, unde

$R_\# = \{(q, q_0), \Sigma, \delta, q\}$

$$\delta(q, a) = (q_0, R), a \in \Sigma$$

şi

$$\delta(q_0, a) = \begin{cases} (q_0, R), a \neq \# \\ (h, a), a = \# \end{cases}$$

Această maşină, pornind din orice poziţie a benzii, îşi mută capul o poziţie la dreapta, şi continuă să îşi mute capul la dreapta, până găseşte un simbol # şi se opreşte.

Dacă $\Sigma = \{\#, a, b, c\}$, atunci $R_\#$ poate fi reprezentată astfel:

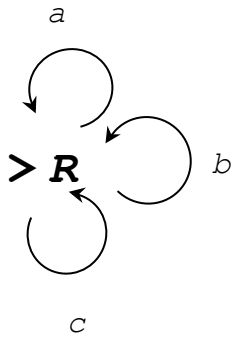


Figura 1-4

Regulile de bază pentru reprezentarea unei scheme (m, η, M_0) sunt următoarele:

1. fiecare maşină Turing $M \in m$ este reprezentată numai o dată
2. maşina iniţială M_0 este marcată cu o săgeată >
3. ori de câte ori $a \in \Sigma$ şi $\eta(M, a)$ este definită, de exemplu, dacă $\eta(M, a) = M'$ o săgeată etichetată cu a este reprezentată între M şi M'

Pentru că aceste reguli pot fi complicat de aplicat, le vom simplifica. Permitem fi două sau mai multe apariţii ale aceleiaşi maşini Turing, care reprezintă copii ale aceleaşi maşini.

Exemplu 1.9

Maşina compusă m constă din două copii ale lui R.

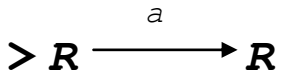


Figura 1-5

Maşina reprezentată în diagramă, îşi mută capul o poziţie la dreapta, apoi, dacă pătratul conţine *a*, îşi mută capul o poziţie la dreapta.

Săgeţile multiple şi subdiagramele similare pot fi eliminate în moduri diferite. O săgeată etichetată cu diferite simboluri înlocuieşte mai multe săgeţi; o săgeată neetichetată corespunde săgeţilor pentru fiecare simbol din Σ.

Exemplu 1.10

Fie Σ = (*a*, *b*, *c*, *#*) , diagrama

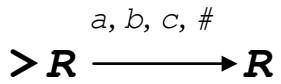


Figura 1-6

poate fi reprezentată

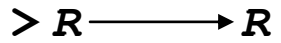


Figura 1-7

Maşina îşi deplasează capul la dreapta două poziţii şi se opreşte. Eliminând arcul, putem scrie *RR* sau R^2. În general R^k reprezintă maşina MM ... M cu *k* apariţii ale lui M.

În continuare vom utiliza notaţia $\overline{\sigma}$ pentru a desemna orice simbol diferit de σ. Cu această notaţie maşina $R_{\#}$ poate fi reprezentată ca:

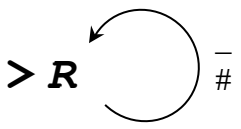
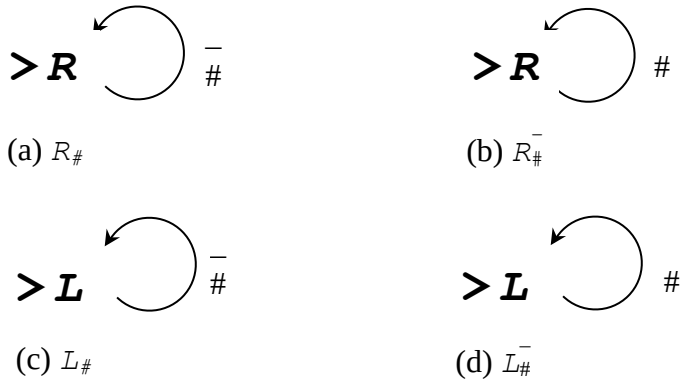


Figura 1-8

deplasează capul la dreapta până la *#*.

Exemplu 1.11

Reprezentarea maşinilor care se deplasează până la un anumit simbol:



(a) $R_{\#}$ (b) $R_{\#}^{-}$

(c) $L_{\#}$ (d) $L_{\#}^{-}$

Figura 1-9

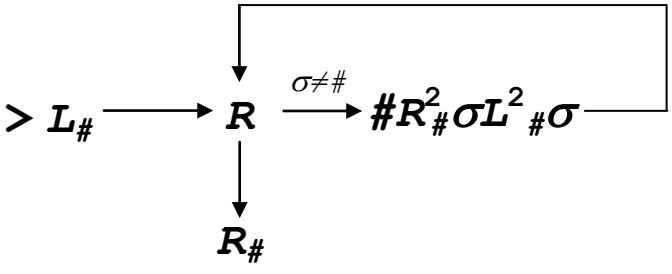
(a) $R_{\#}$ găseşte prima poziție liberă la dreapta celei curente (se deplasează dreapta până la $\#$)
(b) $R_{\#}^{-}$găseşte prima poziție utilizată la dreapta celei curent scanate (se deplasează dreapta până la primul simbol diferit de $\#$)
(c) $L_{\#}$ găseşte prima poziție liberă la stânga celei curente (se deplasează stânga până la $\#$)
(d) $L_{\#}^{-}$ găseşte prima poziție utilizată la stânga celei curent scanate (se deplasează stânga până la primul simbol diferit de $\#$)

Exemplu 1.12

Să reprezentăm maşina de copiere C care primește ca intrare un şir w încadrat de simboluri $\#$, cu capul pe simbolul $\#$ care marchează sfârşitul lui w şi lasă pe bandă şirul $w\#w$ încadrat de simboluri $\#$ cu capul pe simbolul $\#$ care marchează sfârşitul şirului $w\#w$. Prin convenție şirul w, posibil vid, conţine numai simboluri diferite de $\#$.

Formal, dacă s este starea iniţială a lui C, atunci

$(s, \#w\underline{\#}) \vdash_C^* (h, \#w\#w\underline{\#})$.



#

Figura 1-10

Să urmărim operațiile lui C pentru intrarea $\#abc\#$. După $L_{\#}$, pe bandă vom avea șirul $\underline{\#}abc\#$ cu capul poziționat pe simbolul # care delimitează stânga șirul. Capul este mutat o poziţie la dreapta prin execuția mașinii *R* obținându-se $\#\underline{a}bc\#$. Simbolul curent nu este # astfel încât este scris un # în locul lui. Al doilea simbol # la dreapta celui curent scanat este găsit prin execuția mașini R^2 ($\#\#bc\#\underline{\#}$) şi simbolul a este scris în locul lui ($\#\#bc\#\underline{a}$). Apoi capul este poziționat pe al doilea # la stânga simbolului curent prin execuția mașinii L^2 și un a este scris în locul lui ($\#\underline{a}bc\#a$). Acest ciclu se reia până cand se ajunge la sfârșitul șirului inițial.

Câteva dintre configuraţiile mașinii sunt următoarele:

$\#a\underline{b}c\#a$

$\#a\underline{\#}c\#a$

$\#a\#c\#a\underline{\#}$

$\#a\#c\#a\underline{b}$

$\#a\underline{\#}c\#ab$

$\#a\underline{b}c\#ab$ sfârşitul unui ciclu

$\#ab\underline{c}\#ab$

.

.

.

$\#ab\underline{c}\#abc$ sfârşitul celui de al 3 lea ciclu

În acest punct, la mutarea dreapta a capului este găsit un simbol # ($\#abc\underline{\#}abc$). Următorul simbol # la dreapta simbolulu curent este găsit ($\#abc\#abc\underline{\#}$) şi maşina se opreşte.

De notat că Lema 1.1 garantează că maşina C va copia ultimul bloc de simboluri diferite de # pe bandă dacă există mai multe astfel de blocuri. De exemplu, dacă u, v, m sunt şiruri de simboluri diferite de #, C transformă

$\#u\#v\#\underline{w\#}$ în $\#u\#v\#w\#\underline{w\#}$. Desigur, C nu va funcţiona corect dacă porneşte cu simboluri diferite de # la dreapta capului.

1.4. Generalizări ale maşinii Turing

Pentru a înțelege mai bine puterea de calcul a mașinii Turing vom încerca să extindem mașina în diferite moduri. Dar, așa cum vom vedea, noile capabilităţi adăugate nu măresc clasa funcţiilor calculabile sau a limbajelor decidabile, ceea ce întărește ideea că mașina Turing este modelul final de calculabilitate. Extensiile sunt următoarele:

1. permitem benzii să fie infinită în ambele sensuri (nu numai la dreapta);
2. permitem mai multe benzi;
3. permitem mai mute capete pe bandă;
4. banda poate fi bidimensională (în loc de unidimensională);

Pentru fiecare tip de extensie, operaţiile maşinii extinse pot fi simulate printr-o mașina Turing standard.

Bandă infinită în ambele sensuri

Banda nu mai are un capăt stânga și un șir de intrare poate fi scris oriunde pe bandă. Operaţiile se desfăşoară ca în cazul unei mașini Turing obișnuite, cu excepţia faptului că există un spaţiu arbitrar atât la dreapta, cât şi la stânga pentru calcule intermediare.

Formal $M = (K, \Sigma, \delta, s)$; o configuraţie este definită similar (q, w, a, u), $q \in K$, $w, u \in \Sigma^*$, $a \in \Sigma$, unde w nu începe cu # şi u nu se termină cu #; conform notaţiei utilizate pentru mașina Turing, o configurație este descrisă ca (q, wau).

Relaţia dintre configuraţii, $\vdash_M$, este similară celei pentru mașina Turing standard. O diferenţă este că dacă $\delta(q, a) = (p, L)$ atunci $(q, \underline{a}u) \vdash_M (p, \underline{\#}au)$. Adică există bandă disponibilă atât la stânga cât şi la dreapta maşinii, prin urmare aceasta nu se poate agăţa. De asemenea, dacă $\delta(q, a) = (p, R)$ atunci $(q, \underline{\#}au) \vdash_M (p, \underline{a}u)$

Sunt astfel de maşini mai puternice decât mașina Turing clasică? Nu. Orice operaţie efectuată de mașina Turing cu bandă infinită în ambele sensuri, poate fi efectuată de mașina Turing standard.

Lema 1.2

Fie $M_1 = (K_1, \Sigma_1, \delta_1, s_1)$ o maşină Turing cu o bandă infinită în ambele sensuri. Atunci există o maşină Turing standard $M_2 = (K_2, \Sigma_2, \delta_2, s_2)$ astfel încât $\forall\ w \in (\Sigma_1 - \{\#\})^*$, următoarele afirmaţii sunt adevărate :

(a) Dacă M_1 se opreşte pentru w,

$$(s_1, w\underline{\#}) \vdash_M^* (h, u\underline{a}v),\ u, v \in \Sigma_1^*,\ a \in \Sigma_1,$$

atunci M_2 se opreşte pentru w

$$(s_2, \#w\underline{\#}) \vdash_M^* (h, \#u\underline{a}v)$$

(b) Dacă M_1 nu se opreşte, nici M_2 nu se opreşte pentru w.

Pentru a da un răspuns ferm la întrebarea dacă maşinile Turing cu bandă infinită în ambele sensuri, sunt mai puternice, trebuie să definim modul de utilizare al acestor maşini. Vom considera cazul funcţiilor definite pe şiruri. Noţiunile de limbaj decidabil şi acceptat Turing sunt definite analog.

Definiţia 1.8

Spunem că o maşina Turing $M = (K, \Sigma, \delta, s)$ cu bandă infinită în ambele sensuri calculează funcţia $f' : \Sigma_0^* \to \Sigma_1^*$, unde Σ_0, Σ_1 sunt alfabete care nu conţin $\#$, dacă şi numai dacă $\forall\ w \in \Sigma_0^*$, dacă $f(w) = u$, atunci $(s, w\underline{\#}) \vdash_M^* (h, u\underline{\#})$.

Teorema 1.1

Orice funcţie care este calculată sau limbaj care este acceptat sau decis de o maşina Turing cu bandă infinită în ambele sensuri este calculată, decis sau acceptat de o maşină Turing standard.

Demonstraţie. Fie M_1 o maşină Turing cu bandă infinită în ambele sensuri care calculează o anumită funcţie, decide sau acceptă un anumit limbaj. Fie M_2 maşina Turing standard corespunzătoare lui M_1 conform Lemei 1.2. Atunci M_2 calculează aceeaşi funcţie, respectiv decide sau acceptă acelaşi limbaj. ■

Mai multe benzi

Pentru orice număr $k > 0$ fixat, o mașină Turing cu k - benzi constă dintr-o unitate de control şi k benzi (limitate stânga). Fiecare bandă este conectată la dispozitivul de control printr-un cap de citire / scriere.

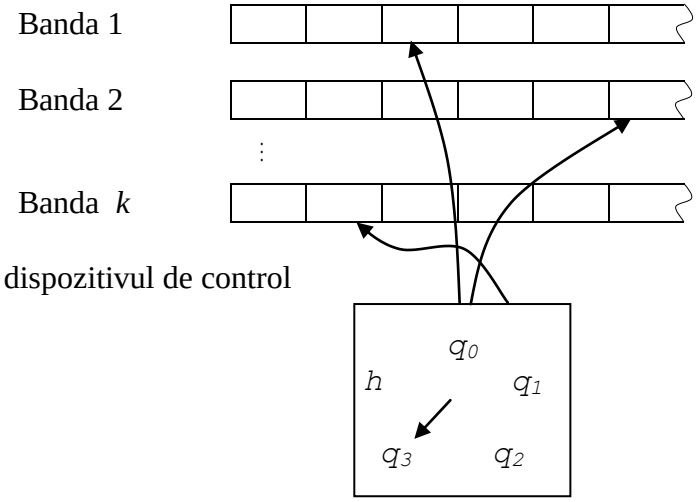


Figura 1-11

Maşina poate într-un singur pas să analizeze toate simbolurile scanate de toate capetele sale şi apoi în funcţie de aceste simboluri şi de starea curentă, poate să rescrie un simbol sau să-şi mute capetele la stânga sau la dreapta.

O mașină Turing cu k - benzi poate fi utilizată pentru a calcula o funcţie sau a decide un limbaj. Convenim ca şirul de intrare să fie plasat pe prima bandă cu capul la limita dreapta. Celelalte benzi sunt iniţial vide, cu capetele la limita stânga a fiecăreia. La sfârşitul operaţiilor, mașina Turing cu k - benzi va avea rezultatul pe prima bandă; conţinutul celorlalte benzi este ignorat.

Cu ajutorul mai multor benzi este deseori mai uşor de construit o mașină Turing pentru o funcţie particulară. Să considerăm copierea unui şir:

$$\#w\underline{\#} \rightarrow \#w\#w\underline{\#},$$

unde w nu conţine simboluri $\#$.

O mașină Turing cu două benzi poate realiza copierea astfel:

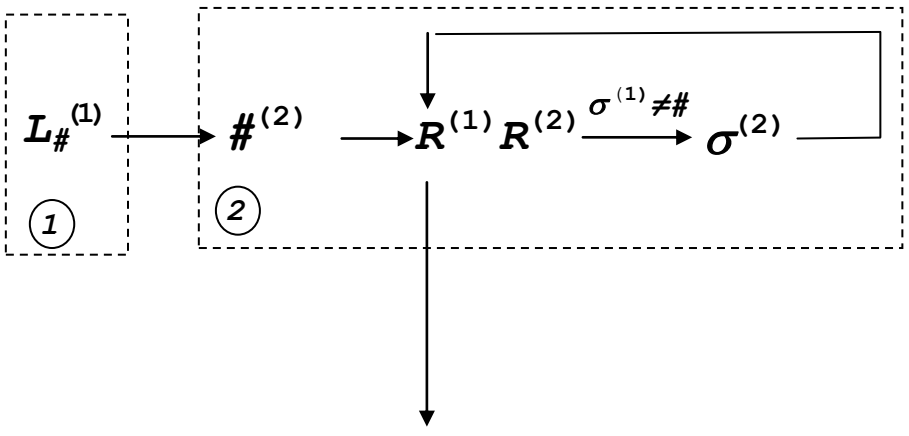
1. mută capul primei benzi la stânga până găseşte un $\#$.

2. mută capetele ambelor benzi la dreapta, copiind fiecare simbol de pe prima bandă pe a doua, până găseşte un # pe prima bandă; prima poziţie a celei de a doua benzi trebuie să conţină #.
3. mută capul celei de a doua benzi la stânga până găseşte un #.
4. mută capete ambelor benzi la dreapta, copiind simboluri de pe a doua bandă pe prima. Se opreşte la găsirea unui # pe a doua bandă.

Secvenţa de acţiuni poate fi reprezentată astfel:

La început:	1*ma* bandă *#w#* 2*a* bandă *#*
După (1)	1*ma* bandă *#w#* 2*a* bandă *#*
După (2)	1*ma* bandă *#w#* 2*a* bandă *#w#*
După (3)	1*ma* bandă *#w#* 2*a* bandă *#w#*
După (4)	1*ma* bandă *#w#w#* 2 bandă *#w#*

Vom ataşa ca indice superior simbolului care reprezintă o maşină numărul benzii pe care operează. De exemplu $\#^{(2)}$ scrie un # pe a doua bandă; $L_{\#}^{(1)}$ caută la stânga un # pe prima bandă. Cu aceste convenţii maşina C de copiere este:



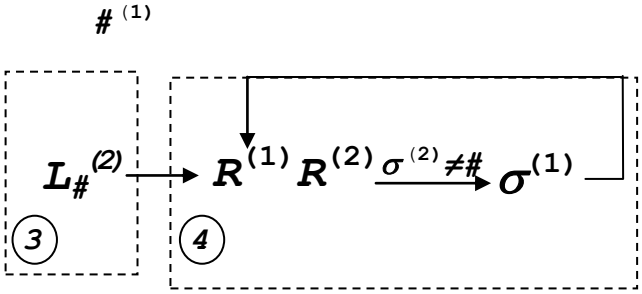


Figura 1-12

Configuraţia unei maşină Turing cu k - benzi este $(q, w_1\underline{a_1}u_1, \ldots, w_k a_k \underline{u_k})$ cu semnificaţia că şirul de pe a i-a bandă este $w_i \underline{a_i} u_i$ cu capul acelei benzi indicând a_i.

Lema 1.3

Fie $k > 0$ şi M_1 o maşină cu k – benzi, alfabetul Σ_1 şi starea iniţială s_1. Atunci există o maşină Turing standard $M_2 = (K_2, \Sigma_2, \delta_2, s_2)$ astfel încât $\Sigma_1 \subseteq \Sigma_2$ şi $\forall\ w \in (\Sigma_1 - \{\#\})^*$,
(a) Dacă M_1 se opreşte pe intrarea w, adică

$$(s_1, \#w\underline{\#}, \underline{\#}, \ldots, \underline{\#}) \vdash_M^* (h, w_1\underline{a_1}u_1, \ldots, w_k\underline{a_k}u_k)$$

pentru $w_1, \ldots, w_K\ u_1, \ldots, u_K$ şi $a_1, \ldots, a_K$, atunci M_2 se opreşte pentru w cu banda având acelaşi conţinut cu prima bandă a lui M_2, respectiv

$$(s_2, \#w\underline{\#}) \vdash_M^* (h, w_1\underline{a_1}n_1)$$

(b) Dacă M_1 se agaţă pentru w, atunci M_2 se agaţă pentru w;
(c) Dacă M_1 nici nu se opreşte nici nu se agaţă pentru w, atunci nici M_2 nu se agaţă nici nu se opreşte pentru w.

Teorema 1.2

Orice funcţie care este calculată sau limbaj care este decis sau acceptat de o maşina Turing cu k – benzi este calculată, decis, respectiv acceptat de o maşină Turing standard.

Demonstraţie. Teorema derivă din Lema 1.3.

Mai multe capete

Într-un pas, pot citi, scrie sau se pot deplasa independent pe aceeași bandă. Trebuie adoptată o anumită convenţie despre ceea ce se întâmplă atunci când două capete încearcă să scrie lucruri diferite pe aceeași poziție a benzii.

Ideea este de a împărți banda în alte benzi, dintre care unele să fie utilizate doar pentru a memora poziția capetelor. Pentru a simula un pas al mașinii cu mai multe capete, banda trebuie scanată de cel puțin două ori. O dată pentru a determina simbolurile indicate de capete, apoi pentru a schimba aceste simboluri sau a deplasa capetele.

Bandă cu două dimensiuni

O altă generalizare a mașinii Turing este de a permite ca banda să aibă două dimensiuni. Mașina are un singur cap care poate fi utilizat pentru citi sau scrie simboluri. Într-un pas, își poate deplasa capul o poziție sus / jos / stânga / dreapta. Intrarea este plasată pe linia de jos a benzii, restul este benzii fiind vidă inițial.

Teorema 1.3

Mașinile Turing cu benzi cu două dimensiuni pot fi simulate de mașini Turing standard.

1.5. Mașina Turing nedeterministă

Dacă automatelor finite li se permite să acţioneze nedeterminist nu rezultă nicio îmbunătăţire a puterii de calcul, dar automatele cu stivă nedeterministe sunt mai puternice decât cele deterministe.

Ne putem imagina şi maşini Turing nedeterministe. Astfel de maşini pot avea pentru anumite combinaţii de stări şi anumite simboluri de intrare mai multe comportamente.

Definiția 1.9

O mașină Turing nedeterministă este un tuplu $M = (K, \Sigma, \Delta, s)$, unde K, Σ, s au aceeași semnificație ca în cazul mașinii Turing standard, iar Δ este o submulţime din

$$(K \times \Sigma) \times ((K \cup \{h\}) \times (\Sigma \cup \{L, R\})).$$

Configuraţiile precum şi relaţiile $\vdash_M$, $\vdash_M^*$ sunt definite în mod analog. Dar în acest caz o configuraţie poate genera mai multe într-un singur pas.

Dacă o mașină Turing acţionează nedeterminist, există o creştere a puterii de calcul? Cum o mașină Turing nedeterminista poate produce două rezultate diferite pentru aceeaşi intrare, trebuie avut grijă la ceea ce este cosiderat a fi rezultatul operaţiilor unei astfel de maşini. Această problemă se rezolvă construind mașinile Turing nedeterministe doar ca acceptori astfel încât singurul fapt semnificativ este dacă maşina se opreşte, nu conţinutul benzii.

Exemplu 1.13

Fie $L = \{w \in \{a, b\}^* : w$ conţine o apariţie a subşirului $abaab\}$. Desigur, L este un limbaj regulat, prin urmare putem construi un automat finit determinist sau nedeterminist pentru a-l accepta.

Maşina Turing primeşte intrarea sub forma $\#w\underline{\#}$, $w \in \{a, b\}^*$. Scanează intrarea de la dreapta la stânga şi alege nedeterminist un punct din w în care să înceapă să verifice subşirul specificat. Din acest moment comportamentul său este determinist. Verifică că următoarele cinci simboluri sunt de la dreapta la stânga, b, a, a, b, a. Dacă acest subşir este detectat, maşina se opreşte şi acceptă şirul de intrare. Dacă nu identifică subşirul, maşina intră într-un ciclu infinit.

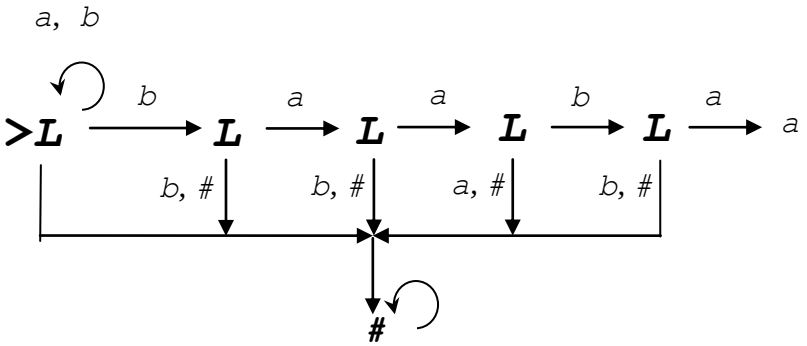


Figura 1-13

Punctul nedeterminist din diagramă este colţul stâng sus, unde dacă simbolul scanat este *b*, există două comportamente posibile. Mută capul spre stânga şi continuă scanare dreapta-stânga sau începe să verifice şirul specificat.

Exemplu 1.14

Un număr compus este un număr care este produsul a două numere naturale fiecare mai mare decât 1; de exemplu, 4, 6, 8, 9, 10, 13 sunt compuse, dar 1, 3, 5, 7, 11, nu sunt.

Fie $L = \{I^n : n$ este un număr compus$\}$. Pentru a proiecta o maşină Turing care să accepte **L**, pare a fi necesar să găsim factorii unui număr. Dacă nedeterminismul este admis, putem proiecta o maşină care să accepte *L*, simplu, ghicind aceşti factori, dacă există. Această maşină operează pentru intrarea $\#I^n\#$ astfel:

1. Alege nedeterminist două numere p, $q > 1$ şi transformă intrarea în $\#I^n\#I^p\#I^q\underline{\#}$
2. Utilizează maşina multiplicativă pentru a transforma rezultatul pasului (1) în $\#I^n\#I^{p\times q}\underline{\#}$.
3. Verifică dacă $I^n = I^{p\times q}$; este uşor de verificat pentru că numai lungimea şirurilor trebuie comparată. Se opreşte dacă lungimile sunt egale, altfel continuă să cicleze la infinit.

Fie *P* - maşina de multiplicare, *E* - maşina care verifica egalitatea a două şiruri: $\forall\ w, u \in \{I\}^*$, E se opreşte pentru intrarea $\#w\#u\underline{\#} \Leftrightarrow$ w=u, şi *G* - maşina care generează un şir de simboluri *I*: scrie I^p, $p \geq 2$. Limbajul *L* este acceptat de maşina *GGPE*.

Maşina *G*:

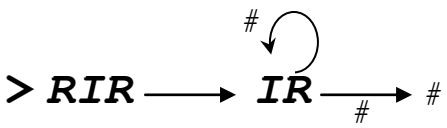
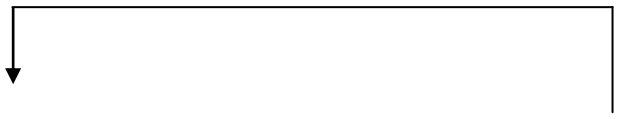


Figura 1-14

Maşina *E*:



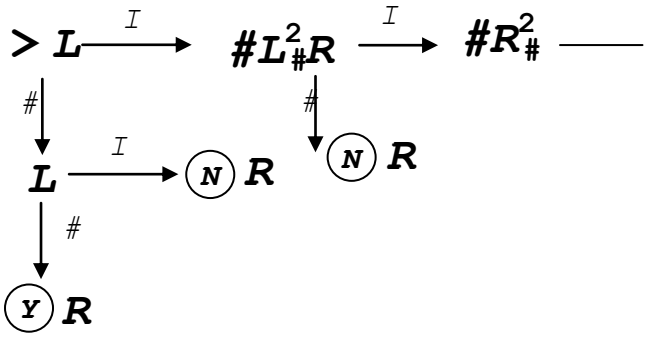


Figura 1-15

Maşina *P*:

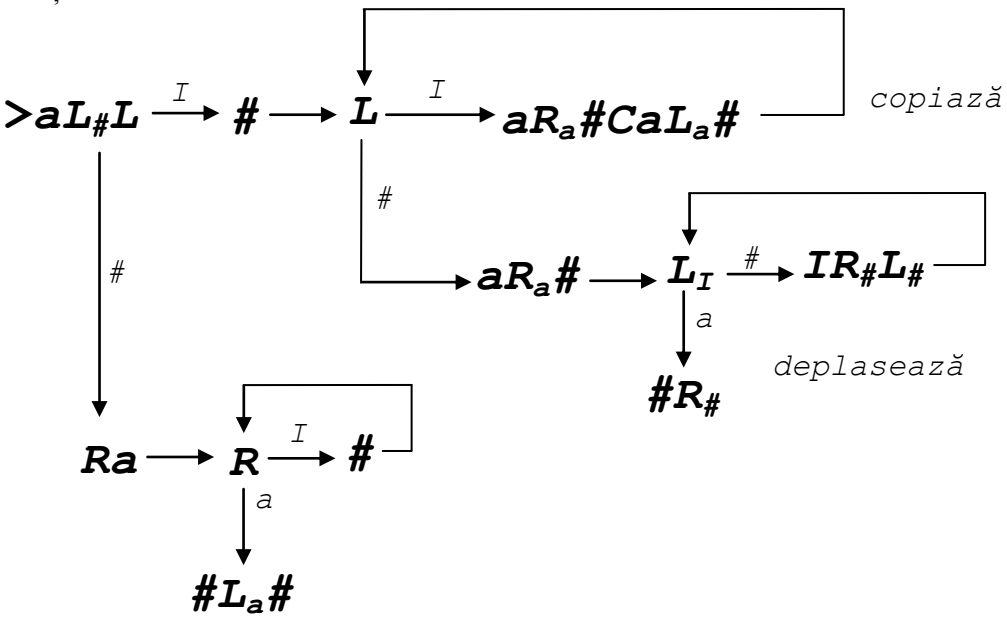


Figura 1-16

Maşina verifică dacă fiecare argument este 0; dacă da, se opreşte cu şirul vid ca rezultat. Altfel, copiază al doilea şir de un număr de ori dat de primul şir; la sfârşit deplasează copiile şi elimină simbolurile #.

Lema 1.4

Pentru orice maşină Turing nedeterministă M_1, putem construi o maşină Turing standard M_2 astfel încât pentru orice şir w care nu conţine #:

(a) Dacă M_1 se opreşte pentru intrarea w, atunci M_2 se opreşte pentru w;

(b) Dacă M_1 nu se opreşte pentru w, atunci nici M_2 nu se opreşte pentru w.

Teorema 1.4

Orice limbaj acceptat de o maşină Turing nedeterministă este acceptat de o maşină Turing deterministă.

1.6. Maşina Turing universală

Una dintre caracteristicile calculatoarelor, pe care maşinile Turing aşa cum au fost prezentate până acum nu o au, este programabilitatea. Până acum, am proiectat o maşină Turing pentru fiecare problemă, în timp ce calculatoarele sunt generale. Vom arăta că şi maşina Turing este în esenţă programabilă.

Pentru început, vom prezenta un mod general de a specifica maşina Turing astfel încât descrierea unei maşini Turing să poată fi folosită ca intrare pentru alte maşini Turing.

O maşină Turing este un tuplu $M = (K, \Sigma, \delta, s)$. Cum K, Σ sunt mulţimi finite, putem scrie acest tuplu ca un şir utilizând simboluri din K şi Σ. Vom codifica stările şi simbolurile de pe bandă ca şiruri peste un alfabet fixat.

Presupunem că există mulţimile infinit numărabile:

$$K_\infty = \{ q_1, q_2, q_3, \ldots \}$$

$$\Sigma_\infty = \{ a_1, a_2, a_3, \ldots \}$$

astfel încât pentru fiecare maşină Turing, mulţimea stărilor este o submulţime finită din K_∞ şi alfabetul de intrare este o submulţime finită din Σ_∞.

Să considerăm următoarea corespondenţă λ între simbolurile componente ale maşinii Turing şi şiruri peste alfabetul $\{I\}$.

σ	$\lambda(\sigma)$
q_i	I^{i+1}
h	I

L	I
R	II
a_i	I^{i+2}

Trebuie remarcat că nu există doi membri din $K_\infty \cup \{h\}$ respectiv din $\{L, R\} \cup \Sigma_\infty$ care să fie reprezentaţi la fel.

Fie c un alt simbol. Vom codifica maşina Turing peste alfabetul $\{c, I\}$.

Fie $M = (K, \Sigma, \delta, s)$, $K \leq K_\infty$, $\Sigma \leq \Sigma_\infty$

$K = \{q_{i_1}, q_{i_2}, \ldots, q_{i_K}\}$, $i_1 < i_2 < \ldots < i_K$

$\Sigma = \{a_{j_1}, a_{j_2}, \ldots, a_{j_1}\}$, $j_1 < j_2 < \ldots < j_l$;

$s = q_{im}$, $1 \leq m \leq k$

Definim kl şiruri S_{pr}, $1 \leq p \leq k$, $1 \leq r \leq l$. Fiecare şir S_{pr} codifică valoarea funcţiei de tranziţie pentru perechea (stare, simbol), respectiv (q_{i_p}, a_{j_r})

$\delta(q_{i_p}, a_{j_r}) = (q', b)$, $q' \in K \cup \{h\}$

$b \in \Sigma \cup \{L, R\}$

$S_{pr} = cw_1cw_2cw_3cw_4c$,

unde $w_1 = \lambda(q_{i_p})$

$w_2 = \lambda(a_{j_r})$

$w_3 = \lambda(q')$

$w_4 = \lambda(b)$

Exemplu 1.15

Fie $M = (K, \Sigma, \delta, s)$, unde

$K = \{q_2\}$

$\Sigma = \{a_1, a_3, a_6\}$

$s = q_2$

$\delta(q_2, a_3) = \delta(q_2, a_6) = (q_2, R)$

$$\delta = (q_2, a_1) = (h, a_3)$$

atunci

$$k = 1,\ i_1 = 2,\ l = 3,\ j_1 = 1,\ j_2 = 3,\ j_3 = 6,$$

şi

$$S_{11} = cI^3cI^3cIcI^5c$$
$$S_{12} = cI^3cI^5cI^3cI^2c$$
$$S_{13} = cI^3cI^8cI^3cI^2c$$

În final, scriem $\rho(M)$ pentru şirul unic

$$cS_0cS_{11}S_{12} \ldots S_{11}S_{21}S_{22} \ldots S_{21} \ldots S_{k1}S_{k2} \ldots S_{kl}c$$

unde $S_0 = \lambda(S)$ este codificarea stării iniţiale.

Acest şir reprezintă codificarea maşinii Turing M. De notat că nu există două şiruri care să corespundă aceleiaşi maşini Turing. Din $\rho(M)$ se poate reconstrui M. $\rho(M)$ are forma cw_0c, …, $w_0 \in \{I\}$; această parte a şirului determină starea iniţială. Restul şirului poate fi spart în blocuri de patru membri din $\{I\}^*$, fiecare bloc fiind separat de vecinii săi prin cc, şi despărţit în interior prin c; din aceste şiruri, valorile δ pot fi reconstruite cu uşurinţă.

Exemplu 1.16

Pentru maşina M utilizată în exemplul anterior, $\rho(M)$ este

$$cI^3ccI^3cI^3cIcI^5ccI^3cI^5cI^3cI^2ccI^3cI^8cI^3cI^2cc$$

Acum putem prezenta maşina Turing universală U, care utilizează codificarea $\rho(M)$ a celorlalte maşini ca programe.

Intuitiv, U primeşte două argumente: o descriere a unei maşini Turing M şi un şir de intrare w, şi execută operaţiile pe care le-ar fi executat M. Desigur, atât M cât şi w, trebuie să fie codificate, la fel va fi şi rezultatul. Şirurile peste alfabetul lui M vor fi codificate astfel:

Dacă

$$w = b_1 \ldots b_n,\ \forall\ b_i \in \Sigma_\infty,\ i = 1 \ldots n$$

atunci

$$\rho(M) = c\lambda(b_1)c\lambda(b_2)c \ldots c\lambda(b_n)c$$

Trebuie remarcat că $\rho(M)$ nu conţine simboluri $\#$. Proprietatea pe care $U = (K_U, \Sigma_U, \delta_U, s_U)$ trebuie să o aibă, poate fi exprimată astfel:

Pentru orice maşină Turing $M = (K, \Sigma, \delta, s)$ şi orice şir $w \in \Sigma^*$

1. dacă $(h, u\underline{a}v)$ este o configuraţie de oprire a lui M astfel încât $(s, \#w\underline{\#}) \vdash_M^* (h, u\underline{a}v)$ atunci $(s_U, \#\rho(M)\rho(w)\underline{\#}) \vdash_U^* (h, \#\rho(uav)\underline{\#})$
2. dacă $(s_U, \#\rho(M)\rho(w)\underline{\#}) \vdash_U^* (h, u'\underline{a}'v')$ pentru o anumită configuraţie de oprire $(h, u'\underline{a}'v')$ a lui U, atunci $a'=\#$, $v'=e$, $u'=\#\rho(uav)$ pentru u, a, v astfel încât $(h, u\underline{a}v)$ este o configuraţie de oprire a lui M şi $(s, \#w\underline{\#}) \vdash_M^* (h, u\underline{a}v)$.

Altfel spus,

1. Dacă M se opreşte pentru w, atunci U se opreşte pentru intrarea formată din codificarea lui M urmată de codificarea lui w şi atunci când U se opreşte, pe bandă există codificarea şirului aflat pe banda lui M, când M se opreşte.
2. Dacă U se opreşte pe intrarea alcătuită din codificarea lui M urmată de codificarea lui w, atunci M se opreşte pe intrarea w, iar conţinutul benzii lui U (la oprirea lui U) conţine codificarea conţinutului benzii lui M când M se opreşte.

Pentru uşurința înțelegerii vom descrie funcționarea unei maşini cu trei benzi U', care este echivalentă cu maşina U (Lema 1.3). U' îşi utilizează benzile astfel:

- 1-ma bandă conţine codificarea benzii lui M
- a 2-a bandă conţine codificarea lui M
- a 3-a bandă conţine codificarea stării lui M la punctul curent al simulării.

Maşina U' porneşte cu şirul $\#\rho(M)\rho(w)$ pe prima bandă şi celelalte benzi goale (este irelevant comportamentul lui U' dacă şirul de intrare nu are această formă). Mai întâi U' pune $\rho(M)$ pe a doua bandă, deplasează $\rho(w)$ la capătul stânga al primei benzi precedându-l de $\#c\rho(\#)$ şi încheindu-l cu $\rho(\#)c$. Deci în acest punct prima bandă conţine $\#\rho(\#w\#)$. (U' nu are nicio dificultate în a determina unde se termină $\rho(M)$ şi începe $\rho(w)$, este singurul punct din $\rho(M)\rho(w)$ unde există trei simboluri c consecutive). Din $\rho(M)$, U' extrage codificarea stării iniţiale a lui M, pe care o copiază pe a treia bandă. Acum U' simulează paşii operaţiilor lui M. Între paşii simulaţi, U' va păstra capetele corespunzătoare benzilor a doua şi a

treia la limita stânga, iar capul primei benzi în dreptul simbolului c care marchează sfârşitul codificării simbolului curent scanat de M. Astfel, dacă M începe scanarea ultimului simbol # din $\#w\#$, U' va începe simularea mutându-şi capul pe prima bandă pe ultimul c de pe această bandă.

Acum U' găseşte pe a doua bandă blocul de forma

$$ccI^icI^jcI^kcI^lcc$$

astfel încât I^i este şirul de simboluri I care se termină la poziţia curentă a capului pe a treia bandă şi I^j este cel de pe prima bandă, şi schimbă prima bandă corespunzător.

Dacă I^l este $\lambda(L)$ sau $\lambda(R)$, aceasta implică mutarea capului cu câteva simboluri la stânga / dreapta; dacă I^l este $\lambda(a)$, $a \in \Sigma_\infty$, atunci poate fi necesară deplasarea părţii dreapta a şirului pe prima bandă într-un sens sau altul pentru a înlocui codificarea unui simbol cu un altul.

În final, U' pune I^k pe a treia bandă şi verifică dacă a treia bandă conţine acum $\lambda(h)$. Dacă nu, atunci U' simulează un alt pas al lui M. Dacă da, U' se opreşte, după ce a mutat capul pe prima bandă la dreapta până la primul #.

2. CALCULABILITATE

Se poate argumenta că dacă reprezentarea limbajelor se face prin şiruri, nu putem reprezenta orice limbaj - există numai o mulţime numărabilă de şiruri peste un alfabet, dar nenumărabilă de limbaje. Automatele finite, automatele cu stivă, maşina Turing, gramaticile sunt exemple de reprezentări finite care pot fi utilizate pentru a specifica limbaje. Automatele şi gramaticile pot fi specificate prin şiruri. Am făcut asta şi pentru maşina Turing. Deci există doar o mulţime numărabilă de limbaje regulate, independente de context, decidabile / acceptate Turing. Deşi am extins maşina Turing, totuşi maşina Turing nu poate accepta / decide decât o parte infinitesimală din toate limbajele posibile. Acum suntem în măsură să spunem ce limbaje nu sunt decidabile / acceptate Turing.

Interesul nostru în găsirea unor astfel de limbaje este justificat de faptul că maşina Turing este formalizarea matematică a noţiunii de algoritm. Deci un limbaj care nu este decidabil Turing, nu este decidabil prin niciun algoritm; nu există niciun mod general şi sistematic de a spune ce şiruri aparţin limbajului.

2.1. Problema opririi maşinii Turing

Teorema 2.1

Orice limbaj decidabil Turing este acceptat Turing.

Demonstraţie. Fie `L` un limbaj decis de o maşină Turing `M`, atunci `L` este acceptat de `M'`:

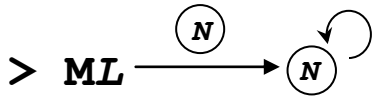


Figura 2-1

care simulează M.
Când M se opreşte, M′ buclează la infinit dacă M (N) lasă pe bandă; dacă M pune (Y) pe bandă, atunci M′ se opreşte. ■

Teorema 2.2

Dacă L este Turing decidabil, atunci $\overline{L}$ (complementul lui L) este Turing decidabil.

Demonstraţie. Dacă L este decis Turing de maşina M, atunci $\overline{L}$ este decis de maşina M'.

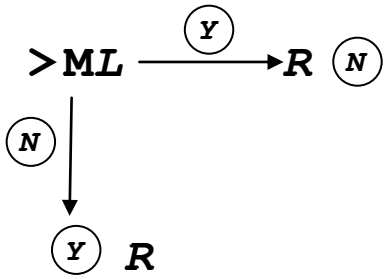


Figura 2-2

M′ simulează M până când se opreşte, apoi interschimbă (Y) cu (N) ■

Dar pentru întrebările:

- este orice limbaj acceptat Turing, decidabil Turing?
- este complementul oricărui limbaj acceptat Turing, acceptat Turing?

Răspunsul în ambele cazuri este NU. Dacă ar fi posibil să se spună pentru orice mașină Turing şi orice şir de intrare dacă respectiva mașină Turing se oprește sau nu pe acea intrare, atunci orice limbaj acceptat Turing ar fi şi decidabil Turing.

Astfel, dacă M_1 ar fi o mașină Turing care acceptă un limbaj L, atunci putem proiecta o mașină Turing M_2 care decide L astfel: M_2 execută operaţiile necesare pentru a prezice eventualul rezultat al operaţiilor lui M_1 pe intrarea w. M_2 s-ar opri cu (Y), (N) pe bandă, în funcţie de acceptarea sau nu a lui w de către M_1. Astfel întrebarea de orice limbaj acceptat Turing este decidabil Turing revine la întrebarea dacă există o super mașină Turing care

poate prezice rezultatul operaţiilor efectuate de maşini Turing arbitrare pe intrări arbitrare. Mai clar, dacă limbajul

$K_0 = \{ \rho(M)\, \rho(w) : M \text{ acceptă şirul } w \}$

este decis de o maşină Turing M_0 atunci orice limbaj acceptat Turing este decidabil Turing.

Atunci pentru o maşină Turing M_1 care acceptă limbajul L, putem construi o maşină Turing, M_2 care decide L, astfel:

1. construim maşina Turing M_1^* care pentru orice şir w, transformă $\#w\underline{\#}$ în $\#\rho(M_1)\,\rho(w)\,\underline{\#}$; adică M_1^* deplasează intrarea peste $|\rho(M_1)|$ poziţii, o codifică peste alfabetul $\{I, C\}$ şi scrie şirul $\#\rho(M_1)$ pe poziţiile cele mai din stânga de benzi. Fie $M_2 = M_1^* M_0$. M_2 transformă intrarea $\#w\underline{\#}$ în $\#\rho(M_1)\,\rho(w)\,\underline{\#}$ şi cedează controlul lui M_0, care, prin ipoteză, decide dacă M_1 acceptă sau nu w.

Mai mult, se poate demonstra că limbajul K_0 este acceptat Turing - este acceptat de o variantă a maşinii Turing universale. Astfel, orice limbaj acceptat Turing este decidabil Turing, dacă şi numai dacă limbajul K_0 acceptat Turing este decidabil Turing.

Dar vom arăta că limbajul K_0 nu este decidabil Turing. Demonstraţia se face prin reducere la absurd. Dacă K_0 ar fi decidabil Turing, atunci

$K_1 = \{ \rho(M) : M \text{ acceptă şirul de intrare } \rho(M) \}$

ar fi decidabil Turing.

Dacă M_0 ar decide K_0, atunci maşina Turing M_1 care să decidă K_1 ar putea fi construită; ar transforma şirul de intrare $\#w\#$ în $\#w\rho(w)\underline{\#}$ şi apoi ar ceda controlul lui M_0.

Acum M_1 va obţine acelaşi rezultat pentru intrarea $\#w\underline{\#}$ ca şi M_0 pentru intrarea $\#w\rho(w)\,\underline{\#}$; dar din definiţia lui K_0, M_0 obţine (Y) pentru intrarea $\#w\rho(w)\,\underline{\#}$, dacă şi numai dacă:

a) w este $\rho(M)$ pentru o maşină Turing M, şi

b) maşina Turing M acceptă şirul w, adică şirul de intrare $\rho(M)$

Dar, aceasta este definiţia lui K_1. Deci este suficient să se demonstreze că limbajul K_1 nu este decidabil Turing.

Dacă limbajul K_1 ar fi decidabil Turing, atunci şi complementul său ar fi decidabil Turing:

$\overline{K_1} = \{ w \in \{I, C\} : w \text{ nu este codificarea } \rho(M) \text{ a niciunei maşini } M, \text{ sau } w = \rho(M) \text{ pentru o anumită maşină } M \text{ care nu acceptă intrarea } \rho(M) \}$

Dar $\overline{K_1}$ nu poate fi nici măcar acceptat Turing. Să presupunem că M^* este o mașină Turing care acceptă $\overline{K_1}$. Este $\rho(M^*)$ în $\overline{K_1}$?

Din definiţia lui $\overline{K_1}$, $\rho(M^*) \in \overline{K_1}$, dacă şi numai dacă M^* nu acceptă şirul de intrare $\rho(M^*)$. Dar se presupune că M^* acceptă $\overline{K_1}$, deci $\rho(M^*) \in \overline{K_1}$ dacă şi numai dacă M^* acceptă $\rho(M^*)$. Ca urmare M^* acceptă $\rho(M^*)$ dacă şi numai dacă M^* nu acceptă $\rho(M^*)$. Ceea ce este absurd, deci presupunerea că există M^* este falsă. ■

Pe scurt, am vrut să vedem dacă orice limbaj acceptat Turing este decidabil Turing. Am văzut că acest lucru ar putea fi adevărat, dacă şi numai dacă limbajul particular K_0 acceptat Turing ar fi decidabil Turing.

Din K_0 am derivat în doi paşi limbajul $\overline{K_1}$, care trebuie să fie decidabil Turing pentru ca limbajul K_0 să fie decidabil Turing. Dar presupunerea că limbajul $\overline{K_1}$ este decidabil Turing duce la o contradicţie.

Am demonstrat teoremele.

Teorema 2.3

Nu orice limbaj acceptat Turing este decidabil Turing.

Teoremă 2.4

Complementele unor limbaje acceptate Turing nu sunt acceptate Turing.

Limbajele K_1 şi K_0 sunt acceptate Turing, dar am arătat că limbajul $\overline{K_1}$ nu este acceptat Turing.

Să revenim asupra aspectului calitativ. Putem vedea K_0 ca un dicţionar infinit care poate răspunde la întrebări de forma: acceptă maşina Turing M şirul w? Cum nu există nicio maşina Turing care să decidă K_0, putem concluziona că nu există niciun algoritm care să decidă pentru o mașină Turing M arbitrară şi un şir w, dacă M acceptă w.

Deci există probleme care nu admit soluţii algoritmice. Astfel de probleme sunt etichetate ca insolvabile. O problemă care poate fi rezolvată de un algoritm este solvabilă. Un algoritm care rezolvă o problemă este numit procedură de decizie pentru acea problemă.

Cea mai faimoasă problemă insolvabilă este cea descrisă de K_0, numită "problema opririi pentru maşina Turing"; ea constă în a determina pentru o maşină Turing arbitrară M şi o intrare w, dacă M se opreşte pentru w.

Acest rezultat de insolvabilitate nu implică însă inexistenţa unor circumstanţe în care este posibilă prezicerea opririi maşinii Turing pentru un şir de intrare.

Putem, de exemplu, examina codificarea maşinii Turing pentru a vedea dacă starea h este reprezentată; dacă nu, maşina nu poate accepta niciun şir de intrare. Aceasta şi analize mai complexe pot oferi anumite informaţii; dar rezultatul secţiei afirmă că astfel de analize sunt fie neconcludente fie dau rezultate proaste. Nu există nicio metodă generală care să decidă corect în toate cazurile.

Mai mult, există anumite maşini particulare pentru care nu există nicio metodă generală de a spune, pentru un şir de intrare, dacă maşina va accepta sau se va opri pentru acel şir de intrare. Aceasta rezultă din faptul că limbajul K_0 este acceptat Turing dar nu este decidabil Turing.

Dacă M_0 este o maşină Turing care acceptă K_0, atunci nu există nicio maşină Turing şi deci niciun algoritm care să determine dacă un şir de intrare oarecare w este acceptat de M_0 sau nu.

Pentru a rezuma, această secţiune stabileşte insolvabilitatea pentru două probleme particulare.

Teorema 2.5

a) Nu există niciun algoritm care să determine, dată fiind o maşină Turing M şi un şir w, dacă M acceptă w.
b) Pentru o anumită maşină Turing fixată M_0, nu există niciun algoritm care să determine pentru un şir w, dacă M_0 acceptă w.

Să subliniem distincţia dintre (a) şi (b). Cum K_0 este $\{\rho(M)\rho(w) : M \text{ acceptă } w\}$, K_0 poate fi spart în infinit de multe submulţimi $K_M = \{\rho(M)\rho(w) : M \text{ acceptă } w\}$, câte una pentru fiecare maşină fixată M. (Fiecare şir din K_M începe la fel, cu $\rho(M)$. Nu rezultă imediat din faptul că limbajul K_0 nu este decidabil Turing că anumite submulţimi K_M nu sunt decidabile Turing. În general, este posibil ca un limbaj care nu este decidabil Turing să fie reuniunea unor limbaje decidabile Turing. De exemplu, orice limbaj ce constă dintr-un singur şir este decidabil Turing; el este decis de maşina Turing care verifică dacă intrarea sa este acel şir particular şi se opreşte cu (Y) sau (N) pe bandă. Astfel, orice limbaj este reuniunea unei

mulţimi numărabile de limbaje decidabile Turing. Totuşi, nu există o metodă generală de a combina infinit de multe proceduri de decizie într-una singură.

În cazul nostru particular, ştim că limbajul K_{M_0} nu este decidabil Turing, unde M_0 este maşina Turing care acceptă K_0. Teorema 2.5 rezultă din asta.

2.2. Enumerabilitate, acceptare, decidabilitate Turing

Aşa cum am văzut, orice limbaj decidabil Turing este acceptat Turing dar cele două clase nu coincid. Ce limbaje acceptate Turing sunt decidabile Turing? Există mai multe modalităţi de a răspunde la această întrebare.

Teorema 2.6

Un limbaj este decidabil Turing, dacă şi numai dacă atât el cât şi complementul său sunt acceptate Turing.

Demonstraţie. Dacă L este un limbaj decidabil Turing, atunci L este acceptat Turing (conform Teoremei 2.1). De asemenea complementul său, $\overline{L}$, este decidabil Turing (conform Teoremei 2.2) şi deci acceptat Turing.
Să presupunem că L este acceptat de M_1 şi $\overline{L}$ este acceptat de M_2. Atunci putem construi o maşină Turing M care decide L; M va fi o maşină cu două benzi (care conform Teoremei 1.2 este echivalentă cu o maşină Turing standard). Maşina M începe prin a pune şirul de intrare w pe ambele benzi şi poziţionează capetele la limita dreapta a fiecărui şir. M simulează M_1 şi M_2 în paralel; la fiecare pas al unei operaţii a lui M, un pas al operaţiilor lui M_1 este pus pe prima bandă, şi un pas al operaţiilor lui M_2 este pus pe a 2-a bandă. Cum fie M_1 fie M_2 trebuie să accepte w, dar nu amândouă, M va ajunge în situaţia în care M_1 sau M_2 se va opri. Când se întâmplă, M determina care dintre M_1 sau M_2 accceptă w şi se opreşte cu benzile configurate pentru a indica dacă $w \in L$ sau $w \notin L$. ■

Am definit un limbaj acceptat Turing ca fiind mulţimea şirurilor pentru care o anumită maşină Turing se opreşte. Dar putem caracteriza un limbaj acceptat Turing ca mulţimea şirurilor rămase pe bandă la sfârşitul operaţiilor.

Definiţia 2.1

Fie Σ_0 un alfabet care nu conţine $\#$. Un limbaj $L \subseteq \Sigma_0^*$ este limbajul de ieşire al unei maşini Turing M dacă şi numai dacă L este mulţimea tuturor şirurilor $w \in \Sigma_0^*$ astfel încât, pentru un anumit şir $u \in \Sigma^*$, $(s, \#u\underline{\#}) \vdash_M^* (h, \#w\underline{\#})$, unde s este starea iniţială a maşinii M.

Cu alte cuvinte, w este în limbajul de ieşire a lui M, dacă există un anumit şir de intrare pentru care M se opreşte, lăsând w pe bandă în modul specificat pentru maşina Turing care calculează funcţii.

Teorema 2.7

Un limbaj este acceptat Turing dacă şi numai dacă este limbajul de ieşire al unei maşini Turing.

Demonstraţie. Mai întâi vom demonstra că dacă L este limbajul de ieşire al unei maşini Turing M, atunci L este acceptat Turing. Din teorema de echivalență a maşinii Turing deterministe şi a celei nedeterministe este suficient să demonstrăm că L este acceptat de o maşină Turing nedeterministă. O maşină Turing nedeterministă M' poate fi proiectată pentru a accepta L astfel: pentru un şir de intrare w, M' scrie nedeterminist şirul u pe bandă, salvând w pe a doua pistă a benzii sale. Apoi M' simulează maşina M. La sfârşitul operaţiilor, M' compară şirul pe care M l-ar fi lăsat pe bandă cu w. Dacă sunt identice atunci M' acceptă, altfel M' intră într-un ciclu infinit. Astfel M' acceptă w doar în cazul în care există un anumit şir u din care şirul w ar fi putut fi calculat.
Implicația inversă: dacă L este mulţimea şirurilor de intrare acceptate de o maşină Turing M, atunci putem modifica M astfel încât la acceptarea unui şir de intrare se opreşte cu acel şir pe bandă (ceea ce implică păstrarea intrării în cursul operaţiilor). Mulţimea şirurilor

rămase pe bandă la sfârşitul operaţiilor este aceeaşi cu mulţimea şirurilor acceptate de M adică L. ■

Un alt mod în care mașina Turing poate fi utilizată pentru a specifica un limbaj este prin generarea membrilor săi.

Definiţia 2.2

O mașină Turing enumeră limbajul L dacă şi numai dacă pentru o anumită stare fixată q a lui M,

$$L = \{ w\text{: pentru un şir } u,\ (s, \#) \vdash_{M}^{*} (q, \#w\underline{\#}u) \}$$

unde s este starea iniţială a lui M.
Un limbaj este enumerabil Turing dacă şi numai dacă este enumerat de o anumită mașină Turing.

M enumeră L pornind cu o bandă vidă şi calculând, trecând periodic printr-o stare specială q (nu este starea de oprire). Intrarea în stare q semnalează că şirul rămas la stânga capului este un membru al lui L; M poate ieşi din q şi reintra mai târziu cu un alt membru din L la stânga capului. Șirul u aflat la dreapta capului poate fi utilizat pentru a păstra informaţie despre reluarea operaţiilor după işirea din starea q.

Teorema 2.8

Un limbaj este acceptat Turing dacă şi numai dacă este enumerabil Turing.

Demonstraţie. Să presupunem că L este acceptat de o anumită mașină Turing M . Atunci putem construi o mașină Turing M' care, în loc să primească un şir ca intrare, porneşte cu o bandă vidă, generează sistematic (de exemplu, în ordine lexicografică) toate şirurile peste alfabetul lui L, şi efectuează asupra fiecăruia operaţiile pe care le-ar fi făcut M. Din păcate, modul evident prin care acest scop poate fi atins nu funcţionează: M' s-ar putea să nu termine operaţiile pe un şir înainte de a începe să-l prelucreze pe următorul, pentru că se poate agăţa pentru un anumit şir pentru care operaţiile lui M nu se termină, deşi există alte şiruri pe care M le acceptă, dar care nu au fost încă generate.

În loc să încerce să termine operaţiile pentru un şir atunci când este generat, M' efectuează următoarea secvenţă de operaţii:

(1) efectuează un pas din operaţiile lui M asupra primului şir peste alfabetul lui M
(2) efectuează doi paşi din operaţiile lui M pe fiecare dintre cele două şiruri generate (primele două)
(3) efectuează trei paşi din operaţiile lui M pe fiecare dintre primele trei şiruri

.
.
.

De fiecare dată când M' descoperă că M ar fi acceptat un şir, M' se opreşte în starea q pentru a semnala acceptarea. Un anumit mecanism trebuie utilizat pentru ca M' să ştie care dintre fazele importante ale operaţiilor sale se desfăşoară şi cât de tare a avansat.

Implicația inversă: dacă M enumeră L, atunci putem modifica M să accepte L. Reconstruim M astfel încât să salveze orice intrare primită înainte să înceapă procesul de enumerare. De fiecare dată când M ar intra în starea q, maşina modificată compară conţinutul curent al benzii cu şirul de intrare salvat. Dacă se potrivesc, şirul de intrare este acceptat, altfel procesul de enumerare continuă. Noua maşină acceptă exact limbajul enumerat de M. ■

Modul în care o mașină Turing enumeră un limbaj este reminiscent modului în care o gramatică generează un limbaj. Vom termina caracterizarea limbajelor acceptate Turing, arătând că această similaritate nu este un accident.

Teoremă 2.9

Un limbaj este acceptat Turing dacă şi numai dacă este generat de o gramatică.

Demonstraţie. Să presupunem că L este generat de o gramatică G. Atunci putem proiecta o maşină Turing M care acceptă L. Fie un şir de intrare w, M salvează w şi apoi sistematic produce toate derivările posibile din gramatică. M poate produce toate şirurile peste acest alfabet în ordine lexicografică şi poate determina care dintre ele reprezintă derivări. Ori de câte ori M produce un şir care nu

reprezintă o derivare compară ultimul şir din derivare cu intrarea w şi se opreşte dacă sunt egale; altfel continuă să genereze lexicografic alte derivări.

Implicaţia inversă: să presupunem că $L \subseteq \Sigma_0^*$ este acceptat Turing. Atunci conform Teoremei 2.7 este limbajul de ieşire al unei maşini Turing $M = (K, \Sigma, \delta, s)$.

$$L = \{w: \text{pentru } u \in \Sigma_0^*, (s, \#u\underline{\#}) \vdash_M^* (h, \#w\underline{\#})\}$$

Fie gramatica G_0 construită astfel încât derivările din gramatică să simuleze tranziţiile maşinii Turing (Lema 2.1). G_0 are proprietatea:

$$(s, \#u\underline{\#}) \vdash_M^* (h, \#w\underline{\#})$$

dacă şi numai dacă

$$[\#us\#] \overset{*}{\Rightarrow}_{G0} [\#wh\#]$$

Pentru a obţine gramatica G, trebuie adăugate următoarele reguli la gamatica G_0.

1. Reguli care permit începerea derivărilor şi generarea şirului de intrare:

   $S \rightarrow [\#S_1\#]$

   $S_1 \rightarrow aS_1; \forall a \in \Sigma_0$

   $S_1 \rightarrow s$

   Neterminalele S şi S_1 sunt nou introduse; S este simbolul de start al lui G. Utilizând doar aceste reguli, poate fi generat orice şir de forma $[\#us\#]$, $u \in \Sigma_0^*$.
2. Reguli pentru a termina derivările şi a şterge simbolurile de sfârşit:

   $h\#] \rightarrow X$

   $aX \rightarrow Xa, \forall a \in \Sigma_0$

   $[\#X \rightarrow e$

unde X este un nou neterminal. Pornind cu un şir de forma $[\#wh\#]$, prima regulă generează $[\#wX$, apoi prin $|w|$ aplicări ale regulilor de tipul al doilea rezultă $[\#Xw$; şi în final, aplicând ultima regulă, w. ■

***Obținerea lui* G_0**

Lema 2.1

Fie $M = (K, \Sigma, \delta, s)$. Atunci există o gramatică G astfel încât pentru orice configuraţii (q, u, a, v) şi (q', u', a', v') ale lui M,

$$(q, u, a, v) \vdash_M^* (q', u', a', v')$$

dacă şi numai dacă:

$$[uqav] \Rightarrow_G^* [u'q'a'v']$$

Unde $[$ şi $]$ sunt simboluri noi care nu există în Σ sau K; presupunem de asemenea că mulțimile K şi Σ sunt disjuncte.

Demonstraţie. Ideea este de a reprezenta configuraţia (q, u, a, v) prin şirul $[uqav]$; poziţia stării în şir serveşte şi ca indicaţie a poziţiei capului. Acestă reprezentare permite ca partea stânga a fiecărei reguli să conţină ca subşir o stare şi simbolul scanat; numai starea, simbolul scanat şi poziţia capului trebuie schimbate la trecerea de la o configuraţie la următoarea.

Simbolurile $[$ şi $]$ apar doar la capetele reprezentării unei configuraţii. Simbolul $]$ este utilizat pentru a trata banda vidă, iar simbolul $[$ este introdus pentru legilibitate.

Formal, $G = (V, \Sigma, R, S)$ (simbolul de start nu are niciun rol în condiţia pe care G trebuie să o îndeplinească).

$$V = K \cup \Sigma \cup \{[,], h, S\}$$

iar R conţine regulile:

(a) $\forall q \in K$, $a \in \Sigma$, dacă $\delta(q, a) = (p, b)$, $p \in K \cup \{h\}$, $b \in \Sigma$ atunci R conţine regula

$$qa \to ph$$

(b) $\forall q \in K$, $a \in \Sigma$, dacă $\delta(q, a) = (p, R)$, $p \in K \cup \{h\}$, R conţine regula

$$qab \to aph, \text{ pentru } \forall\, b \in \Sigma, \text{ şi}$$

$$qa] \to ap\#$$

(c) $\forall q \in K$, $a \in \Sigma$, dacă $\delta(q, a) = (p, L)$, $p \in K \cup \{h\}$, R conţine regula

$$bqac \rightarrow pbac$$

$\forall\ b \in \Sigma$ şi $c \in \Sigma \cup \{]\}$, astfel încât $a \neq \#$ sau $c \neq]$, şi

$$bq\#] \rightarrow pb],$$

$\forall\ b \in \Sigma$, dacă $a = \#$

3. COMPLEXITATE

În capitolul precedent am arătat că există probleme care nu pot fi rezolvate prin algoritmi. Prin urmare putem clasifica problemele în două categorii: probleme care admit, respectiv care nu admit soluții algoritmice. Având în vedere dezvoltarea tehnologică din ultimii ani, ne-am aştepta că toate problemele din prima categorie să poată fi rezolvate cu uşurinţă de calculatoarele actuale. Din păcate, practica arată că deşi teoretic rezolvabile, anumite probleme nu pot fi rezolvate practic datorită timpului foarte mare necesitat.

De exemplu, să presupunem că trebuie să planificăm vizitele pe care un comis voiajor trebuie să le facă în 10 oraşe. Se dă harta cu cele 10 oraşe şi distanţele lor şi trebuie să se obţină itinerariul care minimizează distanța parcursă. Din punct de vedere teoretic problema este rezolvabilă. Dacă sunt n oraşe, numărul itinerariilor posibile este finit, respectiv $(n-1)!$ Deci trebuie proiectat un algoritm care determină itinerariul minim.

Pentru problema celor 10 oraşe, trebuie examinate $9! = 362980$ itinerarii, un număr încă calculabil. Dar dacă trebuie vizitate 30 de oraşe? Numărul intinerariilor este foarte mare: $29! >> 10^{30}$. Chiar dacă s-ar putea examina un miliard de tururi pe secundă (ceea ce este mult peste capabilităţile actuale ale calculatoarelor), timpul necesar pentru a termina este mai mare decât un miliard de vieţi omeneşti.

Deci, faptul că o problemă este teoretic rezolvabilă nu implică că ea poate fi rezolvată practic. Desigur, clasificarea în probleme rezolvabile şi nerezolvabile trebuie rafinată. Problemele rezolvabile trebuie rafinate în concordanţă cu complexitatea lor.

Avem deja o astfel de clasificare. Ştim că un limbaj decidabil Turing poate fi regulat, independent de context dar nu poate fi neregulat sau neindependent de context. În fapt, ştim că există limbaje independente de context care nu sunt independente de context deterministe. Deci avem o ierarhie de limbaje decidabile Turing: decidabile Turing – independente de contex – independente de contex deterministe – regulate. Această ierarhie reflectă o distincţie calitativă între automatele corespunzătoare.

De exemplu, un automat cu stivă este în esenţă o maşină Turing cu două benzi cu restricţii asupra mişcării capetelor şi a scrierii; un automat finit este un automat cu stivă care nu utilizează stiva.

Scopul nostru în acest capitol, este de a stabili o altă clasificare cantitativă a limbajelor decidabile, considerând maşini Turing care nu sunt restricţionate structural, ci în cantitatea de efort pe care o pot consuma în timpul operaţiilor efectuate asupra unui şir de intrare.

O măsură naturală a cantităţii de efort depuse de o maşină Turing este numărul de paşi utilizaţi pentru a ajunge la o concluzie referitoare la intrare.

Definiţia 3.1

Fie M este o maşină Turing şi $t \in \mathbb{N}$, definim relaţia $\vdash_M^t$ între configuraţiile lui M astfel:

$C \vdash_M^0 C'$ dacă şi numai dacă $C = C'$

$C \vdash_M^{t+1} C'$ dacă şi numai dacă există C'' astfel încât $C \vdash_M^t C''$ şi $C'' \vdash_M^t C'$

Adică,

$C \vdash_M^t C'$ dacă şi numai dacă $C \vdash_M^* C'$ printr-un calcul în exact t paşi.

Definiţia 3.2

Fie T o funcţie, $T: \mathbb{N} \to \mathbb{N}$. Fie $L \subseteq \Sigma_0^*$ un limbaj şi $M = (K, \Sigma, \delta, s)$ o maşină Turing cu k benzi şi $\Sigma_0 \subseteq \Sigma$. Spunem că M decide L în timpul T, dacă:

pentru $w \in L$,

$(s, \#w\underline{\#}, \underline{\#}, \ldots, \underline{\#}) \vdash_M^t (h, \#\textcircled{Y}\underline{\#}, \underline{\#}, \ldots, \underline{\#})$, $t \leq T(|w|)$

şi pentru $w \notin L$,

$(s, \#w\underline{\#}, \underline{\#}, \ldots, \underline{\#}) \vdash_M^t (h, \#\textcircled{N}\underline{\#}, \underline{\#}, \ldots, \underline{\#})$, $t \leq T(|w|)$.

Spunem că L este decidabil în timpul T, dacă există $k > 0$ şi o maşină Turing cu k benzi care decide L în timpul T. Clasa limbajelor decidabile în timpul T este notată TIME(T).

Am limitat numărul de paşi ai unei maşini Turing nu printr-o constantă absolută, ci în termenii unei funcţii dependente de lungimea intrării.

Având în vedere că o maşină Turing trebuie să şteargă intrarea şi să scrie Ⓨ sau Ⓝ înainte de a se opri pentru intrarea w trebuie să efectueze $|w| + 1$ scrieri şi $|w| + 3$ mutări ale capului, deci nicio funcţie T astfel încât $T(n) < 2n + 4$, $n \geq 4$ nu este o limită de timp utilă.

Exemplu 3.1

Fie $L = \{w \in \{a, b\}^* : w$ nu conţine nicio apariţie a şirului aa$\}$

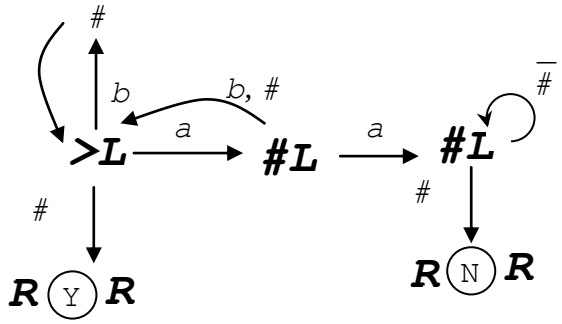


Figura 3-1

Maşina Turing parcurge şirul de la dreapta la stânga, verificând existenţa şirului aa, ştergând în acelaşi timp intrarea. În total, execută $|w| + 1$ scrieri şi $|w| + 3$ deplasări. M decide L în T, $T(n) = 2n + 4$; L $\in$ TIME (T).

Exemplu 3.2

Fie maşina Turing care decide limbajul $L = \{w \in \{a, b, c\}^* : w$ are acelaşi număr de simboluri a, b, c$\}$.

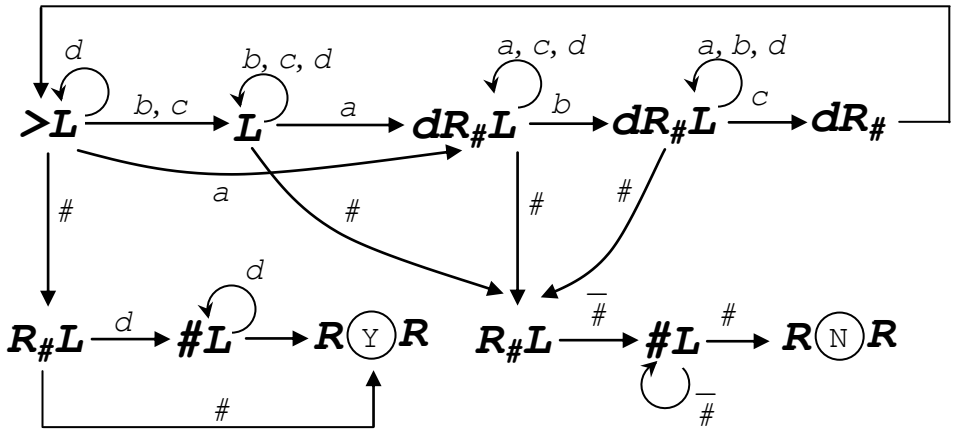


Figura 3-2

M funcţionează în etape. La fiecare etapă începe prin a scana intrarea de la dreapta la stângă, până când întâlneşte un simbol a, care este înlocuit cu simbolul d; apoi M se întoarce înapoi la capătul dreapta, scanează din nou banda, până când întâlneşte un simbol b pe care îl înlocuieşte d. Operaţiile sunt repetate pentru simbolul c, terminând astfel o etapă. Dacă M într-o anumită etapă caută un simbol a dar găseşte numai simboluri d, şterge banda şi se opreşte cu #Y#; în cazul unei căutări nereuşite M şterge banda şi se opreşte cu #N#.

Să numărăm paşii utilizaţi de M pentru o intrare de lungime n. La fiecare etapă, M scanează banda de trei ori; la fiecare scanare, M examinează cel mult n simboluri, înlocuieşte cel mult unul, şi repoziţionează capul la limita dreapta a benzii după cel mult n paşi – în total $(2n + 1)$ paşi. Prin urmare, fiecare etapă constă din cel mult $3(2n + 1)$ paşi. Câte etape sunt? Există cel mult $[n/3] + 1$, pentru că există cel mult $[n/3] + 1$ simboluri a, b sau c. La sfârşit M şterge banda şi scrie Y sau N în $2n + 4$ paşi. Aceasta face ca numărul total de paşi să fie un număr mai mic decât $([n/3] + 1) \infty 3(2n + 1) + 2n + 4$. Pentru a simplifica expresia, să observăm, că pentru orice n, $[n/3] \leq (n + 2) / 3$, deci $T(n) = 2n^2 + 13n + 9$. Cum M decide L în T, $L \in \text{TIME}(2n^2 + 13n + 9)$.

Aproximarea este însă pesimistă pentru anumite intrări. De exemplu pentru b^n, M execută doar $4n + 5$ paşi înainte de a se opri cu #N# pentru că

detectează imediat că nu există a. Deci, limita $T(n)$ reflectă comportarea lui M în cazul cel mai defavorabil pentru un şir de lungime n.

Deşi calculul complexităţii $(O(n))$ pentru o maşină Turing poate fi netrivial, proiectarea unei maşini Turing care să decidă un limbaj L într-un timp dat T, $L \in TIME(T)$, este mai interesant. Scopul calculului de complexitate este de a alege dintre mai multe maşini Turing, pentru a decide un limbaj, pe cea ale cărei operaţii se termină în T paşi, unde T este dat sau dacă nu este posibil, să demonstrăm riguros această imposibilitate.

Exemplu 3.3

Fie L limbajul din exemplul anterior. Există o maşină Turing cu k-benzi, M′ $(k \geq 1)$ astfel încât M′ să decidă L în cazul cel mai defavorabil în mai puţini paşi decât M? Adică $L \in TIME(T')$, unde $T' < T$?

Construim o maşină Turing M′ cu patru benzi. M′ scanează intrarea de la dreapta la stânga, ştergând simbolurile de pe bandă, şi în acelaşi timp scrie un simbol I pe a doua bandă pentru fiecare a pe care îl întâlneşte, un I pe a treia bandă pentru fiecare b întâlnit, şi un I pe a patra bandă pentru fiecare c întâlnit. Apoi M′ compară numărul de simboluri I pe fiecare dintre benzile a doua, a treia şi a patra, mutând capetele benzilor, la stânga, simultan. Dacă pe fiecare bandă simbolul curent este I, continuă să-şi deplaseze capetele, ştergând simbolurile I de pe toate cele trei benzi. Dacă întâlneşte un simbol # (cel mai din stânga) pe toate cele trei benzi, atunci se opreşte cu #Ⓨ# pe prima bandă, altfel se opreşte cu #Ⓝ# după ce şterge simbolurile I rămase.

Vom utiliza următoarele notaţii:

- $[A^{(i)}, B^{(j)}] \Rightarrow$ acţiuni pe benzile i şi j executate simultan
- $A^{(i,j,k)} = [A^{(i)}, A^{(j)}, A^{(k)}]$; de exemplu $[\#^{(1)}, I^{(4)}]$, scrie # pe banda 1 şi I pe banda 4 simultan.
- $L^{(2,3,4)} \Rightarrow$ mută capetele pe benzile 2, 3, 4 la stânga în acelaşi timp.

Intuitiv, M' este mai eficient decât M; să estimăm cazul cel mai defavorabil pentru un şir de intrare de lungime n. Vom analiza trei cazuri corespunzătoare celor trei submaşini (1), (2), (3) .

1. Citirea şi ştergerea intrării se face în $3n + 1$ paşi.
2. Fie p numărul de apariţii în şirul de intrare a unui simbol din $\{a, b, c\}$ care apare de cele mai puţine ori. Atunci compararea

numărului de simboluri I pe cele trei benzi se face în $2p + 1$ paşi.

3. Ştergerea simbolurilor I rămase şi scrierea Ⓨ, Ⓝ se face în cel mult $2(n - 3p) + 2$ paşi.

Prin urmare M' execută $3n + 1 + 2p + 1 + 2(n - 3p) + 2 \leq 5n + 4$ pentru orice şir de lungime n. Deci $L \in \mathrm{TIME}(5n + 4)$.

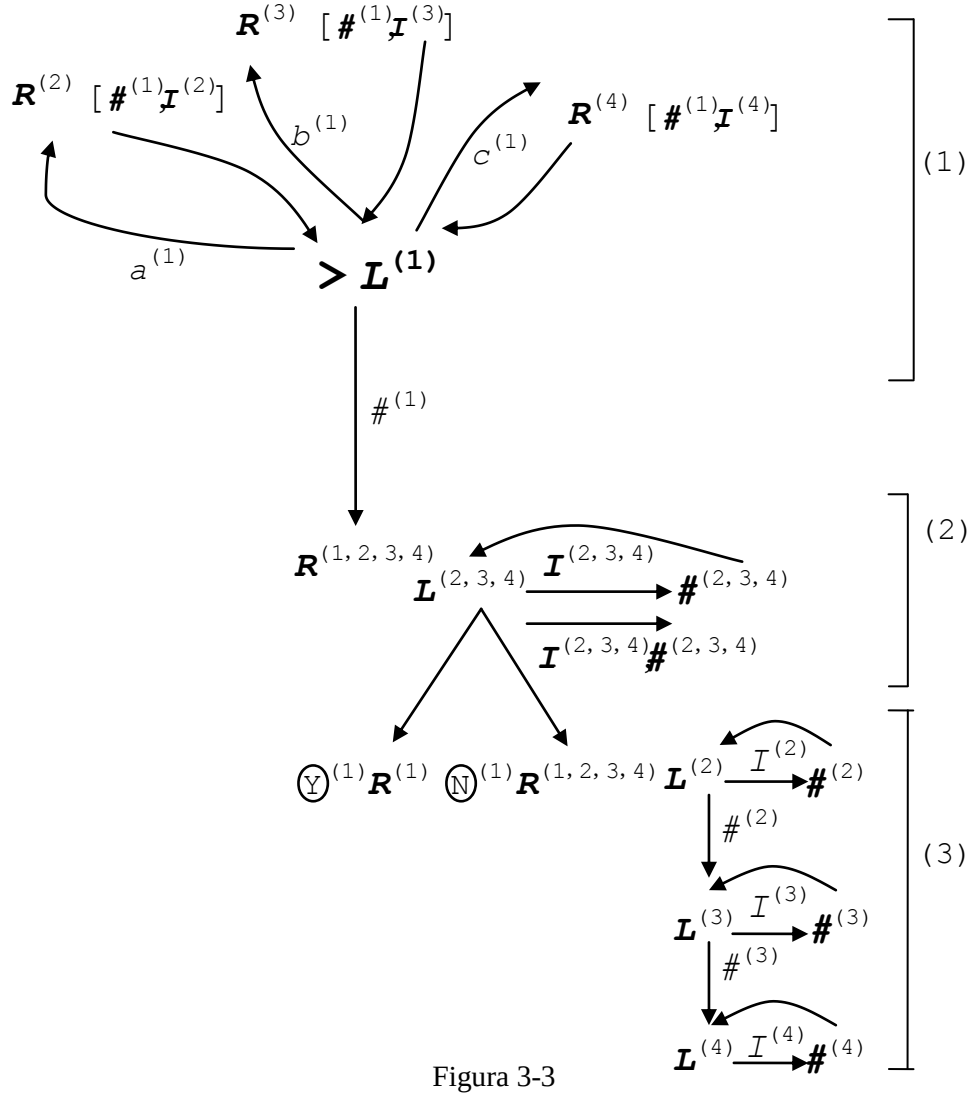


Figura 3-3

Poate fi L decis mai repede? Respectiv $L \in \mathrm{TIME}(cn + d)$, pentru $c < 5$?

Constanta c nu poate fi mai mică decât 2, pentru că sunt necesari cel puţin $2n + 4$ paşi pentru orice mașină Turing, pentru a respecta convenţiile de intrare / ieşire impuse.

Următoarea teoremă demonstrează că c poate fi oricât de apropiat de 2. Mai important, teorema afirmă că întrebări referitoare la coeficienţi constanţi, cum ar fi c, sunt lipsite de importanţă, pentru că aceste constante pot fi întotdeauna micșorate.

Teorema 3.1

Presupunem $L \in \mathrm{TIME}(T)$, şi fie ϵ orice număr real, pozitiv (oricât de mic). Atunci $L \in \mathrm{TIME}(T')$ unde pentru $\forall\, n \in \mathbb{N}$, $T'(n) = 2n + 18 + \lceil \epsilon \infty T(n) \rceil$

Demonstraţie. Fie $M = (K, \Sigma, \delta, s)$ o mașină Turing cu k -benzi care decide L în timpul T. Vom construi o mașină Turing cu M' benzi, $M' = (K', \Sigma', \delta', s')$ care acceptă L în T' unde $T'(n) = 2n + 18 + \lceil \epsilon \times T(n) \rceil$. Dacă $k = 1$ atunci $k' = 2$, altfel $k' = k$.
Operaţiile lui M' au 4 etape. În prima etapă, M' copiază intrarea de pe prima bandă pe a doua, în acest timp comprimând şi inversând intrarea. În a doua etapă, M' copiază versiunea comprimată a intrării înapoi pe prima bandă, reinversând şirul dar păstrând compresia. În a treia etapă, M' simulează M; compresia intrării face posibilă simularea mai multor paşi ai lui M cu mai puţini paşi a lui M', şi acesta este locul unde îmbunătăţirea vitezei se obţine. În a patra etapă M' efectuează paşii necesari pentru a lăsa benzile la fel cu ale lui M. (cu acelaşi conţinut ca în cazul în care M ar fi făcut operaţiile).
M' poate fi descris ca o combinaţie a maşinilor $M_1M_2M_3M_4$; vom descrie aceste maşini pe rând. Pentru a simplifica descrierea, vom presupune că numărul benzilor lui M, k, este 1.
Maşina M_1 porneşte cu şirul de intrare w pe prima bandă. Deplasează capul pe prima bandă la stânga, ştergând w, în acelaşi timp producând o versiune comprimată a intrării $\mathcal{K}(\#w)$ şi scriind intrarea inversată pe a doua bandă. Pentru un anumit întreg pozitiv m (care depinde de ϵ, va fi fixat ulterior în demonstraţie), $\mathcal{K}(\#w)$ este mai scurt decât $\#w$ cu un factor proporţional cu m, respectiv

$|\mathcal{K}(\#w)| = \lceil (1 + |w| + 1) / m \rceil$. Această compresie este obţinută codificând fiecare m–tuplu de simboluri ale lui $\#w$ ca un singur simbol din $\mathcal{K}(\#w)$ (în cazul în care $|w| + 1$ nu este un multiplu exact al lui m, trebuie introduse cel mult $m-1$ simboluri $\#$ în primul simbol din $\mathcal{K}(\#w)$. De exemplu, dacă $w = abbabaa$, $m = 3$, $K(\#w) = (\#, \#, a)(b, b, a)\ (b, a, a)$, unde tripletele sunt simboluri în alfabetul lui M'.

La sfârşitul primei etape, a doua bandă va conţine inversul compresiei $\mathcal{K}(\#w)$, respectiv $(b, a, a)\ (b, b, a)\ (\#, \#, a)$

Formal, etapa de compresie poate fi descrisă astfel. Fie $w \in \Sigma^*$. Dacă $|w| \leq m$, $w = \sigma_1 \ldots \sigma_j$, $j \leq m$, $\sigma_i \in \Sigma$, atunci $\mathcal{K}(w) = (\#, \#, ..., \#, \sigma_1, ..., \sigma_j)$ ($m-j$ apariţii ale simbolului $\#$). Altfel dacă $|w| > m + 1$, $w = w'\sigma_1 \ldots \sigma_m$, $w \in \Sigma^+$ şi $\sigma_1, ..., \sigma_j \in \Sigma$, atunci $\mathcal{K}(w) = \mathcal{K}(w')\tau$, unde $\tau = (\sigma_1, ..., \sigma_m)$. Scopul lui M_1 este să transforme $(\#w\underline{\#}, \underline{\#})$ în $(\underline{\#}, \#\ \mathcal{K}(\#w)^R\underline{\#})$. Este posibilă construcţia lui M_1 astfel încât să facă acest lucru în $2(w) + 1$ paşi.

Maşina M_2 copiază $\mathcal{K}(\#w)$ pe prima bandă în timp ce şterge $\mathcal{K}(\#w)^R$ de pe a doua bandă, adică transformă $(\underline{\#}, \#\mathcal{K}(\#w)^R\underline{\#})$ în $(\#\mathcal{K}(\#w)\underline{\#}, \underline{\#})$. M_2 utilizează doar $2|\mathcal{K}(\#w)^R| + 1$ paşi, respectiv $2\lceil (w)+1/m) \rceil + 1$ paşi. M_2 lasă banda în forma standard.

Maşina M_3 simulează în mod repetat m paşi ai lui M prin 12 paşi proprii. Prima bandă a lui M_3 va conţine – după 12 paşi – $K(x)$, unde x este conţinutul primei benzi a lui M după m paşi. Starea lui M_3 după 12 paşi este de forma (q, j) unde $q \in K$ este starea în care M ar fi ajuns după m paşi, şi j, $1 < j < m$, indică simbolul din m-tuplul scanat de M_3 care corespunde simbolului scanat de M. De exemplu, dacă $m = 3$ şi $7 \infty m = 21$ paşi, M ar fi fost în configuraţia $(q, aab\underline{b}aba)$, atunci M_3 după ce a pornit compresia va fi în configuraţia $((q, b), (\#, \#, a)(a, b, b)(a, b, a))$ după $7 \infty 12 = 84$ paşi.

Simularea următorilor m paşi ai lui M se desfăşoară astfel: mai întâi capul lui M_3 se mută cu o poziţie la stânga, două poziţii la dreapta, o poziţie la stânga, oprindu-se în poziţia din care a pornit. rezultatul este că M_3 a codificat în starea sa cele trei m–tupluri aflate în poziţia curentă, respectiv stânga, dreapta capului său; numim concatenarea acestor m–tupluri şir activ. Acum M_3 are toată informaţia necesară pentru a executa următorii m paşi ai lui M (M nu poate în m paşi să

citească / scrie poziţii în afara şirului activ). Astfel, cunoscând şirul activ şi stările lui M, M_3 poate spune exact care vor fi următorii m paşi ai lui M. Această predicţie poate fi făcută într-un pas al lui M_3, cu ajutorul unei tabele finite, care poate fi construită în avans în funcţia δ_3.Acum M_3 efectuează încă 8 paşi – 3 paşi de scriere, 5 de deplasare – pentru a înregistra noile m–tupluri care trebuie să înlocuiască şirul activ, şi în final să repoziţioneze capul pentru următorul bloc de 12 paşi.

Formal, stările lui M_3 sunt:

$K_3 = K \times \{1, 2 \ldots, m\} \cup K \times \{1, 2 \ldots, m\} \times \Sigma^{3m} \times \{1, 2 \ldots, 11\}$

Starea (q, j, x, p), $1 \le p \le m$, $x \in \Sigma^{3m}$, este starea în care ajunge starea (q, j) după p din cei 12 paşi implicaţi în simularea celor m paşi ai lui M; x este subșirul cunoscut din șirul activ, iar simbolurile # reprezintă partea necunoscută din șir.

Cum M_3 nu consultă / actualizează a doua bandă, vom scrie $\delta_3(q, \sigma) = (q', \sigma')$ în loc de $\delta_3(q, \sigma, \#) = (q', \sigma', \#)$. M_3 are următoarele tranziţii pentru $q \in K$, $1 \le j \le m$, $x, y, z \in \Sigma^m$:

$$\delta_3((q, j), x) = ((q, j, \#^{3m}, 1), L)$$

$$\delta_3((q, j, \#^{3m}, 1), x) = ((q, j, x\#^{2m}, 2), R)$$

$$\delta_3((q, j, x\#^{2m}, 2), y) = ((q, j, xy\#^{m}, 3), R)$$

$$\delta_3((q, j, xy\#^{m}, 3), z) = ((q, j, xyz, 4), L)$$

Fie $xyz = u\sigma v$, $|u| = m + j - 1$, presupunem că $(q, u\underline{\sigma}v) \vdash_M^m (q', u'\underline{\sigma'}v')$ (sau $(q, u\underline{\sigma}v) \vdash_M^t (q', u'\underline{\sigma'}v')$, unde $q' = h$ şi $t < m$) şi $u'\underline{\sigma'}v' = x'y'z'$, unde $x', y', z' \in \Sigma^m$. Atunci

$$\delta_3((q, j, xyz, 4), y) = ((q, j, xyz, 5), L)$$

$$\delta_3((q, j, xyz, 5), x) = ((q, j, xyz, 6), x')$$

$$\delta_3((q, j, xyz, 6), x') = ((q, j, xyz, 7), R)$$

$$\delta_3((q, j, xyz, 7), y) = ((q, j, xyz, 8), y')$$

$$\delta_3((q, j, xyz, 8), y') = ((q, j, xyz, 9), R)$$

$$\delta_3((q, j, xyz, 9), z) = ((q, j, xyz, 10), z')$$

$$\delta_3((q, j, xyz, 10), z') = ((q, j, xyz, 11), L)$$

$$\delta_3((q, j, xyz, 11), y') = \begin{cases} ((q', |u'| + 1), L), & |u'| + 1 \le m \\ ((q', |u'| + 1 - m), y'), & m < |u'| + 1 \le 2m \end{cases}$$

$((q', |u'| + 1 - 2m), R), 2m < |u'| + 1$

În continuare trebuie să ţinem cont de ceea ce se întâmplă când M_3 înâlneşte un nou simbol #. Putem forţa M_3 să trateze un simbol # ca un *m*-tuplu de simboluri #, astfel încât atunci când M_3 întâlneşte un #, simularea să continue fără întârzieri suplimentare pentru compresie. Acelaşi efect se obţine modificând funcţia δ_3 definită anterior înlocuind $\#^m$ cu # oriunde apare, ca argument sau valoare.
La sfârşit, M_4 trebuie să convertească rezultatul lăsat de M_3, când se opreşte, în şirul pe care M l-ar fi lăsat pe bandă. M_4 trebuie să transforme $\#(\#, \underline{Y}, \#, \ldots, \#)$ în $\#\underline{Y}\#$ şi $\#(\#, \underline{N}, \#, \ldots, \#)$ în $\#\underline{N}\#$. Deci M_4 efectuează doar doi paşi. Câţi paşi, în cazul cel mai defavorabil, necesită $M' = M_1M_2M_3M_4$ pentru w, $|w| = n$? M_1 face $2n + 1$ paşi, M_2 - $2\lceil (n + 1) / m \rceil + 1$ paşi, M_4 – doi paşi. În cazul cel mai defavorabil M_3 face $12\lceil T(n)/m \rceil$ paşi, pentru că m paşi ai lui M sunt simulaţi prin 12 paşi ai lui M_3 şi M nu face mai mult de $T(n)$ paşi. Deci în cazul cel mai defavorabil,

$$12\left\lceil \frac{T(n)}{m} \right\rceil + 2\left\lceil \frac{n+1}{m} \right\rceil + 2n + 4 \text{ paşi}$$

Pentru $\forall\, r, s \in \mathbb{N} - \{0\}$,

$$\left\lceil \frac{r}{s} \right\rceil \leq \frac{r + s - 1}{s},$$

numărul de paşi este cel mult:

$$\frac{12T(n) + 12m - 12 + 2n + 2 + 2m - 2}{m} + 2n + 4;$$

cum $T(n) \geq 2n + 4$, acest număr nu este mai mare decât

$$\frac{13T(n)}{m} + 2n + 18;$$

În final, putem alege un m satisfăcător. Dacă $m = \lceil 13/\epsilon \rceil$, atunci M' decide L în T'

$$T'(n) = 2n + 18 + \lceil \varepsilon \times T(n) \rceil, \forall n \geq 0$$

3.1. Rata de creştere a funcţiilor

Teorema precedentă afirmă că atunci când se lucrează cu limita superioară T a unei maşini Turing, cel mai important lucru este rata de creştere a funcţiei T şi nu detalii de tipul coeficienţilor constanţi sau termeni de ordin inferior.

Definiţia 3.3

Fie f, g: N $\rightarrow$ N, $f = O(g)$ dacă şi numai dacă există $c > 0$ şi $n_0 \in$ N astfel încât $f(n) \leq c \infty g(n), n \geq n_0$.

De exemplu:

$50n^2 + 14n + 33 = O(n^2)$

$n \geq 1, n^2 \geq n, n^2 \geq 1$

$50n^2 + 14n + 33 \leq 97n^2, n \geq 1$

$c = 97, n_0 = 1.$

Lema 3.1

Fie $\leq$ relaţia definită pe mulţimea funcţiilor definite pe mulţimea numerelor naturale. $f \leq g$ dacă şi numai dacă $f = O(g)$. Atunci relaţia $\leq$ este reflexivă şi tranzitivă.

Demonstraţie. Relaţia $\leq$ este reflexivă, pentru că $f = O(f), \forall f, c = 1, n_0 = 0$. Este şi tranzitivă. Presupunem că $f = O(g)$ şi $g = O(h)$ atunci $\exists c, c' > 0$ şi $n_0, n_0' \in$ N astfel încât

$f(n) \leq c \infty g(n), n \geq n_0$

$g(n) \leq c' \infty h(n), n \geq n_0'$

$f(n) \leq c \infty c' \infty h(n), n \geq \max(n_0, n_0')$ şi deci $f = O(h)$

Să definim relaţia $\equiv$ astfel:

$f \equiv g$ dacă şi numai dacă $f = O(g)$ şi $g = O(f)$

Cum $\leq$ este reflexivă şi tranzitivă, $\equiv$ este relaţia de echivalenţă ■

Definiţia 3.4

Clasa de echivalenţă a funcţiei f: N $\rightarrow$ N în raport cu relația de echivalență ≡ este numită rata de creştere a funcţiei f.

De exemplu,

$50n^2 + 14n + 33 = O(n^2)$

şi $n^2 = O(50n^2 + 14n + 33) \Rightarrow n^2 \equiv 50n^2 + 14n + 33$

adică n^2 şi $50n^2 + 14n + 33$ au aceeași rată de creştere

Lema 3.2

Fie $f(n) = \sum_{j=0}^{d} a_j n^j$ un polinom de grad d. Atunci:

(a) $f \equiv n^d$;

(b) $f = O(n^{d+1})$ şi $f \not\equiv n^{d+1}$,

Demonstraţie.

(a) Rezultă imediat că

$$f(n) \leq (\sum_{j=0}^{d} b_j) \times n^d,\ b_j = \max(a_j, 0)$$

Deci $f(n) = O(n^d)$, unde

$$c = (\sum_{j=0}^{d} b_j),\ n_0 = 1$$

Fie

$$n_0 = 2 \sum_{j=0}^{d-1} |a_j|$$

este uşor de demonstrat că $n^d \leq 2 \infty f(n)$, $n \geq n_0$

(b) Este evident că $n^d = O(n^{d+1})$

$f = O(n^{d+1})$ pentru că din (a) $f \equiv n^d$.

Pentru a arăta că nu este posibil ca $n^{d+1} = O(n^d)$, să presupunem că $\forall\ n \geq n_0$, $n^{d+1} \leq c \infty n^d$; această inegalitate nu mai este adevărată pentru $n = \max(\lceil c \rceil) + 1, n_0)$ ■

Lema arată că rata de creştere a unui polinom poate fi identificată cu gradul său.

În afară de polinoame, există anumite rate de creştere care deşi nu sunt sub formă polinomială, sunt mărginite superior de polinoame. De exemplu:

$n\lceil \log_2(n+1) \rceil$ şi $n^2\lfloor \mathrm{sqrt}(n) \rfloor$

$n\lfloor \log(n+1) \rfloor = O(n^2)$

$n^2\lfloor \mathrm{sqrt}(n) \rfloor = O(n^3)$

Aceste limite polinomiale ale ratei de creştere sunt interesante prin contrast cu ratele exponenţiale de forma r^n, $r > 1$. Diferenţa este stabilită de următoarea teoremă.

Teorema 3.2

Fie $f(n) = \Sigma_{j=0}^{d} a_j n^j$ şi $r > 1$. Atunci $f = O(r^n)$

Demonstraţie. Este suficient să demonstrăm că $n^d = O(r^n)$.
$\forall\ n > 0$, fie $e_n = r^n$ şi $f_n = n^d$. Atunci $e_{n+1}/e_n = r$,

$$\frac{f_{n+1}}{f_n} = \frac{(n+1)^d}{n^d} = \left(1 + \frac{1}{n}\right)^d$$

$\frac{f_{n+1}}{f_n}$ descreşte cu n şi este mai mică decât r, atunci când

$$\left(1 + \frac{1}{n}\right)^d < r;$$

adică atunci când,

$$n > \frac{1}{r^{1/d} - 1}$$

Fie $n_0 = \lceil 1/(r^{1/d} - 1) \rceil$ şi $c = n_0^d / r^{n0}$. Atunci $\forall\ k \geq 0$,

$$(n_0 + k)^d = n_0^d \left(\frac{n_0 + 1}{n_0}\right)^d \propto \left(\frac{n_0 + 2}{n_0}\right)^d \propto \cdots \propto \left(\frac{n_0 + k}{n_0}\right)^d$$

$$\infty \qquad n_0 \qquad n_0+1 \qquad\qquad n_0+k-1$$

$$= n_0^d \left(1+\frac{1}{n_0}\right)^d \propto \left(1+\frac{1}{n_0+1}\right)^d \infty \ldots \infty 1+\frac{1}{n_0+k-1}\Bigg)^d \leq n_0^d r^k$$

$$\infty$$

$= cr^{n0} r^k = cr^{n0+k}$, $\forall\ n \geq n_0$, $n^d = O(r^n)$.

pentru că:

$$\left(1+\frac{1}{n}\right)^d \leq r,\ n \geq n_0$$

■

Cum pentru orice $d > 0$, n^{d+1} creşte mai repede decât n^d, putem afirma (Teorema 3.1) că orice funcţie exponenţială creşte mai repede decât una polinomială.

Pentru a simplifica notaţia vom nota $\mathrm{TIME}(f)$ pentru

$$\cup \{\mathrm{TIME}(g) : g \equiv f\}$$

pentru a reprezenta clasa limbajelor decidabile într-o limită de timp modelată printr-o funcţie având aceeaşi rată de creştere ca şi f.

3.2. Limite de timp

Teorema 3.3

Fie L un limbaj decis de o maşină Turing M cu bandă infinită în ambele sensuri în timpul T_1. Atunci L este decis de o maşină Turing M_2 în T_2, unde $\forall\ n \in \mathrm{N}$, $T_2(n) = 6T_1(n) + 3n + 8$.

Teorema 3.4

Fie L un limbaj decis în timpul T_1 de o maşină Turing cu k benzi, $M_1 = (K_1, \Sigma_1, \delta_1, s_1)$. Atunci L este decis de o maşină Turing cu o

bandă $M_2 = (K_2, \Sigma_2, \delta_2, s_2)$ în T_2, unde $\forall\ n \in N$, $T_2(n) = 4T_1(n)^2 + (4n + 4k + 3)T_1(n) + 5n + 15$.

Corolar

Dacă L este un limbaj decis în timpul T de o anumită mașină Turing cu k benzi, atunci L este decis în $T' = O(T^2)$ de o mașină Turing cu o bandă.

Teorema 3.5

Dacă L este un limbaj decis în timpul T de o mașină Turing bidimensională, atunci L este decis în $T' = O(T^3)$ de o mașină Turing cu patru benzi.

3.3. Clasele $\mathcal{P}$ și $\mathcal{NP}$

Definiţia 3.5

Definim $\mathcal{P}$ (decidabil în timp polinomial) clasa limbajelor

$$\mathcal{P} = \cup \{TIME(n^d) : d > 0\}$$

unde $TIME(n^d)$ conţine toate limbajele decise de o mașină Turing într-o limită de timp care este un polinom de grad d. Considerând reuniunea peste toate gradele obţinem clasa tuturor limbajelor decise de o mașină Turing în limite polinomiale.

Clasa $\mathcal{P}$ introdusă are un rol important. Definește clasa limbajelor decidabile, din punct de vedere cantitativ. Clasa limbajelor decidabile coincide (Teza lui Church) cu clasa problemelor care pot fi rezolvate algoritmic. $\mathcal{P}$ coincide cu clasa problemelor care pot fi rezolvate practic.

Definiţia 3.6

Fie $T: N \to N$, $L \subseteq \Sigma_0^*$ un limbaj şi $M = (K, \Sigma, \delta, s)$ o mașină Turing nedeterministă. M acceptă L nedeterminist în timpul T dacă:

$\forall\ w \in \Sigma_0^*$, $w \in L$ dacă şi numai dacă $(s, \#w\#) \vdash_M^t (h, v\underline{\sigma}u)$, $v, u \in \Sigma_0^*$, $\sigma \in \Sigma$, $t \le T(|w|)$.

Spunem că L este acceptat nedeterminist în timpul T, dacă există maşina Turing nedeterministă M care acceptă L nedeterminist în timpul T.

Clasa limbajelor acceptate nedeterminist în timpul T este notată cu $\mathrm{NTIME}(T)$.

Notăm $\mathcal{NP} = \cup \{\mathrm{NTIME}(n^d) : d > 0\}$.

Noţiunea de acceptare nedeterministă limitată în timp este analogul cantitativ al acceptării nedeterministe. Spunem că o maşină Turing nedeterministă M acceptă w dacă se opreşte pentru intrarea w; nu acceptă dacă operaţiile sale sunt infinite. În ultimul caz, avem aceeaşi convenţie cu diferenţa că este considerată infinită orice operaţie cu mai mult de $T(|w|)$ paşi pentru intrarea w.

De remarcat o asimetrie fundamentală în definirea calculului nedeterminist limitat de timp. O maşină Turing nedeterministă acceptă şirul w în timpul T dacă şi numai dacă există una sau mai multe operaţii care duc la configuraţia de oprire după $T(|w|)$ sau mai puţini paşi. Nu acceptă dacă operațiile mașinii continuă după mai mult de $T(|w|)$ paşi. În figura 3-4 (a) şirul este acceptat, în (b) nu.

Mulţimea tuturor operaţiilor unei maşini Turing nedeteministe pentru o intrare w poate fi reprezentată ca un arbore. Nodurile reprezintă configurații, arcele paşi. Alegerile nedeterministe sunt reprezentate prin noduri care au mai mult de un arc.

Este imediat faptul că $\mathrm{TIME}(T) < \mathrm{NTIME}(T^2)$ din lema, şi, deci $\mathcal{P} \subseteq \mathcal{NP}$. Este $\mathcal{P} = \mathcal{NP}$? Adică maşinile Turing nedeterministe sunt o variantă a maşinilor Turing în raport cu clasa limbajelor decise în timp polinomial?

Arborii care reprezintă mulţimea operaţiilor unei maşini Turing nedeterministe au multe noduri (configurații) toate de înălţime medie. Singurul mod în care o maşină Turing determinstă poate concura cu o maşină Turing nedeterministă relativ la numărul de configuraţii ce pot fi atinse este prin operaţii cu număr exponenţial de paşi.

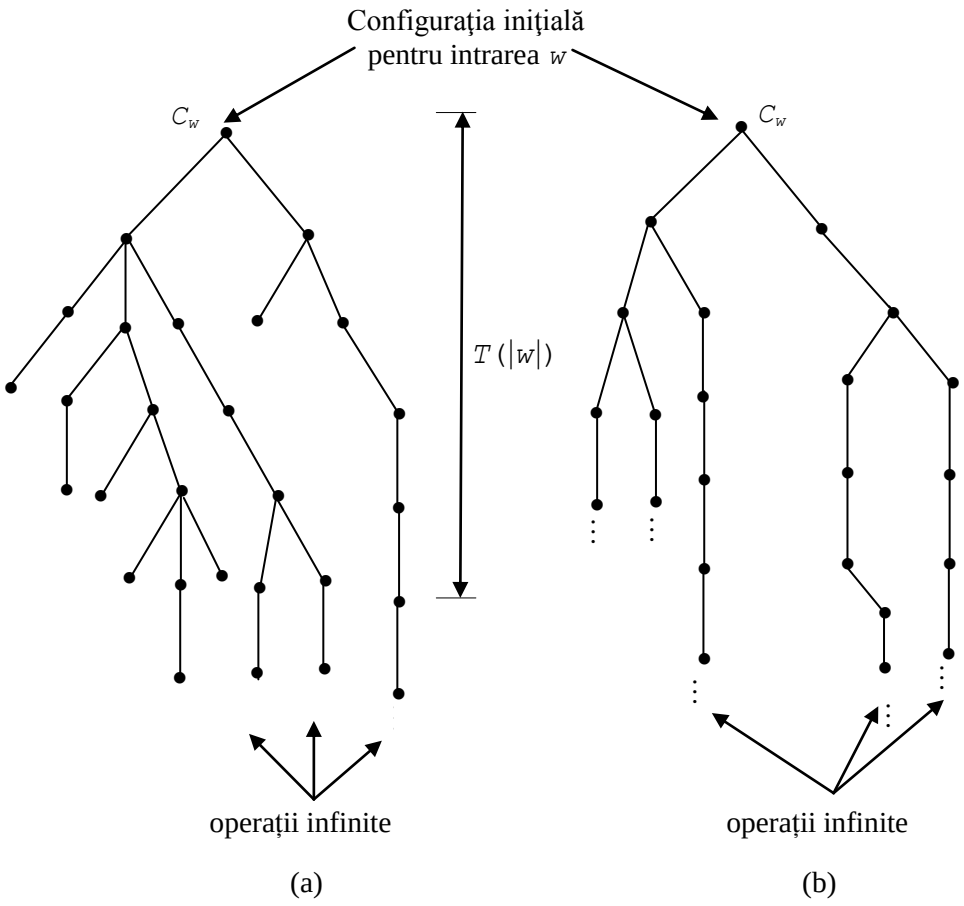


Figura 3-4

Definiţia 3.7

Funcția $f : \mathrm{N} \to \mathrm{N}$ este o funcţie de numărare a paşilor dacă există $k \geq 1$ şi o maşină Turing cu k benzi M astfel încât $\forall\ w \in \Sigma^*$, M se opreşte în $f(|w|)$ paşi pentru w,

$$(s, \#w\underline{\#}, \ldots, \underline{\#}) \vdash_M^{f(|w|)} (h, u_1\underline{a}_1v_1, \ldots, u_K\underline{a}_kv_k)$$

$u_1, \ldots, u_k, v_1, \ldots, v_k \in \Sigma^*$, $a_1, \ldots, a_k \in \Sigma$

Lema 3.3

Dacă p este un polinom cu coeficienţi nenegativi astfel încât

$\forall n \in \mathrm{N}, p(n) \geq 2n + 4,$

atunci p este o funcţie de numărare a pașilor unei mașini Turing.

Teorema 3.6

Fie L un limbaj acceptat în T_1, unde T_1 este o funcţie de numărare a pașilor de o mașină Turing $\mathrm{M}_1 = (K_1, \Sigma_1, \Delta_1, s_1)$. Atunci există numerele întregi $k_2 \geq 1$, $r > 1$ şi o mașină Turing deterministă cu k_2 benzi $\mathrm{M}_2 = (K_2, \Sigma_2, \Delta_2, s_2)$ care decide L în T_2, unde $T_2(n) = r^{T_1(n)}$.

Demonstraţie. Fie M mașina Turing care pentru intrarea w se opreşte după $\mathrm{T}_1(|w|)$ paşi în modul descris în Definiţia 3.7. Este garantată existența lui M pentru că T_1 este o funcţie de numărare a pașilor. Fie k_2 numărul de benzi ale lui M_2, $k_2 = k + 2$, unde k este numărul de benzi ale lui M.

M_2 copiază intrarea pe cea de a k_2-a banda; fie @ maşina care efectuează această funcţie. Apoi M_2 cedează controlul unei versiuni M' a lui M pentru intrarea w. M′ este identică cu M, cu excepţia unei bande suplimentare a (k_2−1)-a pe care scrie câte un simbol d_1 la fiecare pas al lui M.

La sfârşitul operaţiilor lui M′, banda suplimentară conţine şirul $d_1^{T_1(|w|)}$. M_2 îşi şterge benzile cu excepţia celei de a (k_2−1-a şi k_2-a. Fie $\mathcal{E}$ maşina Turing care realizează această funcţie. M_2 scrie $ pe prima bandă, se deplasează la dreapta şi intră într-un ciclu în care simulează determinist operaţiile lui M_1 pentru w. Acest ciclu constă în trei maşini, $\mathcal{G}$, M_1' şi $\mathcal{G}$'. Maşina $\mathcal{G}$ copiază w de pe banda k_2 pe prima bandă. M_1′ este o variantă a lui M_1 care simulează determinist M_1 pe prima bandă, consultând banda (k_2−1) pentru alegeri nedeterministe. Când M_1, s-ar fi oprit, M_1′ şterge benzile şi se opreşte cu #Ⓨ# pe prima bandă. Pentru a trece de la simularea unei operaţii a lui M_1 la alta, M_2 utilizează maşina $\mathcal{G}$' care calculează în baza r_1, unde r_1 este numărul maxim posibil al alegerilor

nedeterministe pentru un pas a lui M_1. Diferenţa dintre $\mathcal{G}$ şi $\mathcal{G}$' este că prima dată $\mathcal{G}$' încearcă să crească lungimea şirului, şterge toate benzile şi scrie Ⓝ pe prima bandă. M_2 este reprezentată în Figura 3-5.

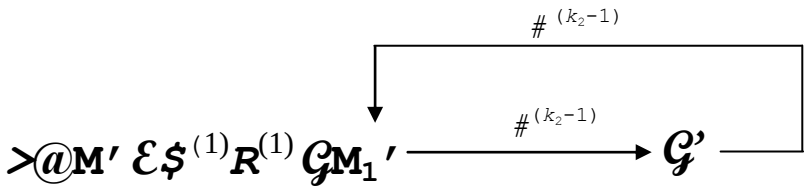


Figura 3-5

Numărul de paşi executaţi de M_2 pentru w, $|w| = n$, este:

1. @ necesită $3n+3$ paşi;
2. M' necesită $2T_1(n)$ paşi, conform definiţiei sale;
3. $\mathcal{E}$ necesită cel mult $4T_1(n)$ paşi
4. Fiecare dintre ciclurile lui $\mathcal{G}$, M_1', $\mathcal{G}$' (cel mult $r_1^{T_1(n)}$) necesită cel mult $4T_1(n)$ paşi.
5. $2T_1(n)+5$ paşi sunt necesari pentru a şterge toate benzile şi a scrie Ⓝ.

În total, $8T_1(n) + 3n + 8 + r_1^{T_1(n)} 4T_1(n)$ paşi. Ţinând cont că:

$$4T_1(n) \leq 2^{4T_1(n)}$$

$$2T_1(n) \geq 3n + 8$$

limita superioară este: $10T_1(n) + (16r_1)^{T_1(n)} \leq (16r_1 + 1)^{T_1(n)}$. Aceasta demonstrează teorema pentru $r = 16r_1+1$. ■

Corolar

$\mathcal{NP} \subseteq \cup \{\text{TIME}(r^{n^d}) : r, d > 0\}$

Definiţia 3.8

Fie $L \subseteq \Sigma^*\$\Sigma^*$ un limbaj, unde $\$ \notin \Sigma$. Spunem că L este balansat polinomial dacă există un polinom p astfel încât $x\$y \in L$, numai dacă $|y| \leq p(|x|)$.

În particular, dacă $L \subseteq \Sigma^*\$\Sigma^*$ şi $\$ \notin \Sigma$, atunci $L/\$\Sigma^*$ este mulţimea tuturor $x \in \Sigma^*$ astfel încât $x\$y \in L$, $y \in \Sigma^*$.

Teorema 3.7

Fie $L \subseteq \Sigma^*$ un limbaj, unde $\$ \notin \Sigma$, $|\Sigma| \geq 2$. Atunci $L \in \mathcal{NP}$ dacă şi numai dacă există un limbaj polinomial balansat $L' \subseteq \Sigma^*\$\Sigma^*$ astfel încât $L' \in \mathcal{P}$ şi $L = L'/\$\Sigma^*$.

Exemplu 3.4

Să formalizăm problema comis voiajorului. Datele sunt:

(a) n - numărul de oraşe care trebuie vizitate, identificate prin $1, 2, \ldots, n$.

(b) distanţele dintre oraşe, date printr-o matice D, de dimensiune $n \times n$, numită matricea distanţelor; intrarea d_{ij} din matrice reprezintă distanţa dintre oraşele i, j (nu se impune ca D să fie simetrică, adică $d_{ij} = d_{ji}$, pentru orice i, j); $d_{ii} = 0$, $\forall\, i$; vom reprezenta matricea printr-o codificare α.

Se cere să se găsească drumul cel mai scurt.

Un tur t poate fi definit ca o bijecţie $t : \{1, 2, \ldots, n\} \rightarrow \{1, 2, \ldots, n\}$. Dacă $t(i) = j$, atunci oraşul i va fi vizitat al j-lea în tur. Lungimea unui tur t este:

$$D(t) = \sum_{j=1}^{n-1} d_{t(j),\,t(j+1)} + d_{t(n),\,t(1)}$$

Determinarea turului t pentru care $D(t)$ este minim nu este o problemă de decizie de limbaj, este o problemă de evaluare de funcţie. Pentru a formula problema în termenii unui limbaj, presupunem că avem un întreg b, reprezentând un buget. Întrebarea este dacă există un tur astfel încât $D(t) \leq b$.

Se poate arăta că dacă avem un algoritm cu timp polinomial pentru a decide dacă există un tur cu lungime într-o limită dată, putem utiliza acel algoritm pentru a dezvolta un nou algoritm în timp polinomial pentru a calcula turul minim. Formal, definim limbajul

$$PCV = \{ I^n @ \alpha(D) @ I^b : \text{există } t \text{ astfel încât } D(t) \le b \}$$

Nu se poate afirma că $PCV \in \mathcal{P}$. Toţi algoritmii cunoscuţi pentru rezolvarea acestei probleme necesită un timp exponenţial.

Este, însă, uşor de arătat că $PCV \in \mathcal{NP}$, utilizând teorema anterioară. Este suficient să observăm că:

$$PCV' = \{ I^n @ \alpha(D) @ I^b \$ \alpha(t) : D(t) \le b \}$$

este polinomial balansat şi este în $\mathcal{P}$, unde prin $\alpha(t)$ am codificat turul t astfel încât

$$\alpha(t) = I^{t(1)} c I^{t(2)} c \ldots c I^{t(n)}$$

PCV' este balansat polinomial pentru că $|\alpha(t)| \le n^2$.

Date fiind t şi D se poate evalua $D(t)$ pentru a decide dacă $D(t) \le b$ în timp polinomial. Prin urmare $PCV \in \mathcal{NP}$.

Putem spune că $\mathcal{P}$ este clasa tuturor mulţimilor L astfel încât apartenenţa unui element x la L poate fi testată eficient.

3.4. $\mathcal{NP}$ Completitudine

Definiţia 3.9

Fie Σ, Δ alfabete. O funcţie $f : \Sigma^* \to \Delta^*$ este calculabilă în timpul T printr-o maşină Turing deterministă $M=(K, \Sigma', \delta, s)$ cu k benzi, dacă şi numai dacă, $\forall x \in \Sigma^*$,

$$(s, \#x\underline{\#}, \underline{\#}, \ldots, \underline{\#}) \vdash_M^t (h, \#f(x)\underline{\#}, \underline{\#}, \ldots, \underline{\#}), t \le T(|x|).$$

Spunem că f este calculabilă în timpul T dacă există maşina Turing M care calculează f în timpul T. Spunem că f este calculabilă polinomial dacă există un polinom T astfel încât f este calculabilă în T.

Rezultă imediat că dacă f este calculabilă în timpul T, atunci $\forall x \in \Sigma^*$, $|f(x)| \le T(|x|) + |x|$, pentru că maşina care calculează f nu poate produce în $T(|x|)$ ca ieşire $f(x)$ cu lungime mai mare decât $T(|x|) + |x|$.

Exemplu 3.5

Dacă L un limbaj din $\mathcal{P}$, atunci funcţia f_L:

$$f_L(x) = \begin{cases} \text{Ⓨ}, & x \in L \\ \text{Ⓝ}, & \text{altfel} \end{cases}$$

este, din definiţia lui $\mathcal{P}$, calculabilă polinomial.

Definiţia 3.10

Fie $L_1 \subseteq \Sigma_1^*$ şi $L_2 \subseteq \Sigma_2^*$ limbaje. O funcţie calculabilă polinomial τ: $\Sigma_1^* \to \Sigma_2^*$ este numită transformare în timp polinomial din L_1 în L_2, dacă şi numai dacă, $\forall\ x \in \Sigma_1^*$, $x \in L_1$ dacă şi numai dacă $\tau(x) \in L_2$.

Dacă există o transformare în timp polinomial a lui L_1 în L_2, este posibil să se adapteze o procedură de decizie pentru L_2 cu scopul de a obţine una pentru L_1.

Pentru a determina dacă x este în L_1, se poate calcula $\tau(x)$ şi se verifică dacă şirul este în L_2.

Lema 3.4

Dacă $\tau_1 : \Sigma_1^* \to \Sigma_2^*$ şi $\tau_2 : \Sigma_2^* \to \Sigma_3^*$ sunt transformări în timp polinomial din L_1 în L_2 şi din L_2 în L_3, atunci $\tau_1 \circ \tau_2 : \Sigma_1^* \to \Sigma_3^*$ este o transformare în timp polinomial din L_1 în L_3.

Demonstraţie. Presupunem că τ_1 este calculat de maşina Turing M_1 în timpul polinomial T_1 şi τ_2 este calculat de M_2 în T_1 (polinomial). Atunci $\tau_1 \circ \tau_2$ poate fi calculat de M_1M_2.

Pentru intrarea $x \in \Sigma_1^*$, M_1M_2 va calcula $\tau_1 \circ \tau_2(x)$ într-un timp mărginit de $T_1(|x|) + T_2(T_1(|x|) + |x|)$ care este polinomial.

Rămâne de arătat că $x \in L_1$ dacă şi numai dacă $\tau_1 \circ \tau_2(x) \in L$, ceea ce este trivial.

$x \in L_1 \Leftrightarrow \tau_1(x) \in L_2 \Leftrightarrow \tau_2(\tau_1(x)) = \tau_1 \circ \tau_2(x) \in L_3$ ■

Definiţia 3.11

Un limbaj $L \subseteq \Sigma^*$ este numit $\mathcal{NP}$ complet, dacă şi numai dacă:
(a) $L \in \mathcal{NP}$
(b) $\forall L' \in \mathcal{NP}$, există o transformare în timp polinomial de la L' la L.

Teorema 3.8

Fie L un limbaj $\mathcal{NP}$ complet. Atunci $\mathcal{P} = \mathcal{NP}$ dacă și numai dacă $L \in \mathcal{P}$.

Demonstraţie. Presupunem $\mathcal{P} = \mathcal{NP}$. Cum L este $\mathcal{NP}$ complet, din Definiţia 3.11, $L \in \mathcal{NP}$, prin urmare $L \in \mathcal{P}$.
Implicația inversă. Presupunem că L este decis de o mașină Turing deterministă M_1 în T_1 (polinomial), şi fie L' un limbaj în $\mathcal{NP}$; vom arăta că $L' \in \mathcal{P}$.
Cum L este $\mathcal{NP}$ complet şi $L' \in \mathcal{NP}$ atunci există o transformare τ în timp polinomial de la L' la L. Să presupunem că τ este calculat de o mașină Turing M_2 în T_2 (polinominal). Atunci afirmăm că mașina Turing M_2M_1 decide L' în timp polinomial.
Pentru a vedea dacă M_2M_1 decide L', trebuie observat că M_2M_1 se opreşte cu Ⓨ pe bandă pentru intrarea x dacă și numai dacă $\tau(x) \in L$; cum τ este o transformare în timp polinomial, $\tau(x) \in L$ dacă și numai dacă $x \in L'$. Mai mult, M_2M_1 se opreşte pentru intrarea x în timpul $T_2(|x|) + T_1(T_2(|x|)+|x|)$ care este polinomial în $|x|$.
Cum L' poate fi orice limbaj în $\mathcal{NP}$, concluzionăm că $L' \in \mathcal{P}$, şi din $\mathcal{NP} \subseteq \mathcal{P}$ rezultă că $\mathcal{P} = \mathcal{NP}$. ■

Limbajele $\mathcal{NP}$ complete joacă în raport cu intrebarea $\mathcal{P} = \mathcal{NP}$, acelaşi rol pe care limbajul K_0 îl joacă în raport cu întrebarea dacă toate limbajele acceptate Turing sunt decidabile Turing.

Dar cum găsim un limbaj $\mathcal{NP}$ complet? Din nou construcţia lui K_0 oferă soluţia. Cum K_0 constă din codificarea maşinii Turing şi a intrării acceptate, vom considera codificarea maşinii Turing nedeterministe, intrările acceptate şi numărul de paşi necesari pentru acceptarea acelor intrări. Fie

$N_0 = \{\rho(M)\rho(w)@I^t$: maşina Turing nedeterministă care acceptă w în cel mult t paşi$\}$.

Teorema 3.9

N_0 este $\mathcal{NP}$ complet.

Noţiunea de $\mathcal{NP}$ completitudine a eşuat în a demonstra $\mathcal{P} = \mathcal{NP}$. Totuşi $\mathcal{NP}$ completitudinea este o idee importantă, pentru că dintre toate limbajele $\mathcal{NP}$ – limbajele $\mathcal{NP}$ complete sunt cel mai puţin probabil de a fi decise în timp polinomial. Conceptul de $\mathcal{NP}$ completitudine a unificat experienţa cercetărilor în diverse domenii arătând că niciuna dintre aceste probleme nu poate fi rezolvată printr-un algoritm polinomial decât dacă $\mathcal{P} = \mathcal{NP}$.

4. MODELE ECHIVALENTE DE CALCULABILITATE

Mașinile Turing reprezintă versiunile formale ale algoritmilor şi nicio procedură nu poate fi considerată un algoritm dacă nu poate fi reprezentată printr-o mașină Turing (Teza lui Church). Este o teză, nu o teoremă, pentru că nu este un rezultat matematic, asociază un obiect matematic unui concept informal.

Sunt mașinile Turing (automatele în general) singura abordare a ideii de calculabilitate? Nu. Vom considera alte două alternative:

- generatoarele de limbaje – gramaticile nerestricționate
- funcţiile definite pe mulțimea numerelor naturale: funcții μ-recursive.

Vom arăta că mașinile Turing pot calcula funcţii μ-recursive, încheind astfel un ciclu de simulări: mașinile Turing poate fi simulate prin gramatici, care pot fi simulate prin funcții μ-recursive care pot fi simulate prin mașini Turing.

Singura concluzie este că toate aceste abordări ale ideii de calculabilitate sunt echivalente. Este teza lui Church extinsă la reprezentări diferite de automate.

4.1. Gramatici

Definiţia 4.1

O gramatică este un tuplu $G = (V, \Sigma, R, S)$, unde

V este un alfabet;

$\Sigma \subseteq V$ este mulţimea simbolurilor terminale; $V - \Sigma$ este numită mulțimea simbolurilor neterminale;

$S \in V - \Sigma$ este simbolul de start;

$R \subseteq (V^*(V-\Sigma)V^*) \times V^*$ este mulţimea regulilor (o submulţime finită);

Scriem $u \to v$, dacă $(u, v) \in R$. Notăm $u \underset{G}{\Rightarrow} v$ dacă şi numai dacă $w_1, w_2 \in V^*$ şi $u' \to v'$, $u = w_1 u' w_2$ şi $v = w_1 v' w_2$. Notaţia $\overset{*}{\underset{G}{\Rightarrow}}$ reprezintă închiderea reflexivă şi tranzitivă a relaţiei $\underset{G}{\Rightarrow}$

Un şir $w \in \Sigma^*$ este generat de gramatica G dacă şi numai dacă $S \overset{*}{\underset{G}{\Rightarrow}} w$. Limbajul generat de gramatică, $L(G)$, este mulţimea tuturor şirurilor din Σ^* generate de G.

O derivare este o secvenţă de forma

$$w_0 \underset{G}{\Rightarrow} w_1 \underset{G}{\Rightarrow} w_2 \underset{G}{\Rightarrow} \dots \underset{G}{\Rightarrow} w_n$$

Exemplu 4.1

Vrem să definim gramatica G care generează limbajul $L(G) = \{ w \in \{a, b, c\}^* \mid w$ are un număr egal de simboluri $a, b, c\}$.
Regulile gramaticii sunt următoarele :

$S \to e$

$S \to ABCS$ // generează şiruri de tipul $(abc)^n$, $n > 0$

Trebuie să introducem reguli pentru a permuta simbolurile A, B, C şi a înlocui apoi neterminalele A, B, C cu a, b, respectiv c :

$AB \to BA$

$AC \to CA$

$BC \to CB$

$BA \to AB$

$CA \to AC$

$CB \to BC$

$A \to a$

$B \to b$

$C \to c$

Să considerăm derivarea şirului *ccbaba*

$$\underline{S} \Rightarrow ABC\underline{S}$$
$$\Rightarrow ABCABC\underline{S}$$
$$\Rightarrow ABCA\underline{BC}$$
$$\Rightarrow ABC\underline{AC}B$$
$$\Rightarrow A\underline{BC}CAB$$
$$\Rightarrow \underline{AC}BCAB$$
$$\Rightarrow CA\underline{BC}AB$$
$$\Rightarrow C\underline{AC}BAB$$
$$\Rightarrow CC\underline{AB}AB$$
$$\Rightarrow CCBA\underline{AB}$$
$$\Rightarrow \underline{C}CBABA$$
$$\Rightarrow c\underline{C}BABA$$
$$\Rightarrow cc\underline{B}ABA$$
$$\Rightarrow ccb\underline{A}BA$$
$$\Rightarrow ccba\underline{B}A$$
$$\Rightarrow ccbab\underline{A}$$
$$\Rightarrow ccbaba$$

Exemplu 4.2

Să definim gramatica *G*, care generează limbajul:

$$L(G) = \{a^{2^n} \mid n \geq 0\}$$

$G = (V, \Sigma, R, S)$
$V = ([,], A, D, S, a)$
$\Sigma = \{a\}$
$R = \{$
$S \rightarrow [A]$
$[\rightarrow [D$
$D] \rightarrow]$

$DA \to AAD$

$[\to e$

$] \to e$

$A \to a$

}

Intuitiv, gramatica generează şirurile astfel: pornind cu simbolul de start S, şirul $[A]$ este obţinut aplicând regula $S \to [A]$; şirul $[D^nA]$ este produs prin $n \geq 0$ aplicări ale regulii $[\to [D$. Simbolurile D sunt pentru a dubla neterminalul A; de câte ori se trece peste un A de la stânga la dreapta, se înlocuieşte neterminalul A cu doi A folosind regula $DA \to AAD$, la sfârşit $[D^nA]$ devine $[A^{2^n}D^n]$. Simbolurile D sunt eliminate prin regula $D] \to]$, obţinând $[A^{2^n}]$; simbolurile [,] sunt eliminate prin regulile $[\to e$, respectiv $] \to e$. La sfârşit se aplică regula $A \to a$ pentru a obţine şirul de simboluri terminale.

Trebuie să arătăm că niciun şir din $\{a\}^*$ care nu este de forma a^{2^n} nu poate fi generat de G. Numărul simbolurilor D din şirurile intermediare este schimbat doar prin aplicarea regulilor $[\to [D$ şi $D] \to]$; simbolurile [şi] pot apărea doar o dată fiecare la stânga şi dreapta şirului. Dacă regula $] \to e$ este aplicată prematur (înainte eliminării simbolurilor D) nu pot fi produse şiruri de simboluri terminale. Deci singurul mod în care pot fi produse şiruri de simboluri terminale este să se genereze simboluri D la capătul stânga, apoi simbolurile A şi la sfârşit să se elimine simbolurile D de la capătul dreapta. Acest proces lasă şirul în forma $\{[, e\}\}A, a\}^{2^n}\{]\}$ unde n este numărul de simboluri D produse. Singurul şir de terminale ce poate fi generat este a^{2^n}.

Vom arăta că operaţiile unei maşini Turing pot fi simulate prin derivări ale unei gramatici. În acest scop vom reprezenta configuraţia maşinii Turing prin şiruri, apoi vom construi o gramatică care va deriva aceste şiruri unul din altul dacă şi numai dacă există relaţia $\vdash_M^*$ între configuraţiile maşinii Turing.

Lema 4.1

Fie $M = (K, \Sigma, \delta, s)$ o mașină Turing. Atunci există o gramatică G astfel încât pentru orice configuraţii (q, u, a, v) şi (q', u', a', v') ale lui M,

$$(q, u, a, v) \vdash_M^* (q', u', a', v')$$

dacă și numai dacă

$$[uqav] \overset{*}{\underset{G}{\Rightarrow}} [u'q'a'v']$$

Unde $[\ ,\ ]$ sunt simboluri care nu aparțin mulțimilor K sau Σ. Se presupune că mulțimile K, și Σ sunt disjuncte.

Demonstraţie. Ideea este să reprezentăm configuraţia (q, u, a, v) prin şirul $[uqav]$. Poziţia stării în şir reprezintă o indicaţie a poziţiei capului mașinii Turing, ceea ce face posibil ca partea stângă a unei reguli să conţină ca subşir o stare şi simbolul scanat. Cu excepţia detaliilor legate de mutarea pe o bandă vidă, doar starea, simbolul scanat şi poziţia capului trebuie schimbate la trecerea de la o configuraţie la alta. Simbourile $[\ ,\ ]$ apar la capetele reprezentării unei configurații. Simbolul $[$ este utilizat pentru a trata zona vidă a benzii, iar $]$ este inclus pentru legibilitate.

Fie $G = (V, \Sigma, R, S)$, unde

$$V = K \cup \Sigma \cup \{[,], h, S)\}$$

și R conţine regulile:

a) $\forall\ q \in K, a \in \Sigma, \delta(q, a) = (p, b), p \in K \cup \{h\}, b \in \Sigma$, atunci R conţine

$$qa \to pb$$

b) $\forall\ q \in K, a \in \Sigma$, dacă $\delta(q, a) = (p, R), p \in K \cup \{h\}, h \in \Sigma$, atunci R conţine

$$qab \to apb, \forall\ b \in \Sigma\ ,\text{ şi}$$

$$qa] \to ap\#]$$

c) $\forall\ q \in$ K, $a \in \Sigma$, $\delta(q, a) = (p, L), p \in K \cup \{h\}$, atunci R conţine

$$bqac \to pbac,$$

$\forall\ b \in \Sigma$, $c \in \Sigma \cup \{\,]\,\}$, astfel încât $a \neq \#$ sau $c \neq]$; şi

$bq\#] \rightarrow pb]$,

$\forall\ b \in \Sigma$, dacă $a = \#$.

Afirmăm că

$(q, u, a, v) \vdash_M^* (q', u', a', v')$ dacă şi numai dacă $[uqav] \overset{*}{\underset{G}{\Rightarrow}} [u'q'a'v']$

Dacă este adevărat atunci este adevărat şi pentru $\vdash_M^*$ şi $\overset{*}{\underset{G}{\Rightarrow}}$ Demonstraţia este imediată şi se bazează pe definiţia $\vdash_M$. ■

Exemplu 4.3

Fie $M = (\{q_0, q_1\}, \{a, \#\}, \delta, q_0)$ şi δ dată prin tabelul:

q	σ	$\delta(q, \sigma)$
q_0	a	(q_1, R)
q_0	$\#$	(q_0, a)
q_1	a	(q_0, L)
q_1	$\#$	(q_1, a)

Gramatica asociată $G = (V, \{a, \#\}, R, S)$, unde

$V = \{q_0, q_1, a, \#, [,], h, S\}$

şi R conţine regulile

(a) 1. $q_0\# \rightarrow q_0a$
2. $q_1\# \rightarrow q_1a$

(b) 1. $q_0aa \rightarrow aq_1a$
2. $q_0a\# \rightarrow aq_1\#$
3. $q_0a] \rightarrow aq_1\#]$

c) 1. $aq_1aa \rightarrow q_0aaa$
2. $\#q_1aa \rightarrow q_0\#aa$
3. $aq_1a\# \rightarrow q_0aa\#$
4. $\#q_1a\# \rightarrow q_0\#a\#$
5. $aq_1a] \rightarrow q_0aa]$
6. $\#q_1a] \rightarrow q_0\#a]$

Mașina M pornind din $(q_0, \underline{\#})$ execută următorii pași:

$$(q_0, \underline{\#}) \vdash_M (q_0, \underline{a}) \vdash_M (q_1, a\underline{\#}) \vdash_M (q_1, a\underline{a}) \vdash_M (q_0, \underline{aa})$$

În gramatică putem scrie următoarele derivări:

$$[q_0\#] \underset{G}{\Rightarrow} [q_0a] \underset{G}{\Rightarrow} [aq_1\#] \underset{G}{\Rightarrow} [aq_1a] \underset{G}{\Rightarrow} [q_0aa]$$

utilizând regulile (a_1), (b_3), (a_2), (c_5).

Definiţia 4.2

a) Fie Σ_0 şi Σ_1 alfabete care nu conţin $\#$, şi fie funcția $f : \Sigma_0^* \to \Sigma_1^*$. Atunci spunem că f este calculabilă gramatical dacă și numai dacă există $G = (V, \Sigma, R, S)$, unde $\Sigma_0, \Sigma_1 \subseteq \Sigma$ şi există $x, y, x', y' \in V^*$ astfel încât $\forall\, u \in \Sigma_0^*$, $v \in \Sigma_1^*$,

$$f(u) = v$$

dacă și numai dacă

$$xuy \overset{*}{\underset{G}{\Rightarrow}} x'vy'$$

Spunem că G calculează f.

b) Fie o funcție $f : \mathrm{N} \to \mathrm{N}$. f este calculabilă gramatical dacă și numai dacă $f' : \{\mathrm{I}^*\} \to \{\mathrm{I}\}^*$ este calculabilă gramatical, unde $f'(I^n) = I^{(fn)}$, $\forall n \in \mathrm{N}$.

Exemplu 4.4

Fie $\Sigma_0 = \{a, b\}$, $f : \Sigma_0^* \to \Sigma_0^*$, $f(w) = w^R$. Atunci f este calculată de gramatica (V, Σ_0, R, S), unde

$$V = \{a, b, A, B, *, S, [,]\}$$

şi R conține regulile :

$$1.\ [a \to [A,\ [b \to [B$$

$$2.\ Aa \to aA,\ Ab \to bA,\ Ba \to aB,\ Bb \to bB$$

$$3.\ A* \to *a \qquad B* \to *b$$

Fie x, y, x', y' şirurile $[,*]$, $[*$ și $]$. G calculează f astfel: pornește cu șirul $[w*]$, $w \in \Sigma_0^*$, și utilizează reguli de tipul 1 și 2 pentru a converti simbolurile termnale (a, b) de la capătul stânga al lui w în simbolurile netermnale (A, B) și le propagă la capătul dreapta. Când ajung la limita

dreapta sunt transformate înapoi în simboluri terminale. Simbolul * este utilizat pentru a le separa de partea stânga a șirului. Dacă un simbol a întâlnit * rămâne neschimbat,

$[abb^*] \Rightarrow [Abb^*]$ $R1$

$\Rightarrow [bAb^*]$ $R2$

$\Rightarrow [bbA^*]$ $R2$

$\Rightarrow [bbx^*]$ $R3$

$\Rightarrow [Bb^*a]$ $R1$

$\Rightarrow [bB^*a]$ $R2$

$\Rightarrow [BB^*a]$ $R1$

$\Rightarrow [B^*ba]$ $R3$

$\Rightarrow [^*bba]$ $R3$

Teorema 4.1

Orice funcţie calculabilă Turing, definită pe şiruri numere, este calculabilă gramatical.

Demonstraţie. Fie $f : \Sigma_0^* \rightarrow \Sigma_1^*$ o funcție calculabilă Turing. Din Definiţia 1.5 există o mașină Turing (K, Σ, δ, s) astfel încât $\forall u \in \Sigma_0^*$, $f(u) = n$ dacă și numai dacă

$(s, \#u, \#, e) \vdash_M^* (h, \#v, \#, e)$

Aplicând construcția din Lema 4.1 se obţine o gramatică G. Fie şirurile x, y, x', y' definite în felul următor:

$x = [\#$

$y = s\#]$

$x' = [\#$

$y' = h\#]$

atunci G, x, y, x', y' respect cerința.

Cum calculabilitatea (gramaticală, respectiv prin mașina Turing) funcțiilor definite pe numere este definită în termenii calculabilității funcțiilor definite pe șiruri, demostrația este valabilă și pentru cele dintâi. ■

4.2. Funcţii primitiv recursive

În cele ce urmează, vom utiliza funcţii definite pe $N^k \to N$, $k \geq 0$, numite funcţii de ordin k.

Definiţia 4.3

Funcţii iniţiale.

a) Definim funcţia de ordin 0, $\xi: N^0 \to N$ astfel încât $\xi() = 0$

b) Fie $k \geq 1$, $1 \leq i \leq k$, π_i^k este funcţia proiecţie i de ordinul k.

$\pi_i^k: N^k \to N$

$\pi_i^k (n_1, \ldots, n_k) = n_i, \ n_1, \ldots, n_k \in N$

Dacă notăm cu $\tilde{n} = n_1, \ldots, n_k$,

$\pi_i^k(\tilde{n}) = n_i, \forall \ \tilde{n} \in N^k$

c) Funcţia succesor $\sigma: N^k \to N$, $\sigma(n) = n + 1$, $n \in N$

d) O funcţie iniţială este ξ, σ sau una dintre π_i^k.

Definiţia 4.4

Compunere şi recursivitate primitivă.

a) Fie $l > 0$, $k \geq 0$, g o funcţie de ordinul l şi $h_1, \ldots, h_l$ funcţii de ordinul k. Fie f o funcţie de ordinul k astfel încât $\forall \ \tilde{n} \in N^k$, $f(\tilde{n}) = g(h_1(\tilde{n}), \ldots, h_l(\tilde{n}))$. Spunem că f este obţinută din funcţiile $g, h_1, \ldots, h_l$ prin compunere.

b) Fie $k \geq 0$, g o funcţie de ordinul k, h o funcţie de ordinul $(k + 2)$. Fie f o funcţie de ordinul $(k + 1)$ astfel încât $\forall \ \tilde{n} \in N^k$, $f(\tilde{n}, 0) = g(\tilde{n})$ şi $\forall \ \tilde{n} \in N^*$, $m \in N$, $f(\tilde{n}, m+1) = h(\tilde{n}, m, f(\tilde{n}, m))$.

Atunci spunem că f este obţinută din g şi h prin recursivitate primitivă.

Definiţia 4.5

Funcţii primitiv recursive.
O funcţie este primitiv recursivă dacă este o funcţie iniţială sau poate fi obţinută dintr-o funcţie iniţială printr-o secvenţă de operaţii de compunere şi recursivitate primitivă.
Altfel spus, funcţiile primitive recursive formează cea mai mică clasă de funcţii ce conţine funcţiile iniţiale şi este închisă în raport cu compunerea şi recursivitatea primitivă.

Exemplu 4.5

Funcţia $\sigma^2 : N \to N$, $\sigma^2(n) = n + 2$, $\forall\ n \in N$ este primitiv recursivă. Fie $k = l = 1$, $g = h_1 = \sigma$:

$$\sigma^2(n) = g(h_1(n_1)) = \sigma(\sigma(n))$$

Funcția σ este o funcţie iniţială, prin urmare este primitiv recursivă. Funcția σ^2 este obţinută din funcţii primitiv recursive prin compunere deci este primitiv recursivă.

Ce justifică ideea, că la nivel intuitiv, orice funcţie primitiv recursivă este calculabilă? Funcţiile iniţiale sunt calculabile. Dacă f este obţinută din $g, h_1, \ldots, h_l$ prin compunere şi funcțiile $g, h_1, \ldots, h_l$ sunt calculabile atunci $f(\tilde{n})$ poate fi evaluată calculând mai întâi $h_1(\tilde{n}), \ldots, h_l(\tilde{n})$; obţinem valorile $m_1, \ldots, m_l$ şi apoi calculăm $g(m_1, \ldots, m_l)$.

Similar, dacă f este obţinută din g şi h prin recursivitate primitivă iar g şi h sunt calculabile atunci $f(\tilde{n}, m)$ poate fi evaluată astfel:

- calculăm $f(\tilde{n}, 0)$, adică $g(\tilde{n})$; dacă $m = 0$, am terminat; altfel fie $g(\tilde{n}) = p_0$
- calculăm $f(\tilde{n}, 1)$, adică $h(\tilde{n}, 0, p_0)$; fie p_1 rezultatul obținut; dacă $m = 1$, am terminat; altfel continuăm calculul
- calculăm $f(\tilde{n}, 2)$, adică $h(\tilde{n}, 1, p_1)$ …

Definiţia 4.6

Funcţii constante.

$\forall\ k \geq 0, j > 0$ definim a j-a funcţie constantă de ordinul k

$K_j^k : \mathrm{N}^k \to \mathrm{N}$, $K_j^k(\tilde{n}) = j$, $\forall\ \tilde{n} \in \mathrm{N}$

Lema 4.2

Funcţiile constante sunt primitiv recursive.

Demonstraţie. Vom demonstra prin inducţie dupa k.

$k = 0$: K_0^0 este ξ, care este o funcţie iniţială;

$K_{j+1}^0() = \sigma(K_j^0())$, $\forall\ j \geq 0$,

astfel K_{j+1}^0 este obţinut din σ şi K_j^0 prin compunere.

$k > 0$: dacă pentru un anumit k, K_j^k este primitiv recursivă $\forall\ j \geq 0$, atunci $\forall\ j \geq 0$ funcţia K_j^{k+1} este primitiv recursivă.

$K_j^{k+1}(\tilde{n}, 0) = K_j^k(\tilde{n})$

$K_j^{k+1}(\tilde{n}, m + 1) = \Pi_{k+2}^{k+2}(\tilde{n}, m, K_j^{k+1}(\tilde{n}, m))$

adică K_j^{k+1} este obţinut din K_j^{k+1} şi Π_{k+2}^{k+2} prin recursivitate primitivă. ■

Definiţia 4.7

Fie g o funcţie de ordinul k şi f o funcţie de ordinul l, unde $k, l \geq 0$. Presupunem că există funcţiile $h_1, \ldots, h_k : \mathrm{N}^l \to \mathrm{N}$, fiecare fiind o funcţie de proiecţie de ordinul l sau o funcţie constantă de ordinul l astfel încât $\forall\ \tilde{n} \in \mathrm{N}^l$,

$f(\tilde{n}) = g(h_1(\tilde{n}), \ldots, h_k(\tilde{n}))$.

Astfel f este obţinută din g prin transformare explicită.

Cu alte cuvinte, funcţia f este obţinută din g prin transformare explicită dacă f este obţinută din g prin duplicare, permutare sau substituire a valorii constante pentru argumentele lui g.

Lema 4.3

Dacă g este primitiv recursivă, atunci orice funcţie obţinută din g prin transformare explicită este primitiv recursivă.

Demonstraţie. Demonstraţia este imediată pentru că proiecţia precum şi funcţiile constante sunt primitive recursive. ■

Definiţia 4.8

a) Un predicat este o relaţie pe numerele naturale, adică o submulţime a lui N^k, pentru $k \geq 0$ (un astfel de predicat, este un predicat de ordin k). Notăm $P\tilde{n}$ pentru a indica $\tilde{n} \in P$.
b) Funcţia caracteristică a unui predicat P de ordinul k, este o funcţie de ordin k, $f : N^k \to \{0, 1\}$, $\forall\, \tilde{n} \in N^k$

$$f(\tilde{n}) = \begin{cases} 1, & P\tilde{n} \\ 0, & not\ P\tilde{n} \end{cases}$$

c) Un predicat este primitiv recursiv dacă şi numai dacă funcţia sa caracteristică este primitiv recursivă.

Lema 4.4

Dacă Q este un predicat primitiv recursiv atunci orice predicat obţinut din Q prin transformare explicită este primitiv recursiv.

Definiţia 4.9

Fie $k, l \geq 0$. Dacă Q este un predicat de ordinul k şi R este un predicat de ordinul l atunci conjuncţia lor este un predicat S de ordin $(k+l)$ astfel încât

$S\tilde{n}\tilde{m}$ dacă şi numai dacă $Q\tilde{n} \wedge R\tilde{m}$, $\forall\, \tilde{n} \in N^k$, $\forall\, \tilde{m} \in N^l$.

Similar, disjuncţia este un predicat T de ordinul $k+l$, astfel încât

$T\tilde{n}\tilde{m}$ dacă şi numai dacă $Q\tilde{n} \vee R\tilde{m}$, $\forall\, \tilde{n} \in N^k$, $\forall\, \tilde{m} \in N^l$.

Negaţia Q este o relaţie P de ordinul k, astfel încât

$P\tilde{n}$ dacă şi numai dacă $not\ Q\tilde{n}$.

Lema 4.5

Dacă Q și R sunt predicate primitiv recursive atunci conjuncţia, disjuncţia, negaţia lor sunt primitiv recursive.

Demonstraţie. Fie g și h funcţiile caracteristice primitiv recursive ale lui Q, R; definim f_1, f_2, f_3, funcţiile caracteristice ale conjuncţiei $Q \wedge R$, disjuncţiei $Q \vee R$, și negației $\bar{Q}$ unde

$f_1(\tilde{n}, \tilde{m}) = g(\tilde{n}) \infty h(\tilde{n}, \tilde{m})$

$f_2(\tilde{n}, \tilde{m}) = \mathrm{sg}(g(\tilde{n}) + h(\tilde{n}, \tilde{m}))$

$f_3(\tilde{n}) = 1 - g(\tilde{n})$ ■

Definiţia 4.8

Fie $k \geq 0$. Dacă Q este un predicat de ordinul $k + 1$ atunci cuantificarea sa existenţială este un predicat R de ordinul $k + 1$, astfel încât $\forall\ \tilde{n} \in \mathrm{N}^k$ şi $m \in \mathrm{N}$, $R\tilde{n}m$ dacă și numai dacă $\exists\ i$, $0 \leq i \leq m$, $Q\tilde{n}i$ este adevărat. Vom scrie:

$R\tilde{n}\tilde{m}$ dacă și numai dacă $\exists\ i \leq m[Q\tilde{n}i]$

Similar, cuantificarea universală a predicatului Q ste un predicat S de ordinul $k + 1$, astfel încât $\forall\ \tilde{n} \in \mathrm{N}^k$ şi $m \in \mathrm{N}$, $S\tilde{n}m$ dacă și numai dacă $\forall\ i$, $0 \leq i \leq m$, $Q\tilde{n}i$ este adevărat. Vom scrie:

$S\tilde{n}\tilde{m}$ dacă și numai dacă $\forall\ i \leq m[Q\tilde{n}i]$

Lema 4.6

Dacă Q este un predicat primitiv recursiv atunci cuantificările sale existenţiale şi universale sunt primitiv recursive.

Demonstraţie. Fie g funcţia caracteristică a lui Q; f_1, f_2 funcţiile caracteristice ale cuantificatorilor universal şi existențial

$$f_1(\tilde{n}, m) = \prod_{i=0}^{m} g(\tilde{n}, i)$$

$$f_2(\tilde{n}, m) = 1 - \prod_{i=1} (1 - g(\tilde{n}, i))$$

■

4.3. Gödelizare

Pentru a demonstra că am construit clasa funcţiilor care includ toate funcţiile calculabile gramatical, respectiv toate funcţiile calculabile Turing trebuie să mai facem doi paşi,

- extindem clasa funcţiilor primitiv recursive
- transformăm funcţiile definite pe mulţimea numerelor naturale în funcţii pe şiruri ($f : \mathrm{N} \to \mathrm{N} \Rightarrow f : \Sigma_0^* \to \Sigma_1^*$)

Fie Δ un alfabet şi $\beta = |\Delta| + 1$. Fixăm o anumită ordine a simbolurilor în Δ astfel încât $\Delta = \{d_1, \ldots, d_{\beta-1}\}$. Orice şir din Δ^* poate fi văzut ca un întreg în baza β.

Definim $g_n : \Delta^* \to \mathrm{N}$. Dacă $w = d_{i1}, \ldots, d_{ik}$, $k \geq 0$, $1 \leq i_j \leq \beta - 1$, $j = 1, \ldots, k$,

$$g_n(w) = \beta^{k-1} \infty i_1 + \beta^{k-2} \infty i_2 + \ldots + \beta^1 \infty i_{k-1} + i_k.$$

$$g_n(e) = 0$$

Spunem că $g_n(w)$ este numărul Gödel al şirului w.

Observaţie. g_n reprezintă codificarea şirurilor în numere; un şir corespunde unui anumit număr (unul singur), dar g_n nu este o bijecţie: un număr (de exemplu β) nu corespunde niciunui şir. Introducem un nou simbol d_0 corespunzând cifrei 0; g_n poate fi extinsă pentru a deveni o bijecţie,

$$g_n : \underbrace{\{e\} \cup \Delta(\Delta \cup \{d_0\})^*}_{\mathcal{D}} \to \mathrm{N}$$

Inversa este acum bine definită:

$$g_n^{-1} : \mathrm{N} \to \mathcal{D}$$

4.4. Funcţii μ-recursive

Definiţia 4.9

Fie $k \geq 0$ şi g o funcţie de ordinul $(k + 1)$. Minimizarea nemărginită a lui g este o funcţie f de ordinul k, astfel încât $\forall \tilde{n} \in N^k$,

$$f(\tilde{n}) = \begin{cases} \text{cel mai mic } m \text{ astfel încât } g(\tilde{n}, m) = 0, \text{ dacă un astfel de } m \text{ există} \\ 0, \text{ altfel} \end{cases}$$

A doua clauză garantează că funcția f este definită peste tot, indiferent de g. Scriem

$f(\tilde{n}) = \mu m[g(\tilde{n}, m)]$ şi spunem că f este obţinut din g prin minimizare nemărginită.

În general, minimizarea nemărginită a unei funcţii primitiv recursive nu trebuie să fie primitiv recursivă. Motivul este acela că nu există nicio metodă pentru a determina dacă un m de tipul cerut există. Totuşi, dacă g are proprietatea că un astfel de m există pentru orice $\tilde{n}$, atunci f este calculabilă şi g este calculabilă. Dându-se $\tilde{n}$, trebuie să evaluăm $g(\tilde{n}, 0)$, $g(\tilde{n}, 1)$, …, până când găsim un m astfel încât $g(\tilde{n}, m)=0$. În acest caz f nu trebuie să fie, în general, primitiv recursivă.

Definiţia 4.10

O funcţie g de ordinul $(k+1)$ este regulată dacă şi numai dacă pentru orice $\tilde{n} \in N^k$, există m astfel încât $g(\tilde{n}, m) = 0$. O funcţie este μ-recursivă dacă şi numai dacă poate fi obţinută din funcţiile iniţiale ξ, π_i^k, σ, prin următoarele operaţii:

- compunere
- recursivitate primitivă
- aplicarea minimizării nemărginite funcţiilor regulate.

Astfel, orice funcţie primitiv recursivă este μ-recursivă. Vom arăta că orice funcţie calculabilă gramatical este μ-recursivă. Vom considera un alfabet Δ şi codificarea Gödel g_n fixate ($\$ \in \Delta$). Fie $\Sigma_0, \Sigma_1 \subseteq \Delta - \{\$\}$.

Definiţia 4.11

O funcţie $f : \Sigma_0^* \to \Sigma_1^*$ este μ-recursivă dacăşi numai dacă funcţia f: N $\to$ N este μ-recursivă, unde $\forall n \in$ N

$$f'(n) = \begin{cases} gn(f(gn^{-1}(n))), & \text{dacă } gn^{-1}(n) \in \Sigma_0^* \\ 0, & \text{dacă } gn^{-1}(n) \notin \Sigma_0^* \end{cases}$$

Cu alte cuvinte, o funcţie f definită pe şiruri este μ-recursivă dacă funcţia corespunzătoare f' definită pe numere Gödel este μ-recursivă. Dacă argumentul nu este un număr Gödel, se consideră că f' are valoarea 0.

Teorema 4.2

Orice funcţie calculabilă gramatical definită pe şiruri sau pe numere este μ-recursivă.

4.5. Calculabilitatea Turing a funcţiilor μ-recursive

Vrem să arătăm că orice funcţie μ-recursivă poate fi calculată de o maşină Turing. Considerăm următoarele maşini Turing:

- S_L – deplasare stânga – transformă $\#w\underline{\#}$ în $w\underline{\#}$ (w nu conţine simboluri $\#$)
- S_R – deplasare dreapta – transformă $\#w\underline{\#}$ în $\#\#w\underline{\#}$ (w nu conţine simboluri $\#$)
- E – maşina de ştergere – transformă $\#w\underline{\#}$ în $\underline{\#}$ (w poate fi vid dar nu conţine simboluri $\#$)

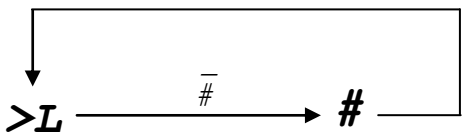


Figura 4-1

- *B* – maşina de backup – transformă $\#u\#w\underline{\#}$ în $\#w\#$; u, w pot fi vide, dar nu conţin simboluri $\#$

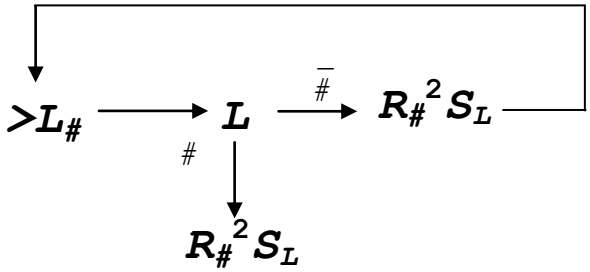


Figura 4-2

- C_k – maşina de copiere – produce o copie a şirului al k-lea de la stânga sa $(k \geq 1)$. Dacă şirurile w_1, …, w_k nu conţin simboluri $\#$, C_k transformă $\#w_1\#w_2\#\ldots\ \#w_k\underline{\#}$ în $\#w_1\#w_2\#\ldots\#w_k\#w_1\underline{\#}$

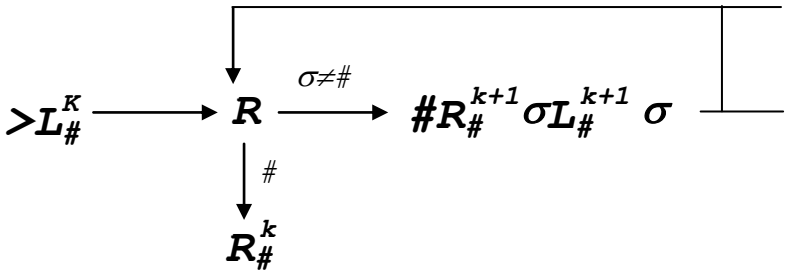


Figura 4-3

Observaţie. Maşina Turing C_k^l ($C_kC_k \ldots C_k$, l copii în total), $l \leq k$ are efectul de a copia şirurile k până la $k-l-1$ de la stânga poziţiei curente; transformă $\#w_1\#w_2\#\ldots\ \#w_k\underline{\#}$ în $\#w_1\#w_2\#\ldots\#w_k\#w_1\#w_2\#\ldots\#w_l\underline{\#}$

Teorema 4.3

Orice funcţie μ-recursivă definită pe numere sau pe şiruri este calculabilă Turing.

Demonstraţie. Vom începe prin a considera funcţii μ-recursive definite pe numere (cazul funcţiilor definite pe şiruri este prezentat printr-un corolar). Funcţiile μ-recursive sunt cele care pot fi obţinute din funcţiile iniţiale ξ, π_i^k, σ prin compunerea, recursivitatea primitivă şi minimizarea nemărginită aplicate funcţiilor regulate. Demonstraţia că orice funcţie μ-recursivă este calculabilă Turing, presupune demonstraţia că funcţiile iniţiale sunt calculabile Turing şi apoi demonstraţia că rezultatul compunerii, recursivităţii primitive şi a minimizării nemărginite aplicate funcţiilor calculabile Turing sunt calculabile Turing. Demonstraţia constă în demonstrarea mai multor leme.

Lema 1

Funcţiile iniţiale sunt calculabile Turing.

Demonstraţie.

a) Funcţia ξ *este* calculată de maşina Turing R care îşi mută capul o poziţie la dreapta şi se opreşte. Maşina porneşte din configuraţia iniţială $(s, \underline{\#})$ şi se opreşte în configuraţia finală $(h, \#\underline{\#})$, calculând funcţia ξ: $\mathrm{N}^0 \to \mathrm{N}^1$.

b) Funcţia de proiecţie π_i^k , $k \geq 1$, $1 \leq i \leq k$ este calculată de maşina Turing $E^{k-i}B^{i-1}$. E^{k-i} transformă $\#w_1\#w_2\#\ldots\#w_k\underline{\#}$ în $\#w_1\#w_2\#\ldots\#w_i\underline{\#}$, după care B^{i-1} transformă în $\#w_i\underline{\#}$. Maşinile E^0 $(k = i)$ şi B^0 $(i = 1)$ sunt considerate maşini triviale care se opresc imediat.

c) Funcţia succesor σ este calculată de maşina Turing IR care transformă $\#I^n\underline{\#}$ în $\#I^{n+1}\underline{\#}$, $(n \geq 0)$ ■

Lema 2

Fie $l > 0$, $k \geq 0$, g o funcţie calculabilă Turing de ordinul l, şi h_1, …, h_l funcţii calculabile Turing de ordinul k. Dacă f este obţinută din g, h_1, …, h_n prin compunere atunci f este calculabilă Turing.

Demonstraţie. Fie G, H_1 … H_l maşini Turing care calculează g, h_1 … h_l. Funcţia f, $f(\tilde{n}) = g(h_1(\tilde{n}), \ldots, h_l(\tilde{n}), \forall\ \tilde{n} \in \mathrm{N}$ este calculată de maşina Turing

$$C_k^k H_1 C_{k+1}^k H_2 \ldots C_{k+l-1}^k H_1 G B^k$$

$C_k^k H_1$ transformă

$\#w_1\#w_2\#...\#w_k\underline{\#}$ în

$w_1\#w_2\#...\#w_k\#w_1\#w_2\#...\#w_k\underline{\#}$ şi apoi în

$\#w_1\#w_2\#...\#w_k\#u_1\underline{\#}$, unde $w_i = I^{ni}$, $i = 1, \ldots, k$, $u_1 = I^{h1(\tilde{n})}$.
Similar, după $C_k^k H_1 C_{k+1}^k H_2 \ldots C_{k+1-1}^k H_1$ banda va fi

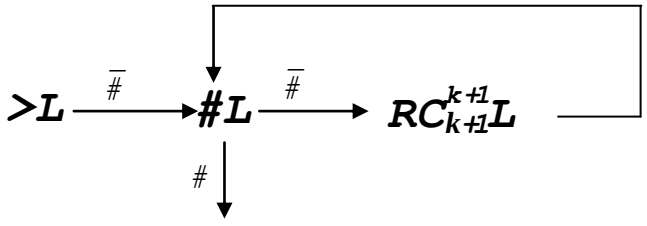
$\#w_1\#w_2\#...\#w_k\#u_1\#u_2\#...\#u_1\underline{\#}$
unde $u_i = I^{hi(\tilde{n})}$, $i = 1, \ldots, l$, care este transformat în

$\#w_1\#w_2\#...\#w_k\#v\underline{\#}$
de G şi apoi în $\#v\underline{\#}$ de B^k, unde $v = I^{f(\tilde{n})}$ ■

Lema 3

Fie $k \geq 0$, g o funcţie calculabilă Turing de ordinul k, şi h o funcţie calculabilă Turing de ordinul $k+2$. Dacă f este o funcţie obţinută din g şi h prin recursivitate primitivă atunci f este calculabilă Turing.

Demonstraţie. Fie G, H maşini Turing care calculează funcţiile g şi h. Construim o maşină Turing F care calculează f unde $\forall\ \tilde{n} \in N^k$, $f(\tilde{n}, 0) = g(\tilde{n})$ şi $\forall\ \tilde{n} \in N^k$, $m \in N^k$, $f(\tilde{n}, m+1) = h(\tilde{n}, m, f(\tilde{n}, m))$. F primeşte $\#w_1\#w_2\#...\#w_k\#u\underline{\#}$, unde pentru $n_1, \ldots, n_k, m \in N$, $w_i = I^{ni}$, $i = 1, \ldots, k$ şi $u = I^m$. Notăm $\overline{w} = w_1\#w_2\#...\#w_k$. F adaugă un simbol $ ca limită stânga a intrării, unde $ este simbol nou, neutilizat de G sau H. Intrarea este transformată în $\#\overline{w}\#u\#$, aplicând maşina Turing $F_1 = (S_R L_\#^2)^{k+1} \$ R_\#^{k+1}$. Conţinutul benzii devine $\#\overline{w}\#I^{m-1}\#\overline{w}\#I^{m-2}\# \ldots \#\overline{w}\#I^1\#\overline{w}\#I^0\#\overline{w}\underline{\#}$, adică $\# urmat de $\#\overline{w}\#I^i\#$, $i = m-1, m-2, \ldots, 0$, urmat de $\overline{w}\#$ cu capul la limita dreapta (dacă $m = 0$ atunci şirul este $\#\overline{w}\underline{\#}$; dacă $m = 1$ atunci şirul este $\#\overline{w}\#\#\overline{w}\underline{\#}$). Maşina Turing care relizează aceste transformări este maşina F_2.



$$RC_{k+1}^{k}$$

Figura 4-4

În final, mașinile Turing G şi H sunt utilizate pentru a calcula valori succesive $f(\tilde{n}, i)$, $i = 0, ..., m$. Notăm v_i pentru $I^{f(\tilde{n}, i)}$. Banda va arăta succesiv astfel:

$\$\#\overline{w}\#I^{m-1}\#\overline{w}\#I^{m-2}\#...\#\overline{w}\#I^{1}\#\overline{w}\#I^{0}\#v_0\underline{\#}$

$\$\#\overline{w}\#I^{m-1}\#\overline{w}\#I^{m-2}\#...\#\overline{w}\#I^{1}\#v_1\underline{\#}$

.
.
.

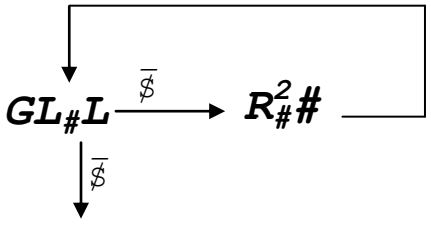
$\$\#\overline{w}\#I^{m-1}\#\overline{w}\#I^{m-2}\#...\#\overline{w}\#I^{i}\#v_i\#$

.
.
.

$\$\#\overline{w}\#I^{m-1}\#v_{m-1}\#$

$\$\#v\#$

La primul pas, $\overline{w}$ este eliminat de la capătul dreapta şi înlocuit prin şirul v_0 de mașina Turing G. La fiecare pas $\overline{w}\#I^{i}\#v_i$ este eliminat din dreapta şi înlocuit cu v_{i+1} de mașina Turing H. Înainte de inițierea fiecărui ciclu, simbolul aflat la stânga celui mai din dreapta şir de pe bandă este verificat; dacă nu este $ trebuie executat alt ciclu; dacă este $ operațiile sunt terminate, trebuie doar deplasat rezultatul la stânga pentru a elimina $. Această parte este executată de mașina Turing F_3.



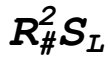


Figura 4-5

Maşina Turing care calculează f este $F_1F_2F_3$. ■

Lema 4

Fie g o funcţie regulată calculabilă Turing de ordinul $k+1$. Dacă f este o funcţie obţinută din g prin minimizare nemărginită atunci f este calculabilă Turing.

Demonstraţie. Fie G maşina Turing care calculează g. Construim o maşină Turing F care calculează f, unde $\forall\ \tilde{n} \in N^k$,

$f(\tilde{n})$ = cel mai mic m astfel încât $g(\tilde{n}, m)=0$.

Cum g este regulată atunci este garantată existenţa unui m pentru orice $\tilde{n}$. Maşina F primeşte intrarea în forma $\#w_1\#w_2\#...\#w_k\underline{\#}$, o transformă în $\#w_1\#w_2\#...\#w_k\#\underline{\#}$ (adică $\#w_1\#w_2\#...\#w_K\#I^0\#$), copiază întregul şir şi aplică G. Dacă rezultatul este diferit de 0 este şters şi un I este adăugat la dreapta şirului rămas pe bandă, care devine $\#w_1\#w_2\#...\#w_K\#I^1\underline{\#}$. Şirul este copiat şi maşina G este aplicată şirului $\#w_1\#w_2\#...\#w_k\#I^m\underline{\#}$. Maşina Turing B este utilizată pentru a elimina argumentele $w_1, ..., w_k$ şi lasă I^m pe bandă. Procesul se repetă până când se obţine 0. Specificarea lui F este următoarea.

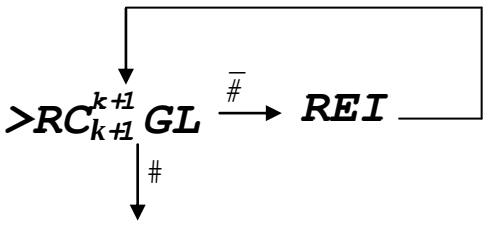




Figura 4-6

Lemele 1 - 4 demonstrează teorema pentru funcţii definite pe mulțimea numerelor naturale. Pentru a încheia demonstraţia, fie f o funcţie μ -recursivă definită pe şiruri. Atunci funcţia f' : N $\rightarrow$ N este μ -recursivă, unde pentru orice n astfel încât $gn^{-1}(n)$ este un şir în domeniul lui f, $f'(n) = gn(f(gn^{-1}(n)))$.
Prin argumentul dat f' este calculabilă Turing; presupunem că f' etse calculată de maşina Turing F'. Pentru a arăta că f este Turing-calculabilă putem proiecta o maşină Turing care primind un şir n din domeniului lui f.

- transformă $\#w\underline{\#}$ în $\#I^{gn(w)}\underline{\#}$ (converteşte şirul în reprezentarea unară a numărului Gödel asociat);
- utilizează F' pentru a transforma $\#I^{g(n)}\underline{\#}$ în $\#I^{f'(gn(w))}\underline{\#}$ adică în $I^{gn(f(w))}\underline{\#}$;
- transformă $\#I^{gn(f(w))}\underline{\#}$ în $\#f(w)\underline{\#}$, adică converteşte reprezentarea numărului Gödel a unui şir în şirul respectiv. ■

Singurele elemente care lipsesc sunt maşinile Turing care convertesc şiruri în/din reprezentări unare ale numărului Gödel (aceste maşini Turing sunt lăsate ca exerciţiu cititorului).

Demonstraţia că funcţiile calculabile Turing, calculabile gramatical şi μ-recursive sunt echivalente este completă. Aceste funcţii sunt numite funcţii recursive.

BIBLIOGRAFIE

1. **Brainerd, W. S., Landweber, H.**, *Theory of Computation*. New York: John Wiley, 1974.
2. **Church, A.**, *An Unsolvable Problem of Elementary Number theory*. American Journal of Mathematics, 58 (1936), 345-363.
3. **Hennie, F.,C.**, *Introduction to Computability*. Reading, Mass: Addison-Wesley, 1977.
4. **Hopcroft., J., E., Motwani, R., Ullman, J., D.**, *Introduction to Automata theory Languages, and Computation*. Addison-Wesley, 2001.
5. **Kleene, S., C.**, *General Recursive Functions of Natural Numbers*. Mathematische Annalen, 112 (1936), 727-742.
6. **Lewis, H., R., Papadimitriou, C., H.**, *Elements of the Theory of Computation*. N.J.: Prentice-Hall, 1998.
7. **Minsky, M., L.**, *Computation*: *Finite and Infinite Machines*. Englewood Cliffs, N.J.: Prentice-Hall, 1967.
8. **Papadimitriou, C., H., Steiglitz, K.**, *Combinatorial Optimization, Algorithms and complexity*. Mineola, New York: Dover Publications, Inc., 1982, 1998.
9. **Rogers Jr., H.**, *The Theory of Recursive Functions and Effective Computability*. New York: McGraw-Hill, 1967.

Nota: formalizarea folosită precum și exemplele prezentate se bazează pe cele prezentate în (4), și (6).

Cuprins

1 Noțiuni generale **3**

1.1 Alfabet, Șir, Limbaj . . . 3

1.1.1 Alfabet . . . 3

1.1.2 Șir . . . 3

1.1.3 Limbaj . . . 3

1.2 Operații pe limbaje . . . 4

1.2.1 Reuniune . . . 4

1.2.2 Concatenare . . . 4

1.2.3 Kleene Star . . . 4

1.2.4 Mulțimea putere . . . 5

1.3 Ierarhia Chomsky . . . 5

2 Limbaje regulate **6**

2.1 Expresii regulate . . . 6

2.1.1 Utilizări . . . 7

2.2 Automate finite deterministe . . . 7

2.2.1 Utilizări . . . 8

2.3 Automate finite nedeterministe . . . 8

2.3.1 Utilizări . . . 10

2.4 Echivalența ER, AFD, AFN . . . 10

2.4.1 Conversie AFD → AFN . . . 10

2.4.2 Conversie AFN → AFD . . . 10

2.4.3 Conversie ER → AFN . . . 11

2.5 Proprietăți algoritmice . . . 12

2.5.1 Închidere în raport cu reuniunea . . . 12

2.5.2 Închidere în raport cu concatenarea . . . 12

2.5.3 Închidere în raport cu Kleene Star . . . 13

2.5.4 Închidere în raport cu diferența . . . 13

2.5.5 Închidere în raport cu intersecția . . . 14

2.5.6 Închidere în raport cu complementarea . . . 15

2.5.7 Decizia dacă un limbaj regulat este vid . . . 15

2.5.8 Decizia dacă un șir este într-un limbaj regulat . . . 15

2.5.9 Decizia dacă un limbaj regulat este finit . . . 16

2.6 Lema de pompare . . . 16

2.6.1 Enunț . . . 16

2.6.2 Negația Lemei de pompare . . . 17

2.6.3 Utilizare 17

3 Limbaje independente de context **18**

3.1 Gramatici independente de context 18
3.1.1 Utilizări 19
3.2 Automate Pushdown nedeterministe 19
3.2.1 Utilizări 20
3.3 Echivalența GIC, APD 20
3.3.1 Conversia GIC → APD 20
3.3.2 Conversia APD → GIC 20
3.4 Proprietăți algoritmice 20
3.4.1 Închiderea în raport cu reuniunea 20
3.4.2 Închiderea în raport cu concatenarea 21
3.4.3 Închiderea în raport cu Kleene Star 22
3.4.4 Clasa limbajelor independente de context nu este închisă în raport cu diferența, intersecția și complementarea . . 23
3.4.5 Decizia dacă un limbaj independent de context este vid 23
3.5 Lema de pompare 24
3.5.1 Enunț 24
3.5.2 Negația Lemei de pompare 24
3.5.3 Utilizare 24

1 Noțiuni generale

1.1 Alfabet, Șir, Limbaj

1.1.1 Alfabet

Un alfabet este orice mulțime nevidă finită de simboluri. Acestea pot fi caractere, semnale sau orice obiect ce poate fi interpretat cumva.
Exemple:

- alfabetul roman: $\Sigma = \{A, B, C, ..., Z\}$
- alfabetul format din cifre: $\Sigma = \{0, 1, 2, ..., 9\}$
- alfabetul semnalelor pentru 3-Way Handshake: $\Sigma = \{$SYN, ACK, SYN-ACK$\}$

1.1.2 Șir

Un șir(cuvânt) este o secvență finită de simboluri dintr-un alfabet. Un șir poate să nu conțină niciun caracter. Acesta, șirul vid, se notează cu e(în alte surse se notează cu ε).
Lungimea unui șir w se notează cu $|w|$ și reprezintă numărul de simboluri din care e format w.
Exemple:

- $\Sigma = \{a, b, c, d\}, w = cbba, |w| = 4$
- $\Sigma = \{0, 1\}, w = 10010101, |w| = 8$
- $\Sigma = \{a, b\}, w = e, |w| = 0$

1.1.3 Limbaj

Un limbaj peste un alfabet Σ este o mulțime de cuvinte formate din simboluri din Σ. Limbajul vid se notează cu $\emptyset$, și este limbajul în care nu se află niciun cuvânt. Acesta este diferit de limbajul care conține doar șirul vid. Un limbaj poate fi finit sau infinit.
Exemple:

- $L = \emptyset, |L| = 0$

- $L = \{e\}, |L| = 1$
- $L = \{e, a, ab\}, |L| = 3$
- $L = \{a, aa, aaa, ...\}, |L| = \infty$

1.2 Operații pe limbaje

1.2.1 Reuniune

Reuniunea a două limbaje este reuniunea normală definită pe mulțimi.

1.2.2 Concatenare

Fie L_1, L_2, două limbaje. Concatenarea se definește în felul următor:
$L_1 \circ L_2 = \{w_1w_2 | w_1 \in L_1 \wedge w_2 \in L_2\}$.
Mai exact, orice concatenare a unui cuvânt din L_1 cu un cuvânt din L_2 se află în $L_1 \circ L_2$. Concatenarea se poate nota mai simplu L_1L_2.
Concatenarea nu este comutativă.
Concatenarea este asociativă și distributivă la stânga, respectiv la dreapta față de reuniune.
Exemple:

- $L_1 = \emptyset$, $L_2 = \{e, a, b\}$. $L_1L_2 = \emptyset$, fiindcă nu există niciun cuvânt în L_1 care să fie concatenat cu vreun cuvânt din L_2
- $L_1 = \{a, b\}$, $L_2 = \{c, d\}$. $L_1L_2 = \{ac, ad, bc, bd\}$
- $L_1 = \{e, a\}$, $L_2 = \{b, bb, bbb, ...\}$. $L_1L_2 = \{b, ab, bb, abb, ...\}$

1.2.3 Kleene Star

Fie L un limbaj. Kleene Star se definește în felul următor:
$L^* = \{w | w = w_1w_2...w_n \wedge w_i \in L, n \geq 0\}$
Kleene Star aplicat pe un limbaj reprezintă concatenarea de zero sau mai multe ori a oricâtor șiruri din L.
Fie Σ un alfabet oarecare. Σ este o mulțime finită de simboluri. Acesta poate fi tratat ca un limbaj format din șiruri de lungime 1. Aplicând Kleene Star pe acest limbaj, obținem Σ^*, mulțimea tuturor șirurilor formate din simboluri din Σ.

Deoarece L^* conține concatenarea tuturor cuvintelor de zero sau mai multe ori, L^* îl va conține întotdeauna pe e.
Exemple:

- $L = \emptyset$. $L^* = \{e\}$.
- $L = \{e\}$. $L^* = \{e\}$.
- $L = \{a\}$. $L^* = \{e, a, aa, aaa, ..\}$
- $L = \{a, b\}$. $L^* = \{e, a, b, aa, ab, ba, bb, aaa, ...\}$

1.2.4 Mulțimea putere

Fie M o mulțime. Notăm cu 2^M mulțimea tuturor submulțimilor lui M.
$2^M = \{A | A \subseteq M\}$.
Exemplu: $M = \{0, 1\}$. $2^M = \{\emptyset, \{0\}, \{1\}, \{0, 1\}\}$
Dacă Σ este un alfabet, Σ^* reprezintă mulțimea tuturor șirurilor peste acel alfabet și 2^{Σ^*} este mulțimea tuturor limbajelor peste acel alfabet. Σ este finit. Σ^* este infinit numărabil. 2^{Σ^*} este infinit și nenumărabil.

1.3 Ierarhia Chomsky

Ierarhia constă în patru mari clase de limbaje:

1. Limbaje regulate
2. Limbaje independente de context
3. Limbaje dependente de context
4. Limbaje zero

2 Limbaje regulate

Clasa limbajelor regulate este mulțimea minimă de limbaje închisă în raport cu operațiile de reuniune, concatenare și Kleene Star.

2.1 Expresii regulate

Expresiile regulate reprezintă o metodă de reprezentare finită a limbajelor. Ele pot descrie orice succesiune finită de operații de reuniune, concatenare și Kleene Star.

Fie Σ un alfabet. O expresie regulată este o secvență finită de simboluri din $\Sigma \cup \{\cup, ^*, (,), \varnothing, e\}$, ținând cont de următoarele proprietăți:

1. $\varnothing$ și e sunt expresii regulate, reprezentând limbajul vid, $L_1 = \emptyset$, respectiv limbajul care conține doar șirul vid, $L_2 = \{e\}$;

2. $\forall a \in \Sigma$, a este expresie regulată, reprezentând limbajul ce conține un singur cuvânt format din simbolul a, $L = \{a\}$;

3. $\forall \alpha, \beta$ expresii regulate, $\alpha \cup \beta$ este expresie regulată, reprezentând reuniunea limbajelor descrise de expresiile regulate α și β, $\mathcal{L}(\alpha \cup \beta) = \mathcal{L}(\alpha) \cup \mathcal{L}(\beta)$;

4. $\forall \alpha, \beta$ expresii regulate, $\alpha\beta$ este expresie regulată, reprezentând concatenarea limbajelor descrise de expresiile regulate α și β, $\mathcal{L}(\alpha\beta) = \mathcal{L}(\alpha) \circ \mathcal{L}(\beta)$;

5. $\forall \alpha$ expresie regulată, α^* este expresie regulată, reprezentând aplicarea operatorului Kleene Star limbajului descris de expresia regulată α, $\mathcal{L}(\alpha^*) = \mathcal{L}(\alpha)^*$;

6. $\forall \alpha$ expresie regulată, (α) este expresie regulată. Parantezele cresc prioritatea operatorilor. Operatorii, ordonați de la prioritate maximă la minimă sunt: Kleene Star, operator de concatenare, operator de reuniune;

7. Orice altceva nu este expresie regulată.

Limbajele acceptate de expresii regulate se numesc limbaje regulate.
Notații ajutătoare:

- Fie α o expresie regulată. Notăm $\alpha^+ = \alpha\alpha^*$.
- Fie α o expresie regulată. Notăm $\alpha? = (\alpha \cup e)$

Exemple:

- $E = e \cup ab \cup bb^*$. $\mathcal{L}(E) = \mathcal{L}(e) \cup \mathcal{L}(ab) \cup \mathcal{L}(bb*) = \{e\} \cup \{ab\} \cup \mathcal{L}(b) \circ \mathcal{L}(b^*) = \{e, ab\} \cup \{b\} \circ \mathcal{L}(b)^* = \{e, ab\} \cup \{b\} \circ \{b\}^* = \{e, ab\} \cup \{b\} \circ \{e, b, bb, bbb, ...\} = \{e, ab\} \cup \{b, bb, bbb, bbbb, ...\} = \{e, ab, b, bb, bbb, ...\}$;
- $E = (a \cup b)^*$. $\mathcal{L}(E) = \mathcal{L}((a \cup b)^*) = \{a, b\}^* = \{e, a, b, aa, ab, ba, bb, aaa, ...\}$;
- $E = a^* \cup b^*$. $\mathcal{L}(E) = \mathcal{L}(a^*) \cup \mathcal{L}(b^*) = \{e, a, aa, aaa, ...\} \cup \{e, b, bb, bbb, ...\} = \{e, a, aa, aaa, ..., b, bb, bbb, ...\}$;

2.1.1 Utilizări

Expresiile regulate sunt folosite în motoare de căutare, unelte de prelucrare text (grep), limbaje de programare (haskell).

2.2 Automate finite deterministe

Un automat finit este un model de calcul care primește la intrare o bandă de simboluri și își modifică starea internă în funcție de ce are la intrare. Acesta poate fi privit ca o cutie neagră cu niște stări între care face tranziții în funcție de intrare, la fiecare tranziție mișcând capul de citire la dreapta.

Formal, un automat finit determinist este un tuplu $M = (K, \Sigma, \delta, s, F)$, cu următoarele proprietăți:

- K este mulțimea stărilor. K este finită, de unde vine și numele automatului;
- Σ este alfabetul din care sunt formate cuvintele acceptate de automat;
- δ se numește funcție de tranziție. $\delta : K \times \Sigma \to K$. $\delta(p, a) = q; p, q \in K; a \in \Sigma$ înseamnă că automatul, dacă se află în starea p și primește pe bandă a, trece în starea q. Fiindcă δ este funcție, toate tranzițiile din fiecare stare pentru fiecare simbol trebuie să fie definite.
- $s \in K$ este starea de start. Aceasta este starea în care se află automatul înainte de a primi ceva pe bandă.

- $F \subseteq K$ este mulțimea stărilor finale. Dacă automatul se află într-o stare $f \in F$ după ce a terminat de citit șirul înseamnă că acceptă șirul.

Un moment din execuția unui automat finit se numește configurație. O configurație completă constă într-un tuplu (q, w) format din starea curentă și șirul de la intrare. O configurație este un element din $K \times \Sigma^*$.

Pentru a se descrie trecerea între configurații se folosește operatorul $\vdash_M$: $K \times \Sigma^* \to K \times \Sigma^*$. $\forall q, q' \in K, \forall w, w' \in \Sigma^*, (q, w) \vdash_M (q', w') \Leftrightarrow \exists \sigma \in \Sigma, w = \sigma w' \wedge \delta(q, \sigma) = q'$

Se notează cu $\vdash_M^*$ închiderea reflexivă și tranzitivă a operatorului de trecere între configurații. Adică $\vdash_M^*$ reprezintă trecerea între configurații în zero sau mai mulți pași.

Un automat finit determinist acceptă un șir dacă și numai dacă la terminarea șirului de la intrare automatul se află într-o stare finală. $\forall w \in \Sigma^*, M$ acceptă $w \Leftrightarrow \exists f \in F, (s, w) \vdash_M^* (f, e)$.

Mulțimea tuturor cuvintelor acceptate de un automat se numește limbaj acceptat de automat. $\forall w \in \Sigma^*, w \in \mathcal{L}(M) \Leftrightarrow \exists f \in F, (s, w) \vdash_M^* (f, e)$. Un limbaj acceptat de un automat finit determinist este regulat.

2.2.1 Utilizări

Automatele finite deterministe sunt folosite la crearea analizoarelor lexicale, la proiectarea circuitelor electrice, la testarea protocoalelor, la crearea unor programe în care este evidentă existența unei stări interne.

2.3 Automate finite nedeterministe

Un automat finit nedeterminist este un automat finit mai puțin restrictiv decât unul determinist. Acesta se bazează pe faptul că se pot rula în paralel o infinitate de ramuri de execuție și dacă cel puțin una se termină într-o stare finală, șirul este acceptat.

Un automat finit nedeterminist permite:

- absența tranzițiilor pe anumite simboluri. Dacă se întâmplă ca la un moment dat să apară un simbol pentru care nu există tranziție, ramura curentă de execuție se termină fără a accepta șirul;

- existența mai multor tranziții din aceeași stare pe același caracter. Dacă se întâmplă acest lucru, automatul lansează mai multe ramuri de execuție, câte una pentru fiecare tranziție posibilă;

- tranziții pe șiruri. Fiindcă se pot neglija tranziții, un lanț de tranziții în care din stările intermediare nu pleacă nicio tranziție se poate considera o singură tranziție pe un șir.
- tranziții pe șirul vid. Automatul își poate modifica spontan starea fără a citi vreun caracter.

Un automat nedeterminist este tot un tuplu $M = (K, \Sigma, \Delta, s, F)$ cu următoarele proprietăți:

- K este mulțimea finită a stărilor;
- Σ este alfabetul din care sunt formate cuvintele acceptate de automat;
- $\Delta \subset (K \times \Sigma^* \times K)$ nu este funcție, este relație. Δ este o finită. $(p, u, q) \in \Delta$ înseamnă că automatul poate trece din starea p în starea q citind șirul u;
- $s \in K$ este starea de start;
- $F \subseteq K$ este mulțimea stărilor finale.

O configurație a unui automat finit nedeterminist se notează la fel ca în cazul determinist, $(q, w) \in (K \times \Sigma^*)$.

Operatorul de trecere între configurații se notează la fel ca în cazul determinist, $\vdash_M$, dar fiindcă se poate ajunge în mai multe configurații sau niciuna nu mai e funcție. Închiderea reflexivă și tranzitivă a operatorului de trecere între configurații se notează tot cu $\vdash_M^*$.

Un automat finit nedeterminist acceptă un șir dacă există cel puțin o cale astfel încât din configurația inițială să se ajungă într-o stare finală fără niciun caracter rămas la intrare. Limbajul acceptat de un automat finit nedeterminist este format din totalitatea șirurilor acceptate de el. $\forall w \in \Sigma^*, w \in \mathcal{L}(M) \Leftrightarrow \exists f \in F, (s, w) \vdash_M^* (f, e)$

Un automat determinist este un automat finit nedeterminist care are niște restricții impuse. Deși mulțimea automatelor finite deterministe este inclusă în mulțimea automatelor finite nedeterministe, limbajele acceptate de ele sunt aceleași, limbajele regulate.

2.3.1 Utilizări

Automatele finite nedeterministe pot fi folosite la modelarea unei probleme pentru a fi ulterior transformat în automat determinist, care e mai greu de scris direct.

2.4 Echivalența ER, AFD, AFN

2.4.1 Conversie AFD $\rightarrow$ AFN

Un automat finit determinist este și nedeterminist, deci conversia este trivială. Îl lăsăm exact asa cum e.

2.4.2 Conversie AFN $\rightarrow$ AFD

Fie $M_1 = (K_1, \Sigma_1, \Delta_1, s_1, F_1)$. Să se găsească un $M_2 = (K_2, \Sigma_2, \delta_2, s_2, F_2), \mathcal{L}(M_1) = \mathcal{L}(M_2), \Sigma_2 = \Sigma_1$.

Pași:

1. Eliminarea tranzițiilor pe șiruri.

   Fie $\Delta_1' = \{(p, c, q) | \forall (p, c, q) \in \Delta_1 \wedge |c| \leq 1\} \cup \{(p, c_1, q_{w1}), (q_{w1}, c_2, q_{w2}), ..., (q_{wn}, c_n, q) | \forall (p, w, q) \in \Delta_1 \wedge |w| = n > 1 \wedge w = c_1 c_2 ... c_n \wedge q_{w1}, q_{w2}, ..., q_{wn} \notin K_1\}$ și $K_1' = \{q_{w1}, q_{w2}, ...q_{w2} | (p, c_1, q_{w1}), (q_{w1}, c_2, q_{w2}), ..., (q_{wn}, c_n, q) \in \Delta_1' \setminus \Delta_1\}$.

   Fie $M_1' = (K_1', \Sigma_1, \Delta_1', s_1, F_1)$.

   $\mathcal{L}(M_1) = \mathcal{L}(M_1')$ fiindcă nu se schimbă nimic la tranzițiile normale, și rezultatul unui șir de tranziții pe câte un caracter e identic cu rezultatul tranziției pe șir.

2. Se construiește automatul determinist folosind ca stări închiderea reflexivă și tranzitivă operatorului de trecere între configurații restricționat la tranziții pe șirul vid. $K_2 = 2^{K_1}$.

   Fie $Q(q) = \{p | (q, w) \vdash_M^* (p, w)\}$. $Q(q)$ reprezintă totalitatea stărilor în care se poate ajunge pornind din q fără a citi vreun caracter.

   $s_2 = Q_0 = Q(s_1)$

   $\delta_2 : K_2 \times \Sigma \rightarrow K_2$. Adică $\delta_2 : 2^{K_1} \times \Sigma \rightarrow 2^{K_1}$. Pentru fiecare element $Q_i \in K_2$ se calculează $\delta(Q_i)$.

$F_2 = \{Q(f) | f \in F_1\}$.

Din cauza modului în care se face transformarea, numărul stărilor unui automat finit determinist tinde să fie crescut exponențial la trecerea de la AFN la AFD. $|K_2| = |2^{K_1}| = 2^{|K_1|}$.

2.4.3 Conversie ER → AFN

Pentru conversia aceasta vom trata expresiile regulate în același mod ca în cazul definirii; întâi constructori de bază și apoi operațiile de reuniune, concatenare și Kleene Star.

Toate automatele create acum vor avea o stare inițială și o singură stare finală. La combinarea a două automate stările inițiale și finale își pierd proprietatea de stare inițială, respectiv finală.

Pentru a fi mai ușor se definesc următoarea notație:

- $\forall E$ expresie regulată, $AFN(E)$ este automatul obținut prin aplicarea algoritmului de aici;

1. $E_1 = \varnothing$. $M_1 = (K_1, \Sigma_1, \Delta_1, s_1, F_1)$; $K_1 = \{s, f\}, \Delta_1 = \emptyset, F = \{f\}$;
   $E_2 = e$. $M_2 = (K_2, \Sigma_2, \Delta_2, s_2, F_2)$; $K_2 = \{s, f\}, \Delta_2 = \{(s, e, f)\}, F = \{f\}$;
2. $\forall a \in \Sigma, E = a$. $M = (K, \Sigma, \Delta, s, F)$; $K = \{s, f\}, \Delta = \{(s, a, f)\}, F = \{f\}$;
3. $\forall \alpha, \beta$ expresii regulate, $E = \alpha \cup \beta$. Fie $M_1 = AFN(\alpha) = (K_1, \Sigma_1, \Delta_1, s_1, \{f_1\})$, $M_2 = AFN(\beta) = (K_2, \Sigma_2, \Delta_2, s_2, \{f_2\})$. $M = (K, \Sigma, \Delta, s, F)$; $K = \{s, f\} \cup K_1 \cup K_2, \Delta = \Delta_1 \cup \Delta_2 \cup \{(s, e, s_1), (s, e, s_2), (f_1, e, f), (f_2, e, f)\}, F = \{f\}$;
4. $\forall \alpha, \beta$ expresii regulate, $E = \alpha\beta$. Fie $M_1 = AFN(\alpha) = (K_1, \Sigma_1, \Delta_1, s_1, \{f_1\})$, $M_2 = AFN(\beta) = (K_2, \Sigma_2, \Delta_2, s_2, \{f_2\})$. $M = (K, \Sigma, \Delta, s, F)$; $K = \{s, f\} \cup K_1 \cup K_2, \Delta = \Delta_1 \cup \Delta_2 \cup \{(s, e, s_1), (f_1, e, s_2), (f_2, e, f)\}, F = \{f\}$;
5. $\forall \alpha$ o expresie regulată, $E = \alpha^*$. Fie $M_1 = AFN(\alpha) = (K_1, \Sigma_1, \Delta_1, s_1, \{f_1\})$. $M = (K, \Sigma, \Delta, s, F)$; $K = \{s, f\} \cup K_1, \Delta = \Delta_1 \cup \{(s, e, f), (s, e, s_1), (f_1, e, f)\}$, $F = \{f\}$;

2.5 Proprietăți algoritmice

2.5.1 Închidere în raport cu reuniunea

Mulțimea limbajelor regulate este închisă în raport cu reuniunea. $\forall L_1, L_2$ limbaje regulate, $L_1 \cup L_2$ este regulat.

1. Expresii regulate:

   $\forall L_1, L_2, \exists E_1, E_2, \mathcal{L}(E_1) = L_1, \mathcal{L}(E_2) = L_2$.

   $\mathcal{L}(E_1 \cup E_2) = \mathcal{L}(E_1) \cup \mathcal{L}(E_2) = L_1 \cup L_2$.

   Se poate scrie o expresie regulată, $E = E_1 \cup E_2$, care descrie limbajul $L_1 \cup L_2$.

2. Automate finite nedeterministe:

   Fie $M_1 = (K_1, \Sigma_1, \Delta_1, s_1, F_1), M_2 = (K_2, \Sigma_2, \Delta_2, s_2, F_2)$, două automate finite, $\mathcal{L}(M_1) = L_1, \mathcal{L}(M_2) = L_2$. $K_1 \cap K_2 = \emptyset$, altfel, redenumim stările dintr-un automat.

   $M = (K, \Sigma, \Delta, s, F)$, $K = K_1 \cup K_2 \cup \{s\}, \Sigma = \Sigma_1 \cup \Sigma_2, \Delta = \Delta_1 \cup \Delta_2 \cup \{(s, e, s_1), (s, e, s_2)\}, F = F_1 \cup F_2$, acceptă $\mathcal{L}(L_1) \cup \mathcal{L}(L_2)$.

   Automatul ori trece în stările corespunzătoare automatului M_1 și acceptă un cuvânt din L_1, ori trece în stările corespunzătoare lui M_2 și acceptă un cuvânt din M_2.

2.5.2 Închidere în raport cu concatenarea

Mulțimea limbajelor regulate este închisă în raport cu concatenarea. $\forall L_1, L_2$ limbaje regulate, $L_1 \circ L_2$ este regulat.

1. Expresii regulate:

   $\forall L_1, L_2, \exists E_1, E_2, \mathcal{L}(E_1) = L_1, \mathcal{L}(E_2) = L_2$.

   $\mathcal{L}(E_1 E_2) = \mathcal{L}(E_1) \circ \mathcal{L}(E_2) = L_1 \circ L_2$.

   Se poate scrie o expresie regulată, $E = E_1 E_2$, care descrie limbajul $L_1 \circ L_2$.

2. Automate finite nedeterministe:

Fie $M_1 = (K_1, \Sigma_1, \Delta_1, s_1, F_1), M_2 = (K_2, \Sigma_2, \Delta_2, s_2, F_2)$, două automate finite, $\mathcal{L}(M_1) = L_1, \mathcal{L}(M_2) = L_2$. $K_1 \cap K_2 = \emptyset$, altfel, redenumim stările dintr-un automat.

$M = (K, \Sigma, \Delta, s, F)$, $K = K_1 \cup K_2, \Sigma = \Sigma_1 \cup \Sigma_2, \Delta = \Delta_1 \cup \Delta_2 \cup \{(f_1, e, s_2) | \forall f_1 \in F_1\}, s = s_1, F = F_2$, acceptă $\mathcal{L}(L_1) \circ \mathcal{L}(L_2)$.

Automatul trece prin stările corespunzătoare lui M_1 până ajunge într-o stare care era finală, apoi trece în s_2 și se oprește într-o stare finală după ce primește și un cuvânt din L_2.

2.5.3 Închidere în raport cu Kleene Star

Mulțimea limbajelor regulate este închisă în raport cu Kleene Star. $\forall L$ limbaj regulat, L'^* este regulat.

1. Expresii regulate:

   $\forall L', \exists E', \mathcal{L}(E') = L'$.

   $\mathcal{L}(E'^*) = \mathcal{L}(E')^* = L'^*$.

   Se poate scrie o expresie regulată, $E = E'^*$, care descrie limbajul L'^*.

2. Automate finite nedeterministe:

   Fie $M' = (K', \Sigma', \Delta', s', F')$, un automat finit, $\mathcal{L}(M') = L'$.

   $M = (K, \Sigma, \Delta, s, F)$, $K = K' \cup \{s\}, \Sigma = \Sigma', \Delta = \Delta' \cup \{(s, e, s')\} \cup \{(f', e, s) | \forall f' \in F'\}, F = \{s\}$, acceptă $\mathcal{L}(L')^*$.

   Automatul termină direct în s sau trece prin stările corespunzătoare lui M până ajunge într-o stare care era finală, de unde ajunge pe o tranziție vidă înapoi în s unde poate termina sau repeta procesul.

2.5.4 Închidere în raport cu diferența

Mulțimea limbajelor regulate este închisă în raport cu diferența. $\forall L_1, L_2$ limbaje regulate, $L_1 \setminus L_2$ este regulat.

1. Automate finite deterministe:

   Fie $M_1 = (K_1, \Sigma_1, \delta_1, s_1, F_1), M_2 = (K_2, \Sigma_2, \delta_2, s_2, F_2)$, două automate finite, $\mathcal{L}(M_1) = L_1, \mathcal{L}(M_2) = L_2$.

$\Sigma_1 = \Sigma_2 = \Sigma$, altfel unele tranziții necesare nu există. Dacă nu există, se pot completa Σ_1 și Σ_2 până la $\Sigma_1 \cup \Sigma_2$, introducând o stare de eroare nefinală în care să rămână pentru orice caracter și tranziții către acea stare din orice altă stare pentru orice caracter nou introdus.

$M = (K, \Sigma, \delta, s, F)$, $K = K_1 \times K_2, \delta = \{((q_1, q_2), c, (\delta(q_1), \delta(q_2)))|\forall q_1 \in K_1, \forall q_2 \in K_2, \forall c \in \Sigma\}, s = (s_1, s_2), F = \{(f_1, q_2)|\forall f_1 \in F_1, \forall q_2 \in K_2 \setminus F_2\}$, acceptă $\mathcal{L}(L_1) \setminus \mathcal{L}(L_2)$.

Automatul simulează execuția ambelor automate în paralel și acceptă un șir dacă doar primul automat ar fi ajuns într-o stare finală.

2.5.5 Închidere în raport cu intersecția

Mulțimea limbajelor regulate este închisă în raport cu intersecția. $\forall L_1, L_2$ limbaje regulate, $L_1 \cap L_2$ este regulat.

Demonstrație:

- Se poate considera $L_1 \cap L_2 = L_1 \setminus (L_1 \setminus L_2)$ și se folosește metoda de mai sus pentru automate.
- Se folosește următoarea metodă:

  1. Automate finite deterministe:

     Fie $M_1 = (K_1, \Sigma_1, \delta_1, s_1, F_1), M_2 = (K_2, \Sigma_2, \delta_2, s_2, F_2)$, două automate finite, $\mathcal{L}(M_1) = L_1, \mathcal{L}(M_2) = L_2$.

     $\Sigma_1 = \Sigma_2 = \Sigma$, altfel unele tranziții necesare nu există. Dacă nu există, se pot completa Σ_1 și Σ_2 până la $\Sigma_1 \cup \Sigma_2$, introducând o stare de eroare nefinală în care să rămână pentru orice caracter și tranziții către acea stare din orice altă stare pentru orice caracter nou introdus.

     $M = (K, \Sigma, \delta, s, F), K = K_1 \times K_2, \delta = \{((q_1, q_2), c, (\delta(q_1), \delta(q_2)))|\forall q_1 \in K_1, \forall q_2 \in K_2, \forall c \in \Sigma\}, s = (s_1, s_2), F = \{(f_1, f_2)|\forall f_1 \in F_1, \forall f_2 \in F_2\}$, acceptă $\mathcal{L}(L_1) \cap \mathcal{L}(L_2)$.

     Automatul simulează execuția ambelor automate în paralel și acceptă un șir când ambele automate ar fi ajuns într-o stare finală.

2.5.6 Închidere în raport cu complementarea

Mulțimea limbajelor regulate este închisă în raport cu complementarea. $\forall L'$ limbaj regulat, $L = L'^c = \bar{L'}$ este regulat.

Demonstrație:

- Se poate considera $L = \Sigma^* \setminus L'$ și se aplică metoda menționată anterior pentru cele două automate.
- Se folosește următoarea metodă:

  1. Automate finite deterministe:

     Fie $M' = (K', \Sigma', \delta', s', F')$, un automat finit determinist, $\mathcal{L}(M') = L'$.

     $M = (K', \Sigma', \delta', s', K' \setminus F')$ acceptă L'^c.

     Complementând mulțimea stărilor finale se obține exact complementul limbajului.

2.5.7 Decizia dacă un limbaj regulat este vid

1. Expresii regulate:

   Fie E o expresie regulată $\mathcal{L}(E) = \emptyset \Leftrightarrow \exists \alpha, \beta \in \Sigma^*, E = \alpha \varnothing \beta$. Dacă $\alpha = \beta = e$, atunci $E = \varnothing$.

2. Automate finite deterministe sau nondeterministe:

   Pentru a stabili dacă automatul acceptă vreun șir, se face o parcurgere a grafului și dacă se ajunge într-o stare finală limbajul nu e vid.

2.5.8 Decizia dacă un șir este într-un limbaj regulat

1. Expresii regulate:

   Se generează în ordinea lungimii șirurile acceptate de expresie și se compară cu șirul căutat.

2. Automate finite deterministe:

   Se lansează în execuție automatul cu șirul la intrare și se așteaptă sfârșitul execuției automatului. Dacă la final este într-o stare finală șirul este acceptat.

3. Automate finite nedeterministe:

   Se lansează în execuție automatul cu șirul la intrare. Dacă vreuna din ramuri se termină într-o stare finală șirul este acceptat. Pentru a se asigura terminarea execuției, ciclurile care ajung în starea de unde au pornit fără a fi citit vreun caracter se execută o singură dată.

2.5.9 Decizia dacă un limbaj regulat este finit

1. Expresii regulate:

   Limbajul descris de o expresie regulată este finit dacă orice operație de Kleene Star din interior este într-o subexpresie care descrie limbajul vid. Limbajul descris de o expresie fără Kleene Star este finit. Limbajul vid este finit.

2. Automate finite deterministe sau nedeterministe:

   Un automat finit care conține o stare finală aflată în/după un ciclu accesibil din starea inițială acceptă o infinitate de șiruri.

2.6 Lema de pompare

Lema de pompare este o proprietate pe care o are orice limbaj regulat.

2.6.1 Enunț

Fie L un limbaj regulat. Atunci $\exists n_0 \in \mathbb{N}$ astfel încât $\forall w \in L, |w| \geq n_0$
$\exists x, y, z \in \Sigma^*, w = xyz, |xy| \leq n_0, |y| > 0$ astfel încât $\forall k \in \mathbb{N}, xy^k z \in L$.

Lema de pompare se poate formula astfel: Pentru orice limbaj regulat, orice șir destul de lung am avea, el se poate împărți în trei părți astfel încât pompând partea din centru de oricâte ori, șirul rezultat să rămână în limbaj.

Pentru limbaje finite se poate alege n_0 lungimea maximă din limbaj. Astfel, pentru toate șirurile mai lungi decât n_0, adică niciunul, se aplică lema.

Pentru o expresie regulată, pentru orice șir se alege un subșir care s-a potrivit o dată pentru o subexpresie căreia i se aplică Kleene Star și poate pompa de oricâte ori, șirul rezultat încă potrivindu-se expresiei regulate.

2.6.2 Negația Lemei de pompare

Lema de pompare nu poate fi folosită pentru a demonstra că un limbaj este regulat fiindcă există limbaje care nu sunt regulate pentru care lema de pompare funcționează.

Fiindcă lema funcționează pentru orice limbaj regulat, limbajele pentru care lema nu se aplică cu siguranță nu sunt regulate. Astfel, dintr-o proprietate fără mare utilitate, lema de pompare a devenit un instrument foarte bun de demonstrare a faptului că un limbaj nu ar fi regulat.

Forma negată a lemei de pompare este:

Fie L un limbaj. Dacă pentru $\forall n_0 \in \mathbb{N}$, $\exists w \in L, |w| \geq n_0$ astfel încât $\forall x, y, z \in \Sigma^*, w = xyz, |xy| \leq n_0, |y| > 0$, $\exists k \in \mathbb{N}$ astfel încât $xy^kz \notin L$, atunci L nu este regulat.

Pentru a demonstra că o proprietate nu e adevărată, căutăm un contraexemplu. Pentru orice dimensiune, căutăm un șir mai lung astfel încât oricum l-am împărți, pomparea șirului din mijloc produce un șir care nu e în limbaj.

2.6.3 Utilizare

Demonstrați folosind lema de pompare că $L = \{a^n b^m | n \neq m\}$ nu este regulat.

Rezolvare: Presupunem L regulat. Știind că $L' = \mathcal{L}(a^*b^*)$ este regulat, $L' \setminus L = \{a^n b^m | n = m\}$ ar trebui să fie regulat, deci ar trebui să respecte lema de pompare.

Fie n_0 oarecare, $w = a^{n_0}b^{n_0}, |w| = 2n_0 \geq n_0$.
$\forall x, y, z \in \Sigma^*, w = xyz$, din cauza restricțiilor $|xy| \leq n_0$ și $|y| > 0$, nu putem pompa decât a. Atunci, considerăm $x = a^i; y = a^j, j > 0; z = a^{n_0-i-j}b^{n_0}$. Atunci, $xy^kz = a^i a^{jk} a^{n_0-i-j} b^{n_0} = a^{n_0+j(k-1)}b^{n_0}$. Pentru $k = 2$, $xy^2z = a^{n_0+j}b^{n_0}$. Pentru $j \neq 0$, $n_0 + j \neq n_0$, adică $a^{n_0+j}b^{n_0} \notin L'$. Contradicție! $\Rightarrow$ presupunerea că L ar fi regulat e greșită.

3 Limbaje independente de context

Din păcate, nu toate limbajele sunt regulate. Pentru că expresiile regulate și automatele finite pot descrie doar limbaje regulate, a fost nevoie de ceva mai mult. Clasa limbajelor independente de context este următoarea clasă principală în ierarhia Chomsky.

3.1 Gramatici independente de context

Gramaticile independente de context reprezintă o altă metodă de reprezentare finită a limbajelor.

Limba română are o gramatică a ei. O gramatică reprezintă o mulțime de reguli de modificare și combinare a cuvintelor în propoziții corecte. Considerând noțiuni definite până acum, limba română poate fi asociată unui limbaj. Propozițiile corecte pot fi considerate șiruri din limbaj și componentele propozițiilor, cuvintele, pot fi considerate litere din alfabet. Astfel, facem următoarele asocieri:

- limbă → limbaj;
- propoziție → șir;
- cuvânt → simbol;
- vocabular → alfabet.

Analizate astfel, noțiunile definite până acum nu sunt suficiente pentru a defini o gramatică fiindcă nu se pot reprezenta regulile.

O gramatică este, din punct de vedere formal, un tuplu $G = (V, \Sigma, R, S)$, unde V reprezintă mulțimea simbolurilor ce pot apărea în reguli, Σ reprezintă alfabetul, R reprezintă mulțimea regulilor și S reprezintă simbolul de start.

Σ este o mulțime finită de simboluri care pot apărea în cuvintele limbajului, simboluri numite și terminali. V este mulțimea tuturor simbolurilor care apar în reguli, dar, pe lângă terminali, conține și niște simboluri care nu apar în cuvintele limbajului, numite neterminali. Un astfel de simbol este $S \in V \setminus \Sigma$.

R este mulțimea finită a regulilor de producție. O regulă este un tuplu, $(A, \gamma), A \in V \setminus \Sigma, \gamma \in V^*$, care înseamnă că neterminalul A poate fi înlocuit cu șirul de terminali și neterminali γ. Regula (A, γ) se notează cu $A \to \gamma$. Modul în care este definit face ca $R \subset (V \setminus \Sigma) \times V^*$. În cazul în care

$\exists A \in V \setminus \Sigma, \exists \gamma, \beta \in V^*, (A, \gamma) \in R \wedge (A, \beta) \in R$, atunci regulile se pot scrie $A \to \gamma|\beta$, însemnând că A se poate înlocui fie cu γ, fie cu β.

Înlocuirea părții din stânga a unei reguli cu partea din dreapta într-un șir se numește derivare. Spunem că v este derivat din $u (u \Rightarrow v)$ dacă $\exists A \in V \setminus \Sigma, \exists \alpha, \beta, \delta \in V^*$ astfel încât $(A, \delta) \in R \wedge u = \alpha A \beta \wedge v = \alpha \delta \beta$.

Închiderea reflexivă și tranzitivă a relației de derivare se notează cu $\Rightarrow^*$. $u \Rightarrow^* v$ înseamnă că v este derivat din u în zero sau mai mulți pași.

O gramatică generează un șir w dacă și numai dacă pornind de la simbolul de start se poate deriva w. Mulțimea tuturor șirurilor generate de o gramatică G se numește limbaj generat de gramatică. $w \in \mathcal{L}(G) \Leftrightarrow S \Rightarrow^* w$.

Un limbaj generat de o gramatică independentă de context se numește limbaj independent de context.

3.1.1 Utilizări

Gramaticile independente de context sunt folosite pentru a descrie formal limbaje de programare. Din cauza asta sunt foarte folosite în compilatoare.

3.2 Automate Pushdown nedeterministe

Din cauza faptului că automatele Pushdown deterministe pot accepta doar o submulțime a limbajelor independente de context nu vor fi folosite deloc.

Un automat Pushdown nedeterminist este un automat finit nedeterminist care are atașată o stivă.

Formal, un APD este un tuplu $M = (K, \Sigma, \Gamma, \Delta, s, F)$, unde K este mulțimea stărilor, Σ este alfabetul, Γ este alfabetul stivei, $\Delta \subset (K \times \Sigma^* \times \Gamma^*) \times (K \times \Gamma^*)$, este relația de tranziție între stări, s este starea de start și F este mulțimea stărilor finale.

Dacă $((p, u, \alpha), (q, \beta)) \in \Delta$, înseamnă că automatul poate trece din starea p în starea q, citind u de la intrare și scoțând α de pe stivă, punând în loc β.

O configurație a unui APD este un element $(p, w, \alpha) \in K \times \Sigma^* \times \Gamma^*$.

Operatorul de trecere între configurații este $\vdash_M \in K \times \Sigma^* \times \Gamma^* \times K \times \Sigma^* \times \Gamma^*$. Închiderea reflexivă și tranzitivă a operatorului de trecere între configurații este $\vdash_M^*$.

Un automat Pushdown nedeterminist acceptă un șir dacă există cel puțin o cale astfel încât din configurația inițială să se ajungă într-o stare finală fără niciun caracter rămas la intrare și cu stiva goală. Limbajul acceptat

de un automat Pushdown nedeterminist este format din totalitatea șirurilor acceptate de el. $\forall w \in \Sigma^*, w \in \mathcal{L}(M) \Leftrightarrow \exists f \in F, (s, w, e) \vdash_M^* (f, e, e)$.

3.2.1 Utilizări

Un APD poate fi folosit pentru implementarea unui parser după o gramatică dată.

3.3 Echivalența GIC, APD

3.3.1 Conversia GIC $\rightarrow$ APD

Fie $G = (V, \Sigma, R, S)$ o gramatică independentă de context. Să se găsească un automat Pushdown nedeterminist $M = (K, \Sigma, \Gamma, \Delta, s, F)$ care să accepte limbajul generat de G, $\mathcal{L}(M) = \mathcal{L}(G)$.

Vom scrie un automat ale cărui tranziții simulează derivările din gramatică.

$K = \{s, f\}$; $\Gamma = V$; $\Delta = \{((s, e, e), (f, S))\} \cup \{((f, e, A), (f, \gamma))|\forall A \in V, \gamma \in \Sigma^* \wedge (A, \gamma) \in R\} \cup \{((f, a, a), (f, e))|\forall a \in \Sigma\}$; $F = \{f\}$.

Automatul pune inițial pe stivă simbolul de start și trece în starea în care face derivările. O derivare este simulată scoțând de pe stivă neterminalul din stânga regulii și punerea pe stivă a părții din dreapta regulii. În cazul în care nu poate face o derivare fiindcă simbolul din vârful stivei este un terminal, automatul poate citi la intrare terminalul respectiv și îl scoate de pe stivă.

3.3.2 Conversia APD $\rightarrow$ GIC

3.4 Proprietăți algoritmice

3.4.1 Închiderea în raport cu reuniunea

Mulțimea limbajelor independente de context este închisă în raport cu reuniunea. $\forall L_1, L_2$ limbaje independente de context, $L_1 \cup L_2$ este independent de context.

1. Gramatici independente de context:

   Fie $G_1 = (V_1, \Sigma_1, R_1, S_1), G_2 = (V_2, \Sigma_2, R_2, S_2), \mathcal{L}(G_1) = L_1, \mathcal{L}(G_2) = L_2$. $(V_1 \setminus \Sigma_1) \cap (V_2 \setminus \Sigma_2) = \emptyset$, altfel redenumim neterminalii identici.

$G = (V, \Sigma, R, S)$. $V = V_1 \cup V_2$; $\Sigma = \Sigma_1 \cup \Sigma_2$; $R = R_1 \cup R_2 \cup \{(S, S_1), (S, S_2)\}$.

Din simbolul de start se poate deriva simbolul de start pentru una din cele două gramatici de unde se poate deriva orice cuvânt.

$\mathcal{L}(G) = \mathcal{L}(G_1) \cup \mathcal{L}(G_2)$.

2. Automate Pushdown nedeterministe:

   Fie $M_1 = (K_1, \Sigma_1, \Gamma_1, \Delta_1, s_1, F_1), M_2 = (K_2, \Sigma_2, \Gamma_2, \Delta_2, s_2, F_2)$, două automate Pushdown nedeterministe, $\mathcal{L}(M_1) = L_1, \mathcal{L}(M_2) = L_2$. $K_1 \cap K_2 = \emptyset$, altfel, redenumim stările dintr-un automat.

   $M = (K, \Sigma, \Gamma, \Delta, s, F)$, $K = K_1 \cup K_2 \cup \{s\}, \Sigma = \Sigma_1 \cup \Sigma_2, \Gamma = \Gamma_1 \cup \Gamma_2, \Delta = \Delta_1 \cup \Delta_2 \cup \{((s, e, e), (s_1, e)), ((s, e, e), (s_2, e))\}, F = F_1 \cup F_2$, acceptă $\mathcal{L}(L_1) \cup \mathcal{L}(L_2)$.

   Automatul ori trece în stările corespunzătoare automatului M_1 și acceptă un cuvânt din L_1, ori trece în stările corespunzătoare lui M_2 și acceptă un cuvânt din M_2.

3.4.2 Închiderea în raport cu concatenarea

Mulțimea limbajelor independente de context este închisă în raport cu concatenarea. $\forall L_1, L_2$ limbaje independente de context, $L_1 \circ L_2$ este independent de context.

1. Gramatici independente de context:

   Fie $G_1 = (V_1, \Sigma_1, R_1, S_1), G_2 = (V_2, \Sigma_2, R_2, S_2), \mathcal{L}(G_1) = L_1, \mathcal{L}(G_2) = L_2$. $(V_1 \setminus \Sigma_1) \cap (V_2 \setminus \Sigma_2) = \emptyset$, altfel redenumim neterminalii identici.

   $G = (V, \Sigma, R, S)$. $V = V_1 \cup V_2$; $\Sigma = \Sigma_1 \cup \Sigma_2$; $R = R_1 \cup R_2 \cup \{(S, S_1 S_2)\}$.

   Din simbolul de start se derivează concatenarea celor două simboluri de start, din fiecare derivându-se câte un cuvânt din limbajul respectiv.

   $\mathcal{L}(G) = \mathcal{L}(G_1) \circ \mathcal{L}(G_2)$.

2. Automate Pushdown nedeterministe:

   Fie $M_1 = (K_1, \Sigma_1, \Gamma_1, \Delta_1, s_1, F_1), M_2 = (K_2, \Sigma_2, \Gamma_2, \Delta_2, s_2, F_2)$, două automate Pushdown nedeterministe, $\mathcal{L}(M_1) = L_1, \mathcal{L}(M_2) = L_2$. $K_1 \cap K_2 = \emptyset$, altfel, redenumim stările dintr-un automat. $\Gamma_1 \cap \Gamma_2 = \emptyset$, altfel

redenumim simbolurile dintr-unul dintre alfabetele stivei unuia dintre automate.

$M = (K, \Sigma, \Gamma, \Delta, s, F)$, $K = K_1 \cup K_2, \Sigma = \Sigma_1 \cup \Sigma_2, \Gamma = \Gamma_1 \cup \Gamma_2, \Delta = \Delta_1 \cup \Delta_2 \cup \{((f_1, e, e), (s_2, e)) | \forall f_1 \in F_1\}, s = s_1, F = F_2$, acceptă $\mathcal{L}(L_1) \circ \mathcal{L}(L_2)$.

Automatul trece prin stările corespunzătoare lui M_1 până ajunge într-o stare care era finală, apoi trece în s_2 și se oprește într-o stare finală după ce primește și un cuvânt din L_2. Fiindcă simbolurile de pe stivă sunt diferite, al doilea automat nu poate curăța stiva dacă primul a lăsat ceva, așa că un șir e acceptat doar dacă amândouă automatele au lăsat stiva goală.

3.4.3 Închiderea în raport cu Kleene Star

Mulțimea limbajelor independente de context este închisă în raport cu Kleene Star. $\forall L'$ limbaj independent de context, $L = L'^*$ este independent de context.

1. Gramatici independente de context:

   Fie $G' = (V', \Sigma', R', S'), \mathcal{L}(G') = L'$.

   $G = (V, \Sigma, R, S)$. $V = V'$; $\Sigma = \Sigma'$; $R = R' \cup \{(S, S'S), (S, e)\}$.

   Din simbolul de start se derivează $S'S$, făcând practic o inserare a unui cuvânt din L' la începutul șirului. După ce se inserează numărul de cuvinte necesare se poate face o derivare a șirului vid din ultimul simbol de start.

   $\mathcal{L}(G) = \mathcal{L}(G_1)^*$.

2. Automate Pushdown nedeterministe:

   Fie $M' = (K', \Sigma', \Gamma', \Delta', s', F')$, un automat Pushdown nedeterminist, $\mathcal{L}(M') = L'$.

   $M = (K, \Sigma, \Gamma, \Delta, s, F)$, $K = K' \cup \{s, f\}, \Sigma = \Sigma', \Gamma = \Gamma' \cup \{Z\}, Z \notin \Gamma', \Delta = \Delta' \cup \{((s, e, e), (s', Z))\} \cup \{((f', e, Z), (s, e)) | \forall f' \in F'\}, F = F_2$, acceptă $\mathcal{L}(L_1)^*$.

   Automatul poate termina direct din starea s sau poate trece prin stările lui M'. Dacă trece prin stările lui M', pune întâi un simbol pe stivă pe

care M' nu îl poate scoate și nu îl scoate decât când dintr-o stare finală se întroarce în s, dar doar dacă automatul a ajuns în acea stare cu stiva curățată. Astfel, fiecare rulare a automatului acceptă un cuvânt.

3.4.4 Clasa limbajelor independente de context nu este închisă în raport cu diferența, intersecția și complementarea

Fie $L_1 = \{a^m b^n c^p | \forall m, n, p \in \mathbb{N} \wedge m = n\}, L_2 = \{a^m b^n c^p | \forall m, n, p \in \mathbb{N} \wedge n \neq p\}$. $L_1 \setminus L_2 = \{a^m b^n c^p | \forall m, n, p \in \mathbb{N} \wedge m = n = p\}$, care nu este independent de context.

Intersecția și complementarea se pot descrie în funcție de diferență astfel:

- $A \cap B = A \setminus (A \setminus B)$
- $A^c = \Sigma^* \setminus A$

3.4.5 Decizia dacă un limbaj independent de context este vid

1. Gramatici independente de context:

   Pentru a verifica dacă o gramatică independentă de context generează cel puțin un șir se marchează pe rând simbolurile din care se pot deriva șiruri de terminali.

   Algoritm:

   (a) Se marchează terminalii ca fiind simboluri din care se pot deriva șiruri formate doar din terminali.

   (b) La fiecare iterație se parcurg regulile și se marchează neterminalii din stânga regulilor care au în dreapta șiruri formate doar din simboluri deja marcate.

   (c) Dacă după o iterație nu s-a mai marcat niciun neterminal, algoritmul se oprește

   Dacă la finalul algoritmului simbolul de start, S, este marcat, înseamnă că gramatica poate genera cel puțin un șir fără neterminali, adică limbajul este nevid

3.5 Lema de pompare

Lema de pompare este o proprietate pe care o respectă orice limbaj independent de context.

3.5.1 Enunț

Fie L un limbaj independent de context. Atunci $\exists n_0 \in \mathbb{N}$ astfel încât $\forall w \in L, |w| \geq n_0$, $\exists u, v, x, y, z \in \Sigma^*, w = uvxyz, |vxy| \leq n_0, |vy| > 0$ astfel încât $\forall k \in \mathbb{N}, uv^kxy^kz \in L$.

Lema de pompare se poate formula astfel: Pentru orice limbaj independent de context, orice șir destul de lung am avea, el se poate împărți în cinci subșiruri astfel încât pompând simultan al doilea și al patrulea subșir de oricâte ori, șirul rezultat să rămână în limbaj.

3.5.2 Negația Lemei de pompare

Lema de pompare nu poate fi folosită pentru a demonstra că un limbaj este independent de context fiindcă există limbaje care nu sunt independente de cotnext pentru care lema de pompare funcționează.

Fiindcă lema funcționează pentru orice limbaj independent de context, limbajele pentru care lema nu se aplică, cu siguranță nu sunt independente de context. Astfel, dintr-o proprietate fără mare utilitate, lema de pompare a devenit un instrument foarte bun de demonstrare a faptului că un limbaj nu ar fi independent de context.

Forma negată a lemei de pompare este:

Fie L un limbaj. Dacă pentru $\forall n_0 \in \mathbb{N}$, $\exists w \in L, |w| \geq n_0$ astfel încât $\forall u, v, x, y, z \in \Sigma^*, w = uvxyz, |vxy| \leq n_0, |vy| > 0$, $\exists k \in \mathbb{N}$ astfel încât $uv^kxy^kz \notin L$, atunci L nu este independent de context.

Pentru a demonstra că o proprietate nu e adevărată, căutăm un contraexemplu. Pentru orice dimensiune, căutăm un șir mai lung astfel încât oricum l-am împărți, pomparea celor două subșiruri produce un șir care nu e în limbaj.

3.5.3 Utilizare

Demonstrați folosind lema de pompare că $L = \{a^nb^nc^n\}$ nu este independent de context.

Rezolvare:

Fie n_0 oarecare, $w = a^{n_0}b^{n_0}c^{n_0}, |w| = 3n_0 \geq n_0$.
$\forall u, v, x, y, z \in \Sigma^*, w = uvxyz$, din cauza restricțiilor $|vxy| \leq n_0$ și $|vy| > 0$, vxy poate să conțină doar a, doar b, doar c, doar a și b sau doar b și c.

Cazuri:

1. $u = a^i$

   $vxy = a^j; v = a^{j_v}, x = a^{j_x}, y = a^{j_y}; \ j_v + j_x + j_y = j \leq n, j_v + j_y > 0$

   $z = a^{n_0-i-j}b^{n_0}c^{n_0}$

   $uv^kxy^kz = a^ia^{kj_v}a^{j_x}a^{kj_y}a^{n_0-i-j}b^{n_0}c^{n_0} = a^{n_0+(j_v+j_y)(k-1)}b^{n_0}c^{n_0}$. Pentru $k = 2$, $n_0 + (j_v + j_y) \neq n_0$, adică $a^{n_0+(j_v+j_y)}b^{n_0}c^{n_0} \notin L$

2. $u = a^{n_0-i}$

   $vxy = a^ib^j; \ i + j \leq n_0; \ i > 0, j > 0$

   $z = b^{n_0-j}c^{n_0}$

   Cazuri:

   (a) $v = a^{i_v}; \ x = a^{i_x}b^{j_x}; \ y = b^{j_y}; \ i_v + i_x = i, j_x + j_y = j, i_v + j_y > 0$
   $uv^kxy^kz = a^{n_0-i}a^{ki_v}a^{i_x}b^{j_x}b^{kj_y}b^{n_0-j}c^{n_0} = a^{n_0+i_v(k-1)}b^{n_0+j_y(k-1)}c^{n_0}$.
   Pentru $k = 2$, $n_0+i_v \neq n_0 \vee n_0+j_y \neq n_0$, adică $a^{n_0+i_v}b^{n_0+j_y}c^{n_0} \notin L$

   (b) $v = a^ib^{j_v}; \ x = b^{j_x}; \ y = b^{j_y}; \ j_v + j_x + j_y = j, i + j_v + j_y > 0, j_v > 0$
   $uv^kxy^kz = a^{n_0-i}(a^ib^{j_v})^kb^{j_x}b^{kj_y}b^{n_0-j}c^{n_0}$. Pentru $k = 2$, se strică forma șirului, $a^{n_0}b^{j_v}a^ib^{n_0+j_y(k-1)}c^{n_0} \notin L$

   (c) $v = a^{i_v}; \ x = a^{i_x}; \ y = a^{i_y}b^j; \ i_v + i_x + i_y = i, i_v + i_y + j > 0, i_y > 0$
   $uv^kxy^kz = a^{n_0-i}a^{ki_v}a^{i_x}(a^{i_y}b^j)^kb^{n_0-j}c^{n_0}$. Pentru $k = 2$, se strică forma șirului, $a^{n_0+i_v(k-1)}b^ja^{i_y}b^{n_0}c^{n_0} \notin L$

3. $u = a^{n_0}b^i$

   $vxy = b^j; \ v = b^{j_v}, x = b^{j_x}, y = b^{j_y}; \ j_v + j_x + j_y = j \leq n, j_v + j_y > 0$

   $z = c^{n_0-i-j}$

   $uv^kxy^kz = a^{n_0}b^ib^{kj_v}b^{j_x}b^{kj_y}b^{n_0-i-j}c^{n_0} = a^{n_0}b^{n_0+(j_v+j_y)(k-1)}c^{n_0}$. Pentru $k = 2$, $n_0 + (j_v + j_y) \neq n_0$, adică $a^{n_0}b^{n_0+(j_v+j_y)}c^{n_0} \notin L$

4. $u = a^{n_0}b^{n_0-i}$

   $vxy = b^ic^j; \ i + j \leq n_0; \ i > 0, j > 0$

   $z = c^{n_0-j}$

   Cazuri:

(a) $v = b^{i_v}$; $x = b^{i_x}c^{j_x}$; $y = c^{j_y}$; $i_v + i_x = i, j_x + j_y = j, i_v + j_y > 0$
$uv^kxy^kz = a^{n_0}b^{n_0-i}b^{ki_v}b^{i_x}c^{j_x}c^{kj_y}c^{n_0-j} = a^{n_0}b^{n_0+i_v(k-1)}c^{n_0+j_y(k-1)}$.
Pentru $k = 2$, $n_0+i_v \neq n_0 \vee n_0+j_y \neq n_0$, adică $a^{n_0}b^{n_0+i_v}c^{n_0+j_y} \notin L$

(b) $v = b^ic^{j_v}$; $x = c^{j_x}$; $y = c^{j_y}$; $j_v + j_x + j_y = j, i + j_v + j_y > 0, j_v > 0$
$uv^kxy^kz = a^{n_0}b^{n_0-i}(b^ib^{j_v})^kc^{j_x}b^{kj_y}c^{n_0-j}$. Pentru $k = 2$, se strică forma șirului, $a^{n_0}b^{n_0}c^{j_v}b^ic^{n_0+j_y(k-1)} \notin L$

(c) $v = b^{i_v}$; $x = b^{i_x}$; $y = b^{i_y}c^j$; $i_v + i_x + i_y = i, i_v + i_y + j > 0, i_y > 0$
$uv^kxy^kz = a^{n_0}b^{n_0-i}b^{ki_v}b^{i_x}(b^{i_y}c^j)^kc^{n_0-j}$. Pentru $k = 2$, se strică forma șirului, $a^{n_0}b^{n_0+i_v(k-1)}c^jb^{i_y}c^{n_0} \notin L$

5. $u = a^{n_0}b^i$

$vxy = c^j$; $v = c^{j_v}, x = c^{j_x}, y = c^{j_y}$; $j_v + j_x + j_y = j \leq n, j_v + j_y > 0$

$z = c^{n_0-i-j}$

$uv^kxy^kz = a^{n_0}b^{n_0}c^ic^{kj_v}c^{j_x}c^{kj_y}c^{n_0-i-j} = a^{n_0}b^{n_0}c^{n_0+(j_v+j_y)(k-1)}$. Pentru $k = 2$, $n_0 + (j_v + j_y) \neq n_0$, adică $a^{n_0}b^{n_0}c^{n_0+(j_v+j_y)} \notin L$.

Cuprins

~ Seminar 1 ~ ..... - 4 -
Automat Finit Determinist ..... - 4 -
Automat Finit Nedeterminist ..... - 4 -
EXEMPLE: ..... - 5 -
L1 = a^* = {a^n, n ≥ 0} ..... - 5 -
L2 = a^+ = {a^n, n ≥ 1} ..... - 5 -
L3 = {a^{2n}, n ≥ 0} ..... - 5 -
L4 = {a^{2n+1}, n ≥ 0} ..... - 5 -
L5 = {a^{2n}, n ≥ 1} ..... - 5 -
L6 = {a^{2n+1}, n ≥ 1} ..... - 6 -
L7 = {a^{2n} b^{3m}, n ≥ 0, m ≥ 0} ..... - 6 -
L8 = {a^{2n} b^{3m}, n ≥ 1, m ≥ 0} ..... - 7 -
L9 = {a^{2n} b^{3m}, n ≥ 0, m ≥ 1} ..... - 7 -
L10 = {a^{2n} b^{3m}, n ≥ 1, m ≥ 1} ..... - 7 -
Operaţii cu limbaje ..... - 8 -
Reuniune $L = L_1 \bigcup L_2$ ..... - 8 -
Concatenare $L = L_1 \cdot L_2$ ..... - 8 -
Stelare $L = L_1^* = \bigcup_{n \geq 0} \left(L_1^n \right)$ ..... - 8 -
Ridicare la putere nenulă $L = L_1^+ = \bigcup_{n \geq 1} \left(L_1^n \right)$ ..... - 8 -
"Compunerea" automatelor finite ..... - 9 -
Reuniune ..... - 9 -
Concatenare ..... - 9 -
Stelare ..... - 9 -
Transformarea AFN→AFD ..... - 9 -
EXEMPLU: ..... - 10 -
Verificare acceptare cuvânt de către automat finit ..... - 12 -

~ Seminar 2 ~ ..... - 14 -
Automat Finit Nedeterminist cu λ–tranziţii ..... - 14 -
λ–închiderea unei stări ..... - 14 -
Verificare acceptare cuvânt de către automat AFN–λ ..... - 14 -
EXEMPLU: ..... - 15 -
Transformarea AFN–λ → AFD ..... - 18 -
EXEMPLU: ..... - 19 -
Automatul minimal ..... - 22 -
EXEMPLU: ..... - 23 -
Condiţii asupra numărului de apariţii ale unui caracter într-un cuvânt ..... - 25 -
EXEMPLE: ..... - 25 -
$L_1 = \{w \in \{0,1\}^* \, a.î. \, |w|_0 = par, |w|_1 = impar\}$ ..... - 25 -
$L_2 = \{w \in \{a,b\}^* \, a.î. \, |w|_a = 2n+1, |w|_b = 3m+2, n \geq 0, m \geq 0\}$ ..... - 26 -

$L_3 = \{w \in \{a,b\}^* \text{ a.î. } aaa \text{ şi } bbb \text{ nu apar în } w\}$ ..... - 26 -

~ Seminar 3 ~ ..... - 27 -
Expresii regulate ..... - 27 -
Precedenţa operaţiilor $*, \cdot, +$ ..... - 27 -
Transformarea expresie_regulată → automat_finit ..... - 27 -
Transformarea automat_finit → expresie_regulată ..... - 28 -
Câteva formule utile ..... - 29 -
EXEMPLU: ..... - 29 -
Gramatici ..... - 31 -
Gramatici independente de context (GIC) / Context-free grammars (CFG) ..... - 31 -
Gramatici liniare ..... - 31 -
Gramatici liniare la dreapta ..... - 31 -
Gramatici liniare la stânga ..... - 31 -
Gramatici regulate ..... - 31 -
EXEMPLE (gramatici regulate): ..... - 32 -
L1 = a^* = $\{a^n, n \geq 0\}$ ..... - 32 -
L2 = a^+ = $\{a^n, n \geq 1\}$ ..... - 32 -
L3 = $\{a^{2n}, n \geq 0\}$ ..... - 32 -
L4 = $\{a^{2n+1}, n \geq 0\}$ ..... - 32 -
L5 = $\{a^{2n}, n \geq 1\}$ ..... - 32 -
L6 = $\{a^{2n+1}, n \geq 1\}$ ..... - 32 -
L7 = $\{a^{2n} b^{3m}, n \geq 0, m \geq 0\}$ ..... - 32 -
Analogia între automate finite şi gramatici regulate ..... - 33 -

~ Seminar 4 ~ ..... - 34 -
EXEMPLE (gramatici independente de context): ..... - 34 -
L1 = $\{a^n b^n, n \geq 0\}$ ..... - 34 -
L2 = $\{a^n b^n, n \geq 1\}$ ..... - 34 -
L3 = $\{a^n b^{2n}, n \geq 1\}$ ..... - 35 -
L4 = $\{a^{2n} b^k c^{3n}, n \geq 0, k \geq 1\}$ ..... - 35 -
L5 = $\{a^{2n} b^k c^{3n}, n \geq 1, k \geq 1\}$ ..... - 35 -
L6 = $\{a^{2n} c^{3n} b^k, n \geq 1, k \geq 1\}$ ..... - 36 -
Verificare generare cuvânt de către gramatică ..... - 36 -
L7 = $\{a^n b^k c^{2k} d^n, n \geq 1, k \geq 0\}$ ..... - 38 -
L8 = $\{a^n b^k c^{2k} d^n, n \geq 0, k \geq 1\}$ ..... - 38 -
L9 = $\{a^n b^{2n} c^{2k} d^k, n \geq 0, k \geq 1\}$ ..... - 39 -
L10 = $\{a^n b^{2n} c^k d^m e^{3k}, n,k,m \geq 0\}$ ..... - 39 -
Transformarea GIC → gramatică în F.N. Chomsky ..... - 40 -
Automatul Push-Down (APD) / Automatul Stivă ..... - 43 -
Modalităţi de acceptare a unui cuvânt de către un APD ..... - 44 -
EXEMPLE (automate push-down): ..... - 45 -
L1 = $\{a^n b^n, n \geq 1\}$ ..... - 45 -
L2 = $\{a^n b^{2n}, n \geq 0\}$ ..... - 46 -
L3 = $\{a^{2n} b^k c^{3k} d^n, n \geq 1, k \geq 1\}$ ..... - 47 -
Verificare acceptare cuvânt de către un APD ..... - 48 -

L4 = $\{a^{2n} b^{3n} c^{2k} d^{k}, n \geq 1, k \geq 1\}$ ..... - 48 -

~ Seminar 5 ~ ..... - 50 -
EXEMPLE (automate push-down) [continuare]: ..... - 50 -
L5 = $\{ w w^{R} \mid w \in \{a,b,c\}^{*} \setminus \{\lambda\} \}$ *(palindroame de lungime pară)* ..... - 50 -
L6 = $\{a^{n} b^{k}, n > k \geq 1\}$ ..... - 52 -
L7 = $\{a^{n} b^{k}, k > n \geq 1\}$ ..... - 52 -
L8 = $\{a^{n} b^{k}, n \geq 1, k \geq 1, n \neq k\}$ ..... - 53 -
L9 = $\{a^{2n} b^{3k}, n \geq 1, k \geq 1, n \neq k\}$ ..... - 54 -
Algoritmul CYK (verificare generare cuvânt de către gramatică în F.N.Chomsky) - 55 -
EXEMPLU: ..... - 56 -

~ Seminar 6 ~ ..... - 59 -
Lema de pompare pentru limbajele regulate: ..... - 59 -
L1 = $\{a^{k\wedge 2}, k \geq 1\} = \{a^{1}, a^{4}, a^{9}, a^{16}, a^{25}, a^{36}, a^{49}, a^{64}, a^{81}, a^{100}, ...\}$ ..... - 59 -
L2 = $\{a^{k}b^{k}, k \geq 1\} = \{ab, a^{2}b^{2}, a^{3}b^{3}, a^{4}b^{4}, a^{5}b^{5}, ...\}$ ..... - 60 -

~ Seminar 1 ~

(AFD; AFN; Operaţii cu limbaje; Compunere automate; Transformarea AFN→AFD)

Automat Finit Determinist

AFD = (Q, Σ, q_0, δ, F)
Q mulţimea de stări
Σ alfabetul de intrare
$q_0 \in Q$ starea iniţială
$F \subseteq Q$ mulţimea de stări finale
$\delta : Q \times \Sigma \to Q$ funcţia de tranziţie („delta")

Pentru a verifica dacă un cuvânt este sau nu acceptat de un automat AFD:
- se porneşte din starea iniţială q_0
- cât timp este posibil:
  - având starea curentă şi caracterul curent din cuvânt, aplicăm funcţia δ pentru a determina noua stare
  - repetăm pentru noua stare şi caracterul următor din cuvânt
- algoritmul se termină în 3 cazuri:
  - chiar dacă nu s-a citit complet tot cuvântul, dar automatul s-a blocat pentru că funcţia de tranziţie nu era definită pentru starea curentă şi caracterul curent → *"cuvânt neacceptat"*
  - dacă întreg cuvântul a fost citit, dar automatul a ajuns într-o stare care nu este finală (adică din mulţimea Q\F) → *"cuvânt neacceptat"*
  - dacă întreg cuvântul a fost citit, iar automatul a ajuns într-o stare care este finală (adică din mulţimea F) → *"cuvânt acceptat"*

Automat Finit Nedeterminist

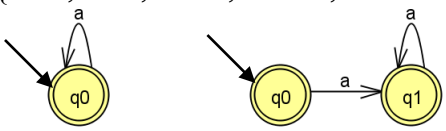
AFN = (Q, Σ, q_0, δ, F)
Q mulţimea de stări
Σ alfabetul de intrare
$q_0 \in Q$ starea iniţială
$F \subseteq Q$ mulţimea de stări finale
$\delta : Q \times \Sigma \to 2^Q$ funcţia de tranziţie („delta")

Diferenţa dintre AFD şi AFN constă în modul în care este definită funcţia de tranziţie.
La AFD: pentru orice pereche de o stare şi un caracter, avem *cel mult* o stare de continuare (putem să nu avem niciuna, adică funcţia să fie nedefinită pentru acele valori).
La AFN: pentru orice pereche de o stare şi un caracter, avem ca posibilitate de continuare o submulţime a mulţimii de stări Q (putem să nu avem niciuna dacă funcţia este nedefinită; putem să avem o singură stare de continuare; sau putem să avem mai multe stări de continuare, aici apare nedeterminismul).

Pentru a verifica dacă un cuvânt este sau nu acceptat de un automat AFN:
Se procedează analog ca în cazul AFD-ului, excepţia fiind că la fiecare aplicare a funcţiei delta trebuie să reţinem toate posibilităţile de stări de continuare, iar la pasul următor să reaplicăm delta pentru toate acestea împreună cu caracterul curent.

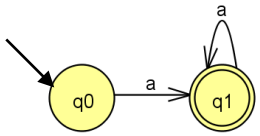
EXEMPLE:

L1 = a^* = {a^n, n ≥ 0} = {λ=a^0, a=a^1, aa=a^2, aaa=a^3, aaaa=a^4, …}



Ambele soluţii sunt AFD-uri.
Obs: De fiecare dată când λ (cuvântul vid) trebuie să fie recunoscut de un AFD sau de un AFN (simplu, adică AFN fără λ-tranziţii), trebuie ca starea iniţială q_0 să fie şi stare finală.

L2 = a^+ = {a^n, n ≥ 1} = {a=a^1, aa=a^2, aaa=a^3, aaaa=a^4, …}



Diferenţa faţă de exemplul anterior?
Aici λ nu trebuie acceptat, deci starea iniţială q_0 trebuie să nu fie stare finală.

L3 = {a^{2n}, n ≥ 0} = {λ=a^0, aa=a^2, aaaa=a^4, aaaaaa=a^6, …}
L4 = {a^{2n+1}, n ≥ 0} = {a=a^1, aaa=a^3, aaaaa=a^5, …}



Cele două automate au aceleaşi stări şi aceleaşi tranziţii. Diferă doar mulţimea stărilor finale. Dacă setăm q_0 finală, atunci vor fi acceptate cuvintele de lungime pară (inclusiv zero), iar dacă setăm în schimb q_1 finală, atunci vor fi acceptate cuvintele de lungime impară.

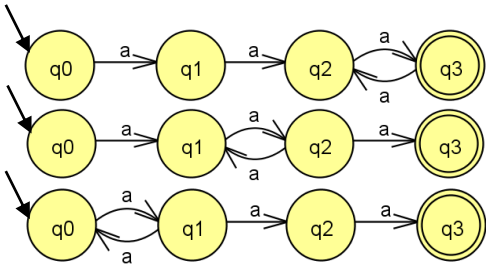
Obs: De fiecare dată când la putere apare un coeficient, înseamnă că vom avea un circuit de lungime egală cu acest coeficient (sau lungime proporţională cu el, în cazul în care puterea era aplicată unui cuvânt, nu doar unei litere).

L5 = {a^{2n}, n ≥ 1} = {aa=a^2, aaaa=a^4, aaaaaa=a^6, …}



În primul rând trebuie să ne asigurăm că obligăm automatul să citească cei doi de "a" (cel mai mic cuvânt din limbaj) şi că nu acceptă ceva mai scurt de atât. Pentru asta, adăugăm cele trei stări şi tranziţiile de la q_0 spre q_1 şi de la q_1 spre q_2, iar apoi setăm q_2 să fie stare finală. Apoi vedem că avem coeficientul 2 la putere, deci trebuie să avem un circuit de lungime 2. Avem două variante: fie adăugăm tranziţie de la q_2 spre q_1, fie adăugăm tranziţie de la q_1 spre q_0. Ambele automate recunosc acelaşi limbaj (dar primul este AFD, iar al doilea este AFN pentru că din q_1 cu "a" putem ajunge şi în q_0 şi în q_2).

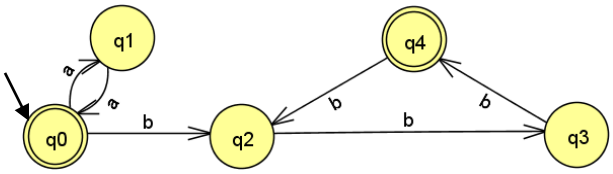
L6 = {a^{2n+1}, n ≥ 1} = {aaa=a^3, aaaaa=a^5, aaaaaaa=a^7, …}



La fel ca la exemplul anterior, ne vom asigura întâi că automatul nu acceptă nimic mai scurt decât cel mai mic cuvânt din limbaj, adică "aaa". Deci vom adăuga cele 4 stări şi tranziţiile către dreapta şi vom seta starea q_3 finală. Apoi pentru a avea circuitul de lungime 2 (egală cu coeficientul lui n), avem cele trei variante. Dacă adăugăm tranziţia de la q_3 spre q_2 vom obţine un AFD, iar dacă alegem una din celelalte două variante vom obţine un AFN.

L7 = {$a^{2n}b^{3m}$, n ≥ 0, m ≥ 0}

= {λ, a^2, a^4, a^6, a^8, …, b^3, b^6, b^9, b^{12}, …, a^2b^3, a^2b^6, a^4b^3,a^4b^6, a^6b^3, a^6b^6, …}

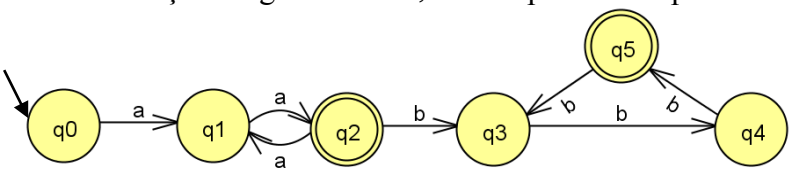


În primul rând vedem că cel mai mic cuvânt ce trebuie acceptat este λ, deci setăm starea iniţială q_0 să fie şi stare finală. Apoi facem circuit de lungime 2 cu litera "a". După ce citim un număr par de a-uri (inclusiv zero), ajungem în starea q_0, deci de aici trebuie să pornim cu citirea b-urilor. Adăugăm q_2, q_3 şi cele două tranziţii cu b. Ştim că va trebui să avem un circuit de lungime 3 cu b-uri, dar trebuie să fim de asemenea atenţi că literele de "a" şi de „b" nu pot fi amestecate între ele, după ce am citit primul "b" nu mai avem voie să citim niciun „a". Deci nu putem avea muchie de întoarcere de la q_3 către q_0 pentru că în q_0 se pot citi a-uri. Înseamnă că avem nevoie de a nouă stare, aşa că adăugăm q_4 şi tranziţii de la q_3 spre q_4 şi de la q_4 spre q_2 pentru a închide circuitul.

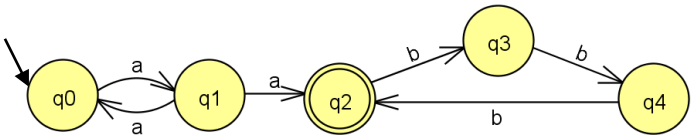
L8 = {$a^{2n} b^{3m}$, n ≥ 1, m ≥ 0}

= {a^2, a^4, a^6, a^8, …, a^2b^3, a^2b^6, a^4b^3,a^4b^6, a^6b^3, a^6b^6, …}

În acest caz, cuvintele conţin obligatoriu a-uri, b-urile putând să apară sau nu.



Trebuie să obligăm automatul să citească număr par de a-uri, dar cel puţin două, deci nu se mai poate întoarce din q_1 în q_0 cu al doilea "a", ci trebuie să continue spre q_2 şi setăm q_2 ca stare finală. Apoi începând din q_2 citim doi de "b", dar pentru al treilea nu putem închide circuitul întorcându-ne în q_2 dacă am ales ca muchie de întoarcere pentru a-uri de la starea q_2 spre starea q_1. Deci trebuie să adăugăm şi starea q5 care va fi şi ea finală şi cele două tranziţii de la q_4 spre q_5 şi de la q_5 spre q_3.

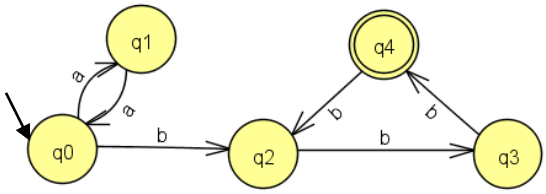


Dar dacă am fi ales ca muchie de întoarcere pentru a-uri de la q_1 spre q_0?
Atunci nu am mai fi avut nevoie de starea q_5, pentru că puteam să închidem circuitul b-urilor de la q_4 spre q_2 şi am fi avut doar starea q_2 finală.
Doar că în acest caz am fi obţinut un AFN (din cauza lui q_1).

L9 = {$a^{2n} b^{3m}$, n ≥ 0, m ≥ 1}

= {b^3, b^6, b^9, b^{12}, …, a^2b^3, a^2b^6, a^4b^3,a^4b^6, a^6b^3, a^6b^6, …}

În acest caz, cuvintele conţin obligatoriu b-uri, a-urile putând să apară sau nu.

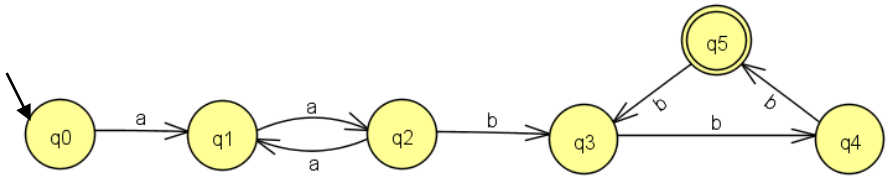


Observăm că automatul este foarte asemănător cu cel pentru limbajul L7.
Diferenţa este că acum starea q_0 nu mai este stare finală, ceea ce înseamnă că putem sau nu să trecem prin circuitul de a-uri, dar sigur suntem obligaţi să citim b-uri înainte de a ajunge într-o stare finală.

L10 = {$a^{2n} b^{3m}$, n ≥ 1, m ≥ 1}

= {a^2b^3, a^2b^6, a^4b^3,a^4b^6, a^6b^3, a^6b^6, …}

În acest caz, cuvintele conţin obligatoriu şi a-urile, şi b-urile.



Acest automat seamănă cu cel pentru L8 (prima variantă), unde eram obligaţi să avem a-uri. În plus, pentru că acum trebuie să avem şi b-uri, starea q_2 nu va mai fi stare finală.

Operaţii cu limbaje

Reuniune $L = L_1 \bigcup L_2$

$L = \{w \mid w \in L_1 \text{ sau } w \in L_2\}$

Exemplu:
L_1 = {a, ab, abc}
L_2 = {a, bd, cd}
→ L = {a, ab, abc, bd, cd}

Concatenare $L = L_1 \cdot L_2$

$L = \{w_i w_j \mid w_i \in L_1 \text{ şi } w_j \in L_2\}$

Exemplu:
L_1 = {a, ab, abc}
L_2 = {a, bd, cd}
→ L = {aa, abd, acd, aba, abbd, abcd, abca, abcbd, abccd}

Stelare $L = L_1^* = \bigcup_{n \geq 0} \left(L_1^n\right)$

$L_1^n = \{w_1 w_2 ... w_n \mid w_i \in L_1 \text{ pentru orice } 1 \leq i \leq n\}$

Exemplu:
L_1 = {a, ab, abc}
→ $(L_1)^*$ = {λ ; a, ab, abc ; aa, aab, aabc, aba, abab, ababc, abca, abcab, abcabc ; …}

Obs: Cuvântul vid λ va aparţine limbajului stelat, indiferent dacă înainte aparţinea sau nu limbajului iniţial.

Ridicare la putere nenulă $L = L_1^+ = \bigcup_{n \geq 1} \left(L_1^n\right)$

La fel ca la stelare, doar că nu mai există cuvântul vid. $L^+ = L^* \setminus \{\lambda\}$

Obs: Atenţie la ridicarea la putere a cuvintelor! *(Să nu distribuiţi puterea ca la înmulţire, ci concatenaţi cuvântul cu el însuşi de câte ori spune puterea.)*
$(abc)^2 = abcabc$; $(abc)^2 \neq aabbcc = a^2b^2c^2$

"Compunerea" automatelor finite

Reuniune

Dacă avem deja construite două automate finite (având stările iniţiale q_{01} şi q_{02} şi mulţimile stărilor finale F1 şi F2) care acceptă respectiv limbajele L1 şi L2 şi dorim să obţinem un automat finit care să accepte limbajul $L = L_1 \bigcup L_2$:

- introducem o nouă stare q_0 care va fi noua stare iniţială
- adăugăm două λ-tranziţii de la q_0 spre q_{01} şi de la q_0 spre q_{02}
- noua mulţime de stări finale va fi $F = F_1 \bigcup F_2$

Concatenare

Dacă avem deja construite două automate finite (având stările iniţiale q_{01} şi q_{02} şi mulţimile stărilor finale F1 şi F2) care acceptă respectiv limbajele L1 şi L2 şi dorim să obţinem un automat finit care să accepte limbajul $L = L_1 \cdot L_2$:

- noua stare iniţială va fi q_{01}, starea iniţială a primului automat
- adăugăm λ-tranziţii de la fiecare stare finală din F1 spre q_{02}
- noua mulţime de stări finale va fi F = F2

Stelare

Dacă avem deja construit un automat finit (având starea iniţială q_{01} şi mulţimea stărilor finale F1) care acceptă limbajul L1 şi dorim să obţinem un automat finit care să accepte limbajul $L = L_1^*$:

- introducem o nouă stare q_0 care va fi noua stare iniţială
- adăugăm o λ-tranziţie de la q_0 spre q_{01}
- adăugăm λ-tranziţii de la fiecare stare finală din F1 spre q_0
- noua mulţime de stări finale va fi $F = \{q_0\}$ *(sau putem lăsa* $F = \{q_0\} \bigcup F_1$ *)*

Transformarea AFN→AFD

Având dat un AFN (reprezentat sub formă de graf), vom calcula funcţia delta completând un tabel astfel:

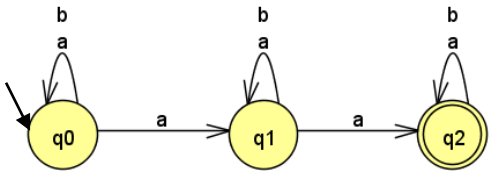
- La capetele de linie vom pune numele stărilor AFN-ului (elementele mulţimii Q): $q_0, q_1, ..., q_{|Q|-1}$
- La capetele de coloană vom pune caracterele alfabetului AFN-ului (elementele mulţimii Σ)
- Calculăm conţinutul primelor |Q| linii astfel: elementul aflat pe linia l_i şi coloana c_j reprezintă mulţimea de stări în care se poate ajunge în AFN dacă pornim din starea de pe linia l_i (în cazul acesta: q_{i-1} dacă numerotăm liniile începând cu 1) şi citim caracterul de pe coloana c_j. Pentru a afla această mulţime, ne uităm la graful

care reprezintă AFN-ul, căutăm starea de la linia l_i şi pentru fiecare tranziţie cu litera de la coloana c_j care porneşte din această stare, adăugăm starea destinaţie a tranziţiei la mulţimea curentă din tabel.

- Apoi trebuie să calculăm această funcţie delta şi pentru noile mulţimi de stări obţinute la paşii anteriori (aceste calcule se fac cel mai simplu folosind doar liniile din tabel calculate la pasul anterior, nu şi graful automatului AFN)
  (a) fie alegem să facem calculele pentru *toate noile mulţimi de stări*, şi abia apoi să vedem care dintre ele vor apărea în AFD-ul obţinut (tot făcând "parcurgerea" în adâncime descrisă mai jos).
  (b) fie alegem să facem calculele *doar pentru noile mulţimi de stări care vor apărea* în AFD: pentru a decide care vor fi acestea, facem un fel de parcurgere în adâncime a grafului care va reprezenta AFD-ul. Adică pornim cu starea q_0 care va fi starea iniţială a AFD-ului, apoi parcurgem prima linie a tabelului (care conţine calculele tranziţiilor ce pleacă din q_0) şi adăugăm la noul automat toate aceste tranziţii împreună cu stările-destinaţie calculate în tabel. Apoi pentru toate aceste stări-destinaţie, dacă nu au deja în tabel un capăt de linie care să le corespundă, îl vom adăuga şi vom calcula acea linie astfel:
    - fie noua stare-mulţime $\{q_{k1}, q_{k2}, \ldots, q_{kp}\}$. Pentru fiecare coloană c_j, calculăm elementul din tabel făcând **reuniunea** elementelor aflate pe aceeaşi coloană c_j şi pe liniile corespunzătoare stărilor $q_{k1}, q_{k2}, \ldots, q_{kp}$.
  Apoi revenim la desenarea grafului AFD-ului, şi adăugăm tranziţiile proaspăt calculate împreună cu stările-destinaţie. Reluăm procedeul până nu mai apar stări noi.
- Stările finale ale noului AFD obţinut, vor fi acele stări-mulţime (pot avea şi un singur element, nu neapărat mai multe) care conţin cel puţin una din stările care erau finale pentru AFN-ul iniţial.

EXEMPLU:

Se dă AFN-ul următor:



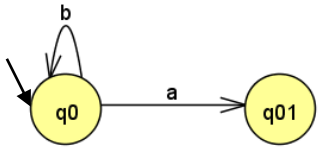
Calculăm tabelul pentru funcţia delta: întâi primele 3 linii folosind graful AFN-ului, apoi următoarele linii folosind aceste prime 3 linii ale tabelului.

Vom avea două coloane, pentru că alfabetul automatului iniţial este $\Sigma = \{a, b\}$.

δ	a	b
q_0	$\{q_0, q_1\}$	q_0
q_1	$\{q_1, q_2\}$	q_1
q_2	q_2	q_2

Alegem varianta (b), adică să calculăm doar acele stări care vor apărea în AFD.

(1) Pornim cu starea q_0 şi îi adăugăm cele două tranziţii şi stările-destinaţie necesare, adică $\{q_0, q_1\} = q_{01}$ (prin notaţie).



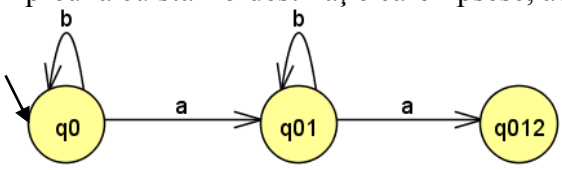
(2) Observăm că a apărut noua stare q_{01} pentru care nu avem o linie în tabel, aşa că o adăugăm şi o calculăm pentru cele două coloane.

$$\delta(q_{01}, a) = \delta(\{q_0, q_1\}, a)$$
$$= \{\delta(q_0, a)\} \cup \{\delta(q_1, a)\}$$
$$= \{q_0, q_1\} \cup \{q_1, q_2\} = \{q_0, q_1, q_2\} = q_{012}$$

$$\delta(q_{01}, b) = \delta(\{q_0, q_1\}, b)$$
$$= \{\delta(q_0, b)\} \cup \{\delta(q_1, b)\}$$
$$= \{q_0\} \cup \{q_1\} = \{q_0, q_1\} = q_{01}$$

δ	a	b
q_0	$\{q_0, q_1\} = q_{01}$	q_0
q_1	$\{q_1, q_2\} = q_{12}$	q_1
q_2	q_2	q_2
q_{01}	$\{q_0, q_1, q_2\} = q_{012}$	$\{q_0, q_1\} = q_{01}$

(3) Revenim la desenul AFD-ului şi adăugăm tranziţiile care lipsesc, adică cele care pleacă din starea q_{01}, împreună cu stările-destinaţie care lipsesc, adică q_{012}.



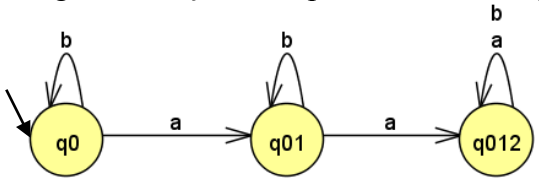
(4) Pentru noua stare apărută, q_{012}, îi adăugăm linia în tabel şi o calculăm.

$$\delta(q_{012}, a) = \delta(\{q_0, q_1, q_2\}, a)$$
$$= \{\delta(q_0, a)\} \cup \{\delta(q_1, a)\} \cup \{\delta(q_2, a)\}$$
$$= \{q_0, q_1\} \cup \{q_1, q_2\} \cup \{q_2\} = \{q_0, q_1, q_2\} = q_{012}$$

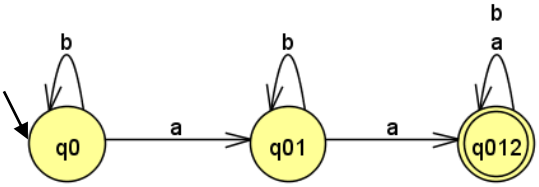
$$\delta(q_{012}, b) = \delta(\{q_0, q_1, q_2\}, b)$$
$$= \{\delta(q_0, b)\} \cup \{\delta(q_1, b)\} \cup \{\delta(q_2, b)\}$$
$$= \{q_0\} \cup \{q_1\} \cup \{q_2\} = \{q_0, q_1, q_2\} = q_{012}$$

δ	a	b
q_0	$\{q_0, q_1\} = q_{01}$	q_0
q_1	$\{q_1, q_2\} = q_{12}$	q_1
q_2	q_2	q_2
q_{01}	$\{q_0, q_1, q_2\} = q_{012}$	$\{q_0, q_1\} = q_{01}$
q_{012}	$\{q_0, q_1, q_2\} = q_{012}$	$\{q_0, q_1, q_2\} = q_{012}$

(5) Revenim la graf şi adăugăm tranziţiile care pleacă din starea q_{012}.



(6) Observăm că nu ne-a mai apărut nicio stare nouă, deci nu ne-a rămas decât să marcăm stările finale şi am obţinul AFD-ul cerut. În AFN-ul iniţial stare finală era doar q_2. Orice stare-mulţime din AFD-ul nostru care îl conţine pe q_2 va fi stare finală. Există una singură: q_{012}.



Obs: Dacă am fi ales varianta (a), atunci am fi calculat în tabel şi linia corespunzătoare stării q_{12}, însă observăm că nici ea, dar nici stările q_1 şi q_2 nu apar în AFD.

Verificare acceptare cuvânt de către automat finit

Folosim "configuraţii" (sau „descrieri instantanee") ale automatului, adică perechi formate din starea curentă şi cât a mai rămas de citit din cuvântul de intrare, şi aplicăm funcţia delta cât timp este posibil (ne oprim fie când nu mai există tranziţie definită, fie când se termină cuvântul). Dacă la final ajungem în stare finală, cuvântul va fi acceptat.

Pentru AFD:

$(q_0, bba) \vdash (q_0, ba) \vdash (q_0, a) \vdash (q_{01}, \lambda)$
Am terminat de citit cuvântul şi am ajuns în starea q_{01}, care nu este finală, deci rezultă că automatul nu acceptă acest cuvânt.

$(q_0, babbaba) \vdash (q_0, abbaba) \vdash (q_{01}, bbaba) \vdash (q_{01}, baba) \vdash (q_{01}, aba) \vdash (q_{012}, ba) \vdash (q_{012}, a) \vdash (q_{012}, \lambda)$

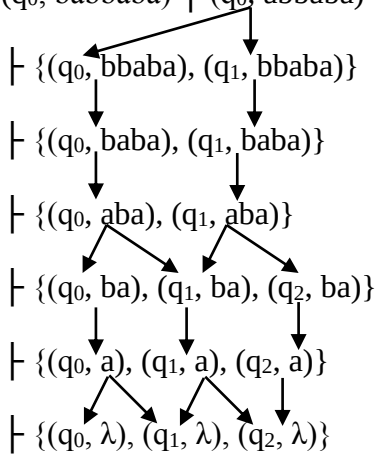
Am terminat de citit cuvântul şi am ajuns în starea q_{012}, care este finală, deci rezultă că automatul acceptă acest cuvânt.

Pentru AFN:

$(q_0, bba) \vdash (q_0, ba) \vdash (q_0, a) \vdash \{(q_0, \lambda), (q_1, \lambda)\}$

Am terminat de citit cuvântul şi am avut două drumuri posibile, care ne-au adus în stările q_0 şi q_1. Nici una din ele nu este stare finală, deci cuvântul nu este acceptat.

$(q_0, babbaba) \vdash (q_0, abbaba)$



Am terminat de citit cuvântul şi am avut şapte drumuri posibile, care ne-au adus în stările q_0, q_1 şi q_2. Există printre ele o stare care este finală, q_2, deci cuvântul este acceptat.

~ Seminar 2 ~

(AFN–λ; Transformarea AFN–λ → AFD; Automatul minimal;
Condiţii asupra numărului de apariţii ale unui caracter într-un cuvânt)

Automat Finit Nedeterminist cu λ–tranziţii

AFN–λ = (Q, Σ, q_0, δ, F)
Q mulţimea de stări
Σ alfabetul de intrare
$q_0 \in Q$ starea iniţială
$F \subseteq Q$ mulţimea de stări finale
$\delta : Q \times (\Sigma \cup \{\lambda\}) \to 2^Q$ funcţia de tranziţie („delta")

Diferenţa faţă de AFN-ul simplu este că suntem într-o stare şi putem citi fie un caracter din alfabetul Σ, fie cuvântul vid λ. Deci se poate întâmpla să ajungem într-o stare nouă fără să fi citit nicio literă nouă din cuvântul de intrare, ci doar aplicând λ–tranziţii.

λ–închiderea unei stări

Mulţimea de stări în care se poate ajunge plecând din starea q şi aplicând zero sau mai multe λ–tranziţii se numeşte „**λ–închiderea stării q**" şi se notează cu **<q>**.

Obs: Orice stare face parte din propria λ–închidere (pentru că $\delta(q, \lambda^0) = q$; practic putem presupune că orice stare are o λ–tranziţie *implicită* către ea însăşi).

$$< q > = \bigcup_{k \geq 0} \{r \mid r \in \delta(q, \lambda^k)\}$$

$$< q > = \{q\} \cup \{q_i \mid q_i \in \delta(q, \lambda^1)\} \cup \{q_{ij} \mid q_{ij} \in \delta(q, \lambda^2)\} \cup ...$$

Observăm că mulţimile se pot calcula inductiv după puterea lui λ :
$\{q_{ij} \mid q_{ij} \in \delta(q, \lambda^2)\} = \{q_{ij} \mid q_i \in \delta(q, \lambda^1), q_{ij} \in \delta(q_i, \lambda^1)\}$.
Sau în general $\{r \mid r \in \delta(q, \lambda^k)\} = \{r \mid s \in \delta(q, \lambda^{k-1}), r \in \delta(s, \lambda^1)\}$.

Verificare acceptare cuvânt de către automat AFN–λ

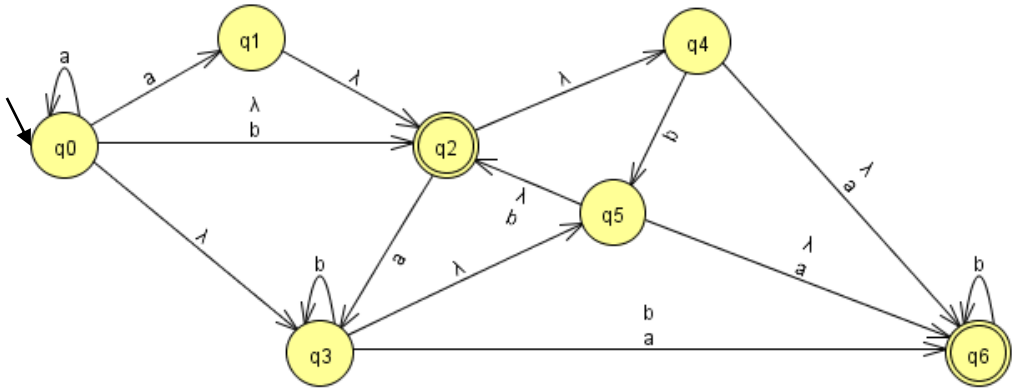
Pentru a verifica dacă un cuvânt este sau nu acceptat de un automat AFN–λ:
Se procedează analog ca în cazul AFN-ului, doar că înainte de a căuta toate stările posibile de continuare cu tranziţii cu caracterul curent, trebuie să facem λ–închiderea mulţimii curente de stări. Iar după ce a fost citit tot cuvântul de intrare, trebuie să facem o ultimă λ–închidere a stărilor curente, pentru a obţine mulţimea finală de stări în care poate ajunge automatul pentru cuvântul dat.

Obs: λ–închiderea unei mulţimi de stări este egală cu reuniunea λ–închiderilor acelor stări.

$<\{q_{i1}, q_{i2}, ..., q_{in}\}> = <q_{i1}> \cup <q_{i2}> \cup ... \cup <q_{in}>$

EXEMPLU:

Se dă următorul AFN–λ:



Calculăm λ–închiderile tuturor stărilor.

$\delta(q_0, \lambda^0) = \{q_0\}$

$\delta(q_0, \lambda^1) = \{q_2, q_3\}$

$\delta(q_0, \lambda^2) = \{\delta(q_2, \lambda)\} \cup \{\delta(q_3, \lambda)\} = \{q_4\} \cup \{q_5\} = \{q_4, q_5\}$

$\delta(q_0, \lambda^3) = \{\delta(q_4, \lambda)\} \cup \{\delta(q_5, \lambda)\} = \{q_6\} \cup \{q_2, q_6\} = \{q_2, q_6\}$ *dar* q_2 *era deja în închidere*

$\delta(q_0, \lambda^4) = \{\delta(q_6, \lambda)\} = \varnothing$

$\rightarrow <q_0> = \{q_0, q_2, q_3, q_4, q_5, q_6\}$

$\delta(q_1, \lambda^0) = \{q_1\}$

$\delta(q_1, \lambda^1) = \{q_2\}$

$\delta(q_1, \lambda^2) = \{\delta(q_2, \lambda)\} = \{q_4\}$

$\delta(q_1, \lambda^3) = \{\delta(q_4, \lambda)\} = \{q_6\}$

$\delta(q_1, \lambda^4) = \{\delta(q_6, \lambda)\} = \varnothing$

$\rightarrow <q_1> = \{q_1, q_2, q_4, q_6\}$

$\delta(q_2, \lambda^0) = \{q_2\}$

$\delta(q_2, \lambda^1) = \{q_4\}$

$\delta(q_2, \lambda^2) = \{\delta(q_4, \lambda)\} = \{q_6\}$

$\delta(q_2, \lambda^3) = \{\delta(q_6, \lambda)\} = \varnothing$

$\rightarrow <q_2> = \{q_2, q_4, q_6\}$

$\delta(q_3, \lambda^0) = \{q_3\}$
$\delta(q_3, \lambda^1) = \{q_5\}$
$\delta(q_3, \lambda^2) = \{\delta(q_5, \lambda)\} = \{q_2, q_6\}$
$\delta(q_3, \lambda^3) = \{\delta(q_2, \lambda)\} \cup \{\delta(q_6, \lambda)\} = \{q_4\} \cup \varnothing = \{q_4\}$
$\delta(q_3, \lambda^4) = \{\delta(q_4, \lambda)\} = \{q_6\}$ *dar* q_6 *era deja în închidere*
$\rightarrow < q_3 > = \{q_2, q_3, q_4, q_5, q_6\}$

$\delta(q_4, \lambda^0) = \{q_4\}$
$\delta(q_4, \lambda^1) = \{q_6\}$
$\delta(q_4, \lambda^2) = \{\delta(q_6, \lambda)\} = \varnothing$
$\rightarrow < q_4 > = \{q_4, q_6\}$

$\delta(q_5, \lambda^0) = \{q_5\}$
$\delta(q_5, \lambda^1) = \{q_2, q_6\}$
$\delta(q_5, \lambda^2) = \{\delta(q_2, \lambda)\} \cup \{\delta(q_6, \lambda)\} = \{q_4\} \cup \varnothing = \{q_4\}$
$\delta(q_5, \lambda^3) = \{\delta(q_4, \lambda)\} = \{q_6\}$ *dar* q_6 *era deja în închidere*
$\rightarrow < q_5 > = \{q_2, q_4, q_5, q_6\}$

$\delta(q_6, \lambda^0) = \{q_6\}$
$\delta(q_6, \lambda^1) = \varnothing$
$\rightarrow < q_6 > = \{q_6\}$

Verificăm dacă un cuvânt este sau nu acceptat de automatul AFN–λ dat.
pas 0: Pornim de la perechea (q_0, abbaa).

pas 1a: Facem λ–închiderea mulţimii curente de stări pentru a obţine noua mulţime de stări.
$<q_0>$ = {q_0, q_2, q_3, q_4, q_5, q_6}
pas 1b: În fiecare stare din mulţimea curentă încercăm să citim simbolul curent din cuvânt şi obţinem o nouă mulţime de stări.
$\delta(q_0, a) = \{q_0, q_1\}$
$\delta(q_2, a) = \{q_3\}$
$\delta(q_3, a) = \{q_6\}$
$\delta(q_4, a) = \{q_6\}$
$\delta(q_5, a) = \{q_6\}$
$\delta(q_6, a) = \varnothing$
Deci noua mulţime de stări este {q_0, q_1, q_3, q_6}, iar cuvântul rămas este *bbaa*.

pas 2a: Facem λ–închiderea mulţimii curente de stări pentru a obţine noua mulţime de stări.

$< q_0, q_1, q_3, q_6 > = < q_0 > \cup < q_1 > \cup < q_3 > \cup < q_6 >$

$= \{q_0, q_2, q_3, q_4, q_5, q_6\} \cup \{q_1, q_2, q_4, q_6\} \cup \{q_2, q_3, q_4, q_5, q_6\} \cup \{q_6\}$

$= \{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}$

pas 2b: În fiecare stare din mulţimea curentă încercăm să citim simbolul curent din cuvânt şi obţinem o nouă mulţime de stări.

$\delta(q_0, b) = \{q_2\}$

$\delta(q_1, b) = \varnothing$

$\delta(q_2, b) = \varnothing$

$\delta(q_3, b) = \{q_3, q_6\}$

$\delta(q_4, b) = \{q_5\}$

$\delta(q_5, b) = \{q_2\}$

$\delta(q_6, b) = \{q_6\}$

Deci noua mulţime de stări este $\{q_2, q_3, q_5, q_6\}$, iar cuvântul rămas este *baa*.

pas 3a: Facem λ–închiderea mulţimii curente de stări pentru a obţine noua mulţime de stări.

$< q_2, q_3, q_5, q_6 > = < q_2 > \cup < q_3 > \cup < q_5 > \cup < q_6 >$

$= \{q_2, q_4, q_6\} \cup \{q_2, q_3, q_4, q_5, q_6\} \cup \{q_2, q_4, q_5, q_6\} \cup \{6_6\}$

$= \{q_2, q_3, q_4, q_5, q_6\}$

pas 3b: În fiecare stare din mulţimea curentă încercăm să citim simbolul curent din cuvânt şi obţinem o nouă mulţime de stări.

$\delta(q_2, b) = \varnothing$

$\delta(q_3, b) = \{q_3, q_6\}$

$\delta(q_4, b) = \{q_5\}$

$\delta(q_5, b) = \{q_2\}$

$\delta(q_6, b) = \{q_6\}$

Deci noua mulţime de stări este $\{q_2, q_3, q_5, q_6\}$, iar cuvântul rămas este *aa*.

pas 4a: Facem λ–închiderea mulţimii curente de stări pentru a obţine noua mulţime de stări.

$< q_2, q_3, q_5, q_6 > = < q_2 > \cup < q_3 > \cup < q_5 > \cup < q_6 >$

$= \{q_2, q_4, q_6\} \cup \{q_2, q_3, q_4, q_5, q_6\} \cup \{q_2, q_4, q_5, q_6\} \cup \{q_6\}$

$= \{q_2, q_3, q_4, q_5, q_6\}$

pas 4b: În fiecare stare din mulţimea curentă încercăm să citim simbolul curent din cuvânt şi obţinem o nouă mulţime de stări.

$\delta(q_2, a) = \{q_3\}$
$\delta(q_3, a) = \{q_6\}$
$\delta(q_4, a) = \{q_6\}$
$\delta(q_5, a) = \{q_6\}$
$\delta(q_6, a) = \varnothing$
Deci noua mulţime de stări este $\{q_3, q_6\}$, iar cuvântul rămas este *a*.

pas 5a: Facem λ–închiderea mulţimii curente de stări pentru a obţine noua mulţime de stări.
$< q_3, q_6 > = < q_3 > \cup < q_6 >$
$= \{q_2, q_3, q_4, q_5, q_6\} \cup \{q_6\}$
$= \{q_2, q_3, q_4, q_5, q_6\}$
pas 5b: În fiecare stare din mulţimea curentă încercăm să citim simbolul curent din cuvânt şi obţinem o nouă mulţime de stări.
$\delta(q_2, a) = \{q_3\}$
$\delta(q_3, a) = \{q_6\}$
$\delta(q_4, a) = \{q_6\}$
$\delta(q_5, a) = \{q_6\}$
$\delta(q_6, a) = \varnothing$
Deci noua mulţime de stări este $\{q_3, q_6\}$, iar cuvântul a fost citit în întregime.

pas 6: Facem ultima λ–închidere a mulţimii curente de stări pentru a obţine mulţimea finală de stări.
$< q_3, q_6 > = < q_3 > \cup < q_6 >$
$= \{q_2, q_3, q_4, q_5, q_6\} \cup \{q_6\}$
$= \{q_2, q_3, q_4, q_5, q_6\}$
pas 7: Verificăm dacă printre stările în care am ajuns este vreo una finală.
$\{q_2, q_3, q_4, q_5, q_6\} \cap F = \{q_2, q_3, q_4, q_5, q_6\} \cap \{q_2, q_6\} = \{q_2, q_6\} \neq \varnothing$
Deci cuvântul *abbaa* este acceptat de automatul AFN–λ dat.

Transformarea AFN–λ → AFD

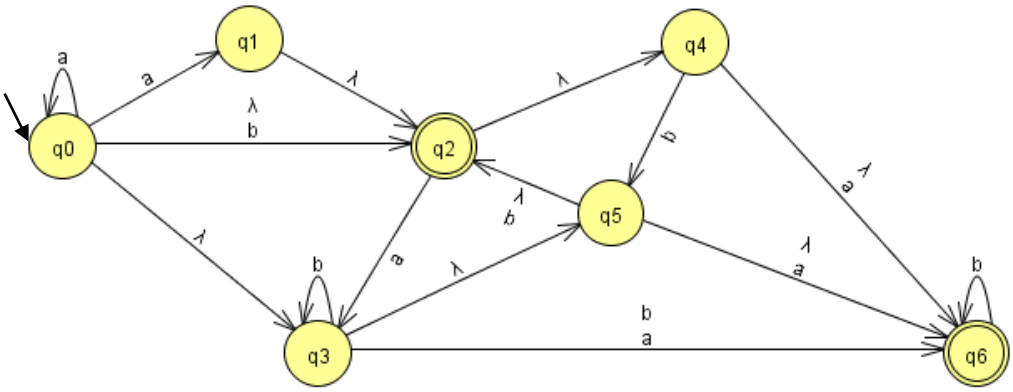
Având dat un AFN–λ (reprezentat sub formă de graf), vom calcula funcţia delta completând un tabel asemănător cu cel de la algoritmul de transformare AFN →AFD.

- La acest algoritm, capetele de coloană vor fi caracterele din alfabet, dar concatenate la stânga şi la dreapta cu λ^*.
- Pentru a calcula primele |Q| linii ale tabelului
  - facem λ–închiderea stării de la capătul de linie curent
  - apoi în toate stările obţinute încercăm să citim caracterul de pe coloana curentă şi obţinem o nouă mulţime de stări
  - facem λ–închiderea acestei mulţimi şi ceea ce obţinem scriem în tabel

- Pentru a calcula următoarele linii (corespunzătoare noilor stări-mulţime care apar), procedăm la fel ca la celălalt algoritm, adică reunim ce găsim în tabel pe aceeaşi coloană şi pe liniile potrivite stărilor componente din starea-mulţime.
- Stările finale ale noului AFD obţinut, vor fi acele stări-mulţime (pot avea şi un singur element, nu neapărat mai multe) care conţin cel puţin una din stările care erau finale pentru AFN-ul iniţial. **În plus**, trebuie să verificăm daca automatul iniţial accepta cuvântul vid λ (adică dacă $< q_0 > \cap F_{AFN-\lambda} \neq \varnothing$). Dacă da, atunci starea iniţială (<q0>) va fi şi stare finală.

EXEMPLU:

Plecăm de la acelaşi automat de mai sus.



Întâi trebuie calculate λ–închiderile tuturor stărilor automatului iniţial.
(Am făcut asta mai sus, deci doar rescriu ce am obţinut.)

$< q_0 > = \{q_0, q_2, q_3, q_4, q_5, q_6\}$

$< q_1 > = \{q_1, q_2, q_4, q_6\}$

$< q_2 > = \{q_2, q_4, q_6\}$

$< q_3 > = \{q_2, q_3, q_4, q_5, q_6\}$

$< q_4 > = \{q_4, q_6\}$

$< q_5 > = \{q_2, q_4, q_5, q_6\}$

$< q_6 > = \{q_6\}$

Avem tabelul pentru primele 7 linii obţinut după cum urmează:

δ	$\lambda^* \cdot a \cdot \lambda^*$	$\lambda^* \cdot b \cdot \lambda^*$
q_0	$\{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}$	$\{q_2, q_3, q_4, q_5, q_6\}$
q_1	$\{q_2, q_3, q_4, q_5, q_6\}$	$\{q_2, q_4, q_5, q_6\}$
q_2	$\{q_2, q_3, q_4, q_5, q_6\}$	$\{q_2, q_4, q_5, q_6\}$
q_3	$\{q_2, q_3, q_4, q_5, q_6\}$	$\{q_2, q_3, q_4, q_5, q_6\}$
q_4	$\{q_6\}$	$\{q_2, q_4, q_5, q_6\}$
q_5	$\{q_2, q_3, q_4, q_5, q_6\}$	$\{q_2, q_4, q_5, q_6\}$
q_6	$\varnothing$	$\{q_6\}$

$$<\delta(<q_0>,a)> = <\delta(\{q_0,q_2,q_3,q_4,q_5,q_6\},a)>$$
$$= <\{\delta(q_0,a)\}\cup\{\delta(q_2,a)\}\cup\{\delta(q_3,a)\}\cup\{\delta(q_4,a)\}\cup\{\delta(q_5,a)\}\cup\{\delta(q_6,a)\}>$$
$$= <\{q_0,q_1\}\cup\{q_3\}\cup\{q_6\}\cup\{q_6\}\cup\{q_6\}\cup\varnothing> = <\{q_0,q_1,q_3,q_6\}>$$
$$= <q_0>\cup<q_1>\cup<q_3>\cup<q_6>$$
$$= \{q_0,q_2,q_3,q_4,q_5,q_6\}\cup\{q_1,q_2,q_4,q_6\}\cup\{q_2,q_3,q_4,q_5,q_6\}\cup\{q_6\} = \{q_0,q_1,q_2,q_3,q_4,q_5,q_6\}$$

$$<\delta(<q_0>,b)> = <\delta(\{q_0,q_2,q_3,q_4,q_5,q_6\},b)>$$
$$= <\{\delta(q_0,b)\}\cup\{\delta(q_2,b)\}\cup\{\delta(q_3,b)\}\cup\{\delta(q_4,b)\}\cup\{\delta(q_5,b)\}\cup\{\delta(q_6,b)\}>$$
$$= <\{q_2\}\cup\varnothing\cup\{q_3,q_6\}\cup\{q_5\}\cup\{q_2\}\cup\{q_6\}> = <\{q_2,q_3,q_5,q_6\}>$$
$$= <q_2>\cup<q_3>\cup<q_5>\cup<q_6>$$
$$= \{q_2,q_4,q_6\}\cup\{q_2,q_3,q_4,q_5,q_6\}\cup\{q_2,q_4,q_5,q_6\}\cup\{q_6\} = \{q_2,q_3,q_4,q_5,q_6\}$$

$$<\delta(<q_1>,a)> = <\delta(\{q_1,q_2,q_4,q_6\},a)>$$
$$= <\{\delta(q_1,a)\}\cup\{\delta(q_2,a)\}\cup\{\delta(q_4,a)\}\cup\{\delta(q_6,a)\}>$$
$$= <\varnothing\cup\{q_3\}\cup\{q_6\}\cup\varnothing> = <\{q_3,q_6\}>$$
$$= <q_3>\cup<q_6>$$
$$= \{q_2,q_3,q_4,q_5,q_6\}\cup\{q_6\} = \{q_2,q_3,q_4,q_5,q_6\}$$

$$<\delta(<q_1>,b)> = <\delta(\{q_1,q_2,q_4,q_6\},b)>$$
$$= <\{\delta(q_1,b)\}\cup\{\delta(q_2,b)\}\cup\{\delta(q_4,b)\}\cup\{\delta(q_6,b)\}>$$
$$= <\varnothing\cup\varnothing\cup\{q_5\}\cup\{q_6\}> = <\{q_5,q_6\}>$$
$$= <q_5>\cup<q_6>$$
$$= \{q_2,q_4,q_5,q_6\}\cup\{q_6\} = \{q_2,q_4,q_5,q_6\}$$

$$<\delta(<q_2>,a)> = <\delta(\{q_2,q_4,q_6\},a)>$$
$$= <\{\delta(q_2,a)\}\cup\{\delta(q_4,a)\}\cup\{\delta(q_6,a)\}>$$
$$= <\{q_3\}\cup\{q_6\}\cup\varnothing> = <\{q_3,q_6\}>$$
$$= <q_3>\cup<q_6>$$
$$= \{q_2,q_3,q_4,q_5,q_6\}\cup\{q_6\} = \{q_2,q_3,q_4,q_5,q_6\}$$

$$<\delta(<q_2>,b)> = <\delta(\{q_2,q_4,q_6\},b)>$$
$$= <\{\delta(q_2,b)\}\cup\{\delta(q_4,b)\}\cup\{\delta(q_6,b)\}>$$
$$= <\varnothing\cup\{q_5\}\cup\{q_6\}> = <\{q_5,q_6\}>$$
$$= <q_5>\cup<q_6>$$
$$= \{q_2,q_4,q_5,q_6\}\cup\{q_6\} = \{q_2,q_4,q_5,q_6\}$$

$$<\delta(<q_3>,a)> = <\delta(\{q_2,q_3,q_4,q_5,q_6\},a)>$$
$$= <\{\delta(q_2,a)\}\cup\{\delta(q_3,a)\}\cup\{\delta(q_4,a)\}\cup\{\delta(q_5,a)\}\cup\{\delta(q_6,a)\}>$$
$$= <\{q_3\}\cup\{q_6\}\cup\{q_6\}\cup\{q_6\}\cup\varnothing> = <\{q_3,q_6\}>$$
$$= <q_3>\cup<q_6>$$
$$= \{q_2,q_3,q_4,q_5,q_6\}\cup\{q_6\} = \{q_2,q_3,q_4,q_5,q_6\}$$

$$<\delta(<q_3>,b)> = <\delta(\{q_2,q_3,q_4,q_5,q_6\},b)>$$
$$= <\{\delta(q_2,b)\}\cup\{\delta(q_3,b)\}\cup\{\delta(q_4,b)\}\cup\{\delta(q_5,b)\}\cup\{\delta(q_6,b)\}>$$
$$= <\varnothing\cup\{q_3,q_6\}\cup\{q_5\}\cup\{q_2\}\cup\{q_6\}> = <\{q_2,q_3,q_5,q_6\}>$$
$$= <q_2>\cup<q_3>\cup<q_5>\cup<q_6>$$
$$= \{q_2,q_4,q_6\}\cup\{q_2,q_3,q_4,q_5,q_6\}\cup\{q_2,q_4,q_5,q_6\}\cup\{q_6\} = \{q_2,q_3,q_4,q_5,q_6\}$$

$$<\delta(<q_4>,a)> = <\delta(\{q_4,q_6\},a)> = <\{\delta(q_4,a)\}\cup\{\delta(q_6,a)\}>$$
$$= <\{q_6\}\cup\varnothing> = <\{q_6\}> = <q_6> = \{q_6\}$$

$$<\delta(<q_4>,b)> = <\delta(\{q_4,q_6\},b)> = <\{\delta(q_4,b)\}\cup\{\delta(q_6,b)\}>$$
$$= <\{q_5\}\cup\{q_6\}> = <\{q_5,q_6\}> = <q_5>\cup<q_6> = \{q_2,q_4,q_5,q_6\}\cup\{q_6\} = \{q_2,q_4,q_5,q_6\}$$

$$<\delta(<q_5>,a)> = <\delta(\{q_2,q_4,q_5,q_6\},a)>$$
$$= <\{\delta(q_2,a)\}\cup\{\delta(q_4,a)\}\cup\{\delta(q_5,a)\}\cup\{\delta(q_6,a)\}>$$
$$= <\{q_3\}\cup\{q_6\}\cup\{q_6\}\cup\varnothing> = <\{q_3,q_6\}>$$
$$= <q_3>\cup<q_6>$$
$$= \{q_2,q_3,q_4,q_5,q_6\}\cup\{q_6\} = \{q_2,q_3,q_4,q_5,q_6\}$$

$$<\delta(<q_5>,b)> = <\delta(\{q_2,q_4,q_5,q_6\},b)>$$
$$= <\{\delta(q_2,b)\}\cup\{\delta(q_4,b)\}\cup\{\delta(q_5,b)\}\cup\{\delta(q_6,b)\}>$$
$$= <\varnothing\cup\{q_5\}\cup\{q_2\}\cup\{q_6\}> = <\{q_2,q_5,q_6\}>$$
$$= <q_2>\cup<q_5>\cup<q_6>$$
$$= \{q_2,q_4,q_6\}\cup\{q_2,q_4,q_5,q_6\}\cup\{q_6\} = \{q_2,q_4,q_5,q_6\}$$

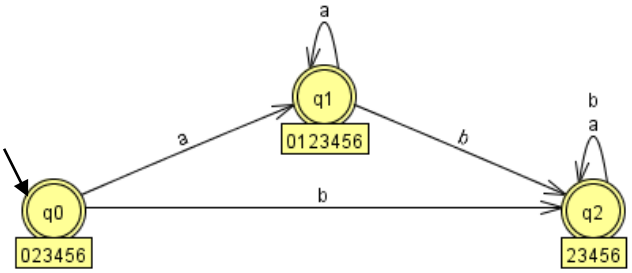
$$<\delta(<q_6>,a)> = <\delta(\{q_6\},a)> = <\{\delta(q_6,a)\}> = <\varnothing> = \varnothing$$

$$<\delta(<q_6>,b)> = <\delta(\{q_6\},b)> = <\{\delta(q_6,b)\}> = <\{q_6\}> = <q_6> = \{q_6\}$$

Apoi calculăm liniile următoare, corespunzătoare noilor stări-mulţime, pe măsură ce ele apar în automat. Începem cu starea iniţială, $<q_0> = q_{023456}$.

δ	$\lambda^* \cdot a \cdot \lambda^*$	$\lambda^* \cdot b \cdot \lambda^*$
q_0	$\{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}$	$\{q_2, q_3, q_4, q_5, q_6\}$
q_1	$\{q_2, q_3, q_4, q_5, q_6\}$	$\{q_2, q_4, q_5, q_6\}$
q_2	$\{q_2, q_3, q_4, q_5, q_6\}$	$\{q_2, q_4, q_5, q_6\}$
q_3	$\{q_2, q_3, q_4, q_5, q_6\}$	$\{q_2, q_3, q_4, q_5, q_6\}$
q_4	$\{q_6\}$	$\{q_2, q_4, q_5, q_6\}$
q_5	$\{q_2, q_3, q_4, q_5, q_6\}$	$\{q_2, q_4, q_5, q_6\}$
q_6	$\varnothing$	$\{q_6\}$
q_{023456}	$q_{0123456}$	q_{23456}
$q_{0123456}$	$q_{0123456}$	q_{23456}
q_{23456}	q_{23456}	q_{23456}

Vom obţine în final automatul AFD următor.



Automatul iniţial avea stările finale $F_{AFN-\lambda} = \{q_2, q_6\}$, deci în noul automat vom avea stările finale $F_{AFD} = \{ q_{023456}, q_{0123456}, q_{23456}\}$.

Obs: Limbajul acceptat de acest automat este $L = \{a,b\}^*$, adică orice cuvinte formate cu literele *a* şi *b* în orice ordine, cuvinte de orice lungime, inclusiv zero.

Obs: Cred că la seminar am uitat să punem şi tranziţia de la q_5 la q_2 cu b, aşa că pot exista mici diferenţe la calculele intermediare, dar rezultatul final este acelaşi.

Automatul minimal

Se porneşte de la un AFD complet definit (adică pentru orice stare din mulţimea Q şi pentru orice caracter din alfabetul Σ, funcţia δ este definită).

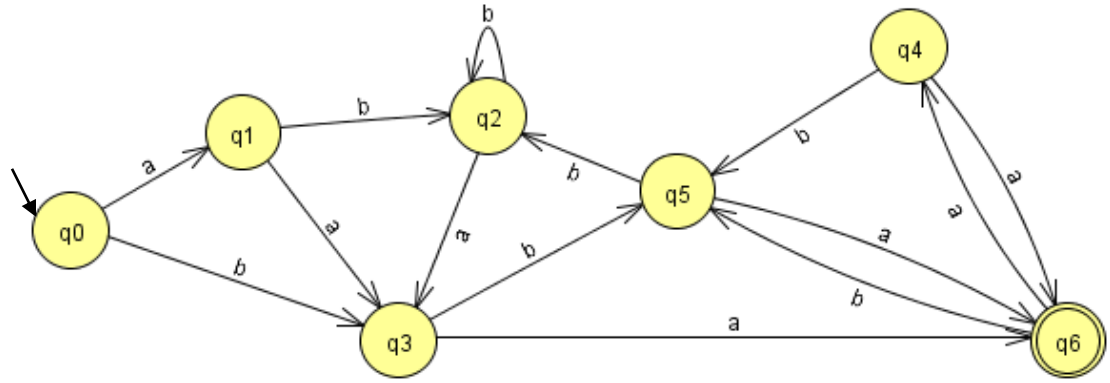
Ideea algoritmului este de a găsi acele stări care au comportament echivalent, pentru a le grupa şi a obţine o unică stare nouă în locul acestora.

- Două stări sunt „**echivalente**" dacă pentru orice cuvânt am alege, plecând din cele două stări, fie ajungem în două stări finale, fie ajungem în două stări nefinale.
  $\forall p,q \in Q,\ p \equiv q \Leftrightarrow [\forall w \in \Sigma^*, \delta(p,w) \in F \leftrightarrow \delta(q,w) \in F]$
- Două stări sunt „**separabile**" dacă există un cuvânt pentru care plecând din cele două stări ajungem într-o stare finală şi într-una nefinală.
  $\forall p,q \in Q,\ p \not\equiv_w q \Leftrightarrow [\exists w \in \Sigma^*, \delta(p,w) \in F \leftrightarrow \delta(q,w) \notin F]$

Vom construi un tabel, pe linii şi pe coloane având stările automatului iniţial. Vom completa tabelul pentru fiecare pereche de stări (q_i, q_j), având i > j, cu un cuvânt prin care cele două stări sunt separabile. Dacă nu vom găsi un astfel de cuvânt atunci acele stări sunt echivalente.

Vom completa tabelul căutând cuvintele recursiv, în ordinea crescătoare a lungimii lor. Cuvintele de lungime *k* le vom obţine cu ajutorul celor de lungime *k*–1 calculate anterior.
Obs: Două stări sunt separabile prin cuvântul vid λ (de lungime zero), dacă una din ele este finală şi cealaltă este nefinală în automatul iniţial.

EXEMPLU:
Fie automatul AFD complet definit:



Pas 0: Căutăm perechile de stări separabile prin cuvântul λ de lungime zero. Avem o singură stare finală în acest exemplu, deci ea va fi separabilă prin λ de toate celelalte stări, care sunt nefinale. Completăm în tabel, pentru toate perechile (q_6, q_i), $0 \le i \le 5$.

	q_0	q_1	q_2	q_3	q_4	q_5	q_6
q_0							
q_1							
q_2							
q_3							
q_4							
q_5							
q_6	λ	λ	λ	λ	λ	λ	

Pas 1: Căutăm cuvinte de lungime 1.

Avem $\delta(q_0, a) = q_1 \notin F$; $\delta(q_1, a) = q_3 \notin F$; $\delta(q_2, a) = q_3 \notin F$;
$\delta(q_3, a) = q_6 \in F$; $\delta(q_4, a) = q_6 \in F$; $\delta(q_5, a) = q_6 \in F$

Rezultă că orice stare din mulţimea $\{q_0, q_1, q_2\}$ va fi separabilă de orice stare din mulţimea $\{q_3, q_4, q_5\}$ prin cuvântul "a".
(Observăm că toate tranziţiile cu "b" merg către stări nefinale, deci nu va exista nicio pereche de stări care să fie separabile prin cuvântul "b".)

Obs: E posibil să fie separabile şi prin altceva, dar este suficient să găsim un singur cuvânt.

	q_0	q_1	q_2	q_3	q_4	q_5	q_6
q_0							
q_1							
q_2							
q_3	a	a	a				
q_4	a	a	a				
q_5	a	a	a				
q_6	λ	λ	λ	λ	λ	λ	

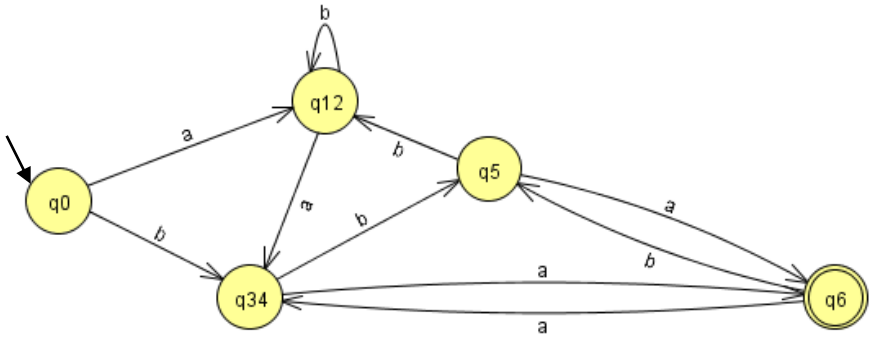
Pas 2: Căutăm cuvinte de lungime 2.

- Perechea (q_1, q_0) cu caracterul "a" ajunge în perechea (q_3, q_1). Dar q_3 şi q_1 erau separabile prin "a". Rezultă că stările q_1 şi q_0 vor fi separabile prin cuvântul "aa" (*obţinut prin concatenarea caracterului încercat şi a fostului cuvânt calculat în tabel*).
- Perechea (q_2, q_0) cu caracterul "a" ajunge în perechea (q_3, q_1). Dar q_3 şi q_1 erau separabile prin "a". Rezultă că stările q_1 şi q_0 vor fi separabile prin cuvântul "aa".
- Perechea (q_2, q_1) cu caracterul "a" ajunge în perechea (q_3, q_3), iar cu caracterul "b" ajunge în perechea (q_2, q_2). *Observăm* că dacă după citirea primului caracter (indiferent dacă este "a" sau "b"), ambele drumuri pornesc din acelaşi loc, nu au cum să se mai ramifice în continuare. Rezultă că aceste două stări, q_2 şi q_1, sunt echivalente.
- Perechea (q_4, q_3) cu caracterul "a" ajunge în perechea (q_6, q_6), iar cu caracterul "b" ajunge în perechea (q_5, q_5). Cu aceeaşi observaţie ca mai devreme, rezultă că stările q_4 şi q_3 sunt echivalente.
- Perechea (q_5, q_3) cu caracterul "a" ajunge în perechea (q_6, q_6), dar cu caracterul "b" ajunge în perechea (q_2, q_5). Perechea (q_2, q_5) era separabilă prin "a". Rezultă, prin concatenare, că stările q_5 şi q_3 sunt separabile prin "ba".
- Perechea (q_5, q_4) cu caracterul "a" ajunge în perechea (q_6, q_6), dar cu caracterul "b" ajunge în perechea (q_2, q_5). Perechea (q_2, q_5) era separabilă prin "a". Rezultă, prin concatenare, că stările q_5 şi q_4 sunt separabile prin "ba".

	q_0	q_1	q_2	q_3	q_4	q_5	q_6
q_0							
q_1	aa						
q_2	aa	∅					
q_3	a	a	a				
q_4	a	a	a	∅			
q_5	a	a	a	ba	ba		
q_6	λ	λ	λ	λ	λ	λ	

Am terminat de completat tabelul şi am obţinut stări echivalente: $q_1 \equiv q_2$ *şi* $q_3 \equiv q_4$.

Automatul AFD minimal obţinut va avea Q = {q_0, q_{12}, q_{34}, q_5, q_6} şi F = {q_6}.
Tranziţiile le desenăm conform automatului iniţial, dar ţinând cont de această grupare a stărilor.



Condiţii asupra numărului de apariţii ale unui caracter într-un cuvânt

Notăm cu $|w|_a$ numărul de apariţii ale caracterului „a” în cuvântul „w”.

Obs: În general, când avem limbaje definite cu astfel de condiţii asupra numărului de apariţii ale unui caracter, sugestia pentru a putea construi mai uşor automatul este să folosim nişte indici la stări pentru o codificare cât mai intuitivă a cuvintelor din limbaj.

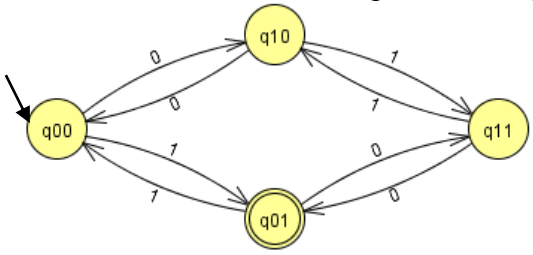
Atenţie! Nu confundaţi exemplele următoare cu cele din primul seminar, în sensul de a încerca să descrieţi limbajele acestea cu expresii asemănătoare cu acelea cu puteri. Acolo existau în plus restricţii asupra poziţiei literelor în cuvânt (de exemplu toate a-urile la început şi apoi toate b-urile). Aici nu contează ordinea literelor, ci doar numărul lor.

EXEMPLE:

$L_1 = \{w \in \{0,1\}^* \; a.î. \; |w|_0 = par, |w|_1 = impar\}$

Ideea de rezolvare este ca fiecare stare să aibă doi indici, egali cu restul împărţirii la 2 a numărului de apariţii ale caracterului “0”, respectiv ale caracterului “1”.

Starea finală va fi q_{01} pentru că ea codifică numărul par de 0-uri şi impar de 1-uri.



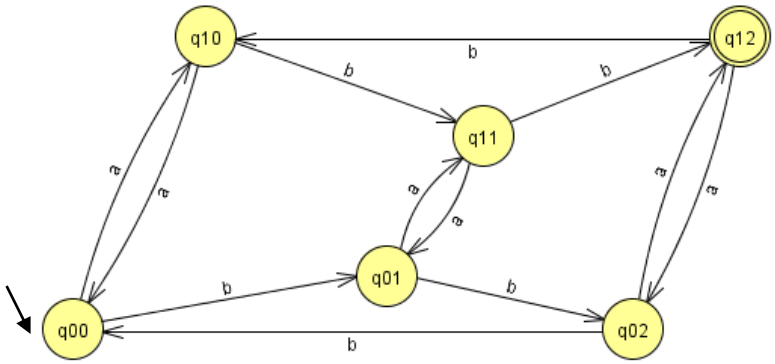
Obs: Pentru că nu există nicio altă restricţie asupra formei cuvântului, automatul va fi *complet definit*, adică în orice stare am fi trebuie să avem tranziţii cu toate caracterele din alfabet.

$L_2 = \{w \in \{a,b\}^* \; a.î. \; |w|_a = 2n+1, |w|_b = 3m+2, \; n \geq 0, \; m \geq 0\}$

Obs:

- Fiecare stare va avea doi indici pentru că alfabetul are două elemente şi ambele au condiţii ataşate.
- Primul indice, corespunzător caracterului "a", va lua valori în mulţimea resturilor împărţirii la 2, deoarece coeficientul necunoscutei "n" este 2. Al doilea indice, corespunzător caracterului "b", va lua valori în mulţimea resturilor împărţirii la 3, deoarece coeficientul necunoscutei "m" este 3.
- Numărul de stări va fi egal cu produsul numerelor ce reprezintă cardinalele mulţimilor în care iau valori indicii, în cazul acesta 2·3=6.

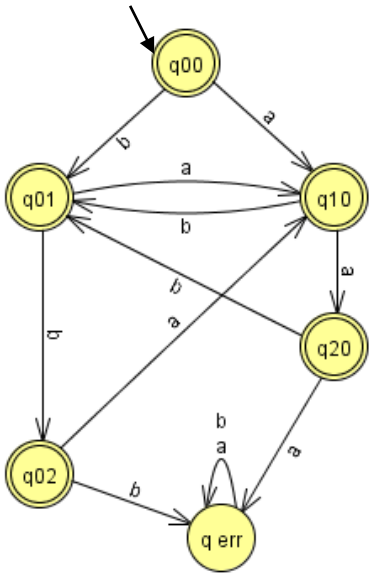
Starea finală va fi q_{12}, deoarece termenii liberi sunt "+ 1" şi "+ 2".

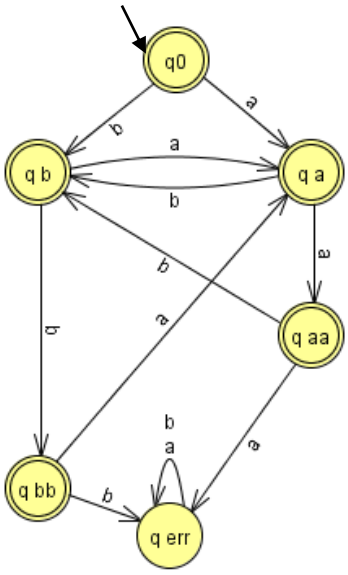


$L_3 = \{w \in \{a,b\}^* \; a.î. \; aaa \; şi \; bbb \; nu \; apar \; în \; w\}$

Obs: Acest limbaj ar mai putea fi scris şi $L_3 = \{\lambda, b, bb\} \cdot (\{a, aa\} \cdot \{b, bb\})^* \cdot \{\lambda, a, aa\}$.

Sunteţi de acord că e adevărat? Ştiţi să explicaţi cum am ajuns la acea expresie ?





~ Seminar 3 ~

(Expresii regulate; Transformarea expresie_regulată → automat_finit
Transformarea automat_finit → expresie_regulată ; Gramatici ; Gramatici regulate)

Expresii regulate

Se numeşte familia expresiilor regulate peste Σ şi se notează REX(Σ) mulţimea de cuvinte peste alfabetul $\Sigma \cup \{(,),+,\cdot,*,\varnothing,\lambda\}$ definită recursiv astfel:

i) $\varnothing, \lambda \in REX$; $a \in REX$ *pentru* $\forall a \in \Sigma$

ii) $e_1, e_2 \in REX \Rightarrow (e_1 + e_2) \in REX$

iii) $e_1, e_2 \in REX \Rightarrow (e_1 \cdot e_2) \in REX$

iv) $e \in REX \Rightarrow (e^*) \in REX$

Precedenţa operaţiilor $*, \cdot, +$

Obs: Atenţie la ordinea în care se evaluează operaţiile: întâi stelarea, apoi concatenarea şi apoi reuniunea. *(Dacă vreţi să fiţi siguri că nu le încurcaţi, puteţi să faceţi o analogie cu operaţiile aritmetice, unde se evaluează întâi ridicarea la putere, apoi înmulţirea şi apoi adunarea.)*

Transformarea expresie_regulată → automat_finit

Expresie regulată	Limbaj	Automat finit
$\varnothing$	$\varnothing$	$Q = \{q_0\}, F = \varnothing, \delta = \varnothing$
λ	$\{\lambda\}$	$Q = \{q_0\}, F = \{q_0\}, \delta = \varnothing$
a	$\{a\}$, $a \in \Sigma$	$Q = \{q_0, q_1\}, F = \{q_1\}, \delta(q_0, a) = q_1$
$e_1 + e_2$	$L(e_1 + e_2)$ $= L(e_1) \cup L(e_2)$	$Q = Q_1 \cup Q_2 \cup \{q_0\}; F = F_1 \cup F_2; V = V_1 \cup V_2;$ $\delta = \delta_1 \cup \delta_2 \cup \{\delta(q_0, \lambda) = \{q_{01}, q_{02}\}\}$
$e_1 \cdot e_2$	$L(e_1 \cdot e_2)$ $= L(e_1) \cdot L(e_2)$	$Q = Q_1 \cup Q_2, q_0 = q_{01}; F = F_2; V = V_1 \cup V_2;$ $\delta = \delta_1 \cup \delta_2 \cup \{\delta(q_{f1}, \lambda) = q_{02}, pentru \forall q_{f1} \in F_1\}$
$(e_1)^*$	$L(e_1^*) = (L(e_1))^*$	$Q = Q_1 \cup \{q_0\}; F = \{q_0\}; V = V_1;$ $\delta = \delta_1 \cup \{\delta(q_0, \lambda) = q_{01}\} \cup \{\delta(q_{f1}, \lambda) = q_0, pentru \forall q_{f1} \in F_1\}$

Descrierea în cuvinte a ultimelor trei linii ale tabelului o găsiţi în acest document la pagina 9 (compunerea automatelor finite).

Obs: La construirea automatelor compuse se ţine cont de precedenţa operaţiilor (întâi stelarea, apoi concatenarea, apoi reuniunea) şi de parantezele din expresie.

Transformarea automat_finit → expresie_regulată

Se numeşte *EFA (automat finit extins),* $M = (Q, \Sigma, et, q_0, F)$, unde, la fel ca la celelalte automate finite, Q este mulţimea stărilor, Σ este alfabetul, q_0 este starea iniţială, F este mulţimea stărilor finale. Aici avem *funcţia de etichetare* $et : Q \times Q \to REX(\Sigma)$.
Notăm $et(p,q)$ prin e_{pq}.

Ideea algoritmului este de a transforma automatul finit într-un automat finit extins, şi apoi a elimina una câte una stările până ajungem la o expresie regulată echivalentă cu automatul iniţial.

(a) Dacă există tranziţii care ajung în starea iniţială, atunci se adaugă la automat o nouă stare care va fi iniţială şi va avea o λ–tranziţie către fosta stare iniţială.
(b) Dacă există mai multe stări finale, atunci se adaugă la automat o nouă stare care va fi singura finală şi va avea λ–tranziţii din toate fostele stări finale către ea.
(c) Se elimină pe rând, una câte una, toate stările în afară de cea iniţială şi cea finală. Presupunem că vrem să eliminăm starea q_k şi că există etichetele $et(q_i, q_k), et(q_k, q_j)$ şi eventual bucla $et(q_k, q_k)$. Atunci obţinem noua etichetă între stările q_i şi q_j reunind (fosta etichetă directă de la q_i la q_j) cu ("eticheta de la q_i la q_k" concatenată cu "stelarea etichetei buclei de la q_k la q_k" concatenată cu "eticheta de la q_k la q_j").
(d) Atunci când rămân doar două stări, se elimină bucla stării finale, dacă există. Concatenând (expresia obţinută între starea iniţială şi cea finală) cu (stelarea expresiei de pe bucla stării finale) obţinem răspunsul final.

În practică, vom calcula etichetele folosind un tabel cu numărul de coloane complete egal cu numărul de stări ale automatului, şi cu numărul de linii egal cu pătratul numărului de stări ale automatului.
Obs: Atenţie, stările automatului vor fi numerotate începând de la q_1 (care va fi starea iniţială).
Notăm cu R^k_{ij} expresia regulată ce se obţine plecând din starea q_i, ajungând în starea q_j şi având ca stări intermediare primele k stări (adică q_1, q_2, ..., q_k).

Tabelul va conţine la capete de linii toate perechile posibile (q_i, q_j) cu i şi j între 1 şi |Q|, iar la capete de coloană toţi k, cu k de la 0 la |Q| – 1.
Tabelul va fi calculat coloană cu coloană, pentru că valorile coloanei curente se calculează în funcţie de valori aflate pe coloana precedentă.
Prima coloană se calculează astfel:

$$R^0_{ij} = \begin{cases} \{a \in V \mid \delta(q_i, a) = q_j\}, dacă\ i \neq j \\ \{\lambda\} \cup \{a \in V \mid \delta(q_i, a) = q_j\}, dacă\ i = j \end{cases}$$

Următoarele coloane se calculează astfel:
(coloana curentă k+1 în funcţie de coloana precedentă k)
$R_{i\,j}^{k+1} = R_{i\,j}^{k} \cup R_{i\,k+1}^{k} \cdot (R_{k+1\,k+1}^{k})^{*} \cdot (R_{k+1\,j}^{k})$

Răspunsul final, expresia regulată căutată, va fi obţinută calculând $\bigcup_{f \in F} R_{1\,f}^{|Q|}$, adică reuniunea etichetelor de la starea iniţială la toate stările finale, având ca stări intermediare toate stările automatului.

Câteva formule utile

(A) $e \cdot \varnothing = \varnothing$; $\varnothing \cdot e = \varnothing$ ($\varnothing$ este pentru concatenare cum este 0 pentru înmulţire)
(B) $e \cdot \lambda = e$; $\lambda \cdot e = e$ (λ este pentru concatenare cum este 1 pentru înmulţire)
(C) $e^{*} \cdot e = e^{+}$; $e \cdot e^{*} = e^{+}$
(D) $\{e_1, e_2\}^{*} = (e_1 + e_2)^{*} = (e_1^{*} \cdot e_2^{*})^{*}$ *(formulă valabilă pentru oricâte expresii, nu doar 2)*
(E) $e_1 \cdot (e_2 + e_3) = (e_1 \cdot e_2) + (e_1 \cdot e_3)$; $(e_1 + e_2) \cdot e_3 = (e_1 \cdot e_3) + (e_2 \cdot e_3)$

EXEMPLU:

Se dă automatul finit următor şi se doreşte obţinerea expresiei regulate care îi corespunde, folosind tabelul descris anterior.



Avem două stări, deci tabelul va avea $2^2 = 4$ linii şi 2 coloane complete (plus eventual una incompletă pentru răspunsul final).

$R_{i\,j}^{k+1}$	k+1 = 0	k+1 = 1	k+1 = 2
$r_{ij} = r_{11}$	$1 + \lambda$		
$r_{ij} = r_{12}$	0		
$r_{ij} = r_{21}$	$\varnothing$		
$r_{ij} = r_{22}$	λ		

(a) Calculăm prima coloană, pentru k+1 = 0.
- Dacă indicii i şi j sunt diferiţi, atunci căutăm tranziţiile de la q_i la q_j şi scriem în tabel (pe linia corespunzătoare lui r_{ij}) expresia obţinută prin reuniunea acelor caractere de pe tranziţii.
Deci $R_{12}^{0} = 0$, $R_{21}^{0} = \varnothing$.
- Dacă indicii i şi j sunt egali, căutăm buclele stării q_i=q_j şi scriem în tabel expresia obţinută prin reuniunea acelor caractere de pe bucle şi a caracterului λ.
Deci $R_{11}^{0} = 1 + \lambda$, $R_{22}^{0} = \lambda$.

(b) Calculăm a doua coloană, pentru k+1 = 1.

$R_{11}^1 = R_{11}^0 \cup R_{11}^0 \cdot (R_{11}^0)^* \cdot R_{11}^0 = (1+\lambda) \cup (1+\lambda) \cdot (1+\lambda)^* \cdot (1+\lambda)$

Dacă notăm $(1+\lambda)$ cu b,

avem $b \cup b \cdot b^* \cdot b = b^1 \cup \{b^n \mid n \geq 2\} = \{b^n \mid n \geq 1\} = b^+ = (1+\lambda)^+$

Putem aplica formula (C) şi avem $(1+\lambda)^+ = (1+\lambda)^* \cdot (1+\lambda)$

Apoi putem aplica formula (D) pentru prima parte şi obţinem $(1^* \cdot \lambda^*)^* \cdot (1+\lambda)$

Dar $\lambda^* = \lambda$ şi conform formulei (B) prima paranteză este egală cu 1^*, deci vom avea $(1^*)^* \cdot (1+\lambda) = 1^* \cdot (1+\lambda)$. Aplicăm formula (E) (si apoi (C) şi (B)) şi obţinem $(1^* \cdot 1) + (1^* \cdot \lambda) = 1^+ + 1^* = \{1^n \mid n \geq 1\} \cup \{1^n \mid n \geq 0\} = \{1^n \mid n \geq 0\} = 1^*$ pe care îl trecem în tabel.

$R_{12}^1 = R_{12}^0 \cup R_{11}^0 \cdot (R_{11}^0)^* \cdot R_{12}^0 = 0 \cup (1+\lambda) \cdot (1+\lambda)^* \cdot 0$

Aplicăm formula (C) şi obţinem $0 \cup (1+\lambda)^+ \cdot 0$, care este echivalent după cum am aflat mai sus cu $0 \cup 1^* \cdot 0 = \lambda \cdot 0 \cup 1^* \cdot 0 = \lambda \cdot 0 + 1^* \cdot 0 = (\lambda + 1^*) \cdot 0 = 1^* \cdot 0$

$R_{21}^1 = R_{21}^0 \cup R_{21}^0 \cdot (R_{11}^0)^* \cdot R_{11}^0 = \varnothing \cup \varnothing \cdot (1+\lambda)^* \cdot (1+\lambda)$

Aplicăm formula (A) şi avem $\varnothing \cup \varnothing = \varnothing$

$R_{22}^1 = R_{22}^0 \cup R_{21}^0 \cdot (R_{11}^0)^* \cdot R_{12}^0 = \lambda \cup \varnothing \cdot (1+\lambda)^* \cdot 0 = \lambda \cup \varnothing = \lambda$

R_{ij}^{k+1}	k+1 = 0	k+1 = 1	k+1 = 2
$r_{ij} = r_{11}$	$1 + \lambda$	1^*	
$r_{ij} = r_{12}$	0	$1^* \cdot 0$	$1^* \cdot 0$
$r_{ij} = r_{21}$	$\varnothing$	$\varnothing$	
$r_{ij} = r_{22}$	λ	λ	

(c) Calculăm ultima coloană, cea incompletă, pentru k+1 = 2.

Singura stare finală este q_2, deci avem de calculat doar eticheta de la starea q_1 (cea iniţiala) la starea q_2. *(Dacă aveam mai multe stări finale, calculam pentru fiecare câte o expresie şi apoi le reuneam.)*

$R_{12}^2 = R_{12}^1 \cup R_{12}^1 \cdot (R_{22}^1)^* \cdot R_{22}^1 = (1^* \cdot 0) \cup (1^* \cdot 0) \cdot \lambda^* \cdot \lambda = (1^* \cdot 0) \cup (1^* \cdot 0) = 1^* \cdot 0$

Deci expresia regulată corespunzătoare automatului dat este $1^* \cdot 0$.

Gramatici

G = (N, T, S, P)
N = {A, B, C, …} mulţimea de simboluri neterminale
T = {a, b, c, ...} mulţimea de simboluri terminale
S ∈ N simbolul de start
P mulţimea de producţii (sau reguli de producţie)

Modul în care este definită funcţia P determină diferite tipuri de gramatici.

Gramatici independente de context (GIC) / Context-free grammars (CFG)

$P \subseteq N \times (N \cup T)^*$

Cazuri particulare de gramatici independente de context:

Gramatici liniare

Au producţii de tipurile $A \to x, \quad cu\ x \in T^*\ şi\ A \in N$
$A \to xBy, \quad cu\ x, y \in T^*\ şi\ A, B \in N$

Gramatici liniare la dreapta

Au producţii de tipurile $A \to x, \quad cu\ x \in T^*\ şi\ A \in N$
$A \to xB, \quad cu\ x \in T^*\ şi\ A, B \in N$

Gramatici liniare la stânga

Au producţii de tipurile $A \to x, \quad cu\ x \in T^*\ şi\ A \in N$
$A \to Bx, \quad cu\ x \in T^*\ şi\ A, B \in N$

Gramatici regulate

O gramatică se numeşte regulată dacă este liniară la dreapta sau liniară la stânga.
Dar pentru că gramaticile liniare la dreapta şi cele liniare la stânga definesc acelaşi set de limbaje, atunci în general ne referim la cele liniare la dreapta.

Deci gramaticile regulate au producţii de tipurile $A \to \lambda$
$A \to a$
$A \to aB, \quad cu\ a \in T\ şi\ A, B \in N$

Obs: Putem grupa mai multe producţii care au acelaşi membru stâng şi putem scrie pe scurt $A \to \lambda \mid a \mid aB, \quad cu\ a \in T\ şi\ A, B \in N$

EXEMPLE (gramatici regulate):

Vom relua acele exemple din primul seminar pentru care am construit automate finite *(pagina 5 din acest document)*, pentru a vedea corespondenţa strânsă dintre automate finite si gramatici regulate (dintre stări şi neterminale; tranziţii şi producţii).

Obs: Vom scrie pe scurt, doar producţiile, acolo unde nu există risc de confuzii (adică se înţelege clar că literele mari sunt neterminale, literele mici sunt terminale, iar simbolul de start este S).

L1 = $\mathbf{a^* = \{a^n, n \geq 0\}}$ = {$\lambda=a^0$, $a=a^1$, $aa=a^2$, $aaa=a^3$, $aaaa=a^4$, …}

$S \to aS \mid \lambda$

L2 = $\mathbf{a^+ = \{a^n, n \geq 1\}}$ = {$a=a^1$, $aa=a^2$, $aaa=a^3$, $aaaa=a^4$, …}

$S \to aS \mid a$

L3 = $\mathbf{\{a^{2n}, n \geq 0\}}$ = {$\lambda=a^0$, $aa=a^2$, $aaaa=a^4$, $aaaaaa=a^6$, …}

$S \to aA \mid \lambda$

$A \to aS$

L4 = $\mathbf{\{a^{2n+1}, n \geq 0\}}$ = {$a=a^1$, $aaa=a^3$, $aaaaa=a^5$, …}

$S \to aA \mid a$ sau $S \to aA$

$A \to aS$ $A \to aS \mid \lambda$

L5 = $\mathbf{\{a^{2n}, n \geq 1\}}$ = {$aa=a^2$, $aaaa=a^4$, $aaaaaa=a^6$, …}

$S \to aA$ sau $S \to aA$ sau $S \to aA$

$A \to aS \mid a$ $A \to aB \mid a$ $A \to aB$

$B \to aA$ $B \to aA \mid \lambda$

L6 = $\mathbf{\{a^{2n+1}, n \geq 1\}}$ = {$aaa=a^3$, $aaaaa=a^5$, $aaaaaaa=a^7$, …}

$S \to aA$ $S \to aA$

$A \to aB$ sau $A \to aS \mid aB$

$B \to aA \mid a$ $B \to a$

L7 = $\mathbf{\{a^{2n} b^{3m}, n \geq 0, m \geq 0\}}$

= {λ, a^2, a^4, a^6, a^8, …, b^3, b^6, b^9, b^{12}, …, a^2b^3, a^2b^6, a^4b^3,a^4b^6, a^6b^3, a^6b^6, …}

$S \to aA \mid bB \mid \lambda$ $S \to aA \mid bB \mid \lambda$

$A \to aS$ $A \to aS$

$B \to bC$ sau $B \to bC$

$C \to bD$ $C \to bD \mid b$

$D \to bB \mid \lambda$ $D \to bB$

Analogia între automate finite şi gramatici regulate

i) La limbajele L1 şi L2, unde avem cuvinte de forma a^n:
- la automate avem buclă (adică tranziţie de la o stare către ea însăşi)
- la gramatici avem o producţie în care acelaşi neterminal apare şi în membrul stâng şi în cel drept al producţiei

ii) La limbajele L3 – L10, unde la putere avem şi un coeficient număr natural (2 pentru a-uri şi 3 pentru b-uri):
- la automate avem circuit de lungime egală cu coeficientul între stări diferite
- la gramatici avem circuit de lungime egală cu coeficientul între neterminale diferite

iii) Dar ce se întâmplă dacă avem limbaje definite folosind condiţii asupra numărului de apariţii ale caracterelor în cuvinte ?
- la automate ideea era să folosim indici la stări, câte unul pentru fiecare condiţie
- la gramatici ideea este să folosim indici la neterminale, câte unul pentru fiecare condiţie

~ Seminar 4 ~

(Exemple cu gramatici independente de context GIC ;
Transformarea GIC → gramatică în F.N. Chomsky ;
Automatul Push-Down / Automatul Stivă)

EXEMPLE (gramatici independente de context):

În general, trebuie să lucrăm cu gramatici independente de context, pentru că nu putem cu gramatici regulate (respectiv, trebuie să lucrăm cu automate push-down, pentru că nu putem cu automate finite), atunci când limbajul conţine cuvinte în care numărul de litere dintr-o parte a cuvântului depinde de numărul de litere din altă parte a cuvântului şi deci avem nevoie să „controlăm" cumva acest număr de litere.

L1 = {$a^n b^n$, n ≥ 0}
= {$\lambda=a^0b^0$, $ab=a^1b^1$, $aabb=a^2b^2$, $aaabbb=a^3b^3$, $aaaabbbb=a^4b^4$, …}

$S \to \overset{(1)}{aSb} \mid \overset{(2)}{\lambda}$ sau $S \to \overset{(1)}{aAb} \mid \overset{(2)}{\lambda}$, $A \to \overset{(3)}{aAb} \mid \overset{(4)}{\lambda}$

Ideea acestei gramatici este de a genera cuvântul dinspre margini spre mijlocui lui, astfel pentru fiecare aplicare a producţiei (1) vor apărea un „a" şi un „b", iar între ele va rămâne S-ul pentru a continua generarea. La final, se aplică producţia (2) pentru a nu mai avea neterminal în cuvântul obţinut.
Cel mai mic cuvând este într-adevăr cuvântul vid, pentru că putem aplica direct a doua producţie fără să o aplicăm deloc pe prima.

L2 = {$a^n b^n$, n ≥ 1}
= {$ab=a^1b^1$, $aabb=a^2b^2$, $aaabbb=a^3b^3$, $aaaabbbb=a^4b^4$, …}

$S \to \overset{(1)}{aSb} \mid \overset{(2)}{ab}$ sau $S \to \overset{(1)}{aAb}$, $A \to \overset{(2)}{aAb} \mid \overset{(3)}{\lambda}$

Aici, cel mai mic cuvânt trebuie să fie „ab", deci observăm că din ambele gramatici de la exemplul anterior au dispărut producţiile „S→λ".
În prima soluţie avem ca pas se oprire cuvântul „ab", iar în a doua soluţie suntem obligaţi să generăm cel puţin un „a" şi un „b" (prin prima producţie) înainte de a ne opri (cu a treia producţie).

L3 = {$a^n b^{2n}$, n ≥ 1}
= {abb=a^1b^2, aabbbb=a^2b^4, aaabbbbbb=a^3b^6, …}

$$S \xrightarrow{(1)} aSbb \mid \xrightarrow{(2)} abb$$

Pentru fiecare „a” din prima parte a cuvântului trebuie să avem doi de „b” în a doua parte a cuvântului.
Deoarece avem „n ≥ 1”, pasul de oprire, a doua producţie este cu „abb” în membrul drept.
Dacă era „n ≥ 0”, atunci a doua producţie ar fi avut ca membru drept cuvântul vid.

L4 = {$a^{2n} b^k c^{3n}$, n ≥ 0, k ≥ 1}
= { b=b^1, bb=b^2, bbb=b^3, bbbb=b^4, …,
aabccc=$a^2b^1c^3$, aabbccc=$a^2b^2c^3$, aabbbccc=$a^2b^3c^3$, aabbbbccc=a^2b^4c3, …,
aaaabccccccc=$a^4b^1c^6$, aaaabbccccccc=$a^4b^2c^6$, aaaabbbccccccc=$a^4b^3c^6$, ... }

$$S \xrightarrow{(1)} aaSccc \mid \xrightarrow{(2)} bA$$
$$A \xrightarrow{(3)} bA \mid \xrightarrow{(4)} \lambda$$

Pentru fiecare „aa” de la începutul cuvântului trebuie să avem „ccc” la finalul lui, iar între ele vom avea „b”-uri. Deci aici vom folosi întâi neterminalul S pentru a genera „a”-urile şi „c”-urile, iar apoi vom trece în alt neterminal cu ajutorul căruia vom genera „b”-urile.
Observăm că avem „n ≥ 0”, deci nu trebuie să-l obligăm să aibă „a”-uri şi „c”-uri, ci trebuie să îl lăsăm să treacă direct la „b”-uri.
Cel mai mic cuvânt este „b”: aplicăm producţia (2) şi apoi pe (4).

L5 = {$a^{2n} b^k c^{3n}$, n ≥ 1, k ≥ 1}
= { aabccc=$a^2b^1c^3$, aabbccc=$a^2b^2c^3$, aabbbccc=$a^2b^3c^3$, aabbbbccc=$a^2b^4c^3$, …,
aaaabccccccc=$a^4b^1c^6$, aaaabbccccccc=$a^4b^2c^6$, aaaabbbccccccc=$a^4b^3c^6$, ... }

$$S \xrightarrow{(1)} aaSccc \mid \xrightarrow{(2)} aaAccc \qquad A \xrightarrow{(3)} bA \mid \xrightarrow{(4)} b$$

sau

$$S \xrightarrow{(1)} aaSccc \mid \xrightarrow{(2)} aaAbccc \qquad A \xrightarrow{(3)} bA \mid \xrightarrow{(4)} \lambda$$

sau

$$S \xrightarrow{(1)} aaSccc \mid \xrightarrow{(2)} aabAccc \qquad A \xrightarrow{(3)} bA \mid \xrightarrow{(4)} \lambda$$

Diferenţa faţă de exemplul anterior este că aici şi n este strict pozitiv, de aceea trebuie ca înainte de a trece la neterminalul A să generăm obligatoriu minim doi de „a” şi trei de „c” (producţia a doua). Apoi în neterminalul A să generăm cel puţin un „b” (de aceea observăm că s-a modificat şi producţia a patra).
Cel mai mic cuvânt este „aabccc”: aplicăm producţia (2) şi apoi (4).

L6 = {$a^{2n}c^{3n}b^{k}$, n ≥ 1, k ≥ 1}
= { aacccb=$a^2c^3b^1$, aacccbb=$a^2c^3b^2$, aacccbbb=$a^2c^3b^3$, aacccbbbb=$a^2c^3b^4$, …,
aaaabccccccb=$a^4c^6b^1$, aaaaccccccbb=$a^4c^6b^2$, aaaaccccccbbb= $a^4c^6b^3$, ... }

$S \xrightarrow{(1)} AB$

$A \xrightarrow{(2)} aaAccc \mid \xrightarrow{(3)} aaccc$

$B \xrightarrow{(4)} bB \mid \xrightarrow{(5)} b$

Observăm că nu întotdeauna putem genera cuvintele dinspre exterior spre centru, având maxim un singur neterminal în membrul drept al producţiilor. Uneori este necesar să generăm cuvintele pe bucăţi, folosind pentru fiecare parte un neterminal separat.
Aici, vom folosi neterminalul A pentru a genera „a”-urile şi „c”-urile spre centru simultan câte doi de „a” şi câte trei de „c”, iar din neterminalul B vom genera „b”-urile.
Dacă am fi avut condiţia „n ≥ 0”, atunci producţia (3) ar fi fost „A→λ”, iar dacă am fi avut condiţia „k ≥ 0”, atunci producţia (5) ar fi fost „B→λ”.

Verificare generare cuvânt de către gramatică

Obs: Atenţie la terminologia folosită:
- automatul *acceptă* un (cuvânt din) limbaj
- cuvântul / limbajul *este acceptat de* automat
- gramatica *generează* un (cuvânt din) limbaj
- cuvântul / limbajul *este generat de* gramatică

Pentru a verifica dacă un cuvânt este sau nu generat de o gramatică, se folosesc „***arborii de derivare***”, care au ca rădăcină simbolul de start al gramaticii (adică S, dacă nu se specifică altul).
Pentru a trece de la un nod la altul al arborelui, ne alegem un neterminal care face parte din cuvântul curent, căutăm acele producţii care îl au ca membru stâng, alegem una dintre ele şi obţinem noul cuvânt curent prin înlocuirea neterminalului cu membrul drept al acelei producţii.
- Fie ne oprim când am găsit un drum prin care am ajuns să generăm cuvântul verificat (deci suntem într-o frunză a arborelui, nu mai avem neterminale în cuvânt).
- Fie ne oprim când toate drumurile posibile au creat cuvinte mai lungi decât cel verificat, dar fără să întâlnim frunza corectă (deci nu are rost să continuăm pentru că vom găsi cuvinte din ce în ce mai lungi, deci diferite de cel verificat)

(a) Fie îi reprezentăm chiar sub formă de arbore, de sus în jos începând cu rădăcina, etichetând nodurile cu cuvântul curent, iar muchiile cu numărul producţiei pe care am aplicat-o pentru a ajunge din nodul-tată în nodul-fiu.

(b) Fie îi reprezentăm de la stânga la dreapta cu aceeaşi convenţie pentru etichetele nodurilor şi muchiilor. În acest caz, ori decidem să urmăm toate ramificaţiile posibile, ori

scriem direct doar drumul care ne duce la rezultat (fiind cuvinte scurte şi gramatici destul de mici, putem „vedea” drumul fară prea mare greutate). Preferabil alegem a doua variantă când avem de verificat un singur cuvânt.

Obs: Uneori se poate întâmpla ca *pentru un cuvânt să existe mai mulţi arbori de derivare distincţi*.
(i) Dacă *gramatica este regulată*, asta poate însemna că în gramatică există unele producţii redundante (care ar putea fi eliminate fără să apară probleme, adică după eliminare să se obţină o gramatică echivalentă cu cea iniţială).
(ii) Iar dacă *gramatica este independentă de context* şi are cel puţin o producţie în al cărei membru drept apar mai multe neterminale, atunci apar mai mulţi arbori pentru un cuvânt care conţine la un moment dat mai multe neterminale, pentru că nu este obligatoriu să încercăm să rescriem doar neterminalul cel mai din stânga/dreapta, ci putem să le alegem în orice ordine.

EXEMPLU:
Pentru gramatica anterioară, vom verifica dacă generează cuvântul $a^4c^6b^3 \in L6$.

$S \xrightarrow{(1)} AB$

$A \xrightarrow{(2)} aaAccc \mid \xrightarrow{(3)} aaccc$

$B \xrightarrow{(4)} bB \mid \xrightarrow{(5)} b$

(a) Aveţi aici (http://bit.ly/arbore_derivare) poza arborelui de derivare pentru a găsi toate drumurile posibile prin care poate fi obţinut cuvântul dat.
În desen, muchiile sunt etichetate cu numărul producţiei care a fost aplicată, iar *nodurile interioare* cu neterminalul care a fost rescris (în interiorul cercului) şi cu noul cuvânt obţinut (în dreptunghiul de sub cerc).
Avem mai multe tipuri de *noduri fără fii*:

- Cele marcate cu „--” reprezintă începutul unor subarbori care mai pot fi extinşi, dar în care nu vom mai găsi soluţii pentru cuvântul dat. Acest lucru se poate întâmpla din două motive: fie am aplicat de prea multe ori una dintre producţiile recursive (2 sau 4), fie am aplicat producţia pentru pasul de oprire (3 sau 5) înainte de a o aplica de suficiente ori pe cea recursivă.
- Cele marcate cu „cerc dublu” reprezintă frunze ale arborelui, adică sfârşitul drumurilor în urma cărora am obţinut cuvinte fără neterminale, adică unele dintre cuvintele care sunt generate de gramatică.
- Cel marcate cu „cerc dublu în care este scris OK” reprezintă toate drumurile prin care putem genera cuvântul dat.

Observăm (direct din cuvânt şi din structura gramaticii) că producţiile pe care trebuie să le aplicăm sunt: o dată (1), apoi o dată (2), o dată (3) (dar numai după ce a fost aplicată 2), de două ori (4), o dată (5) (dar numai după ce a fost aplicată 4 de două ori). Dar ordinea pentru 2,3,4,5 poate fi oricare în afara restricţiilor menţionate. De aceea vom obţine cele 10 soluţii: 23445, 24345, 24435, 24453, 42345, 42435, 42453, 44235, 44253, 44523.

(b) A doua variantă, scriem direct pe scurt un drum prin care ştim că vom obţine cuvântul dat. Îl vom alege pe primul, în care vom rescrie mereu cel mai din stânga neterminal.

$\underline{S} \overset{(1)}{\Rightarrow} \underline{A}B \overset{(2)}{\Rightarrow} aa\underline{A}cccB \overset{(3)}{\Rightarrow} a^4c^6\underline{B} \overset{(4)}{\Rightarrow} a^4c^6b\underline{B} \overset{(4)}{\Rightarrow} a^4c^6b^2\underline{B} \overset{(5)}{\Rightarrow} a^4c^6b^3$

Obs: Dacă cerinţa este doar de a verifica pentru un anumit cuvânt dacă este sau nu generat de gramatică, atunci este preferabilă varianta (b). Dar dacă cerinţa este să se afle toate cuvintele generate de o gramatică, pentru o lungime de cuvânt dată sau pentru un număr maxim de aplicări ale producţiilor, atunci trebuie folosită varianta (a) cu extinderea mai multor drumuri.

Continuăm cu exemplele de gramatici independente de context.

Dacă avem cuvinte formate din 4 porţiuni (a, b, c, d) şi vrem ca lungimile porţiunilor (puterile la care sunt literele) să depindă unele de altele grupate două câte două atunci avem doar două posibilităţi de grupare: *prima cu a patra* (şi a doua cu a treia) sau *prima cu a doua* (şi a treia cu a patra).
Deci nu vom putea să avem grupe intersectate (prima cu a treia; a doua cu a patra)!

L7 = {$a^n b^k c^{2k} d^n$, n ≥ 1, k ≥ 0}
= { ad, a^2d^2, a^3d^3, a^4d^4, ..., abc^2d, ab^2c^4d, ab^3c^6d, ..., $a^2bc^2d^2$, $a^2b^2c^4d^2$, $a^2b^3c^6d^2$, ... }

$S \to \overset{(1)}{aSd} \mid \overset{(2)}{aAd}$
$A \to \overset{(3)}{bAcc} \mid \overset{(4)}{\lambda}$

sau

$S \to \overset{(1)}{aSd} \mid \overset{(2)}{ad} \mid \overset{(3)}{aAd}$
$A \to \overset{(4)}{bAcc} \mid \overset{(5)}{\lambda}$

(în a doua soluţie, producţia (2) este redundantă pentru că poate fi obţinută aplicând (3) şi apoi (5))

Aici avem grupate porţiunile 1-4 şi 2-3, deci vom genera cuvintele dinspre exterior spre centru, întâi „a”-urile simultan cu „d”-urile, iar apoi „b”-urile simultan cu „c”-urile.
Atenţie, trebuie să folosim *neterminale diferite*, pentru că altfel s-ar genera şi cuvinte cu literele amestecate („a” şi „b” între ele şi „c” şi „d” între ele).
Avem „k ≥ 0”, deci „b”-urile şi „c”-urile pot lipsi, de aceea avem producţia (4) pe care o aplicăm pentru cazurile când avem k = 0.
Avem „n ≥ 1”, deci producţia (2), pentru că trebuie să ne asigurăm că avem cel puţin un „a” şi un „d” înainte de a trece la neterminalul A pentru a genera mijlocul cuvântului.

Dacă am fi avut „k ≥ 1”, atunci producţia (4) s-ar fi transformat în „A→bcc”.

L8 = {$a^n b^k c^{2k} d^n$, n ≥ 0, k ≥ 1}
= { bc^2, b^2c^4, b^3c^6, ..., abc^2d, ab^2c^4d, ab^3c^6d, ..., $a^2bc^2d^2$, $a^2b^2c^4d^2$, $a^2b^3c^6d^2$, ... }

$S \to \overset{(1)}{aSd} \mid \overset{(2)}{bAcc}$

$A \to \overset{(3)}{bAcc} \mid \overset{(4)}{\lambda}$

Structura cuvintelor este aceeaşi ca la exemplul anterior, s-au schimbat doar condiţiile. Avem „n ≥ 0”, deci „a”-urile şi „d”-urile pot lipsi, dar mijlocul nu. De aceea avem producţia (2) în care folosim direct „bcc”-ul pentru a trece la neterminalul A.
Avem „k ≥ 1”, dar deja am citit un „bcc” cu neterminalul S, deci pasul de oprire pentru A va fi cu cuvântul vid.

Dacă am fi avut „k ≥ 0”, înseamnă că şi mijlocul („b”-urile şi „c”-urile) ar fi putut lipsi, deci am fi avut în plus producţia „S→λ”.

L9 = {$a^n b^{2n} c^{2k} d^k$, n ≥ 0, k ≥ 1}
= { c^2d, c^4d^2, c^6d^3, ..., ab^2c^2d, $ab^2c^4d^2$, $ab^2c^6d^3$, ..., $a^2b^4c^2d$, $a^2b^4c^4d^2$, $a^2b^4c^6d^3$, ... }

$S \to \overset{(1)}{AB}$

$A \to \overset{(2)}{aAbb} \mid \overset{(3)}{\lambda}$

$B \to \overset{(4)}{ccBd} \mid \overset{(5)}{ccd}$

Aici avem grupate porţiunile 1-2 şi 3-4, deci din simbolul de start S ne vom ramifica în două părţi disjuncte: vom folosi un neterminal A pentru a genera dinspre exterior spre interior porţiunea cu „a”-uri şi „b”-uri, iar separat vom folosi un neterminal B pentru a genera dinspre exterior spre interior porţiunea cu „c”-uri şi „d”-uri.
Avem „n ≥ 0”, deci pasul de oprire pentru A va fi cuvântul vid.
Avem „k ≥ 1”, deci pasul de oprire pentru B va fi „ccd”.

L10 = {$a^n b^{2n} c^k d^m e^{3k}$, n,k,m ≥ 0}

$S \to \overset{(1)}{AB} \mid \overset{(2)}{\lambda}$

$A \to \overset{(3)}{aAbb} \mid \overset{(4)}{\lambda}$

$B \to \overset{(5)}{cBeee} \mid \overset{(6)}{\lambda} \mid \overset{(7)}{dC}$

$C \to \overset{(8)}{dC} \mid \overset{(9)}{\lambda}$

(producţia (2) este reduntantă pentru că poate fi obţinută din 1, 4, 6)

Vom folosi neterminaul A pentru a genera spre interior „a”-urile şi „b”-urile. Separat, vom folosi neterminalul B pentru a genera spre interior „c”-urile şi „e”-urile, iar apoi între ele „d”-urile.

Pentru „n ≥ 0” avem producţia (4).
Pentru „m ≥ 0” avem producţia (6).
Pentru „k ≥ 0” avem producţia (7) (în care trecem direct cu „d” mai departe fără să fi citit „ceee”) sau producţia (6) (atunci când avem şi m = 0).

Transformarea GIC → gramatică în F.N. Chomsky

O gramatică este (scrisă) în ***forma normală Chomsky*** dacă are doar producţii care au în membrul stâng un neterminal, iar în membrul drept fie un terminal, fie două neterminale.

$A \to a$

$A \to BC$

EXEMPLU:
Avem următoarea gramatică independentă de context:

$S \to \overset{(1)}{aBcDeF} \mid \overset{(2)}{HF} \mid \overset{(3)}{HBc}$

$B \to \overset{(4)}{b} \mid \overset{(5)}{\lambda}$

$D \to \overset{(6)}{d} \mid \overset{(7)}{\lambda}$

$F \to \overset{(8)}{G}$

$G \to \overset{(9)}{f} \mid \overset{(10)}{g}$

$H \to \overset{(11)}{\lambda}$

Pasul 1 – *Eliminarea λ–producţiilor*
Vrem să nu mai avem în gramatică producţii care au ca membru drept cuvântul vid.
Pot exista două situaţii:

(i) Dacă neterminalul care are o λ–producţie NU are şi alte producţii, atunci
- va fi eliminată producţia lui
- toate producţiile care au ca membru drept un cuvânt de lungime minim 2 în care apare acest neterminal → vor fi înlocuite prin eliminarea neterminalului din cuvinte
- dacă membrul drept avea lungime 1 (adică era doar acest neterminal), atunci se înlocuieşte cu λ (dar şi această producţie va fi ulterior eliminată tot la „pasul 1”)

În EXEMPLU:
Neterminalul H este singurul de acest gen, având doar λ–producţia (11), care va fi eliminată. În plus, producţia (2) va fi înlocuită cu „S→F” (12), iar producţia (3) va fi înlocuită cu „S→Bc” (13).

$S \xrightarrow{(1)} aBcDeF \mid \overset{(12)}{F} \mid \overset{(13)}{Bc}$

$B \xrightarrow{(4)} b \mid \overset{(5)}{\lambda}$

$D \xrightarrow{(6)} d \mid \overset{(7)}{\lambda}$

$F \xrightarrow{(8)} G$

$G \xrightarrow{(9)} f \mid \overset{(10)}{g}$

(ii) Dacă neterminalul care are o λ–producţie are şi alte producţii, atunci
- va fi eliminată doar λ–producţia lui
- toate producţiile care au ca membru drept un cuvânt de lungime minim 2 în care apare acest neterminal → vor fi înlocuite atât de varianta în care cuvântul conţine neterminalul, cât şi de varianta în care neterminalul este eliminat din cuvânt
- dacă membrul drept avea lungime 1 (adică era doar acest neterminal), atunci această producţie este o redenumire şi va fi eliminată la „pasul 2"

În EXEMPLU:
Neterminalele B şi D au mai multe producţii printre care şi câte o λ–producţie. Deci (5) şi (7) vor fi eliminate.
În producţia (13) apare doar B, deci va fi înlocuită cu „S→Bc" (14) şi „S→c" (15).
În producţia (1) apar şi B şi D, deci va fi înlocuită de toate variaţiile posibile:
-- B şi D prezente: „S→aBcDeF" (16)
-- B lipsă, D prezent: „S→acDeF" (17)
-- B prezent, D lipsă: „S→aBceF" (18)
-- B şi D lipsă: „S→aceF" (19)

$S \xrightarrow{(16)} aBcDeF \mid \overset{(17)}{acDeF} \mid \overset{(18)}{aBceF} \mid \overset{(19)}{aceF} \mid \overset{(11)}{F} \mid \overset{(14)}{Bc} \mid \overset{(15)}{c}$

$B \xrightarrow{(4)} b \quad ; \quad D \xrightarrow{(6)} d \quad ; \quad F \xrightarrow{(8)} G \quad ; \quad G \xrightarrow{(9)} f \mid \overset{(10)}{g}$

Pasul 2 – *Eliminarea redenumirilor*
Vrem să nu mai avem în gramatică producţii care au ca membru drept un neterminal.

Înlocuim neterminalul din dreapta cu toate cuvintele care sunt membru drept în producţiile sale.
Verificăm dacă acest neterminal din dreapta mai apare în dreapta în cadrul altor producţii
- Daca nu, atunci eliminăm toate producţiile pe care le avea.
- Dacă mai apare în cuvinte de lungime minim 2, arunci trebuie să i le păstrăm
- Dacă apare în cuvânt de lungime 1, înseamnă că este tot o redenumire şi va fi eliminată tot la „pasul 2"

În EXEMPLU:
Avem două şiruri de redenumiri:
F → G (8) → f (9) | g (10)
Producţia (8) va fi înlocuită cu „F→f " (20) şi „F→g" (21).

Neterminalul G nu mai apare nicăieri în dreapta, deci vom elimina (9) şi (10).
S → F (11) → f (20) | g (21)

Producţia (11) va fi înlocuită cu „S→f ” (22) şi „S→g” (23).

Dar neterminalul F mai apare în dreapta în producţiile 16–19, deci NU vom elimina producţiile lui F.

$$S \to \overset{(16)}{aBcDeF} \mid \overset{(17)}{acDeF} \mid \overset{(18)}{aBceF} \mid \overset{(19)}{aceF} \mid \overset{(22)}{f} \mid \overset{(23)}{g} \mid \overset{(14)}{Bc} \mid \overset{(15)}{c}$$

$$B \to \overset{(4)}{b} \quad ; \quad D \to \overset{(6)}{d} \quad ; \quad F \to \overset{(20)}{f} \mid \overset{(21)}{g}$$

Pasul 3 – *Adăugare neterminale noi pentru terminalele din cuvinte*
Vrem ca terminalele să apară doar singure în membrul drept. De aceea, peste tot unde apar în componenţa unui cuvânt de lungime minim 2, le vom înlocui cu un neterminal nou şi vom adăuga producţia de la neterminalul nou la terminalul pe care l-a înlocuit.

În EXEMPLU:
Producţiile 22, 23, 15, 4, 6, 20, 21 au deja forma dorită.
Producţia (16) va fi înlocuită cu „S → X_a B X_c D X_e F” (24).
Producţia (17) va fi înlocuită cu „S → X_a X_c D X_e F” (25).
Producţia (18) va fi înlocuită cu „S → X_a B X_c X_e F” (26).
Producţia (19) va fi înlocuită cu „S → X_a X_c X_e F” (27).
Producţia (14) va fi înlocuită cu „S → B X_c” (28).
În plus, vom avea „X_a→a” (29); „X_c→c” (30); „X_e→e” (31).

$$S \to X_a B \overset{(24)}{X_c} D X_e F \mid X_a \overset{(25)}{X_c} D X_e F \mid X_a B \overset{(26)}{X_c} X_e F \mid X_a \overset{(27)}{X_c} X_e F \mid \overset{(22)}{f} \mid \overset{(23)}{g} \mid B \overset{(28)}{X_c} \mid \overset{(15)}{c}$$

$$B \to \overset{(4)}{b} \quad ; \quad D \to \overset{(6)}{d} \quad ; \quad F \to \overset{(20)}{f} \mid \overset{(21)}{g}$$

$$X_a \to \overset{(29)}{a} \quad ; \quad X_c \to \overset{(30)}{c} \quad ; \quad X_e \to \overset{(31)}{e}$$

Pasul 4 – *Adăugare neterminale noi pentru „spargerea” cuvintelor lungi (>2)*
Vrem ca în dreapta să avem cuvinte formate din exact două neterminale. De aceea, unde avem cuvinte mai lungi, păstrăm doar primul neterminal din cuvânt şi îi alipim un neterminal nou, iar noul neterminal va avea o producţie cu membrul drept cuvântul pe care l-a înlocuit. Reluăm procedeul până când toate cuvintele ajung la lungimea 2.

***Obs**:* Fiecare producţie care avea un cuvânt de lungime k va fi înlocuită de k–1 producţii cu cuvinte de lungime 2.

În EXEMPLU:
Producţia (24) va deveni „S→X_a Y_1” (32), iar Y_1 ~ B X_c D X_e F.
Apoi „Y_1→B Y_2” (33), iar Y_2 ~ X_c D X_e F.
Apoi „Y_2→X_c Y_3” (34), iar Y_3 ~ D X_e F.
Apoi „Y_3→D Y_4” (35), iar Y_4~ X_e F.
În final „Y_4→ X_e F” (36).
Observăm că putem avea sufixe comune ale cuvintelor şi atunci am putea folosi neterminale deja adăugate anterior.

Producţia (25) va deveni „$S \to X_a Y_2$” (37), pentru că aveam $Y_2 \sim X_c D X_e F$.
Producţia (26) va deveni „$S \to X_a Y_5$” (38), iar $Y_5 \sim B X_c X_e F$.
Apoi „$Y_5 \to B Y_6$” (39), iar $Y_6 \sim X_c X_e F$.
Apoi „$Y_6 \to X_c Y_4$” (40), pentru că aveam $Y_4 \sim X_e F$.
Producţia (27) va deveni „$S \to X_a Y_6$” (41), pentru că aveam $Y_6 \sim X_c X_e F$.

$$S \xrightarrow{(32)} X_a Y_1 \mid X_a Y_2 \mid X_a Y_5 \mid X_a Y_6 \mid f \mid g \mid B X_c \mid c$$

$$B \xrightarrow{(4)} b \quad ; \quad D \xrightarrow{(6)} d \quad ; \quad F \xrightarrow{(20)} f \mid g$$

$$X_a \xrightarrow{(29)} a \quad ; \quad X_c \xrightarrow{(30)} c \quad ; \quad X_e \xrightarrow{(31)} e$$

$$Y_1 \xrightarrow{(33)} B Y_2 \quad ; \quad Y_2 \xrightarrow{(34)} X_c Y_3 \quad ; \quad Y_3 \xrightarrow{(35)} D Y_4 \quad ; \quad Y_4 \xrightarrow{(36)} X_e F \quad ; \quad Y_5 \xrightarrow{(39)} B Y_6 \quad ; \quad Y_6 \xrightarrow{(40)} X_c Y_4$$

Automatul Push-Down (APD) / Automatul Stivă

APD = $(Q, \Sigma, q_0, F, \delta, \Gamma, Z_0)$
Q mulţimea de stări
Σ alfabetul de intrare
$q_0 \in Q$ starea iniţială
$F \subseteq Q$ mulţimea de stări finale
Γ alfabetul stivei („gamma”)
$Z_0 \in \Gamma$ simbolul iniţial al stivei
$\delta : Q \times \Sigma \times \Gamma \to Q \times \Gamma^*$ funcţia de tranziţie („delta”)

Modul în care funcţionează tranziţiile pentru un APD
- La intrare avem nevoie de 3 parametri:
  - starea curentă *(element din mulţimea Q)*
  - caracterul curent din cuvântul de intrare *(element din mulţimea Σ)*
  - **simbolul** aflat în vârful stivei *(element din mulţimea Γ)*
- La ieşire vom avea 2 parametri:
  - starea în care ajungem *(element din mulţimea Q)*
  - **cuvântul** cu care înlocuim **simbolul** din vârful stivei *(element din mulţimea Γ^*)*

Atenţie! Simbolul din vârful stivei este mereu *eliminat* din stivă, înainte de a se adăuga noul cuvânt în locul lui.

Deci avem mai multe variante pentru această înlocuire:
- Dacă scriem λ în stivă, înseamnă că doar „am şters” simbolul din vârful stivei fără a adăuga nimic în locul lui.
- Dacă scriem acelaşi caracter în stivă (cel care fusese eliminat), înseamnă că dorim să „păstrăm stiva nemodificată”.
- Dacă scriem în stivă un cuvânt de lungime minim 2 avem două cazuri:

-- dacă dorim să păstrăm şi caracterul care era deja în stivă, atunci cuvântul introdus va avea ultima literă exact cea care fusese eliminată *(deci partea de început, din stânga, a cuvântului reprezintă vârful stivei)*.
-- dacă nu dorim să păstrăm caracterul care era în vârful stivei, ci chiar să-l înlocuim, atunci cuvântul se poate termina cu alt caracter decât cel eliminat.

Obs: La fel ca automatele finite, şi automatele push-down pot fi:
- *„deterministe"* (definite anterior, dar sunt acceptate şi λ–tranziţiile! Dar dacă între două stări avem λ–tranziţie, nu putem avea tranziţie şi cu un caracter din Σ.)
- *„nedeterministe"* (când avem mai multe posibilităţi de continuare: fie ajungând în stări diferite, fie scriind pe stivă cuvinte diferite, fie ambele (ajungând în stări diferite şi scriind pe stivă cuvinte diferite))
- *„nedeterministe cu λ–tranziţii"* (când între două stări avem voie să avem tranziţii şi cu λ şi cu caractere din Σ şi în plus avem nedeterminismul descris anterior)

Modalităţi de acceptare a unui cuvânt de către un APD

(a) cu stări finale

$$L_F(APD) = \{w \mid (q_0, w, Z_0) \overset{*}{\mapsto} (\underline{p}, \underline{\lambda}, \alpha),\ p \in F, \alpha \in \Gamma^*\}$$

(Obs: semnul cu steluţă deasupra ar trebui să fie doar T-ul întors, fără săgeată în dreapta, dar nu am găsit simbolul potrivit...)

Citim: Cuvântul w este acceptat de automat dacă pornind din configuraţia având starea q_0, cuvântul de intrare w şi pe stivă simbolul iniţial Z_0, după ce aplicăm un număr oarecare de tranziţii ajungem în configuraţia finală.

În acest caz (a) ne interesează:
- prima componentă să fie o stare finală;
- a doua componentă să fie cuvântul vid (adică să fi terminat de parcurs tot cuvântul de intrare)

În acest caz (a) NU ne interesează a treia componentă (conţinutul stivei): poate să fie vidă sau poate să conţină un cuvânt cu simboluri din alfabetul Γ.

(b) cu vidarea stivei

$$L_\lambda(APD) = \{w \mid (q_0, w, Z_0) \overset{*}{\mapsto} (p, \underline{\lambda}, \underline{\lambda}),\ p \in Q\}$$

În acest caz (b) ne interesează:
- a doua componentă să fie cuvântul vid (adică să fi terminat de parcurs tot cuvântul de intrare)
- a treia componentă să fie cuvântul vid (adică stiva să fie vidă)

În acest caz (b) NU ne interesează prima componentă: starea în care ajungem poate să fie finală sau poate să fie nefinală.

Obs: Atenţie, dacă ştergeţi tot conţinutul stivei, *automatul îşi opreşte funcţionarea*, pentru că nu poate aplica funcţia delta dacă nu mai are parametrul al treilea (simbolul din vârful stivei). Deci asiguraţi-vă că nu se va întâmpla asta înainte de a fi citit tot cuvântul de intrare. În general e bine să-l păstrăm pe Z_0 la început şi să-l eliminăm abia la final (eventual folosind o λ–tranziţie).

(c) cu stări finale şi cu vidarea stivei
În unele cărţi apare şi acest al treilea caz, care este de fapt intersecţia primelor două.

$$L_{F\&\lambda}(APD) = \{w \mid (q_0, w, Z_0) \overset{*}{\vdash\!\!\rightarrow} (\underline{p}, \underline{\lambda}, \underline{\lambda}),\ p \in F\}$$

În acest caz (c) ne interesează:
- prima componentă să fie o stare finală;
- a doua componentă să fie cuvântul vid (adică să fi terminat de parcurs tot cuvântul de intrare)
- a treia componentă să fie cuvântul vid (adică stiva să fie vidă)

EXEMPLE (automate push-down):

De fiecare dată vom scrie în cuvinte *modul în care vom folosi stiva*.
Adică pentru un caracter anume citit din cuvântul de intrare ce dorim să se întâmple pe stivă:
- să eliminăm de tot vârful stivei sau să-l păstrăm?
- să scriem şi ceva în plus sau să lăsăm stiva exact cum era înainte?

Putem reprezenta automatul push-down în două moduri:
(i) Fie alegem să scriem lista cu cazurile în care definim funcţia de tranziţie şi menţionăm dacă avem stări finale care sunt ele.
(ii) Fie alegem să facem ca la automatele finite, să desenăm sub formă de graf cu nodurile stări şi muchiile tranziţii.

Obs: *Avantajul* alegerii variantei (i) ar fi faptul că putem pune ”comentarii” în dreptul rândurilor pentru a urmări mai uşor logica automatului, iar dacă greşim ceva când scriem pe foaie e mult mai uşor de corectat prin tăierea/adăugarea unui rând (mai ales pentru că nu contează ordinea în care sunt scrise rândurile) decât modificarea pe desen.
Totuşi, pentru unele limbaje se poate să vi se pară mai intuitivă varianta cu graful şi atunci o puteţi folosi.

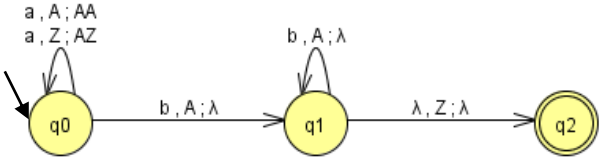
L1 = {$a^n b^n$, n ≥ 1}
Vrem ca numărul de „a”-uri să fie egal cu numărul de „b”-uri, deci vom reţine acest număr cu ajutorul stivei astfel:
- pentru fiecare „a” citit din cuvântul de intrare, vom adăuga un „A” în stivă;
- pentru fiecare „b” citit vom şterge un „A” din stivă;

Trebuie să ne asigurăm că citim cel puţin un „a” şi un „b” (pentru că n ≥ 1) şi că literele nu sunt amestecate. De aceea, după ce citim primul „b” va trebui să schimbăm starea.

$\delta(q_0, a, Z_0) = (q_0, AZ_0)$ // citim primul a, *mereu iniţial îl găsim pe Z_0 în stivă*
// vrem să scriem A, dar să-l păstrăm şi pe Z_0, deci scriem „AZ_0”
$\delta(q_0, a, A) = (q_0, AA)$ // citim oricare alt a şi avem A în stivă (introdus anterior),
// vrem să scriem A, dar fără să-l ştergem pe anteriorul, deci scriem „AA”
$\delta(q_0, b, A) = (q_1, \lambda)$ // citim primul b, găsim A în stivă
// *schimbăm starea*, vrem să-l ştergem pe A, deci scriem în locul lui „λ”
$\delta(q_1, b, A) = (q_1, \lambda)$ // citim oricare alt b, găsim sigur A în stivă; scriem „λ” pe stivă

Pentru a termina execuţia şi a accepta cuvintele prin cele 3 metode:
(a) cu stări finale:
Adăugăm $\delta(q_1, \lambda, Z_0) = (q_2, Z_0)$ sau $\delta(q_1, \lambda, Z_0) = (q_2, \lambda)$.
Avem $F = \{q_2\}$ stare finală. Putem să-l păstrăm pe Z_0 pe stivă sau să-l ştergem.
(b) cu vidarea stivei:
Adăugăm $\delta(q_1, \lambda, Z_0) = (q_1, \lambda)$ sau $\delta(q_1, \lambda, Z_0) = (q_2, \lambda)$.
Putem avea $q_1 \notin F, q_2 \in F$. Ne interesează doar să avem λ pe stivă.
(c) cu stări finale şi cu vidarea stivei:
Adăugăm $\delta(q_1, \lambda, Z_0) = (q_2, \lambda)$.
Avem $F = \{q_2\}$ stare finală şi stiva vidă.

Aceeaşi soluţie (c), sub formă de graf:

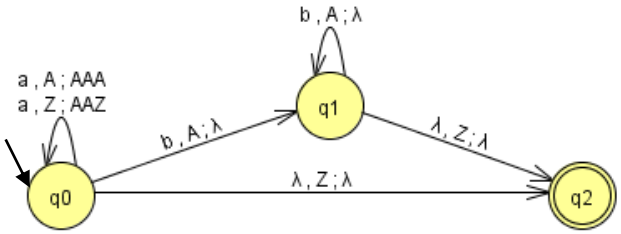


Obs: *Tranziţiile se reprezintă cu structura următoare:* caracterul citit la intrare, apoi virgulă, apoi caracterul care va fi eliminat din vârful stivei, apoi punct şi virgulă (sau slash „ / ”), apoi cuvântul care va fi scris în stivă (cu aceeaşi convenţie, prima literă din cuvânt este cea care va fi în vârful stivei după adăugarea cuvântului).

L2 = {$a^n b^{2n}$, n ≥ 0}
Pentru fiecare „a” citit, vom adăuga doi de A în stivă (adică AA).
Pentru fiecare „b” citit, vom şterge un A din stivă.
Astfel ne asigurăm că numărul de „b”-uri este dublul celui de „a”-uri.
Schimbăm starea după ce citim primul „b”, pentru a nu amesteca literele.
Trebui să ne asigurăm că acceptăm şi cuvântul vid (pentru că avem „n ≥ 0”).

$\delta(q_0, a, Z_0) = (q_0, AAZ_0)$ // *adaugam AA pentru primul "a"*
$\delta(q_0, a, A) = (q_0, AAA)$ // *adaugam AA pentru oricare "a"*
$\delta(q_0, b, A) = (q_1, \lambda)$ // *stergem un A pentru primul "b"*
$\delta(q_1, b, A) = (q_1, \lambda)$ // *stergem un A pentru oricare "b"*
$\delta(q_1, \lambda, Z_0) = (q_2, \lambda)$ // *gata cuvantul, stare finala si stiva vida*
$\delta(q_0, \lambda, Z_0) = (q_2, \lambda)$ // *acceptam cuvantul vid, stare finala si stiva vida*
$F = \{q_2\}$

Aceeaşi soluţie, sub formă de graf:



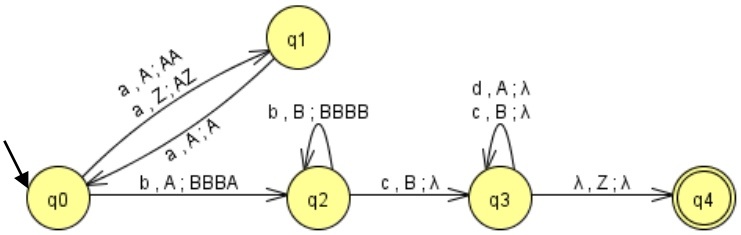
L3 = {$a^{2n} b^k c^{3k} d^n$, n ≥ 1, k ≥ 1}
Pentru fiecare „aa” de la începutul cuvântului trebuie să avem „d” la finalul cuvântului.
*(**Obs:** Dacă am vrea să adăugăm câte un A pentru fiecare „a”, atunci ar trebui ca pentru fiecare „d” să dispară doi de A; nu putem face asta direct, ci ar trebui un circuit de lungime 2 între două stări, cu o muchie se elimină un A pentru „d”-ul citit, iar cu muchia de întoarcele se elimină un A făcând o λ–tranziţie)*
Cel mai simplu, pentru pentru fiecare „aa” citiţi, vom scrie în stivă un singur A (pentru primul „a” din pereche se adaugă A, pentru al doilea „a” nu se modifică stiva). Avem nevoie de circuit de lungime 2 pentru a controla ca numărul de „a”-uri să fie par.
La mijlocul cuvântului, pentru fiecare „b” din stânga trebuie să avem „ccc” în dreapta.
Deasupra A-urilor din stivă, pentru fiecare „b” vom adăuga câte 3 de B.
Apoi pentru fiecare „c” vom şterge câte un B din stivă.
*(**Obs:** Pentru „c”-uri nu vom avea nevoie de circuit de lungime 3 între stări pentru că în stivă avem deja un număr de B-uri multiplu de 3.)*
La final, pentru fiecare „d” vom şterge câte un „A”.

$\delta(q_0, a, Z_0) = (q_1, AZ_0)$ // pentru primul „a”, adăugăm un A
$\delta(q_1, a, A) = (q_0, A)$ // pentru „a”-ul al 2-lea (4-lea, 6-lea, ...) nu modificăm stiva
$\delta(q_0, a, A) = (q_1, AA)$ // pentru „a”-ul al 3-lea (5-lea, 7-lea, ...) adăugăm un A
$\delta(q_0, b, A) = (q_2, BBBA)$ // pentru primul „b” schimbăm starea, adăugăm BBB
$\delta(q_2, b, B) = (q_2, BBBB)$ // pentru orice alt „b”, adăugăm BBB
$\delta(q_2, c, B) = (q_3, \lambda)$ // pentru primul „c” schimbăm starea, şterge un B
$\delta(q_3, c, B) = (q_3, \lambda)$ // pentru orice alt „c”, şterge un B
$\delta(q_3, d, A) = (q_3, \lambda)$ // pentru orice „d”, şterge un A

$\delta(q_3, \lambda, Z_0) = (q_4, \lambda)$ // am terminat cuvântul, stare finală şi stivă vidă
$F = \{q_4\}$

Obs: *Când am început să citim „d”-urile nu a fost neapărat necesar să schimbăm starea faţă de cea în care am citit „c”-urile, deoarece nu există problema amestecării lor. Putem citi „c”-uri doar atât timp cât găsim B-uri în stivă, iar apoi putem citi „d”-uri doar pentru A-urile din stivă.*

Aceeaşi soluţie, sub formă de graf:



Verificare acceptare cuvânt de către un APD

Folosim „configuraţii” (sau „descrieri instantanee”) având 3 componente: starea curentă, cât a mai rămas de citit din cuvântul de intrare şi (întreg!) conţinutul stivei (partea stângă fiind vârful stivei, unde se fac ştergerile şi scrierile).

Fie n = 3 şi k = 2, deci avem cuvântul $a^6b^2c^6d^3$.
Scriu de fiecare dată numărul tranziţiei pe care am aplicat-o (a câta era în lista scrisă anterior).
$(q_0, a^6b^2c^6d^3, Z_0) \vdash^{(1)} (q_1, a^5b^2c^6d^3, AZ_0) \vdash^{(2)} (q_0, a^4b^2c^6d^3, AAZ_0)$
$\vdash^{(3)} (q_1, a^3b^2c^6d^3, AAAZ_0) \vdash^{(2)} (q_0, a^2b^2c^6d^3, AAAAZ_0)$
$\vdash^{(3)} (q_1, ab^2c^6d^3, AAAAAZ_0) \vdash^{(2)} (q_0, b^2c^6d^3, AAAAAZ_0)$
$\vdash^{(4)} (q_2, bc^6d^3, BBBAAAAZ_0) \vdash^{(5)} (q_2, c^6d^3, B^6A^3Z_0)$
$\vdash^{(6)} (q_3, c^5d^3, B^5A^3Z_0) \vdash^{(7)} (q_3, c^4d^3, B^4A^3Z_0) \vdash^{(7)} (q_3, c^3d^3, B^3A^3Z_0)$
$\vdash^{(7)} (q_3, c^2d^3, B^2A^3Z_0) \vdash^{(7)} (q_3, cd^3, BA^3Z_0) \vdash^{(7)} (q_3, d^3, A^3Z_0)$
$\vdash^{(8)} (q_3, d^2, A^2Z_0) \vdash^{(8)} (q_3, d, AZ_0) \vdash^{(8)} (q_3, \lambda, Z_0)$
$\vdash^{(9)} (q_4, \lambda, \lambda)$, $q_4 \in F$, deci cuvântul este acceptat.

L4 = {$a^{2n} b^{3n} c^{2k} d^k$, n ≥ 1, k ≥ 1}

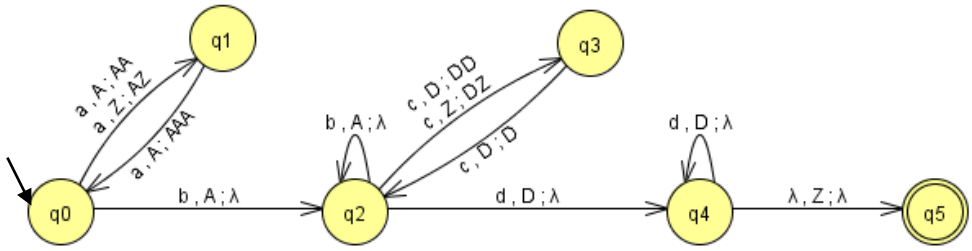
Acum avem grupate primele două părţi şi ultimele două părţi.
Pentru fiecare „aa” trebuie să avem apoi „bbb”. Pentru primul „a” din fiecare pereche vom adăuga A în stivă, iar pentru al doilea „a” din fiecare pereche vom adăuga AA în stivă. Deci pentru fiecare „b” vom şterge câte un A din stivă.
(Obs: Am fi putut la fel de bine pentru unul dintre „a” să scriem AAA iar pentru celălalt nimic. Orice variantă am alege, important e ca după citirea „aa” să apară AAA în stivă.)
Apoi, după ce toţi A au fost şterşi, trebuie să avem pentru fiecare „cc” câte un „d”. Pentru primul „c” din fiecare pereche vom adăuga un D în stivă, iar pentru al doilea „c” din fiecare pereche vom lăsa stiva nemodificată. Apoi pentru fiecare „d” citit vom şterge un D din stivă.

Atât pentru „a”-uri cât şi pentru „c”-uri vom avea nevoie de câte un circuit de lungime 2 între stări.

$\delta(q_0, a, Z_0) = (q_1, AZ_0)$ // pentru primul „a”, adăugăm un A
$\delta(q_1, a, A) = (q_0, AAA)$ // pentru „a”-ul al 2-lea (4-lea, 6-lea, ...) adăugăm AA
$\delta(q_0, a, A) = (q_1, AA)$ // pentru „a”-ul al 3-lea (5-lea, 7-lea, ...) adăugăm un A
$\delta(q_0, b, A) = (q_2, \lambda)$ // pentru primul „b” schimbăm starea, ştergem un A
$\delta(q_2, b, A) = (q_2, \lambda)$ // pentru orice alt „b”, ştergem un A
$\delta(q_2, c, Z_0) = (q_3, DZ_0)$ // pentru primul „c” schimbăm starea, adăugăm un D
$\delta(q_3, c, D) = (q_2, D)$ // pentru „c”-ul al 2-lea (4-lea, 6-lea, ...) nu modificăm stiva
$\delta(q_2, c, D) = (q_3, DD)$ // pentru „c”-ul al 3-lea (5-lea, 7-lea, ...) adăugăm un D
$\delta(q_2, d, D) = (q_4, \lambda)$ // pentru primul „d” schimbăm starea, ştergem un D
$\delta(q_4, d, D) = (q_4, \lambda)$ // pentru orice alt „d”, ştergem un D
$\delta(q_4, \lambda, Z_0) = (q_5, \lambda)$ // am terminat cuvântul, stare finală şi stivă vidă
$F = \{q_5\}$

Obs: Când am vrut să închidem circuitul pentru „c”-uri nu a trebuit neapărat să schimbăm starea, am putut folosi q_2 pentru că aveam „b” cu A şi „c” cu D (sau Z_0), deci nu exista risc de amestecare. Dar când am început citirea „d”-urilor a trebuit să o schimbăm pentru că aveam „c” cu D şi „d” cu D, deci s-ar fi putut amesteca adăugările şi ştergerile D-urilor (adică „c”-urile cu „d”-urile).

Aceeaşi soluţie, sub formă de graf:



~ Seminar 5 ~

(Exemple cu APD (continuare) ;
Algoritmul CYK: Verificare generare cuvânt de către gramatică în F.N.Chomsky)

EXEMPLE (automate push-down) [continuare]:

Notăm cu $\mathbf{w^R}$ ***cuvântul oglindit*** al lui w. *(R vine de la reverse)*
De exemplu: $(abcde)^R = edcba$

L5 = { w w^R | w $\in$ {a,b,c}* \ {λ} } ***(palindroame de lungime pară)***
= {aa, bb, cc, abba, acca, baab, bccb, caac, cbbc,
aaaaaa, aabbaa, aaccaa, abaaba, abbbba, abccba, acaaca, acbbca, acccca, ...}

Pentru prima jumătate de cuvânt am dori ca pentru fiecare „a" să adăugăm un A în stivă, pentru fiecare „b" să adăugăm un B, iar pentru fiecare „c" că adăugăm un C. Vrem ca literele să fie amestecate deci trebuie să folosim aceeaşi stare pentru toate citirile.
Pentru a doua jumătate de cuvânt am dori ca pentru fiecare „a" să ştergem un A, pentru fiecare „b" să ştergem un B, iar pentru fiecare „c" să ştergem un C. La fel, folosind aceeaşi stare pentru toate ştergerile.

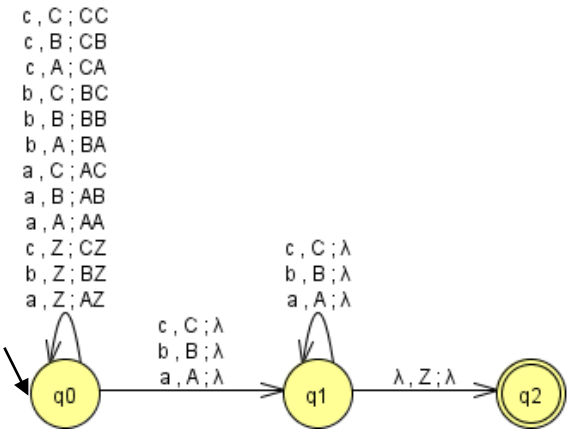
*Dar putem ca **exact** după prima literă din a doua jumătate a cuvântului să schimbăm starea? NU, pentru că nu avem de unde şti unde este jumătatea cuvântului!*
Deci dacă cuvântul conţine undeva „aa", nu vom şti dacă exact acolo este mijlocul (deci ar trebui să trecem în starea pentru ştergeri) sau dacă suntem cumva tot în prima jumătate (deci trebuie să scriem şi pentru al doilea).
De aceea, acesta va fi un ***APD nedeterminist*** !

$\delta(q_0, a, Z_0) = (q_0, AZ_0)$ // cuvântul începe cu „a"
$\delta(q_0, b, Z_0) = (q_0, BZ_0)$ // sau cuvântul începe cu „b"
$\delta(q_0, c, Z_0) = (q_0, CZ_0)$ // sau cuvântul începe cu „c"
$\delta(q_0, a, A) = \{(q_0, AA), (q_1, \lambda)\}$ // fie scriem; fie am ajuns la mijloc, altă stare şi ştergem
$\delta(q_0, a, B) = (q_0, AB)$ // păstrăm B-ul, scriem A
$\delta(q_0, a, C) = (q_0, AC)$ // păstrăm C-ul, scriem A
$\delta(q_0, b, B) = \{(q_0, BB), (q_1, \lambda)\}$ // fie scriem; fie am ajuns la mijloc, altă stare şi ştergem
$\delta(q_0, b, A) = (q_0, BA)$ // păstrăm A-ul, scriem B
$\delta(q_0, b, C) = (q_0, BC)$ // păstrăm C-ul, scriem B
$\delta(q_0, c, C) = \{(q_0, CC), (q_1, \lambda)\}$ // fie scriem; fie am ajuns la mijloc, altă stare şi ştergem
$\delta(q_0, c, A) = (q_0, CA)$ // păstrăm A-ul, scriem C

$\delta(q_0, c, B) = (q_0, CB)$ // păstrăm B-ul, scriem C
$\delta(q_1, a, A) = (q_1, \lambda)$ // ştergem A când citim „a” din a doua jumătate
$\delta(q_1, b, B) = (q_1, \lambda)$ // ştergem B când citim „b” din a doua jumătate
$\delta(q_1, c, C) = (q_1, \lambda)$ // ştergem C când citim „c” din a doua jumătate
$\delta(q_1, \lambda, Z_0) = (q_2, \lambda)$ // gata cuvântul, stare finală, stivă vidă
$F = \{q_2\}$

Înainte de a decide că nu acceptă un cuvânt, APD-ul nedeterminist va încerca toate drumurile posibile.
De exemplu, dacă avem cuvântul „aaaa”.
Plecăm din configuraţia (q_0, aaaa, Z_0), aplicăm tranziţia (1) şi ajungem în (q_0, aaa, AZ_0).
Apoi avem două variante de continuare:
(i) Dacă aplicăm tranziţia (4), ajungem în (q_0, aa, AAZ_0).
Acum avem din nou două variante de continuare:
(i.i) Dacă aplicăm tranziţia (4), ajungem în (q_0, a, $AAAZ_0$).
Din nou, avem cele două variante de continuare:
(i.i.i) Dacă aplicăm tranziţia (4), ajungem în (q_0, λ, $AAAAZ_0$).
De aici, nu avem nicio posibilitate de continuare, deci cuvântul nu va fi acceptat folosind acest drum.
(i.i.ii) Dacă aplicăm tranziţia (5), ajungem în (q_1, λ, AAZ_0).
Am terminat de citit cuvântul, dar avem A în vârful stivei, deci cuvântul nu va fi acceptat folosind acest drum.
(i.ii) Dacă aplicăm tranziţia (5), ajungem în (q_1, a, AZ_0).
Aplicăm tranziţia (16), ajungem în (q_1, λ, Z_0).
Aplicăm (19), ajungem în (q_2, λ, λ) şi *acceptăm* cuvântul.
(ii) Dacă aplicăm tranziţia (5), ajungem în (q_1, aa, Z_0).
De aici, nu avem nicio posibilitate de continuare. Nu am terminat de citit tot cuvântul de intrare, deci nu va fi acceptat folosind acest drum.

Aceeaşi soluţie, sub formă de graf:

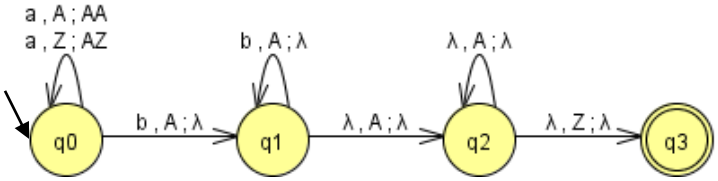


Obs: Până acum am avut doar condiţii cu egalităţi între puteri (sau divizori ai puterilor), dar putem avea şi ***inegalităţi***.

L6 = {$a^n b^k$, n > k ≥ 1}
Pentru fiecare „a” vom adăuga un A în stivă, pentru fiecare „b” vom şterge un A din stivă.
Numărul de „a”-uri trebuie să fie strict mai mare decât numărul de „b”-uri. De aceea, după ce am terminat de citit cuvântul de intrare, trebuie să mai existe cel puţin un A în stivă. Schimbăm starea, ştergem toată stiva şi acceptăm cuvântul.

$\delta(q_0, a, Z_0) = (q_0, AZ_0)$
$\delta(q_0, a, A) = (q_0, AA)$
$\delta(q_0, b, A) = (q_1, \lambda)$
$\delta(q_1, b, A) = (q_1, \lambda)$
$\delta(q_1, \lambda, A) = (q_2, \lambda)$ // obligatoriu mai găseşte cel puţin un A în stivă
$\delta(q_2, \lambda, A) = (q_2, \lambda)$ // altfel nu poate ajunge din q_1 în q_3
$\delta(q_2, \lambda, Z_0) = (q_3, \lambda)$, $F = \{q_3\}$

Aceeaşi soluţie, sub formă de graf:



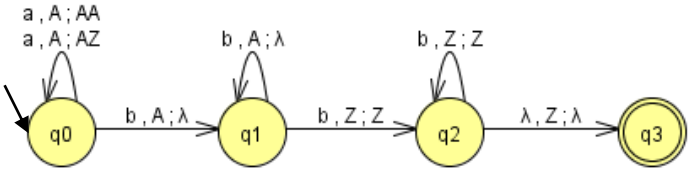
Obs: Dacă am fi avut în schimb condiţia „**n ≥ k ≥ 1**”, am fi avut în plus tranziţia $\delta(q_1, \lambda, Z_0) = (q_3, \lambda)$ pentru cazul de egalitate (n=k).
Pentru condiţia „**n > k ≥ 0**”, am fi avut şi $\delta(q_0, \lambda, A) = (q_2, \lambda)$ (în plus faţă de cele iniţiale), pentru k=0.
Pentru (pentru „**n ≥ k ≥ 0**”) am fi avut $\delta(q_1, \lambda, Z_0) = (q_3, \lambda)$ (pentru n=k) şi $\delta(q_0, \lambda, Z_0) = (q_2, Z_0)$ (pentru k=0).

L7 = {$a^n b^k$, k > n ≥ 1}
Pentru fiecare „a” vom adăuga un A în stivă, pentru fiecare „b” vom şterge un A din stivă.
Numărul de „a”-uri trebuie să fie strict mai mic decât numărul de „b”-uri. De aceea, după ce vom elimina toţi de A din stivă va trebui să mai citim cel puţin un „b” din cuvânt şi să lăsăm stiva nemodificată. Când am terminat de citit cuvântul, schimbăm starea, îl ştergem şi pe Z_0 din stivă şi acceptăm cuvântul.

$\delta(q_0, a, Z_0) = (q_0, AZ_0)$

$\delta(q_0, a, A) = (q_0, AA)$
$\delta(q_0, b, A) = (q_1, \lambda)$
$\delta(q_1, b, A) = (q_1, \lambda)$
$\delta(q_1, b, Z_0) = (q_2, Z_0)$ // obligatoriu mai citeşte cel puţin un „b”
$\delta(q_2, b, Z_0) = (q_2, Z_0)$ // dar nu uităm că trebuie să-l păstrăm pe Z_0 până la final
$\delta(q_2, \lambda, Z_0) = (q_3, \lambda), F = \{q_3\}$

Aceeaşi soluţie, sub formă de graf:

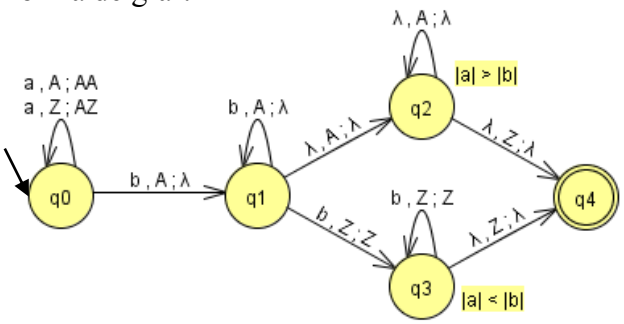


Obs: Dacă era condiţia „k ≥ n ≥ 1” aveam şi $\delta(q_1, \lambda, Z_0) = (q_3, \lambda)$ (pentru k=n).
Dacă era „k > n ≥ 0” aveam şi $\delta(q_0, b, Z_0) = (q_2, Z_0)$ (pentru n=0).
Dacă era „k ≥ n ≥ 0” aveam $\delta(q_1, \lambda, Z_0) = (q_3, \lambda)$ (pentru k=n) şi
$\delta(q_0, \lambda, Z_0) = (q_2, Z_0)$ (pentru n=0).

L8 = {$a^n b^k$, n ≥ 1, k ≥ 1, n ≠ k}
Putem avea fie numărul de „a”-uri strict mai mare decât numărul de „b”-uri fie invers (numărul de „b”-uri strict mai mare decât numărul de „a”-uri).
Pentru fiecare „a” vom adăuga un A, iar pentru fiecare „b” vom şterge un A. Fie ne vor rămâne A-uri în stivă după ce terminăm de citit cuvântul de intrare (dacă n > k), fie vom întâlni Z_0 dar vom mai citi cel puţin un „b” din cuvânt (dacă n < k).

$\delta(q_0, a, Z_0) = (q_0, AZ_0)$
$\delta(q_0, a, A) = (q_0, AA)$
$\delta(q_0, b, A) = (q_1, \lambda)$
$\delta(q_1, b, A) = (q_1, \lambda)$
// n > k
$\delta(q_1, \lambda, A) = (q_2, \lambda)$ // obligatoriu mai găseşte cel puţin un A în stivă
$\delta(q_2, \lambda, A) = (q_2, \lambda)$ // altfel nu poate ajunge din q_1 în q_4
$\delta(q_2, \lambda, Z_0) = (q_4, \lambda)$
// n < k
$\delta(q_1, b, Z_0) = (q_3, Z_0)$ // obligatoriu mai citeşte cel puţin un „b”
$\delta(q_3, b, Z_0) = (q_3, Z_0)$ // dar nu uităm că trebuie să-l păstrăm pe Z_0 până la final
$\delta(q_3, \lambda, Z_0) = (q_4, \lambda)$
$F = \{q_4\}$

Aceeaşi soluţie, sub formă de graf:



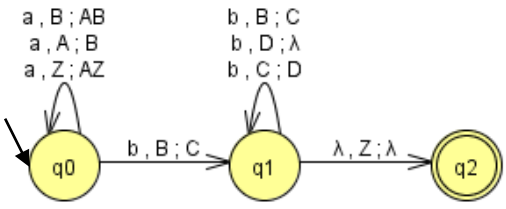
L9 = {$a^{2n} b^{3k}$, n ≥ 1, k ≥ 1, n ≠ k}

Obs: Există şi *o altă modalitate de a avea circuite de o anumită lungime*. În loc să folosim stări diferite, putem folosi simboluri diferite în stivă pe care le înlocuim unul cu altul.

Trebuie să avem grupuri de câte doi „a”. Când îl citim pe primul din grup adăugăm un A în stivă. Când îl citim pe al doilea din grup vom înlocui A-ul cu un B. La final vom avea n de B în stivă.
Trebuie să avem grupuri de câte trei „b”. Când citim primul „b” din grup înlocuim un B cu C în stivă. Când citim al doilea „b” din grup înlocuim C-ul cu un D. Iar când citim al treilea „b” din grup ştergem D-ul din stivă.
Deci avem circuit între A-B-A pentru 2n şi circuit între B-C-D-B pentru 3k.

$\delta(q_0, a, Z_0) = (q_0, AZ_0)$ // pentru primul ”a”, adăugăm A
$\delta(q_0, a, A) = (q_0, B)$ // pentru al 2-lea (4-lea, 6-lea, ...) ”a”, înlocuim A cu B
$\delta(q_0, a, B) = (q_0, AB)$ // pentru al 3-lea (5-lea, 7-lea, ...) ”a”, adăugăm A
// *schimbăm starea când începem citirea ”b”-urilor*
$\delta(q_0, b, B) = (q_1, C)$ // pentru primul ”b”, înlocuim B cu C
$\delta(q_1, b, C) = (q_1, D)$ // pentru al doilea (al 3m+2 -lea) ”b”, înlocuim C cu D
$\delta(q_1, b, D) = (q_1, \lambda)$ // pentru al treilea (al 3m+3 -lea) ”b”, ştergem D-ul
$\delta(q_1, b, B) = (q_1, C)$ // pentru al patrulea (al 3m+1 -lea) ”b”, înlocuim B cu C
$\delta(q_1, \lambda, Z_0) = (q_2, \lambda), F = \{q_2\}$

Aceeaşi soluţie, sub formă de graf:



Algoritmul CYK (verificare generare cuvânt de către gramatică în F.N.Chomsky)

Am văzut că dacă gramatica avea producţii specifice uneia independente de context, atunci foloseam arbori de derivare şi porneam de la simbolul de start S şi încercam să ajungem la cuvântul dat. *(pagina 36 în acest document)*

Dar dacă gramatica are producţiile scrise în forma normală Chomsky, atunci vom folosi acest algoritm CYK, şi vom porni de la cuvântul dat încercând să ajungem la simbolul de start S.

$V_{ij} = \{A \in N \mid \exists B \in V_{ik}, \exists C \in V_{i+k,\, j-k}, (A \to BC) \in P, k \in \{1,2,..., j-1\}\}$

Notăm cu $V_{i\,j}$ mulţimea de neterminale care pot genera acea porţiune din cuvântul de intrare care începe de pe poziţia i şi are lungimea j.

Ideea este de a împărţi cuvântul în două părţi în toate modurile posibile. Dacă prima parte poate fi generată de neterminalul B, a doua parte poate fi generată de neterminalul C şi avem în gramatică producţia "A→BC", atunci înseamnă că întreg cuvântul poate fi generat de neterminalul A.
Observăm că avem împărţiri posibile pentru k=1 (prima literă în prima parte, iar restul în a doua), k=2 (primele două litere în prima parte, iar restul în a doua), ... până la k=j–1 (doar ultima literă în a doua parte, iar restul în prima).

Obs: *Nu reţineţi indicii pe dinafară, ci încercaţi să îi deduceţi logic!*
Valoare pe care o calculăm este $V_{i\,j}$ pentru cuvântul care începe la poziţia i şi are lungimea j.
Prima parte (generată de neterminalul B) conţine primele k litere din cuvânt. Deci va începe la poziţia i (la fel ca întreg cuvântul) şi va avea lungime k, deci $V_{i\,k}$.
A doua parte (generată de neterminalul C) conţine restul de litere, adică j–k, şi începe cu k poziţii mai în dreapta faţă de cuvântul total, adică la i+k, deci avem $V_{i+k,\,j-k}$.

Răspunsul final pe care îl căutăm este $V_{1,\,|w|}$ adică mulţimea de neterminale din care putem genera cuvântul dat începând de pe prima poziţie şi având lungimea egală cu întreg cuvântul. Dacă în această mulţime se va găsi şi simbolul de start S, înseamnă că cuvântul este generat de gramatică. Dacă nu, atunci nu este generat.

La pasul iniţial vom căuta neterminalele care generează fiecare literă mică (terminal) din cuvânt: $V_{i1} = \{A \in N \mid (A \to x_i) \in P\}$, unde x_i este litera de pe poziţia i din cuvântul dat.

EXEMPLU:

Avem o gramatică în F.N.Chomsky: $\begin{aligned} S &\to BA \\ A &\to a \mid BA \mid CA \\ B &\to b \mid AA \mid AB \\ C &\to c \mid BC \mid BB \end{aligned}$ şi un cuvânt: baabca.

Avem un tabel cu i pe coloane şi j pe linii în care vom completa neterminalele din mulţimile $V_{i\,j}$. Variabilele i şi j iau valori între 1 şi 6 (lungimea cuvântului dat). Observăm că putem avea **maxim i + j – 1 = |w|** = 6. Deci de exemplu V_{62} nu va fi calculat (6 + 2 – 1 = 7 > 6) pentru că dacă începem pe poziţia 6 (ultima) nu vom putea avea lungimea 2 în cuvânt.

Pentru prima linie, j = 1, vom căuta în gramatică pentru fiecare literă mică de ce neterminale poate fi generată.
Vom avea $V_{21} = V_{31} = V_{61} = \{A\}$ pentru că litera ”a” se află în cuvânt pe poziţiile 2, 3, 6, iar neterminalul A este singurul care generează ”a”.
Vom avea $V_{11} = V_{41} = \{B\}$ pentru că litera ”b” se află în cuvânt pe poziţiile 1, 4, iar neterminalul B este singurul care generează ”b”.
Vom avea $V_{51} = \{C\}$ pentru că litera ”c” se află în cuvânt pe poziţia 5, iar neterminalul C este singurul care generează ”c”.

$V_{i\,j}$	i = 1	i = 2	i = 3	i = 4	i = 5	i = 6
j = 1	{B}	{A}	{A}	{B}	{C}	{A}
j = 2						
j = 3						
j = 4						
j = 5						
j = 6						

Pentru a doua linie, j = 2, avem cuvinte de lungime 2, deci nu le putem separa decât într-un singur mod, pentru k = 1.
$V_{12} = V_{11} \times V_{21} = \{B\} \times \{A\} = \{BA\}$
În gramatică apare BA ca membru drept pentru neterminalele S şi A, deci în tabel vom completa cu mulţimea {S, A}.
$V_{22} = V_{21} \times V_{31} = \{A\} \times \{A\} = \{AA\}$
Singurul neterminal care se rescrie în AA este B, deci îl trecem în tabel.
$V_{32} = V_{31} \times V_{41} = \{A\} \times \{B\} = \{AB\}$ rezultă B în tabel.
$V_{42} = V_{41} \times V_{51} = \{B\} \times \{C\} = \{BC\}$ rezultă C în tabel.
$V_{52} = V_{51} \times V_{61} = \{C\} \times \{A\} = \{CA\}$ rezultă A în tabel.

Obs: Pentru a calcula orice valoare de pe linia j=2 se face produsul cartezian între mulţimile aflate la nord şi la nord-est de acea valoare.

V_{ij}	i = 1	i = 2	i = 3	i = 4	i = 5	i = 6
j = 1	{B}	{A}	{A}	{B}	{C}	{A}
j = 2	{S, A}	{B}	{B}	{C}	{A}	
j = 3						
j = 4						
j = 5						
j = 6						

Pentru a treia linie, j = 3, avem cuvinte de lungime 3, deci putem avea k=1 sau k=2 şi reunim cele două cazuri.

$V_{13} = V_{11} \times V_{22} \cup V_{12} \times V_{31} = \{B\} \times \{B\} \cup \{S, A\} \times \{A\} = \{BB, SA, AA\}$

Nu avem SA ca membru drept în gramatică, dar avem BB generat de C, iar AA de B, deci în tabel completăm {B, C}.

$V_{23} = V_{21} \times V_{32} \cup V_{22} \times V_{41} = \{A\} \times \{B\} \cup \{B\} \times \{B\} = \{AB, BB\}$ rezultă {B, C} în tabel.

$V_{33} = V_{31} \times V_{42} \cup V_{32} \times V_{51} = \{A\} \times \{C\} \cup \{B\} \times \{C\} = \{AC, BC\}$ rezultă {C} în tabel.

$V_{43} = V_{41} \times V_{52} \cup V_{42} \times V_{61} = \{B\} \times \{A\} \cup \{C\} \times \{A\} = \{BA, CA\}$ rezultă {S, A} în tabel.

V_{ij}	i = 1	i = 2	i = 3	i = 4	i = 5	i = 6
j = 1	{B}	{A}	{A}	{B}	{C}	{A}
j = 2	{S, A}	{B}	{B}	{C}	{A}	
j = 3	{B, C}	{B, C}	{C}	{S, A}		
j = 4						
j = 5						
j = 6						

Pentru a patra linie, j = 4, avem cuvinte de lungime 4, deci putem avea k=1, k=2 sau k=3.

$V_{14} = (V_{11} \times V_{23}) \cup (V_{12} \times V_{32}) \cup (V_{13} \times V_{41})$

$= \{B\} \times \{B, C\} \cup \{S, A\} \times \{B\} \cup \{B, C\} \times \{B\} = \{BB, BC, SB, AB, CB\}$

Nu avem SB sau CB ca membru drept în gramatică, dar pentru restul avem {B,C} în tabel.

$V_{24} = (V_{21} \times V_{33}) \cup (V_{22} \times V_{42}) \cup (V_{23} \times V_{51})$

$= \{A\} \times \{C\} \cup \{B\} \times \{C\} \cup \{B, C\} \times \{C\} = \{AC, BC, CC\}$

Nu avem AC sau CC, dar pentru BC rezultă {C} în tabel.

$V_{34} = (V_{31} \times V_{43}) \cup (V_{32} \times V_{52}) \cup (V_{33} \times V_{61})$

$= \{A\} \times \{S, A\} \cup \{B\} \times \{A\} \cup \{C\} \times \{A\} = \{AS, AA, BA, CA\}$

Nu avem AS, dar pentru celelalte rezultă {S,A,B} în tabel.

V_{ij}	i = 1	i = 2	i = 3	i = 4	i = 5	i = 6
j = 1	{B}	{A}	{A}	{B}	{C}	{A}
j = 2	{S, A}	{B}	{B}	{C}	{A}	
j = 3	{B, C}	{B, C}	{C}	{S, A}		
j = 4	{B, C}	{C}	{S, A, B}			
j = 5						
j = 6						

Pentru a cincea linie, avem cuvinte de lungime j = 5, iar k ia valori între 1 şi 4.

$V_{15} = (V_{11} \times V_{24}) \cup (V_{12} \times V_{33}) \cup (V_{13} \times V_{42}) \cup (V_{14} \times V_{51})$

$= \{B\} \times \{C\} \cup \{S, A\} \times \{C\} \cup \{B, C\} \times \{C\} \cup \{B, C\} \times \{C\} = \{BC, SC, AC, CC\}$

Nu avem SC, AC sau CC, dar pentru BC rezultă {C} în tabel.

$V_{25} = (V_{21} \times V_{34}) \cup (V_{22} \times V_{43}) \cup (V_{23} \times V_{52}) \cup (V_{24} \times V_{61})$

$= \{A\} \times \{S, A, B\} \cup \{B\} \times \{S, A\} \cup \{B, C\} \times \{A\} \cup \{C\} \times \{A\} = \{AS, AA, AB, BS, BA, CA\}$

Nu avem AS, BS dar pentru AA, AB, BA, CA rezultă {S,A,B} în tabel.

V_{ij}	i = 1	i = 2	i = 3	i = 4	i = 5	i = 6
j = 1	{B}	{A}	{A}	{B}	{C}	{A}
j = 2	{S, A}	{B}	{B}	{C}	{A}	
j = 3	{B, C}	{B, C}	{C}	{S, A}		
j = 4	{B, C}	{C}	{S, A, B}			
j = 5	{C}	{S, A, B}				
j = 6						

Pentru linia a şasea, avem cuvântul total de lungime j=6, variabila k ia valori între 1 şi 5.

$V_{16} = (V_{11} \times V_{25}) \cup (V_{12} \times V_{34}) \cup (V_{13} \times V_{43}) \cup (V_{14} \times V_{52}) \cup (V_{15} \times V_{61})$

$= \{B\} \times \{S, A, B\} \cup \{S, A\} \times \{S, A, B\} \cup \{B, C\} \times \{S, A\} \cup \{B, C\} \times \{A\} \cup \{C\} \times \{A\}$

$= \{BS, BA, BB, SS, SA, SB, AS, AA, AB, CS, CA\}$

Pentru BA, BB, AA, AB, CA avem în tabel {S,A,B,C}.

V_{ij}	i = 1	i = 2	i = 3	i = 4	i = 5	i = 6
j = 1	{B}	{A}	{A}	{B}	{C}	{A}
j = 2	{S, A}	{B}	{B}	{C}	{A}	
j = 3	{B, C}	{B, C}	{C}	{S, A}		
j = 4	{B, C}	{C}	{S, A, B}			
j = 5	{C}	{S, A, B}				
j = 6	{S,A,B,C}					

În mulţimea finală obţinută pentru V_{16} există S, deci cuvântul este generat de gramatică.

Obs: Dacă mulţimea finală conţine şi alte elemente, înseamnă că acel cuvânt dat ar putea fi generat şi pornind din acele neterminale (A, B sau C). Dar dacă existau doar ele şi nu exista şi S, acest lucru nu ne ajuta la nimic, cuvântul nu ar fi fost generat de gramatică.

Obs: Unde apare S în tabel înseamnă că gramatica generează şi cuvintele corespunzătoare pentru acele perechi (i, j).

Pentru ij = 12 avem cuvântul "ba": $\underline{S} \Rightarrow \underline{B}A \Rightarrow b\underline{A} \Rightarrow ba$.

Pentru ij = 25 avem "aabca", pentru ij = 34 avem "abca", pentru ij = 43 avem "bca".

~ Seminar 6 ~

(Leme de pompare pentru limbajele regulate)

Lema de pompare pentru limbajele regulate:

Fie L un limbaj regulat. Atunci $\exists n \in \mathrm{N}$ (număr natural) astfel încât pentru orice cuvânt $\alpha \in L$, cu $|\alpha| \geq n$, putem scrie $\alpha = u \cdot v \cdot w$ cu proprietăţile:

i) $|u \cdot v| \leq n$

ii) $|v| \geq 1$

iii) $u \cdot v^i \cdot w \in L, \forall i \geq 0$

Vrem să demonstrăm că următoarele limbaje nu sunt regulate.
Schema demonstraţiei este să presupunem prin absurd că limbajul este regulat şi atunci putem aplica lema anterioară. Alegem un cuvânt α din limbajul dat care să respecte ipoteza lemei, adică să aibă lungimea cel puţin n, iar descompunerea sa $\alpha = u \cdot v \cdot w$ să respecte condiţiile i) şi ii). Apoi alegem convenabil un număr natural i pentru care să obţinem o contradicţie a condiţiei iii), adică să rezulte că cuvântul $u \cdot v^i \cdot w \notin L$, şi deci presupunerea făcută este falsă.

L1 = {a^k^2, k ≥ 1} = {a^1, a^4, a^9, a^16, a^25, a^36, a^49, a^64, a^81, a^100, ...}

Vrem să demonstrăm că L1 nu este limbaj regulat. Observăm că L1 conţine cuvinte formate doar din litera ”a” şi având lungimi egale cu pătratele perfecte strict pozitive. Presupunem prin absurd că L1 este limbaj legulat. Atunci $\exists n \in \mathrm{N}$ şi putem aplica lema de pompare. Alegem cuvântul $\alpha = a^{n^2}$, cu $|\alpha| = n^2 \geq n$ *(deci lungimea cuvântului respectă ipoteza lemei)*.
Conform lemei, cuvântul poate fi scris sub forma $\alpha = u \cdot v \cdot w$, iar $1 \leq |v| \leq n$ *(cele două inegalităţi rezultă din condiţiile ii) şi i) şi din faptul că nu se spune că ”u” nu ar putea fi cuvântul vid, deci de lungime 0).*
De asemenea, condiţia iii) spune că $u \cdot v^i \cdot w \in L, \forall i \geq 0$. Alegem $i = 2$. Atunci avem cuvântul $u \cdot v^2 \cdot w$ de lungime $|u \cdot v^2 \cdot w| = |u \cdot v \cdot w| + |v| = |\alpha| + |v| = n^2 + |v|$.
La inegalitatea de mai sus adunăm peste tot n^2 şi obţinem: $n^2 + 1 \leq n^2 + |v| \leq n^2 + n$, adică $n^2 + 1 \leq |u \cdot v^2 \cdot w| \leq n^2 + n$.
Dar $n^2 < n^2 + 1$ şi $n^2 + n < (n+1)^2$. Rezultă că $n^2 < |u \cdot v^2 \cdot w| < (n+1)^2$, adică $u \cdot v^2 \cdot w \notin L1$ (pentru că lungimea cuvântului este inclusă strict între două pătrate perfecte alăturate, deci $|u \cdot v^2 \cdot w|$ nu poate fi pătrat perfect).

L2 = {$a^k b^k$, k ≥ 1} = {ab, a^2b^2, a^3b^3, a^4b^4, a^5b^5, ...}

Vrem să demonstrăm că L2 nu este limbaj regulat. Observăm că L2 conține cuvinte formate din k litere de ”a” urmate tot de k litere de ”b”, deci având lungimi numere pare strict pozitive.

Presupunem prin absurd că L2 este limbaj legulat. Atunci $\exists n \in \mathrm{N}$ și putem aplica lema de pompare. Alegem cuvântul $\alpha = a^n b^n$, cu $|\alpha| = 2n \geq n$ *(deci lungimea cuvântului respectă ipoteza lemei)*.

Conform lemei, cuvântul poate fi scris sub forma $\alpha = u \cdot v \cdot w$.

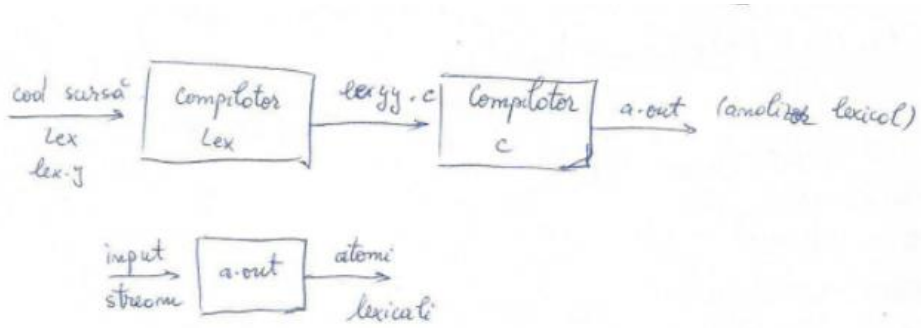
Din condiția i), avem $|u \cdot v| \leq n$. Rezultă că cuvântul $u \cdot v$ conține doar litere de ”a” (pentru că $u \cdot v$ este un prefix al primelor n caractere din α).

Atunci fie $v = a^p$. Din condițiile i) și ii) avem $1 \leq |v| \leq n$. Deci $1 \leq |a^p| \leq n$, adică $1 \leq p \leq n$.

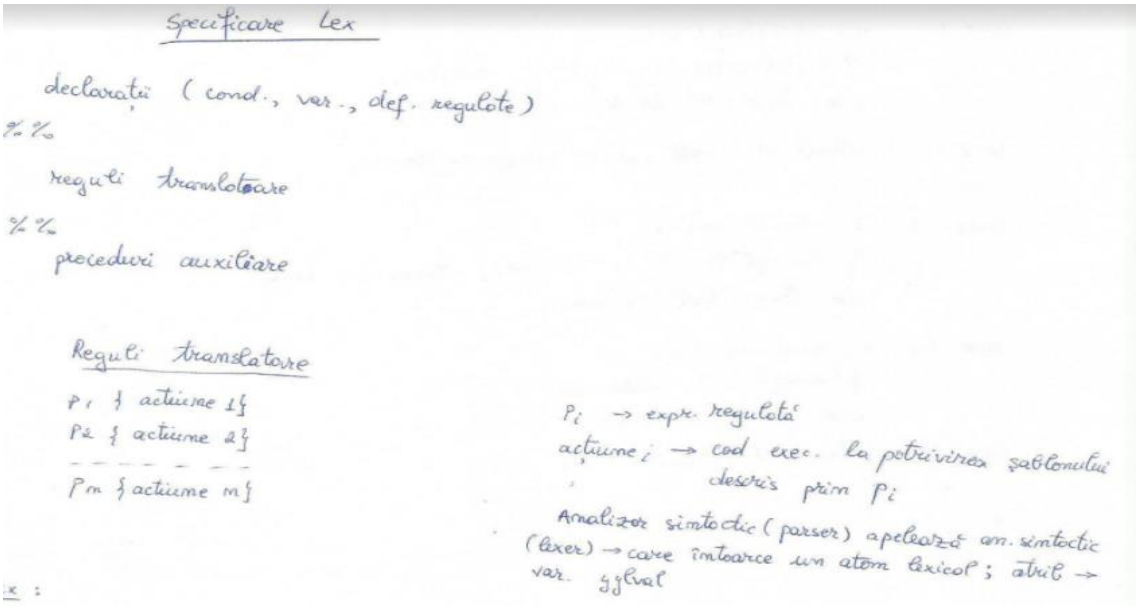
De asemenea, condiția iii) spune că $u \cdot v^i \cdot w \in L, \forall i \geq 0$. Alegem $i = 0$. Atunci avem cuvântul $u \cdot v^0 \cdot w = u \cdot w = a^{n-p} b^n \notin L2$ (pentru că p>0, deci numărul de ”a”-uri este strict mai mic decât numărul de ”b-uri”).

Analiza lexicala – lex

(cod sursa, compilator lex, compilator c. analizator lexical)



Specificare Lex (reguli translatare)



Def constante DELIM, WS, LIT, CIF, ID, NUM, WS, THEN, ELSE, <, >, LONG INSTAL_ID

```
%{ * def. constante LT, LE, EQ, NE, GT, GE, IF, THEN, ELSE, ID, NUM */
%}

delim   [ \t \n]
ws      {delim}+
lit     [A-Z a-z]
cif     [0-9]
id      {lit}({lit}|{cif})*
num     {cif}+ (\.{cif}+)?(E[+\-]?{cif}+)?

%%
{ws}    {/* nu executa nimic */}
if      {return IF;}
then    {return THEN;}
else    {return ELSE;}
{id}    {yylval = instal_id(); return ID;}
{num}   {yylval = instal_num(); return NUM;}
"<"     {yylval = LT; return OPREL;}
"<="    {yylval = LE; return OPREL;}
">"     {yylval = GT; return OPREL;}
```

+ = o dată, cel puțin

? = care poate să lipsească


```
%%
long instal_id() {/* pune în TS lexema cuprinsă între yytext și yytext+
                    yyleng */}
long instal_num() {/* pune lexema asoc. unui nr. în tabelă */}
```

obs: trebuie să prefixăm punctul (\.) pt. a avea semnificația de pct, altfel = înseamnă toate caracterele cu excepția NL (new line)

\- (minus), altfel → interval

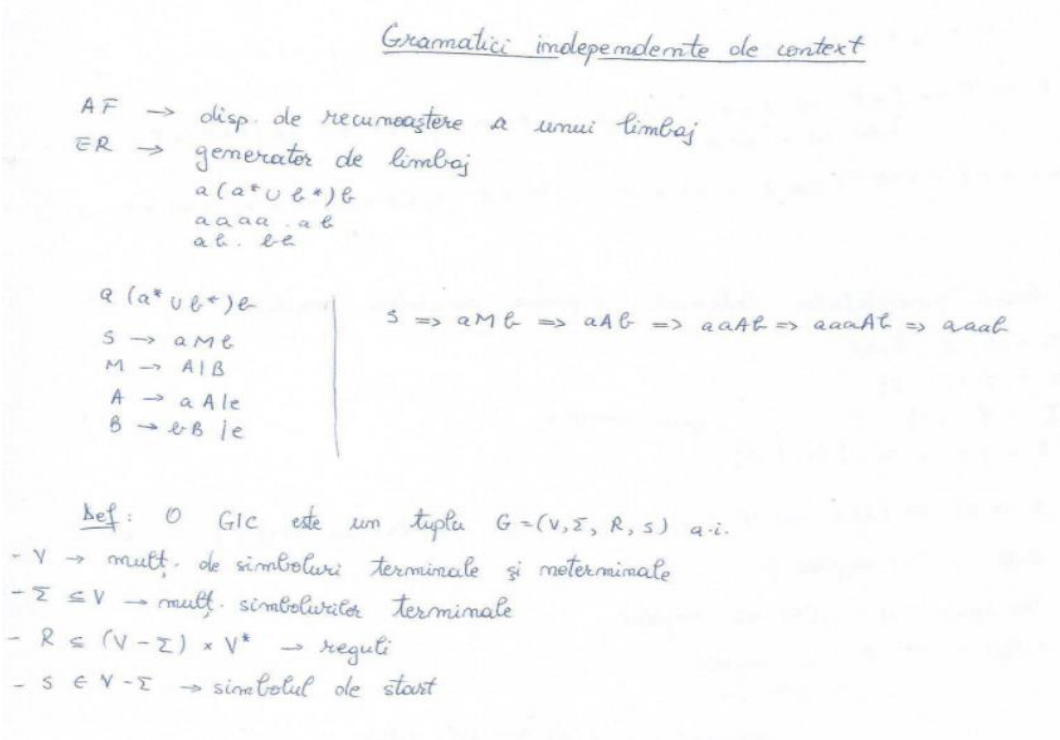
Reguli translatare: {} → definiția
șir de caractere (" eventual") → potrivire

yylval, yytext, yyleng } tip long

Gramatici independente de context

AF, ER, generator de limbaj

GIC este un tuplu (multimer de simboluri terminale si neterminale)



Gramatici independente de context

AF → disp. de recunoaștere a unui limbaj

ER → generator de limbaj

$a(a^* \cup b^*)b$

aaaa . ab

ab . bb

$a(a^* \cup b^*)b$

$S \to aMb$

$M \to A \mid B$

$A \to aA \mid \varepsilon$

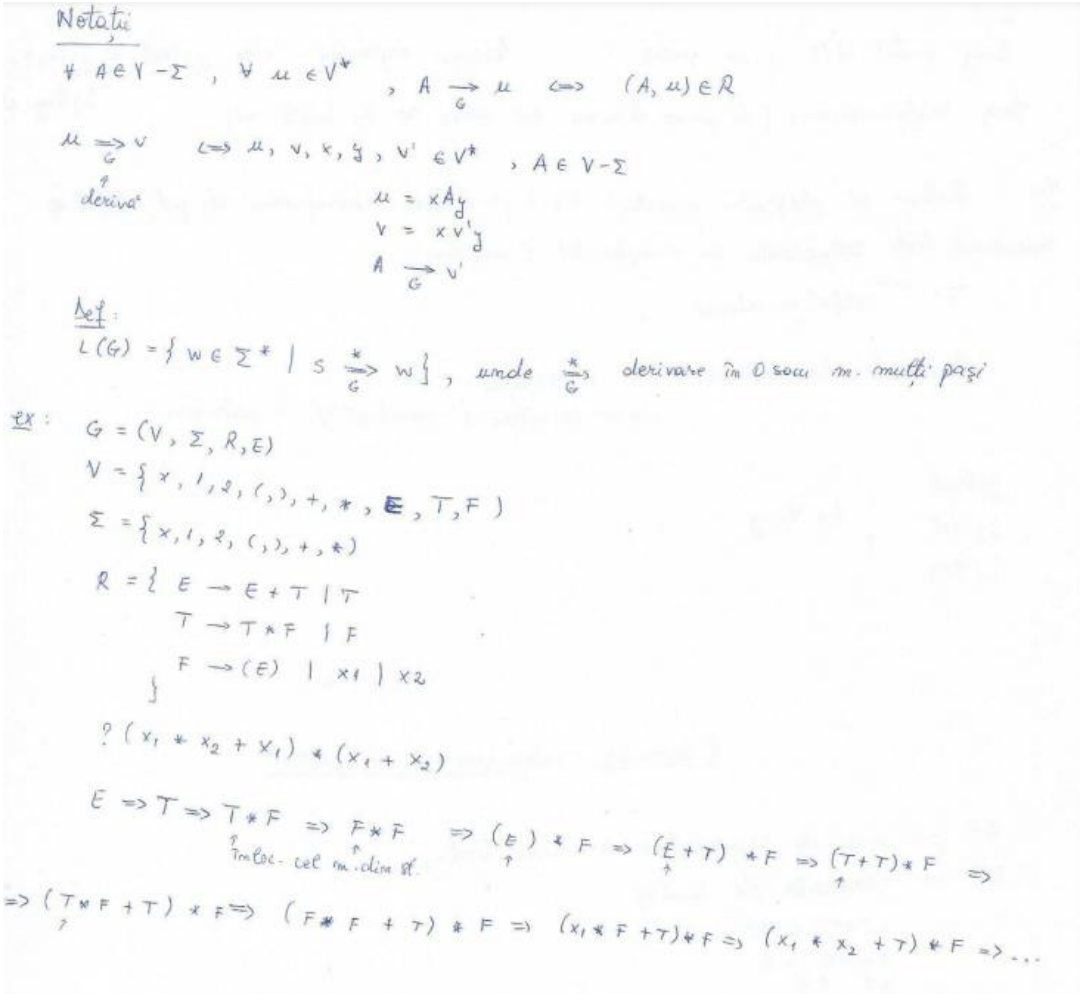
$B \to bB \mid \varepsilon$

$S \Rightarrow aMb \Rightarrow aAb \Rightarrow aaAb \Rightarrow aaaAb \Rightarrow aaab$

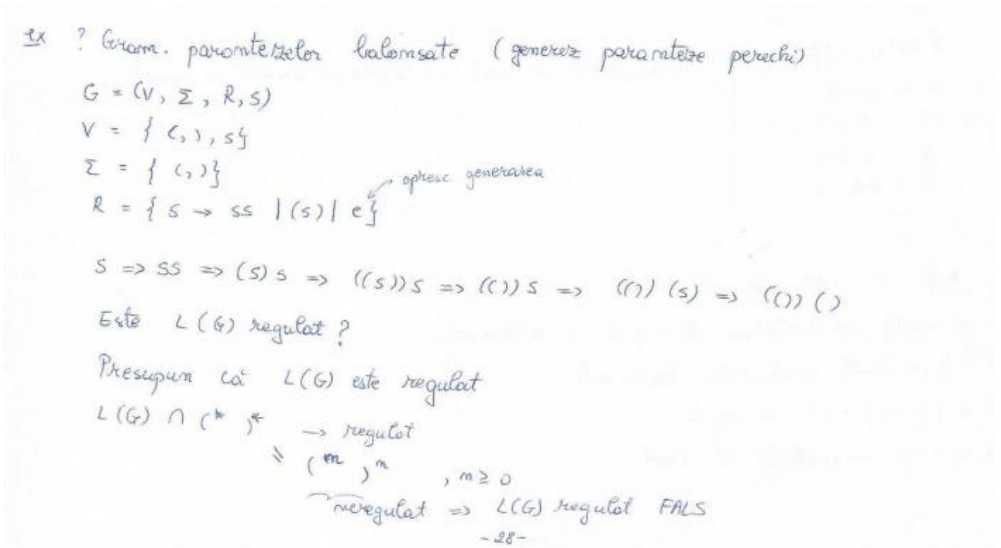
Def: O GIC este un tuplu $G = (V, \Sigma, R, S)$ a.i.

- V → mult. de simboluri terminale și neterminale
- $\Sigma \subseteq V$ → mult. simbolurilor terminale
- $R \subseteq (V - \Sigma) \times V^*$ → reguli
- $S \in V - \Sigma$ → simbolul de start

Notatii, ex G = (v, r, e)

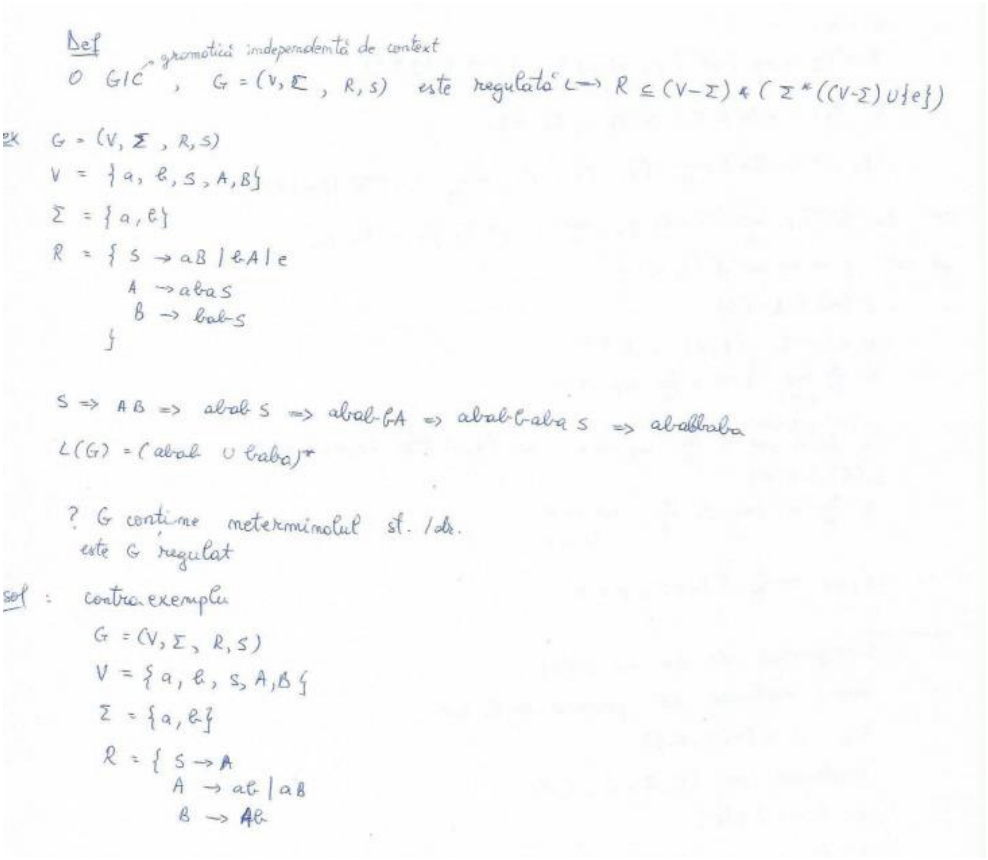


Gram paranteze lansate (paranteze perechi)



Gramatici regulate

O gic este regulata (exemplu, contraexemplu)



Def
O GIC (gramatică independentă de context), $G = (V, \Sigma, R, S)$ este regulată $\Leftrightarrow R \subseteq (V-\Sigma) \times (\Sigma^* ((V-\Sigma) \cup \{\varepsilon\}))$

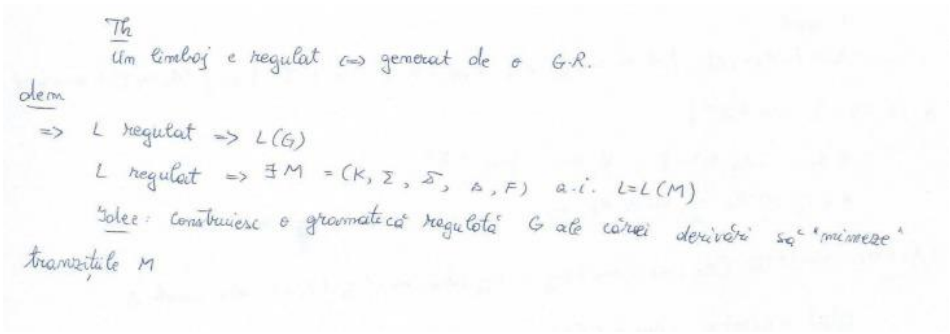
ex $G = (V, \Sigma, R, S)$
$V = \{a, b, S, A, B\}$
$\Sigma = \{a, b\}$
$R = \{ S \to aB \mid bA \mid \varepsilon$
$A \to abaS$
$B \to babS$
$\}$

$S \Rightarrow AB \Rightarrow ababS \Rightarrow ababbA \Rightarrow ababbaba S \Rightarrow ababbaba$
$L(G) = (abab \cup baba)^*$

? G conține neterminalul st. / dr.
este G regulat

sol : contra-exemplu
$G = (V, \Sigma, R, S)$
$V = \{a, b, S, A, B\}$
$\Sigma = \{a, b\}$
$R = \{ S \to A$
$A \to ab \mid aB$
$B \to Ab$

Teorema : un limbaj este regulat ⇔ generat de o G.R(dem)



Th
Un limbaj e regulat $\Leftrightarrow$ generat de o G.R.

dem
$\Rightarrow$ L regulat $\Rightarrow L(G)$
L regulat $\Rightarrow \exists M = (K, \Sigma, \delta, s, F)$ a.î. $L = L(M)$
Idee: Construiesc o gramatică regulată G ale cărei derivări să "mimeze" tranzițiile M

$G = (V, \Sigma, R, S)$

$V = K \cup \Sigma$

$S = s$

$R = \{q \to ap \mid \delta(q, a) = p\} \cup \{q \to e \mid q \in F\}$

$\forall p_0, \dots, p_m \in K, \forall \sigma_1, \dots, \sigma_m \in \Sigma$

$(p_0, \sigma_1 \dots \sigma_m) \vdash_M (p_1, \sigma_2 \dots \sigma_m) \vdash_M \dots \vdash_M (p_m, e)$

$\Leftrightarrow p_0 \Rightarrow_G \sigma_1 p_1 \Rightarrow_G \sigma_1 \sigma_2 p_2 \Rightarrow_G \dots \Rightarrow_G \sigma_1 \sigma_2 \dots \sigma_m p_m$

pt. că $p \to aq \Leftrightarrow \delta(p, a) = q$

$L(M) \subseteq L(G)$

$(s, w) \vdash_M^* (p, e)$, $p \in F$

$s \Rightarrow_G^* wp \mid \Rightarrow s \Rightarrow_G^* wp \Leftrightarrow w$
$p \to e$

$L(G) \subseteq L(M)$

$s \Rightarrow_G^* w \Leftrightarrow s \Rightarrow_G^* wp \Rightarrow w \Rightarrow (s, w) \vdash_M^* (p, e)$, $p \in F$
$\uparrow p \to e$

$L(G) \subseteq L(M)$

$s \Rightarrow_G^* w \Leftrightarrow s \Rightarrow_G^* wp \Rightarrow w$
$\uparrow p \to e$

$\Rightarrow (s, w) \vdash_M^* (p, e)$, $p \in F$

L → generat de $G_R \Rightarrow L(M)$

Idee: construiesc AF pornind de la G_R

Fie $G = (V, \Sigma, R, S)$

Construiesc $M = (K, \Sigma, \Delta, s, F)$

$K = (V - \Sigma) \cup \{f\}$

$s = S$

$F = \{f\}$

$\Delta = \{(A, w, B) \mid A \to wB \in R, A, B \in V - \Sigma, w \in \Sigma^*\} \cup \{(A, w, f) \mid A \to w \in R, A, B \in V - \Sigma, w \in \Sigma^*\}$

$\forall A_1, \dots, A_m \in V - \Sigma, \forall w_1, \dots, w_m \in \Sigma^*$

$A_1 \Rightarrow_G w_1 A_2 \Rightarrow_G w_1 w_2 A_3 \Rightarrow_G \dots \Rightarrow_G w_1 \dots w_{m-1} A_m \Rightarrow_G w_1 \dots w_m \Leftrightarrow$

$(A_1, w_1 \dots w_m) \vdash_M (A_2, w_2 \dots w_m) \vdash_M \dots \vdash_M (A_m, w_m) \vdash_M (f, e)$ doar cuvinte Δ

$L(G) \subseteq L(M)$ și $L(M) \subseteq L(G)$

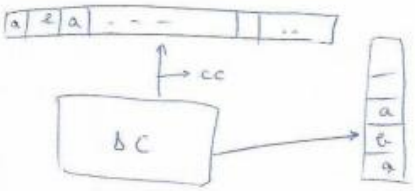
$S \Rightarrow^* w \Leftrightarrow (s, w) \vdash_M^* (f, e)$ și $(s, w) \vdash_M^* (f, e) \Leftrightarrow S \Rightarrow_G^* w$

$L(AFD) = L(AFN) = L(ER) = L(GR)$

Automate cu stiva (push-down)
Def: un Apd este un tuplu M = (k, …)
Multimea, alfabetul, starilor finale, rel de tranzitie, submultim finite

Automate cu Stivă (push-down)

$L = \{w \in \Sigma^* \mid w = w^R\}$



Def

Un APD este un tuplu $M = (K, \Sigma, \Gamma, \Delta, s, F)$

K → mulțimea finită a stărilor
Σ → alfabetul de intrare
Γ → alfabetul stivei
$s \in K$ → st. inițială
$F \in K$ → mult. stărilor finale
$\Delta \subseteq ((K \times \Sigma^* \times \Gamma^*) \times (K \times \Gamma^*))$ → rel. de tranziție
(submult. finită)

$((p, \beta), (q, \delta, \gamma))$

$((p, u, e), (q, a))$ → pune a pe stivă
$((p, u, a), (q, e))$ → scoate a de pe stivă

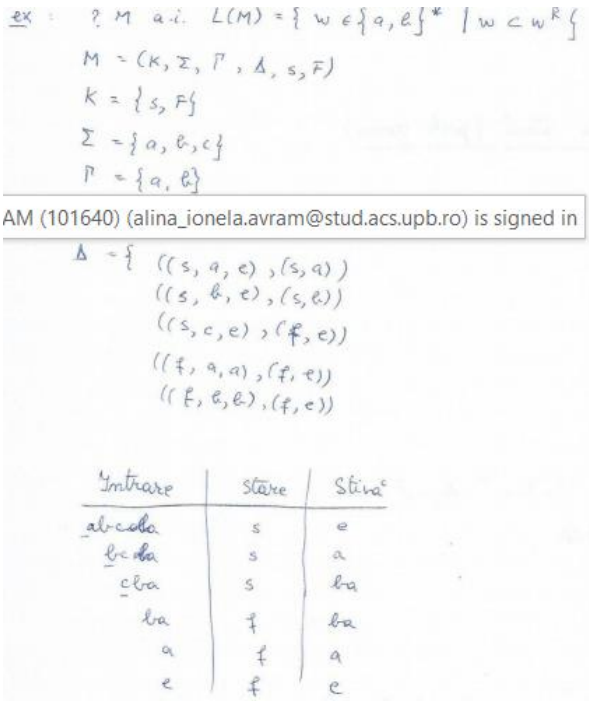
Config unui apd este un element din
Un sir w este acceptat de un apd
Un AF este un Apd care nu lucreaza cu stiva

Def
Config. unui APD este un element din $K \times \Sigma^* \times \Gamma^*$

Def
Un șir w este acceptat de un APD, M, $(s, w, e) \vdash_M^* (f, e, e)$, $f \in F$

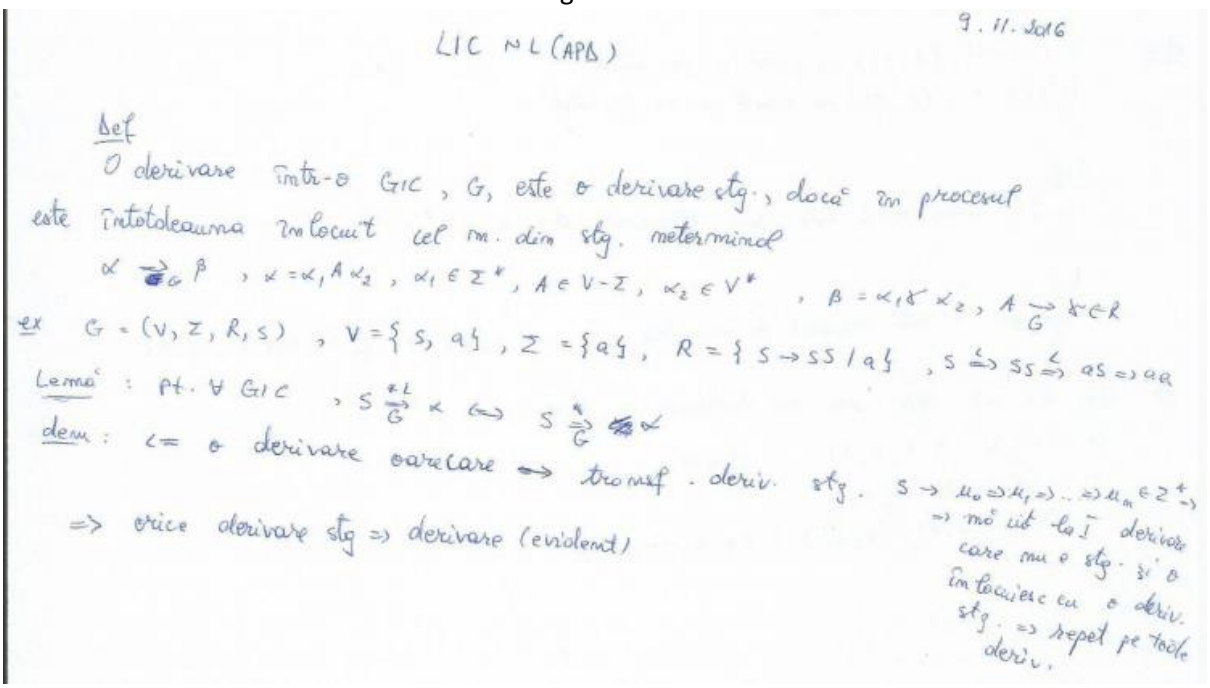
ex: Un AF este APD care nu lucrează cu stiva
$M = (K, \Sigma, \Gamma, \Delta, s, F)$, $\Gamma = \emptyset$
$M' = (K, \Sigma, \Delta', s, F)$
$\Delta = \{((p, u, e), (q, e)) \mid (p, u, q) \in \Delta'\}$

M astfel incat L(M) = {w apartine …w inclus ..} intrare stare stiva



Lic ~ L(APD)

Def: o derivare intr-o GIC …G este o derivare stg



Th: Clasa LIC este clasa L(APD)

Demonstratie, lema: oricare lic este acceptat de un apd

Th
Clasa LIC este clasa L(APΔ)

demo :

Lemă
∀ LIC este acceptat de un APΔ

demo :

Fie $G = (V, \Sigma, R, S)$ o GIC

Vreau să constr. $M = (K, \Sigma, \Gamma, \Delta, s, F)$ a.i. $L(M) = L(G)$

M are 2 stări p, q

$M = (\{p,q\}, \Sigma, V, \Delta, p, \{q\})$

$\Delta =$ 1° $((p, e, e)(q, S))$

2° $((q, e, A), (q, x))$, $A \xrightarrow{G} x \in R$

3° $((q, a, a)(q, e))$, $a \in \Sigma$

ex : $G = (V, \Sigma, R, S)$

$V = \{S, a, b, c\}$

$\Sigma = \{a, b, c\}$

$R = \{S \to aSa \mid bSb \mid c\}$, $L(G) = \{wcw^R \mid w \in \{a,b\}^*\}$

? M a.i. $L(M) = L(G)$

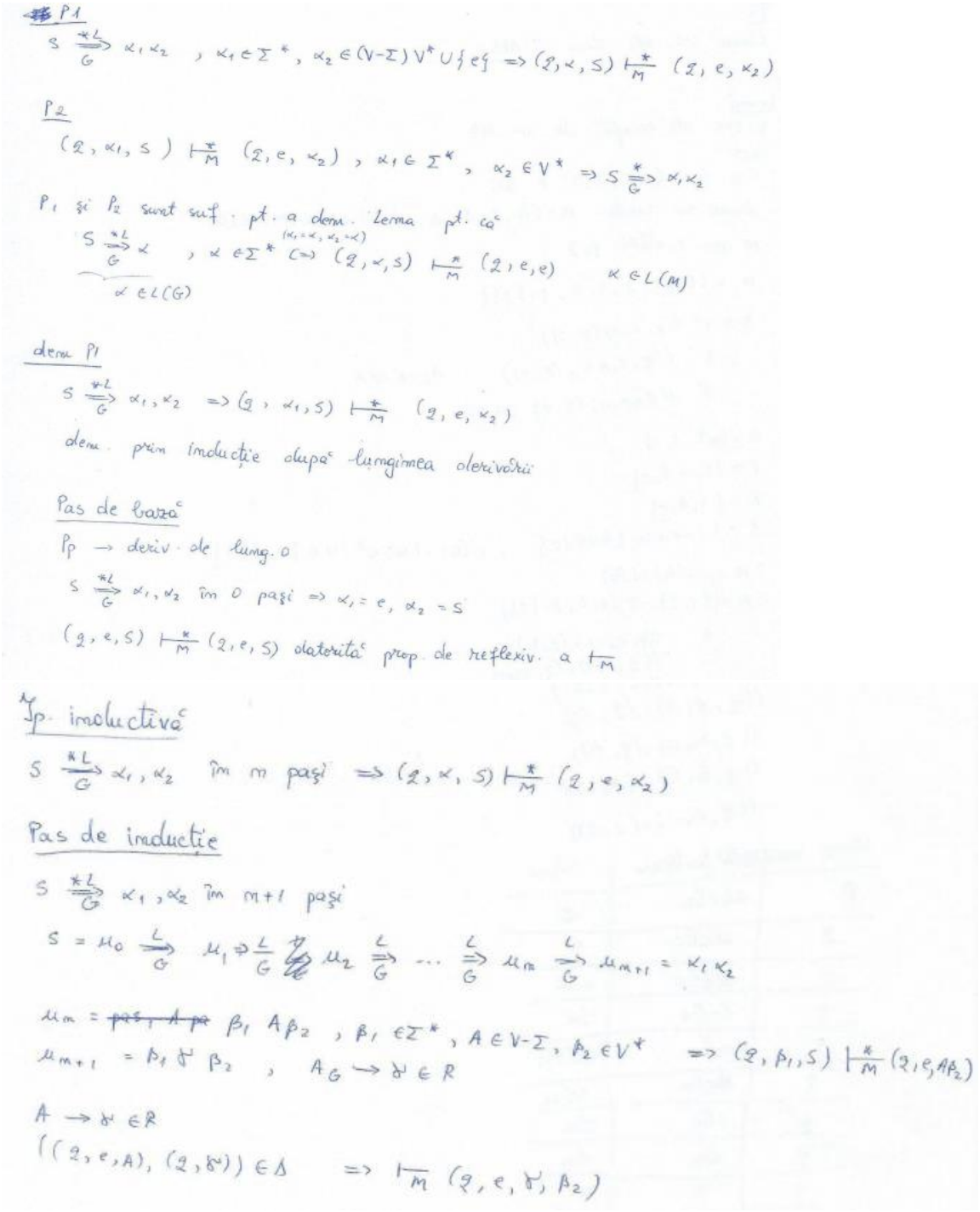
$M = (\{p, q\}, \Sigma, V, \Delta, p, \{q\})$

Δ : $((p, e, e), (q, S))$
$((q, e, S), (q, aSa))$
$((q, e, S), (q, bSb))$
$((q, e, S), (q, c))$
$((q, a, a), (q, e))$
$((q, b, b), (q, e))$
$((q, c, c), (q, e))$

Stare ~~intrare~~	Intrare	Stivă
p	abcba	e
q	abcba	S
q	abcba	aSa
q	bcba	Sa
q	bcba	bSba
q	~~b~~cba	Sba
q	cba	cba
q	ba	ba
q	a	a
q	e	e

P1 si P2 pentru a demonstra lema

Dem prin inductie dupa lungimea derivarii



$\beta_1 \gamma \beta_2 = \alpha_1 \alpha_2$

$|\alpha_1| \geq |\beta_1|$

$|\alpha_2| \leq |\gamma \beta_2|$

$\alpha_1 = \beta_1 \delta$

$\delta \alpha_2 = \gamma \beta_2$

$(q, \alpha_1, s) = (q, \beta_1 \delta, s) \vdash_M^* (q, \delta, A\beta_2) \vdash_M (q, \delta, \gamma\beta_2) \vdash_M^* (q, \delta, \delta\alpha_2) \vdash_M^* (q, e, \alpha_2)$

bim P_2 → temă

Lema: pentru orice limbaj acceptat de un apd pot construe o gic care sa-l genereze
Automat push-down stiva
Tranzitii intermediare / stari intermediare

Lemă

Pt. orice limbaj acceptat de un APD, pot construi o GIC care să-l genereze.

dem: $M = (K, \Sigma, \Gamma, \Delta, s, F)$ → APD simplu

1°. $((q, u, \beta), (p, \gamma))$, $|\beta| \leq 1$

2°. $((q, u, e), (p, \gamma)) \Rightarrow ((q, u, A), (q, \gamma A))$ | nu lucrează cu stiva ≡ pop, push

APD => transf. APDs (automat pop/push stivă)

$M = (K, \Sigma, P, \Delta, s, F)$

1° transf. $((q, u, \beta), (p, \gamma))$ pt. $|\beta| > 1$ transf. $\beta = B_1 \cdot B_m$

adaug tranziții interm. și stări intermediare

$(q, e, B_1), (t_1, e))$

$((t_1, e, B_2), (t_2, e))$

$((t_{m-1}, u, B_m), (p, \gamma)$ | → $e \in \Delta$, $t_1, \dots, t_{m-1} \in K$

pt. toate tranzițiile $|\beta| > 1$

2° $((q, u, e), (\beta, \gamma))$, înloc. $((q, u, A), (\beta, \gamma A))$

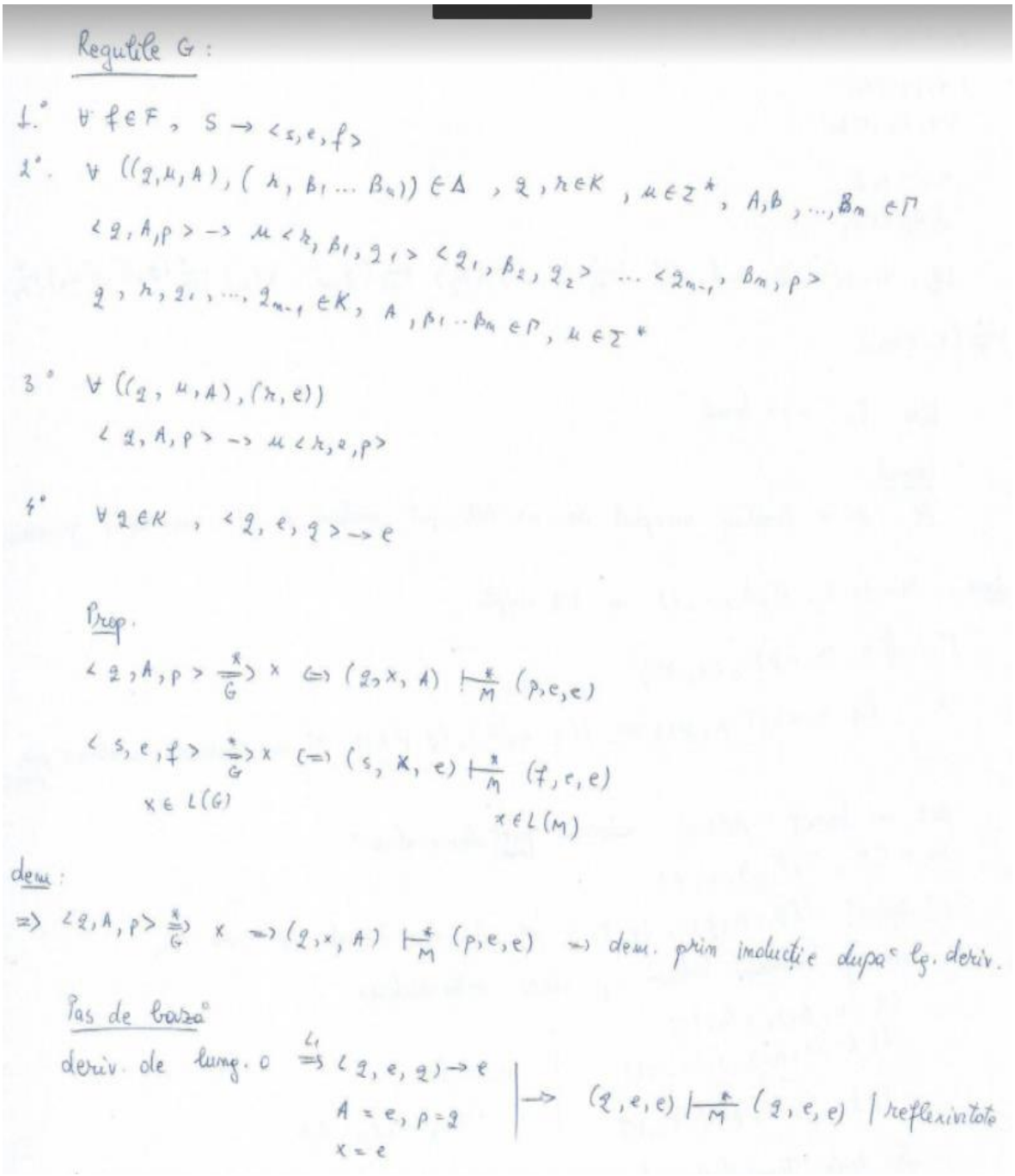
Fie M → APDs. Constr. G a.i. $L(M) = L(G)$

$G = (V, \Sigma, R, S)$

G conține pe lângă S și $\forall \in \Sigma$ neterm. $\langle q, A, p \rangle$ → șirul care e prelucrat de M între mom. de timp în care se află în starea q cu A pe stivă și ajunge în P, scoțând A de pe stivă.

$\langle q, e, p \rangle$ → gen. șirul care este prelucrat de M între mom. de timp în care se află în stările q și p, lăsând stiva nemodif.

Regulile G, pas de baza, ipoteza inductive, pas de inductie



Regulile G:

1° $\forall f \in F,\ S \to \langle s, e, f \rangle$

2° $\forall ((q, u, A), (r, B_1 \dots B_n)) \in \Delta,\ q, r \in K,\ u \in \Sigma^*,\ A, B, \dots, B_n \in \Gamma$

$\langle q, A, p \rangle \to u \langle r, B_1, q_1 \rangle \langle q_1, B_2, q_2 \rangle \dots \langle q_{n-1}, B_n, p \rangle$

$q, r, q_1, \dots, q_{n-1} \in K,\ A, B_1 \dots B_n \in \Gamma,\ u \in \Sigma^*$

3° $\forall ((q, u, A), (r, e))$

$\langle q, A, p \rangle \to u \langle r, e, p \rangle$

4° $\forall q \in K,\ \langle q, e, q \rangle \to e$

Prop.

$\langle q, A, p \rangle \overset{*}{\underset{G}{\Rightarrow}} x \iff (q, x, A) \overset{*}{\underset{M}{\vdash}} (p, e, e)$

$\langle s, e, f \rangle \overset{*}{\underset{G}{\Rightarrow}} x \iff (s, x, e) \overset{*}{\underset{M}{\vdash}} (f, e, e)$

$x \in L(G)$ $\qquad$ $x \in L(M)$

dem:

$\Rightarrow$ $\langle q, A, p \rangle \overset{*}{\underset{G}{\Rightarrow}} x \to (q, x, A) \overset{*}{\underset{M}{\vdash}} (p, e, e)$ → dem. prin inductie dupa lg. deriv.

Pas de baza°

deriv. de lung. 0 $\overset{L_1}{\Rightarrow} \langle q, e, q \rangle \to e$, $A = e$, $p = q$, $x = e$ $\to (q, e, e) \overset{*}{\underset{M}{\vdash}} (q, e, e)$ | reflexivitate

Pas de inductie

$\langle q, A, p \rangle \overset{*}{\underset{G}{\Rightarrow}} x$ în $k+1$ pași

Primul pas $< \begin{matrix} R_2 \\ R_3 \end{matrix}$

R2

$\langle q, A, p \rangle \overset{R_2}{\Rightarrow} u \, t \, \langle r, B_1, q_1 \rangle \langle q_1, B_2, q_2 \rangle \dots \langle q_{m-1}, B_m, p \rangle \overset{*}{\underset{G}{\Rightarrow}} x$

$\langle q_{i-1}, B_i, q_i \rangle \overset{*}{\underset{G}{\Rightarrow}} z_i$ în cel mult k pași $\overset{ip.ind.}{\Rightarrow}$

$(q_{i-1}, z_i, B_i) \vdash_M^* (q_i, e, e)$

$(q, u z \dots z_m, A) \vdash_M^* (r, z \dots z_m, B \dots B_m) \vdash_M^* (q_1, z_2 \dots z_m, b_2 \dots b_m) \vdash_M^*$

$\vdash_M^* (q_{m-1}, z_m, B_m) \vdash_M^* (p, e, e)$

R3

$\langle q, A, p \rangle \overset{R_3}{\underset{G}{\Rightarrow}} u \langle r, e, p \rangle \overset{+}{\underset{G}{\Rightarrow}} x$

Ψ z în cel mult k pași $\overset{ip.ind.}{\Rightarrow}$

$(q, u z, A) \vdash_M^* (r, z, e) \vdash_M^* (p, e, e)$

Proprietati de inchidere ale LIC

Reuniune/concatenare/ Kleene

Prop. de închidere ale LIC

Th

LIC sunt închise în rap. cu operațiile de:

a) reuniune
b) concatenare
c) kleene star

dem: a) $G_1 = (V_1, \Sigma_1, R_1, S_1)$

$G_2 = (V_2, \Sigma_2, R_2, S_2)$

$(V_1 - \Sigma_1) \cap (V_2 - \Sigma_2) = \emptyset$

$G = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, R_1 \cup R_2 \cup \{S \to S_1 \mid S_2\}, S)$

$S \overset{*}{\underset{G}{\Rightarrow}} w$, $w \in (\Sigma_1 \cup \Sigma_2)^* \iff S_1 \overset{*}{\underset{G}{\Rightarrow}} w$ sau $S_2 \overset{*}{\underset{G_2}{\Rightarrow}} w$

b) $G_1 = (V_1, \Sigma_1, R_1, S_1)$

$G_2 = (V_2, \Sigma_2, R_2, S_2)$

$G = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, R_1 \cup R_2 \cup \{S \to S_1 S_2\}, S)$

c) $G_1 = (V_1, \Sigma_1, R_1, S_1)$

$G = (V_1, \Sigma_1, R_1 \cup \{S_1 \to S_1 S_1 \mid e\}, S_1)$

Intersectia dintre LIC si LR este LIC

Th

Intersecția dintre LIC și LR este LIC.

dem: Fie $M_1 = (K_1, \Sigma, \Pi, \Delta_1, s_1, F_1)$

$M_2 = (K_2, \Sigma, \delta, s_2, F_2)$

Constr. M a.i. $L(M) = L(M_1) \cap L(M_2)$

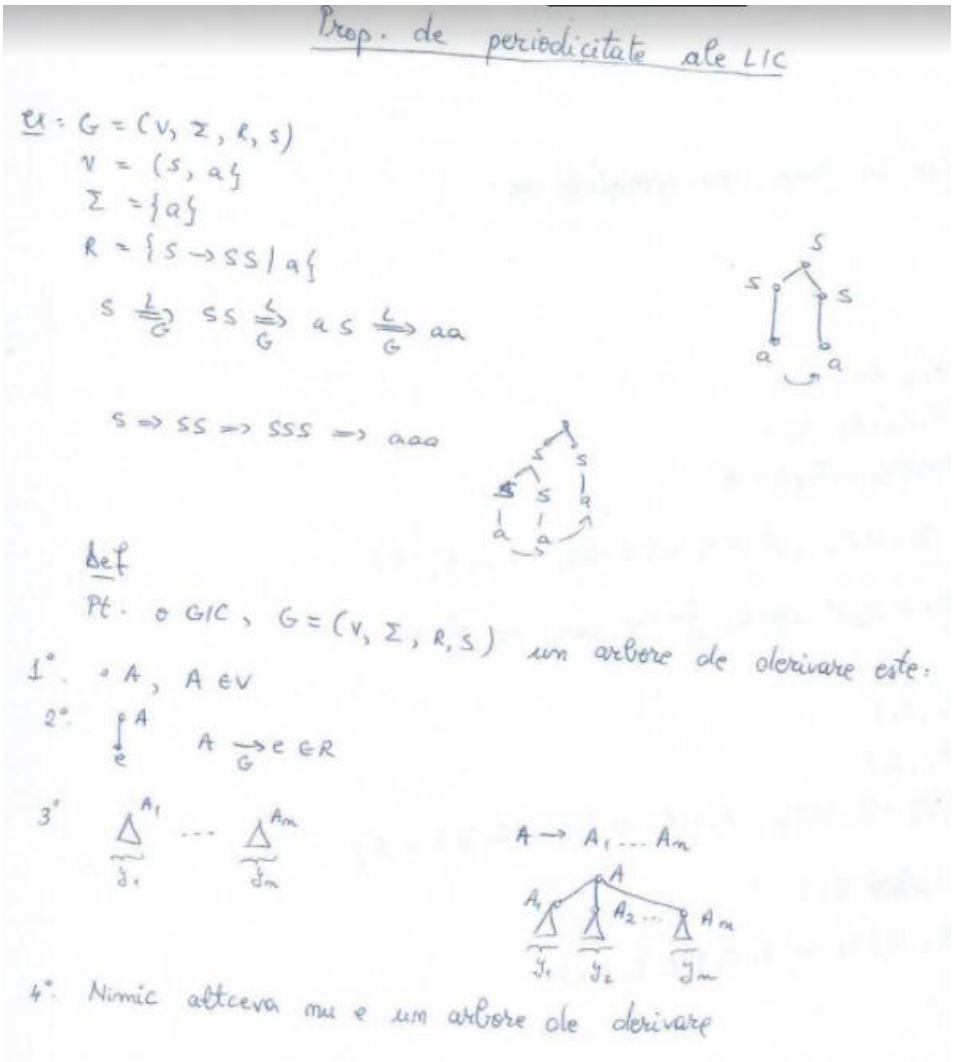
$M = (K_1 \times K_2, \Sigma, \Pi, \Delta, (s_1, s_2), F_1 \times F_2)$

$\Delta = (((q_1, q_2), u, \gamma), ((p_1, p_2), \beta)$

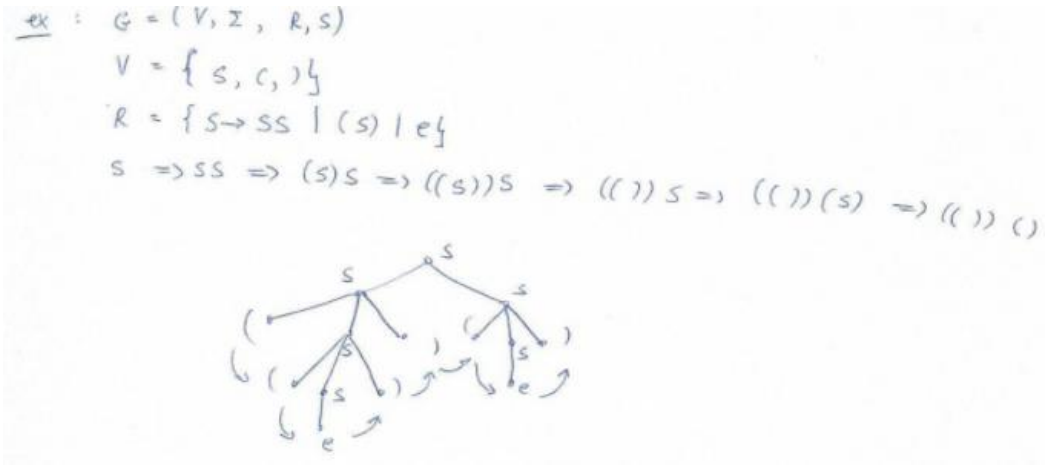
$((q_1, u, \gamma), (p_1, \beta)) \in \Delta_1$

$(q_2, u) \overset{*}{\underset{M}{\vdash}} (p_2, e)$

Proprietatea de peridiocitate ale LIC
Un arbore de derivare este



Ex ss



Lema de pompare
Constanta K dependenta
Un arbore de inaltime m are cel mult frunze
Se reduce

Th (Lema de pompare)

Fie $G \to$ GIC. At. $\exists$ o constanta K dependenta de G a.i. $\forall w \in L(G)$, $|w| > K$, $w = uvxyz$, $v \neq e$ sau $y \neq e$ și $uv^n xy^n \in L(G)$, $\forall n \geq 0$

dem

Fie $G = (V, \Sigma, R, S)$, $\exists K(G)$ a.i. $\forall w \in L(G)$ poate fi obț. printr-o derivare

$S \xrightarrow{*} uAz \xrightarrow{*} uvAyz \xrightarrow{*} w$

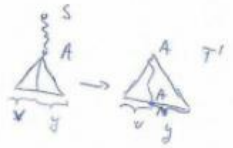
și $A \Rightarrow vAy$ poate fi repetata de ori cate ori

Notăm $p = \max\{|\alpha| \mid A \to \alpha \in R\}$

Un arb. de $\geq$ înălțime m are cel mult p^m frunze

$m = |V - \Sigma|$, $K = p^m$

Fie $w \in L(G)$, $|w| > K$



Nu se poate ca $v = e$ și $y = e$ pt. O alegere a căii.

$\Rightarrow$ arb. T' se reduce la o sg. cale de $\geq$ neterminale $\Rightarrow$ imposibil $|w| > K$

Limbajul independent context

Teorema

Limbajul $\{a^n b^n c^n \mid n \geq 0\}$ nu este ind. de context.

dem: Pres. $L \to$ LIC $\xrightarrow{\text{L. pompare}}$ $\exists K(G)$ a.i. $w \in L$, $|w| \geq K$, $w = uvxyz$, $v \neq e$ sau $y \neq e$ și $uv^i xy^i z \in L$, $\forall i \geq 0$

Fie $K \to$ const. (def. cf. LP) $\Rightarrow n > K/3$

$w = a^n b^n c^n$

v, y format ab sau bc $\to$ nu se poate $\to$ se strică forma șirurilor

v, y a/b/c $\to$ nu se poate, pt. că $u_a = u_b = u_c$, $\forall i \geq c$

$\Rightarrow L \to$ NU este LIC

Teorema

Limb. Ind. de Context nu sunt închise în rap. cu intersecția și complementarea.

dem: $L_1 = \{a^n b^n c^m \mid n, m > 0\}$

$L_2 = \{a^m b^n c^n \mid m, n > 0\}$

$L_1 \cap L_2 = \{a^n b^n c^n \mid n > 0\} \to$ ~~LIC~~

Complementare / proprietati algoritmice ale lic
Algoritmi care pot raspunde la urmatoarele intrebari l(gic)

Complementare

$L_1 \cap L_2 = \Sigma^* - ((\Sigma^* - L_1) \cup (\Sigma^* - L_2))$

(închis) (închis)

Proprietăți algoritmice ale LIC

Teoremă

Există alg. care pot răspunde la urm. întrebări referitoare la $L(G_{IC})$:

1. $w \in \Sigma^*$, $G \to G_{IC}$, $w \in L(G)$?
2. $G \to G_{IC}$, $L(G) = \emptyset$?

Obs: L(apd) automat push-down/ L(g1) = l(g2)
Deriv de lung 2 w – 1
Sirul vid /

Obs:

1. Ac. întrebări sunt valabile și pt. $L(APD)$ (APD poate fi obținut din GIC)
   ↓ autom. push-down
2. Lipsa întrebărilor de forma $L(G_1) = L(G_2)$ nu este întâmplătoare => nu pot fi sol. alg.

dem:

1. Analizez toate deriv. până la o anumită lung. / toți arb. de derivare până la o anum. h.

Fie $G = (V, \Sigma, R, S) \to G_{IC}$

Dacă (pt. w) regulile sunt numai de forma $A \to u$, $u \in \Sigma$ sau $|u| \geq 2$, atunci este suficient să inspectez deriv. de lung. $\leq 2|w| - 1$ / arb. de derivare de înălțime $|w|$

Elimin → $A \to e$, $A \to B$, $A, B \in V - \Sigma$

Dacă $L(G)$ conține șirul vid e, atunci trebuie să păstrez $S \to e$

Elim $A \to e$

$A \to e$
$B \to uAv$ | → adaug $B \to uv$, $u, v \in V^*$

Se repetă până când nu se mai pot adăuga noi reguli.
La sfârșit se elimină $A \to e$.

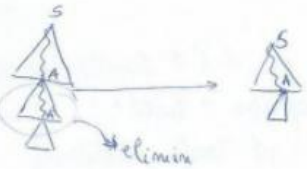
Elim. $A \to B$

$A \to B$
$B \to u$ | → adaug $A \to u$

Procesul se repetă pt. toate regulile
La sf. se elim. $A \to B$

2. $L(G) = \emptyset$?

Orice șir din $L(G)$ va avea un arbore de derivare.



=> e suficient să inspectez arb. de derivare de $h \leq |V - \Sigma|$

Tehnici de constructie a GIC

Deschideți cu

1° a^*, $a \in \Sigma$, $A \to aA \mid e$

2° $a^n b^n$, $a, b \in \Sigma$, $n \geq 0$, $X \to aXb \mid e$

3° $a^n b^{3n}$, $a, b \in \Sigma$, $n \geq 0$, $X \to aXbbb \mid e$

4° $a^m b^m$, $a, b \in \Sigma$, $m > n \geq 0$

$a^n b^n b^k$

$S \to XY$
$X \to aXb \mid e$
$Y \to bY \mid b$

5° $a^n b^{nr}$, $n \neq nr$

$L = L_1 \cup L_2$
$L_1 = \{a^n b^m \mid n > m\}$
$L_2 = \{a^n b^m \mid n < m\}$

6° $a^n b^m c^m d^n$

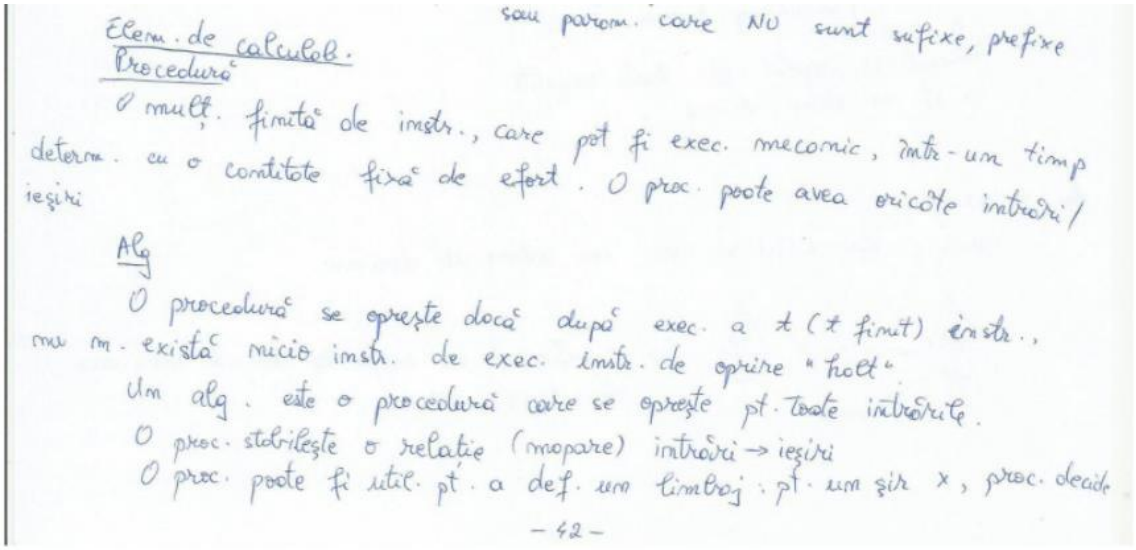
$S \to aSd \mid X$
$X \to bXc \mid e$

Nu se poate cu GIC: 1° să controlezi m. mult de 2 parom.
$a^m b^n c^m$ LIC

2° $a^n b^m c^n d^m$
sau parom. care NU sunt sufixe, prefixe

Elem. de calculab.
Procedură

Elem de calculi
Procedura/ efort/ instructiune de oprire halt / un algoritm este o procedura/ mapare/ utilizare/ recursiv enumerabila / recursiva



sau parcurs. care NU sunt sufixe, prefixe

Elem. de calculab.

Procedură

O mulț. finită de instr., care pot fi exec. mecanic, într-un timp determ. cu o cantitate fixă de efort. O proc. poate avea oricâte intrări/ ieșiri

Alg

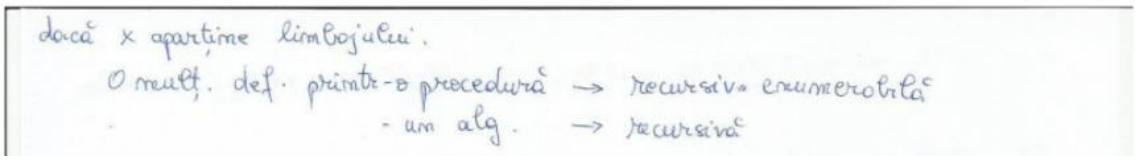
O procedură se oprește dacă după exec. a t (t finit) instr., nu mai există nicio instr. de exec. instr. de oprire "halt"

Un alg. este o procedură care se oprește pt. toate intrările.

O proc. stabilește o relație (mapare) intrări → ieșiri

O proc. poate fi utilă pt. a def. un limbaj: pt. un șir x, proc. decide

– 42 –



dacă x aparține limbajului.

O mulț. def. printr-o procedură → recursiv enumerabilă

- un alg. → recursivă

Formalizarea notiunii de procedra
M turing
Ierarhia chausky, markov, lambda/ taj/ post/ gramatici regulate/ gic/ gdc (gramatici dependente de context) / gramatici nerestrictionate

Formalizarea noțiunii de procedură

→ M. Turing (1936 - 1937)
→ Ierarhia Chomsky (1959, 1963)
→ Alg. Markov (1951)
→ Calc. Lambda (1941)
→ Sist. Tag / Post (~ 1950)

Toate formalism. sunt echiv. atât cât al calculabilității.

Ierarhia Chomsky

1) Gramatici regulate
$A \to xB \mid x$, $x \in \Sigma^*$, $A, B \in V - \Sigma$

2) GIC
$A \to \alpha$, $A \in V - \Sigma$, $\alpha \in V^*$

3) GDC (gram. dependente de context)
$\alpha A \beta \to \alpha \sigma \beta$, $|\alpha A \beta| \leq |\alpha \sigma \beta|$, $A \in V - \Sigma$, $\alpha, \beta, \sigma \in V^*$

4) Gramatici nerestricționate (GO) (= zero)
$V^*(V-\Sigma)V^* \to V^*$

Pt. fiecare tip de gramatică ∃ acceptare.

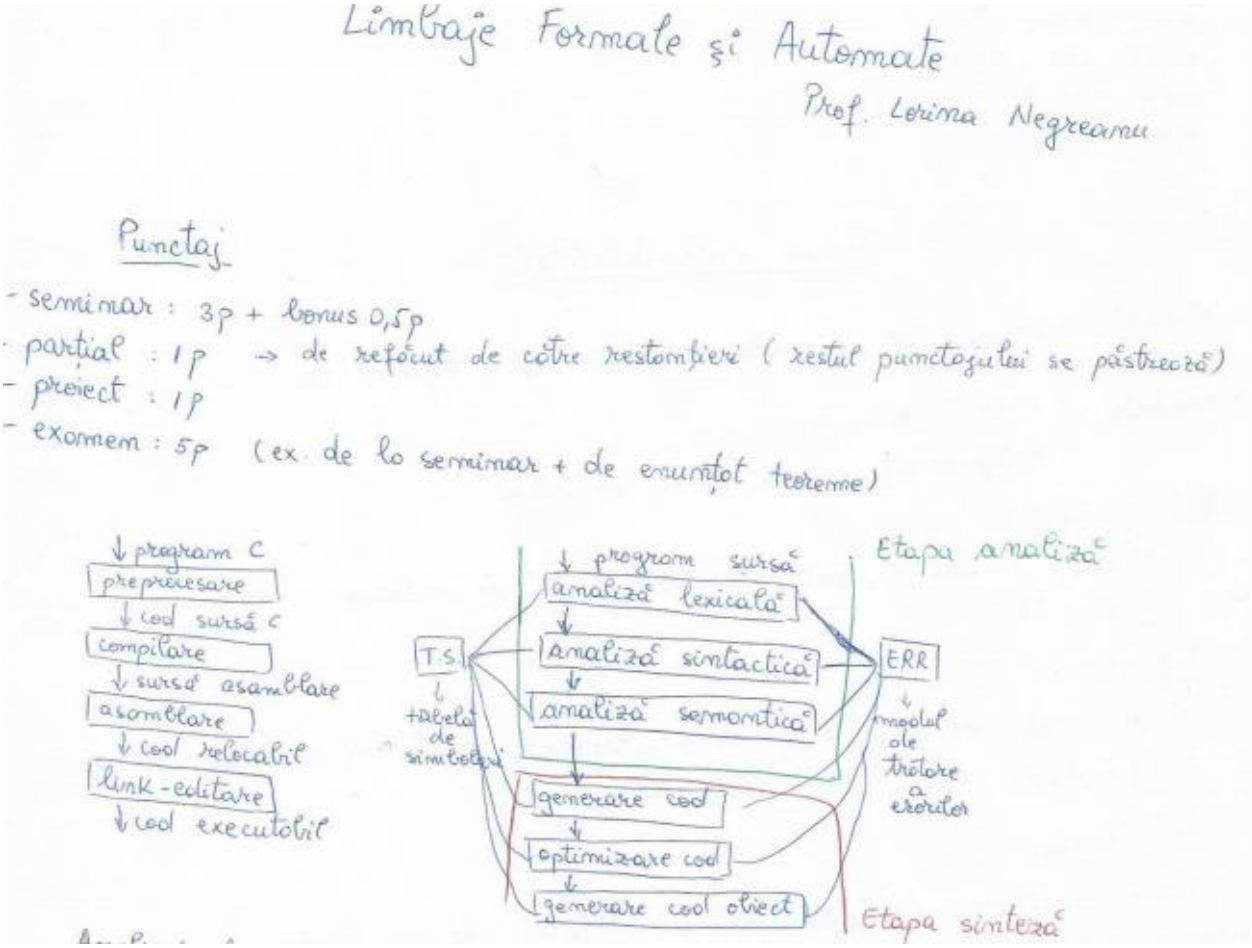
1) LR → AF
2) LIC → APD (nedeterm.)
3) LDC → 2 way nedeterm. linear bounded automatic
4) L. Nerestricționate → M. Turing

ex GDC → $L = \{ a^m b^m c^m \mid m > 0 \}$

$S \to aSBC \mid abC$
$CB \to BC$
$bB \to bb$
$cC \to cc$
$bC \to bc$

$S \Rightarrow aSBC \Rightarrow aabCBC \Rightarrow aabBCC \Rightarrow aabbCC \Rightarrow aabbcC \Rightarrow aabbcc$

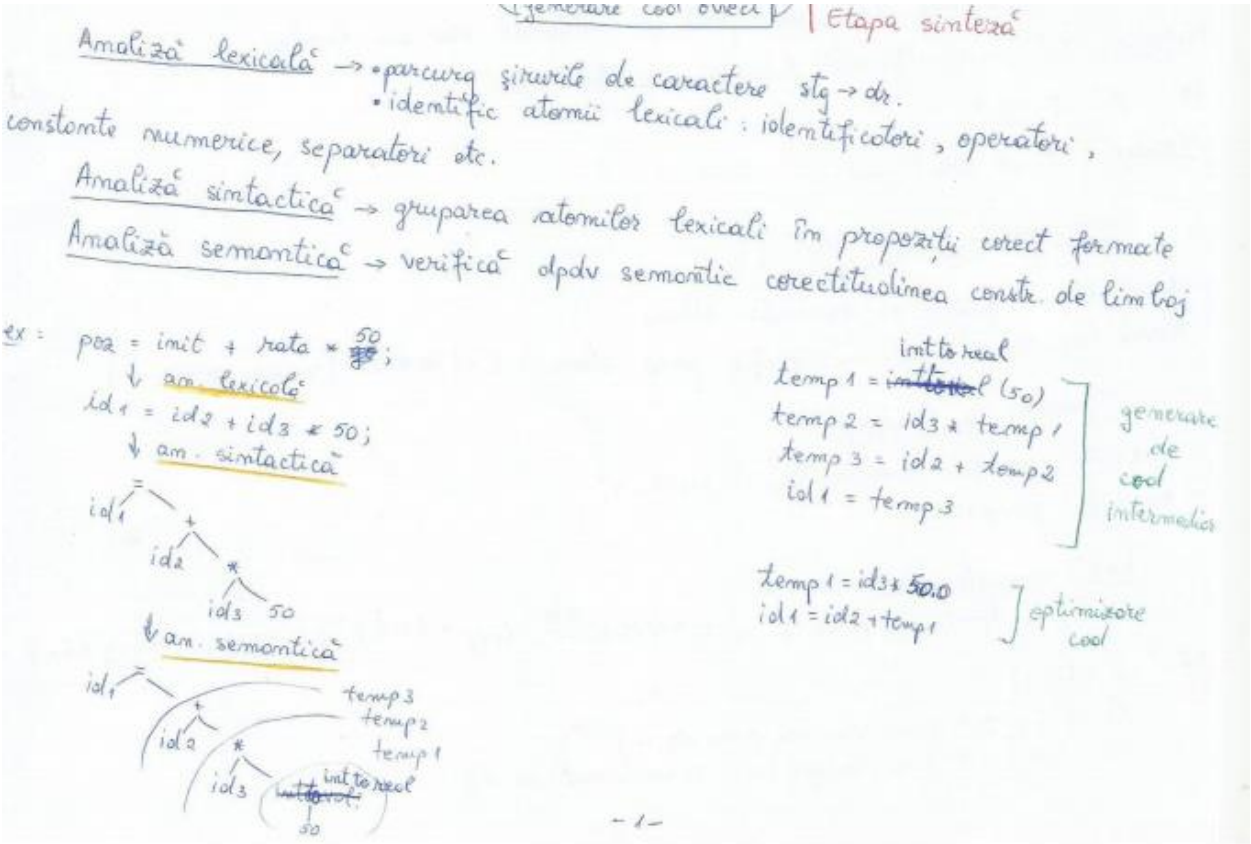
LFA CUPRINS



Analiza lexicala

Analiza sintactica

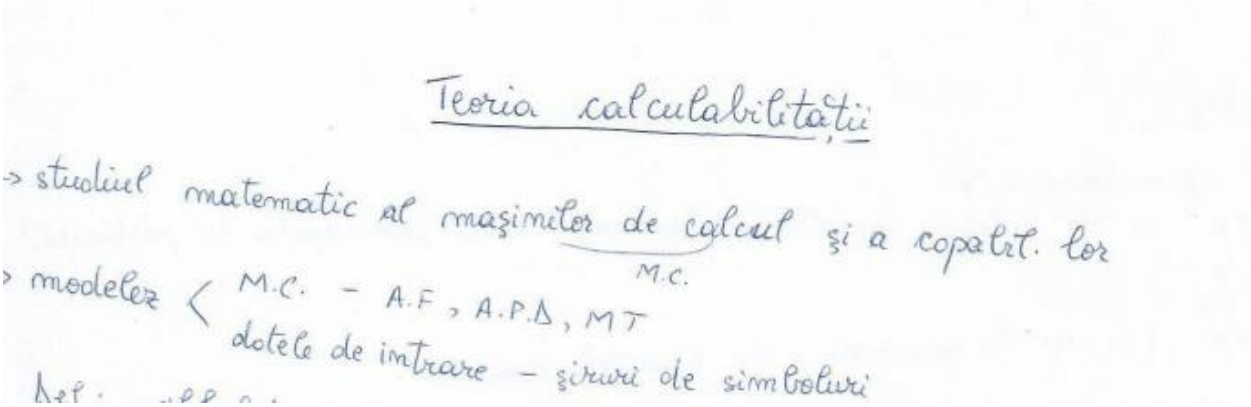
Analiza semnatica



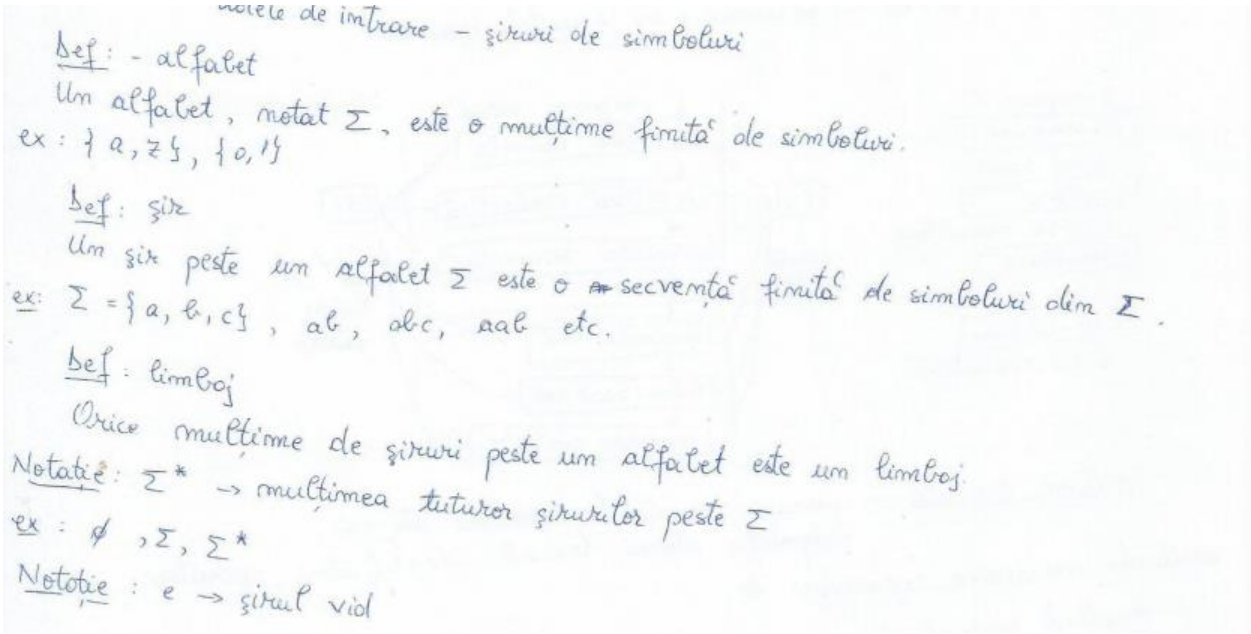

```
MOVF   id3, R1
MULF   #50.0, R1
MOVF   id2, R2
ADDF   R2, R1
MOVF   R1, id1
```

Teoria calculabilitatii

- Studiul matematic al masinilor de calcul
- Modelez -MC – AF, APD, MT
- Datele de intrare : siruri de simboluri

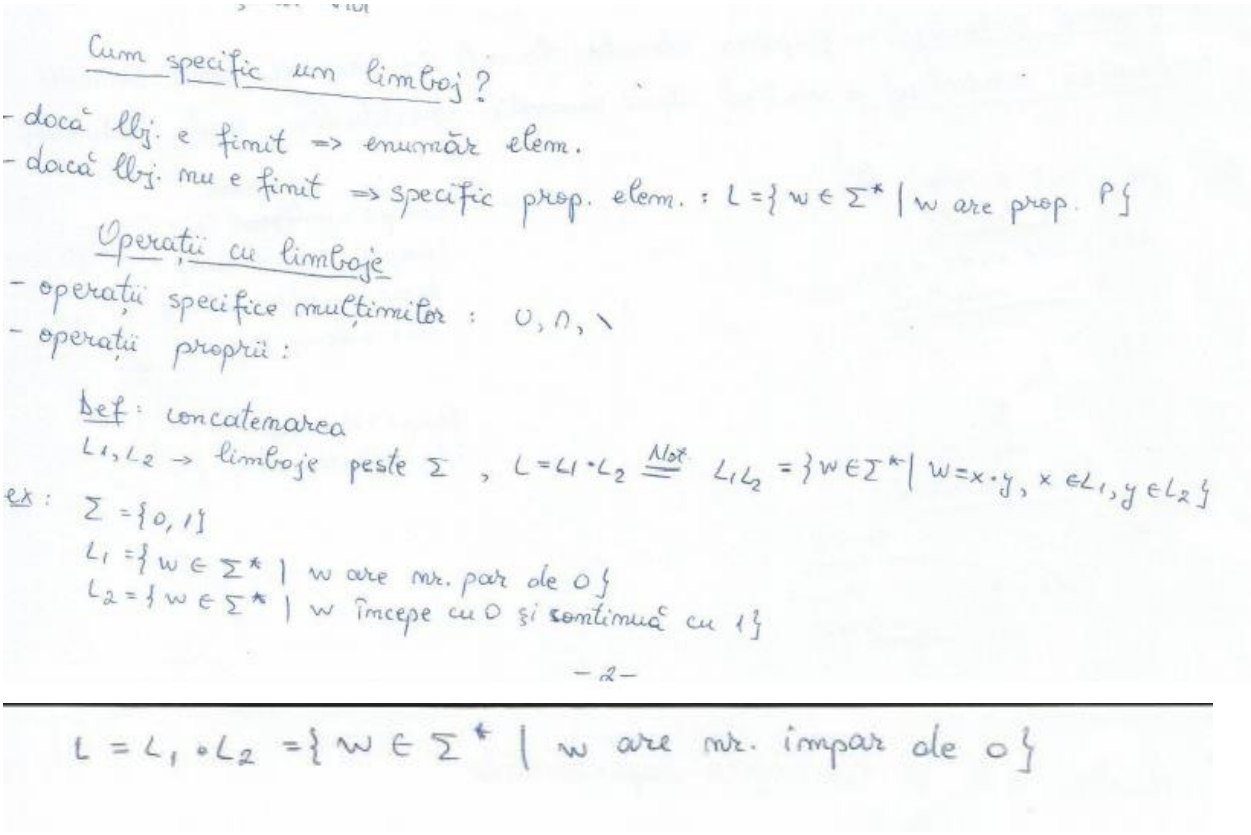


Def : alfabet, sir, limbaj



Limbaj finit/ nu e finit
Operatii cu limbaje:

- Concatenarea :



Cum specific un limbaj ?

- dacă lbj. e finit => enumăr elem.
- dacă lbj. nu e finit => specific prop. elem. : $L = \{ w \in \Sigma^* \mid w \text{ are prop. } P \}$

Operații cu limbaje

- operații specifice mulțimilor : $\cup, \cap, \setminus$
- operații proprii :

Def: concatenarea

$L_1, L_2 \to$ limbaje peste Σ, $L = L_1 \cdot L_2 \overset{\text{not}}{=\!=} L_1 L_2 = \{ w \in \Sigma^* \mid w = x \cdot y, x \in L_1, y \in L_2 \}$

ex: $\Sigma = \{0, 1\}$

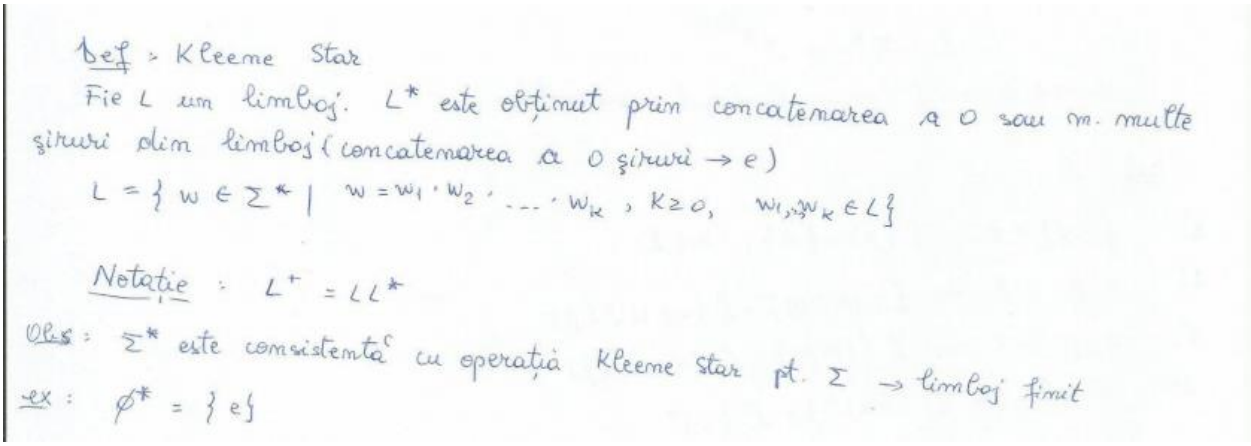
$L_1 = \{ w \in \Sigma^* \mid w \text{ are nr. par de } 0 \}$

$L_2 = \{ w \in \Sigma^* \mid w \text{ începe cu } 0 \text{ și continuă cu } 1 \}$

– 2 –

$L = L_1 \circ L_2 = \{ w \in \Sigma^* \mid w \text{ are nr. impar de } 0 \}$

Kleene Star : Fie L un limbaj. L* este obtinut prin concatenarea a 0 sau mai multe siruri din limbaj. Notatie.



Def: Kleene Star

Fie L un limbaj. L^* este obținut prin concatenarea a 0 sau nr. multe șiruri din limbaj (concatenarea a 0 șiruri $\to \varepsilon$)

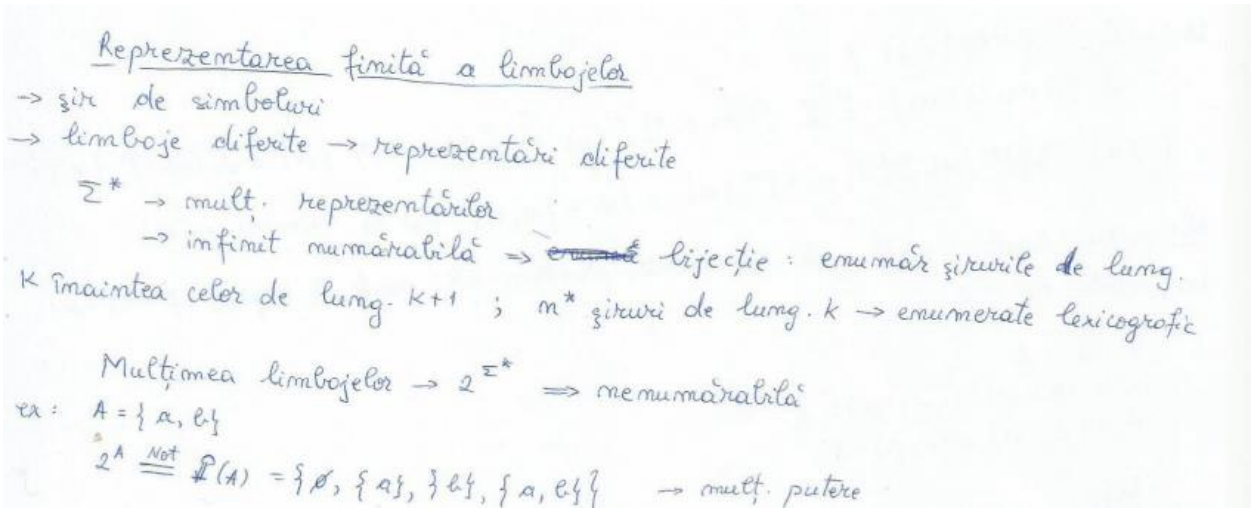
$L = \{ w \in \Sigma^* \mid w = w_1 \cdot w_2 \cdot \ldots \cdot w_k, k \geq 0, w_1, \ldots, w_k \in L \}$

Notație: $L^+ = LL^*$

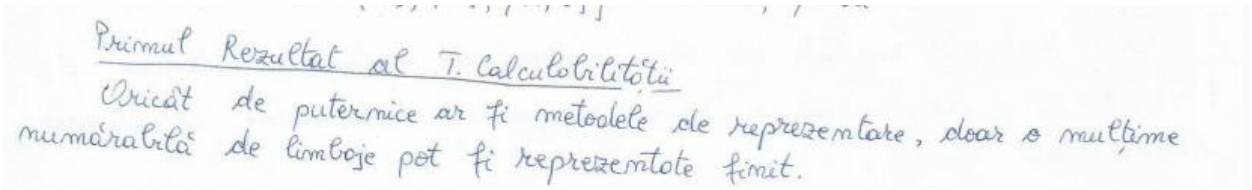
Obs: Σ^* este consistentă cu operația Kleene star pt. $\Sigma \to$ limbaj finit

ex: $\emptyset^* = \{\varepsilon\}$

Reprezentarea finita a limbajelor:
-sir de simboluri
-limbaje diferite -> reprezentari diferite
-multimea reprezentarilor
-numarabila / nenumarabila / bijectie / lexicografic

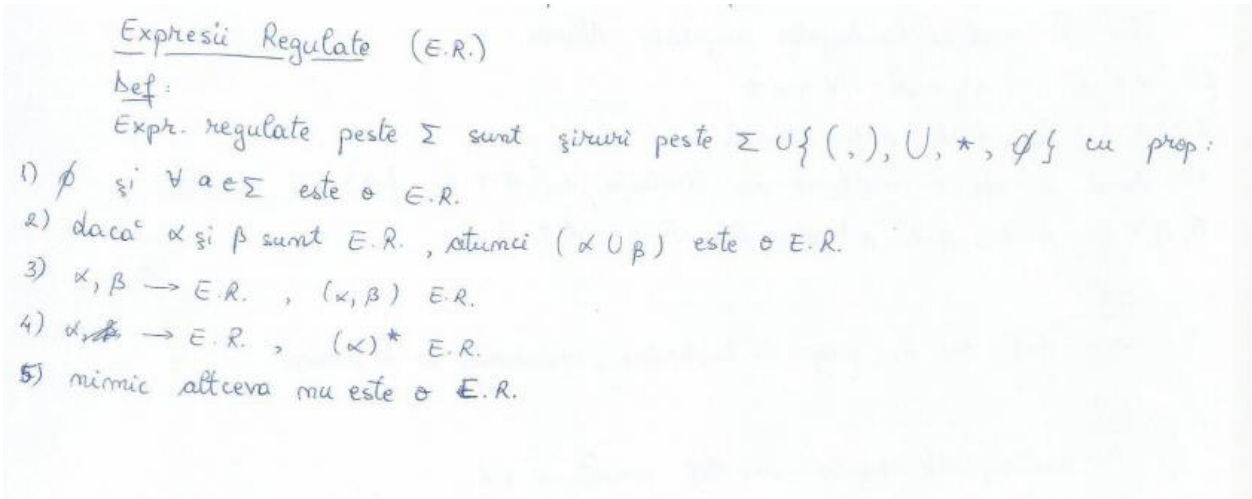


Primul rezultat al Teoremei Calculabilitatii
Oricat de puternice ar fi metodele de reprezentare, doar o multime numarabila de limbaje pot fi reprezentate finit.

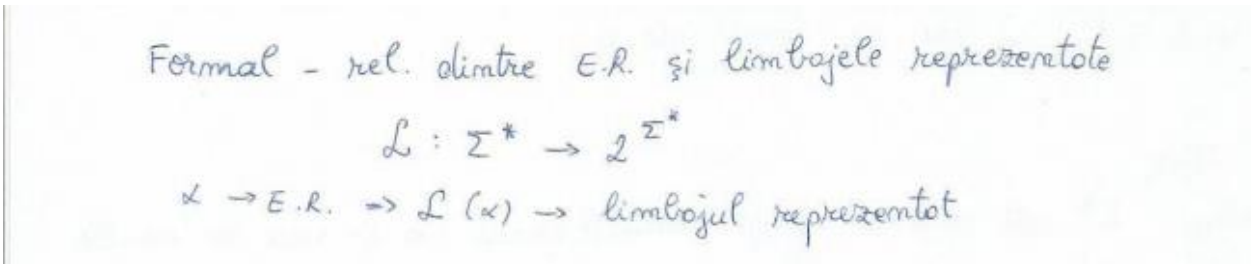


EXPRESII REGULATE

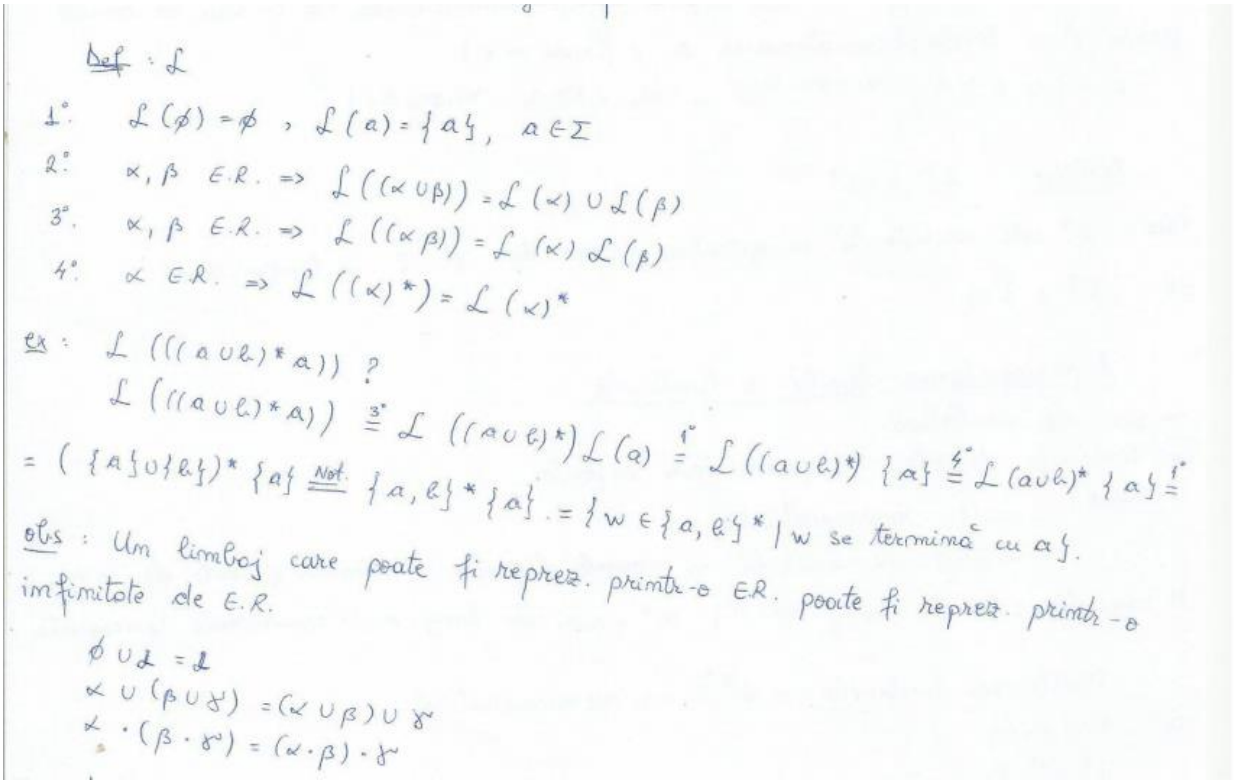
Proprietati:



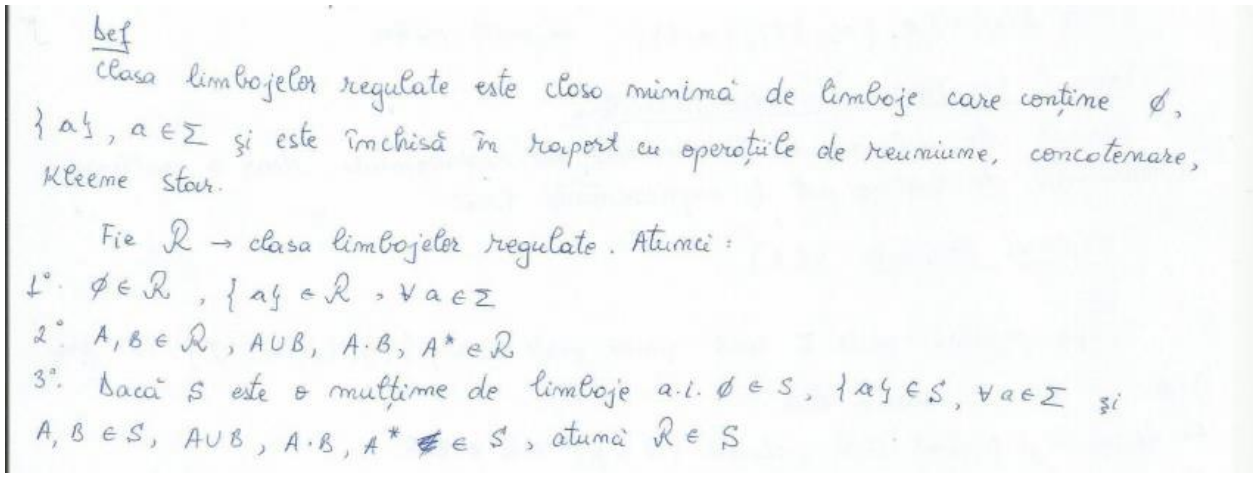
Relatia dintre expresii regulate / ER si limbaje reprezentate :



Def : expresii regulate / ER
Un limbaj care poate fi reprezentat printr-o expresie regulata, poate fi reprezentat printr-o infinitate de expresii regulate.



Def :
Clasa limbajelor regulate este clasa minima de limbaje si este inchisa in raport cu operatiile de reunine, concatenare, Kleene Star.
Clasa limbajelor regulate :



Def
Clasa limbajelor regulate este cea mai mica clasa de limbaje care contine $\emptyset$, $\{a\}$, $a \in \Sigma$ si este inchisa in raport cu operatiile de reuniune, concatenare, Kleene star.

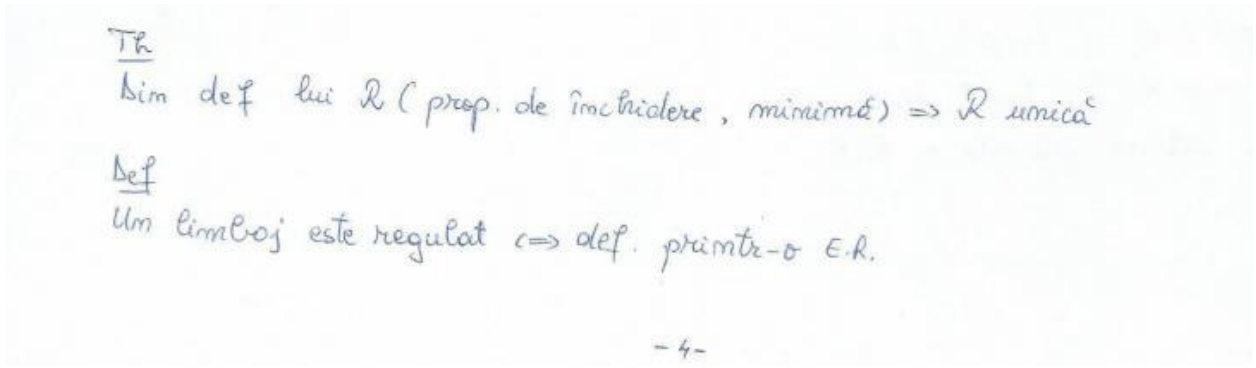
Fie $\mathcal{R}$ → clasa limbajelor regulate. Atunci:

1° $\emptyset \in \mathcal{R}$, $\{a\} \in \mathcal{R}$, $\forall a \in \Sigma$

2° $A, B \in \mathcal{R}$, $A \cup B$, $A \cdot B$, $A^* \in \mathcal{R}$

3° Daca S este o multime de limbaje a.i. $\emptyset \in S$, $\{a\} \in S$, $\forall a \in \Sigma$ si $A, B \in S$, $A \cup B$, $A \cdot B$, $A^* \in S$, atunci $\mathcal{R} \in S$

Teorema : Din definitia lui R (proprietate de inchidere , minima) => R unica
Def : Un limbaj este regulat daca si numai daca este definit printr-o expresie regulata

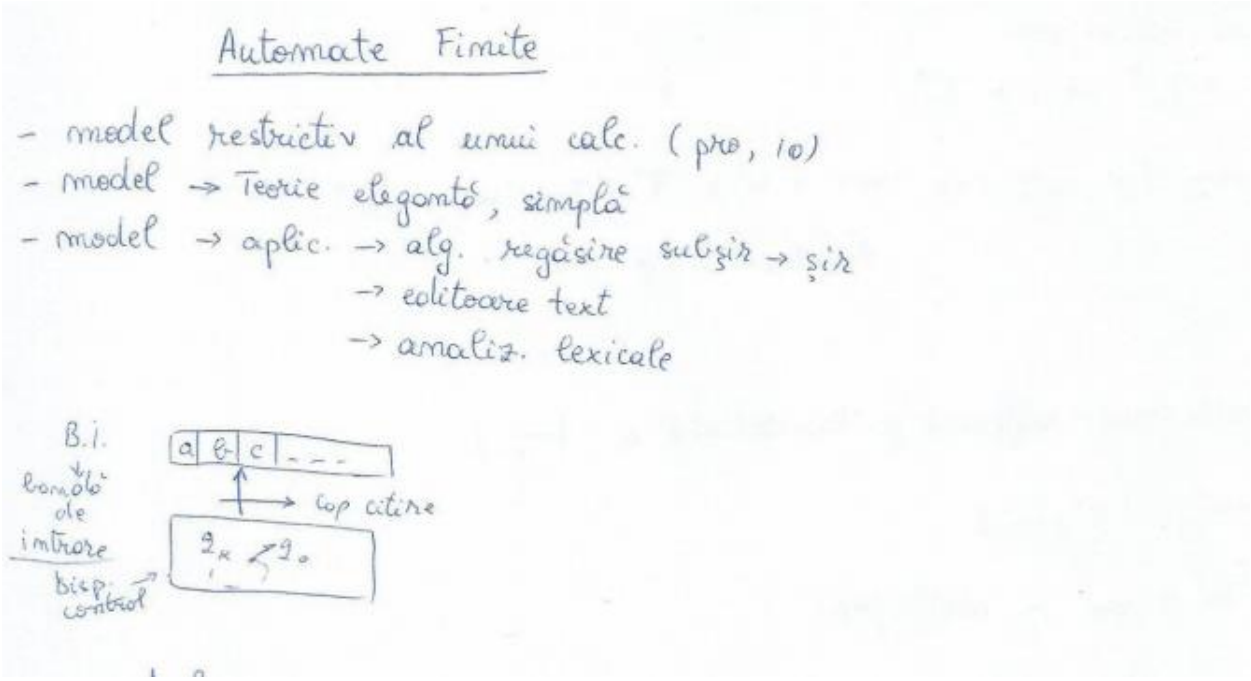


Th
Din def lui $\mathcal{R}$ (prop. de inchidere, minimă) => $\mathcal{R}$ unică

Def
Un limbaj este regulat <=> def. printr-o E.R.

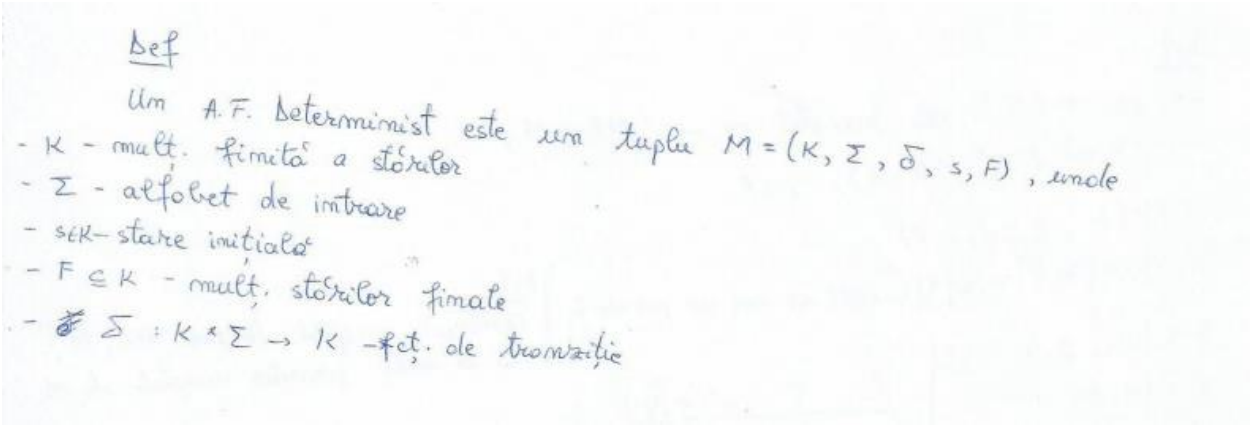
AUTOMATE FINITE :

- Model restrictiv al unui calc
- Model : teorie eleganta/simpla
- Editoare text/ analiza lexicala/ regasire sir/ subsir



Un automat determinist este un tuplu:

- Multime finita a starilor
- Alfabet de intrare
- Stare initiala
- Multimea starilor finale
- Fct de tranzitie

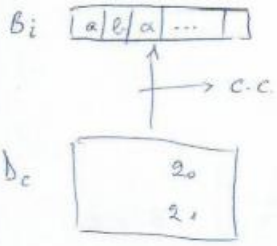


Configuratia unui AFD / Automat finit determinist :

12.10.2016

Curs 2

Automate finite deterministe



$M = (K, \Sigma, \delta, s, F)$

$K \rightarrow$ mult. finită de stări

$\Sigma \rightarrow$ alfabet

$s \in K \rightarrow$ st. inițială

$F \subseteq K \rightarrow$ mult. st. finale

$\delta : K \times \Sigma \rightarrow K$ fct. tranziție

Def:

Configurația unui AFD, $M = (K, \Sigma, \delta, s, F)$, este un elem. din $K \times \Sigma^*$

-trecerea intr-un pas

-inchiderea reflexiva

- un sir este acceptat de un AFD

Def trecere într-un pas

$\vdash_M : K \times \Sigma^* \to K \times \Sigma^*$

$(q, w) \vdash_M (q', w') \iff w = \tau w', \ \tau \in \Sigma, \ w, w' \in \Sigma^*$

$\delta(q, \tau) = q'$

Def

$\vdash_M^*$ (închiderea reflexivă și tranzitivă a $\vdash_M$)

$(q, w) \vdash_M^* (q', w')$

↑ în 0 sau m. multi pași

Def

Un șir $w \in \Sigma^*$ este acceptat de un AFD M,

$(s, w) \vdash_M^* (q, e)$, $q \in F$

ex: ? $M = (K, \Sigma, \delta, s, F)$

$L(M) = \{w \in \{a, b\}^* \mid w$ conține un nr. par de $b\}$

$K = \{q_0, q_1\}$

$\Sigma = \{a, b\}$

$s = q_0$

$F = \{q_0\}$

q	τ	$\delta(q, \tau)$
q_0	a	q_0
q_0	b	q_1
q_1	a	q_1
q_1	b	q_0

Def

Limbajul acceptat de un AFD, M, este mult. șirurilor acceptate de M.

Activati Windows

Reprezentare AFD prin diagrama de tranzitie : graf directionat :

Reprezentare AFA prin diagr. de tranziție → graf orientat
→ noduri → stări
→ arce → tranziții
→ st. init. >0
→ st. finală ◎

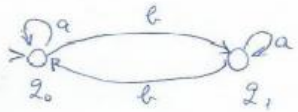


abb : $>q_0 \xrightarrow{a} q_0 \xrightarrow{b} q_1 \xrightarrow{b} q_0 \in F$

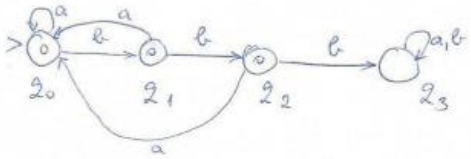
ab : $>q_0 \xrightarrow{a} q_0 \xrightarrow{b} q_1 \notin F$

→ pt. că nu ajung într-o stare finală

Ex : w nu contine 3 b consecutive : trb sa am in fiecare stare tranzitii dif pt toate simbolurile :

ex. ? $M = (K, \Sigma, \delta, s, F)$

$L_{\neq}(M) = \{ w \in \{a, b\}^* \mid w$ nu conține 3 b consecutive $\}$



→ trb. să am în fiecare stare tranziții def. pt. toate simbolurile

$K = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{a, b\}$

$s = q_0$

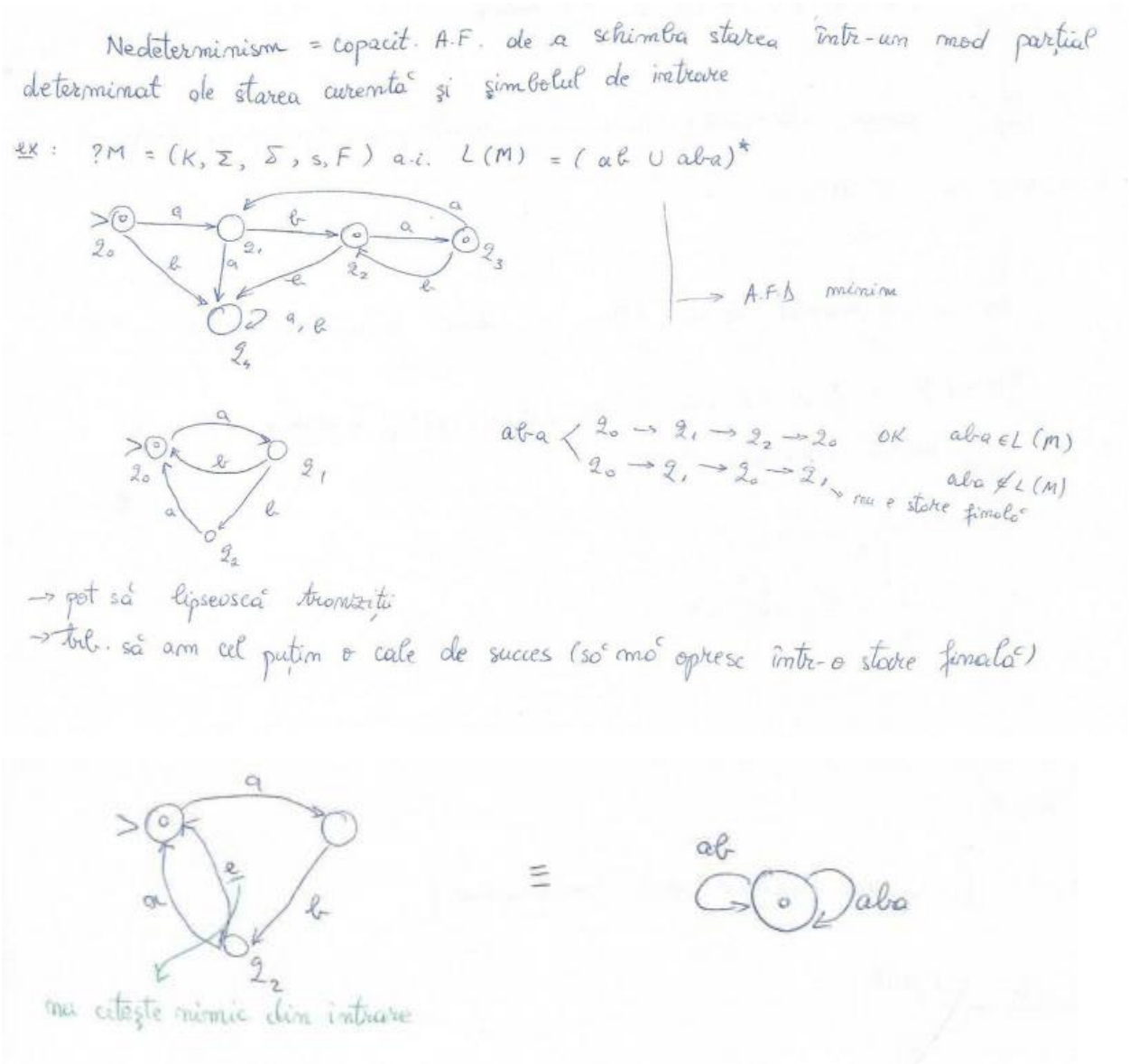
$F = \{q_0, q_1, q_2\}$

δ → def. prin diagrame de tranziție

AUTOMATE FINITE NEDETERMINISTE :

NEDETERMINISM = capacitatea unui Automat Finit de a schimba starea intr-un mod partial determinat de starea curenta si simbolul de intare :

- Pot sa lipseasca tranzitii
- Trb sa am cel putin o cale de succes (sa ma opresc intr-o stare finala)



DEF : Un automat finit nedetrminist / AFN este un tuplu

Configuratie AFN

RELATIE

Trecerea intre configuratii in 0 sau mai multi pasi (inchiderea refexiva si tranzitiva)

Def

Un automat finit nedeterminist este un tuplu $M = (K, \Sigma, \Delta, s, F)$

$K \rightarrow$ mult. finită a stărilor

$\Sigma \rightarrow$ alfabet

$s \in K \rightarrow$ st. inițială

$F \subseteq K \rightarrow$ mult. st. finale

$\Delta \subseteq K \times \Sigma^* \times K$

↑ submult. finită

Def

O config. a unui AFN este un elem. din $K \times \Sigma^*$.

Def

$\vdash_M : (K \times \Sigma^*) \times (K \times \Sigma^*) \rightarrow$ relație

Def

$\vdash_M^*$ Trecerea între config. în 0 sau mai multi pași (închiderea reflex. și tranzitivă a $\vdash_M$)

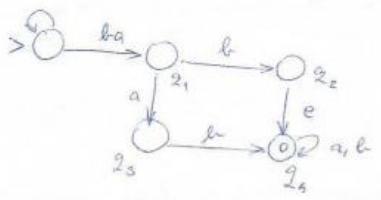
Un sir este acceptat de un AFN daca :

Ex : w contine cel putin o aparitie a subsirului bab sau baab

Def

Un șir este acceptat de un AFN, M, dacă $(s, w) \vdash_M^* (q, e)$, $q \in F$

ex : $M = (K, \Sigma, \Delta, s, F)$ a.î. $L(M) = \{ w \in \{a, b\}^* \mid w$ conține cel puțin o apariție a subșirului bab sau baab

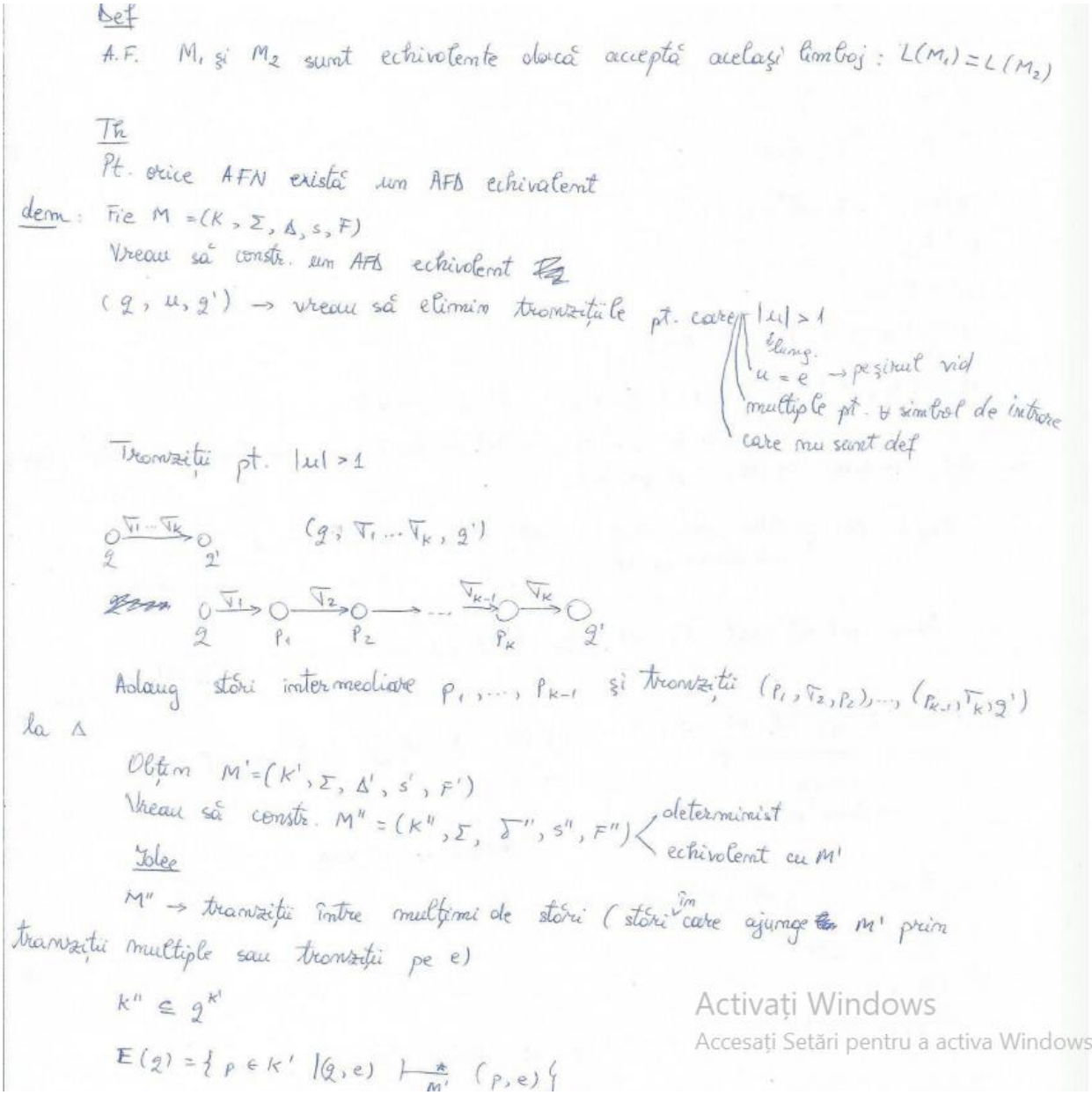


ECHIVALENTA AFN SI AFD

Def : 2 automate finite sunt echivalente daca accepta acelasi limabj

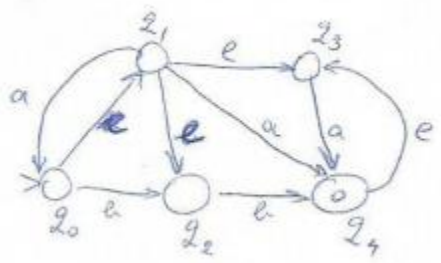
Th : Pt orice AFN exista un AFD echivalent

Dem : Tranzitii / stari intermediare



Ex :

ex



$E(q_0) = \{q_0, q_1, q_2, q_3\}$

$E(q_1) = \{q_1, q_2, q_3\}$

$E(q_2) = \{q_2\}$

$E(q_3) = \{q_3\}$

$E(q_4) = \{q_3, q_4\}$

$M'' = (K'', \Sigma, \delta'', s'', F'')$

$K'' \subseteq 2^{K'}$

$s'' = E(s')$

$F'' = \{Q \subseteq K' \mid Q \cap F' \neq \emptyset\}$

$\delta''(Q, \nabla) = \bigcup \{E(p) \mid (q, \nabla, p) \in \Delta', q \in Q, \nabla \in \Sigma\}$

Pt. fiecare q din Q, mă uit la tranz. care mă duce într-o stare, i.e. la închiderea cu tranz. pe ε a lui

Arăt M'' < determinist → dim constr. $\delta'' \rightarrow \emptyset \in K'' \subseteq 2^{K'}$
echivalent cu M' ↳ starea vidă e o stare validă

INDUCTIE

Pas de baza

Ipoteza inductiva

Dacă pot să arăt: $(q, w) \vdash_{M'}^{*} (p, e) \iff (E(q), w) \vdash_{M''}^{*} (P, e)$, $p \in P$

$(s, w) \vdash_{M'}^{*} (f, e)$, $f \in F \iff (E(s), w) \vdash_{M''}^{*} (P, e)$, $f \in P$

$w \in L(M')$ acceptarea pt. AFN

$w \in L(M'')$ acceptarea pt AFD

Dem. prin inducție după $|w|$

Pas de bază

$|w| = 0$

$(q, e) \vdash_{M'}^{*} (p, e) \iff (E(q), e) \vdash_{M''}^{*} (P, e)$, $p \in P$

$p \in E(q)$

s'', P''

$w \in L(M'')$

Ip. inductivă: PP adev pt $|w| = k$

Pas de inducție

$|w| = k+1$, $w = v\,a$, $v \in \Sigma^*$, $a \in \Sigma$

$(q, w) \vdash_M^* (p, e) \iff \exists\, r_1, r_2 \in K'$ a.i. $(q, va) \vdash_{M'}^* (r_1, a) \vdash_{M'} (r_2, e) \vdash_{M'}^* (p, e)$

I $(q, va) \vdash_{M'}^* (p, e) \Rightarrow (E(q), va) \vdash_{M''}^* (P, e)$, $p \in P$

$(q, va) \vdash_{M'}^* (r_1, a) \vdash_{M'} (r_2, e) \vdash_{M'}^* (p, e)$

$(q, v) \vdash_{M'}^* (r_1, e)$, $|v| = K$ $\xrightarrow{\text{ip. inductivă}}$ $(E(q), v) \vdash_{M''}^* (R_1, e)$, $r_1 \in R_1$

$(r_1, a) \vdash_{M'} (r_2, e) \Rightarrow (r_1, a, r_2) \in \Delta'$

dim constr. δ''

$\Rightarrow$ $E(r_2) \subseteq \delta''(r_1, a)$ | $\delta''(Q, \nabla) = \cup \{ E(p) \mid (q, a, p) \in \Delta' \}$
$q \in Q$, $\nabla \in \Sigma$, $p \in K'$

$(r_2, e) \vdash_{M'}^* \Rightarrow$ $p \in E(r_2)$

$E(r_2) \subseteq \delta''(R_1, a)$ | $\Rightarrow p \in \delta''(R_1, a) \Rightarrow (R_1, a) \vdash_{M''} (P, e)$, $p \in P$

$\Rightarrow (E(q), va) \vdash_{M''}^* (R_1, a) \vdash_{M''} (P, e)$, $p \in P$

II $\Leftarrow (E(q), va) \vdash_{M''}^* (R_1, a) \vdash_{M''} (P, e)$, $p \in P \Rightarrow (q, va) \vdash_{M'}^* (r_1, a)$

TEMĂ (pași invers echiv.) → ole acelas data viitoare la curs

$s'' = E(s') = E(q_0) = \{q_0, q_1, q_2, q_3\} = Q_0$

$s''(Q_0, a) = E(q_0) \cup E(q_4) =$ → care stări au tranz. pe a

$= \{q_0, q_1, q_2, q_3, q_4\} = Q_1$

$s''(Q_0, b) = E(q_2) \cup E(q_4) = \{q_2, q_3, q_4\} = Q_2$

$\delta''(Q_1, a) = Q_1$

$\delta''(Q_1, b) = Q_2$

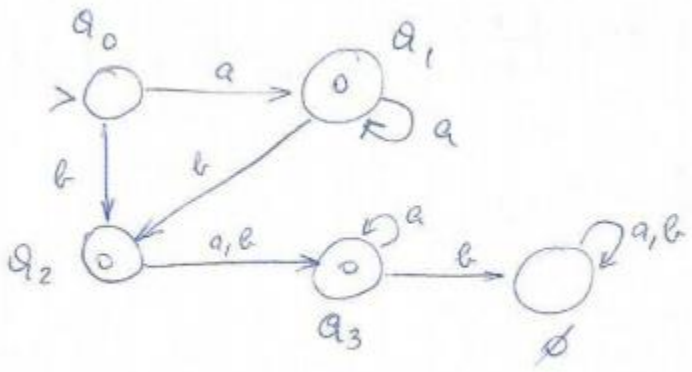
$\delta''(Q_2, a) = E(q_4) = \{q_3, q_4\} = Q_3$ → doar q_4 face tranz. pe a

$\delta''(Q_2, b) = E(q_4) = Q_3$

$\delta''(Q_3, a) = Q_3$

$\delta''(Q_3, b) = \emptyset$

$s''(Q, T) = \cup \{E(p) \mid (q, T, p) \in \Delta', q \in Q, p \in K', T \in \Sigma\}$



Proprietati de inchidere ale L(AF)

Teorema : Clasa limbajelor acceptate de AF este inchisa in raport cu op de

-reuniune

- concatenare

- Kleene Star

-Complementare

-Intersectie

DEM:

Th

Clasa limbajelor acceptate de AF este închisă în raport cu op. de:

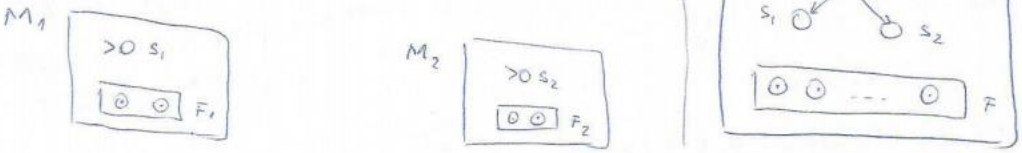
a) reuniune
b) concatenare
c) Kleene star
d) complementare
e) intersecție

dem:

a) $M_1 = (K_1, \Sigma, \Delta_1, s_1, F_1)$

$M_2 = (K_2, \Sigma, \Delta_2, s_2, F_2)$

$?M, L(M) = L(M_1) \cup L(M_2)$



$M = (K, \Sigma, \Delta, s, F)$

$K = K_1 \cup K_2 \cup \{s\}$

$s = s$

$F = F_1 \cup F_2$

$\Delta = \Delta_1 \cup \Delta_2 \cup \{(s, e, s_1), (s, e, s_2)\}$

$w \in L(M) \iff (s, w) \vdash_M^* (f, e)$, $f \in F$

$\iff (s_1, w) \vdash_{M_1}^* (f, e)$, $f \in F_1$

sau

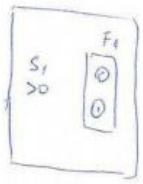
$(s_2, w) \vdash_{M_2}^* (f, e)$, $f \in F_2$

b) concatenare

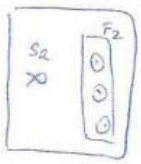
$M_1 = (K_1, \Sigma, \Delta_1, s_1, F_1)$

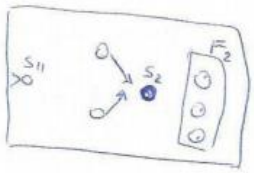
$M_2 = (K_2, \Sigma, \Delta_2, s_2, F_2)$ $\longmapsto$? M a.i. $L(M) = L(M_1) \cdot L(M_2)$

M_1



M_2





$M = (K, \Sigma, \Delta, s, F)$

$K = K_1 \cup K_2$

$s = s_1$

$F = F_2$

$\Delta = \Delta_1 \cup \Delta_2 \cup (f_1 \times \{e\} \times \{s_2\})$

$(s, w) \vdash_M^* (f, e)$, $f \in F$

$\Updownarrow$

$w \in L(M)$

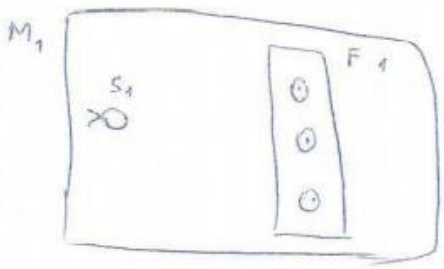
$\Leftrightarrow$ $\exists w_1, w_2 \in \Sigma^*$ a.i. $w = w_1 w_2$ și $p \in F_1$

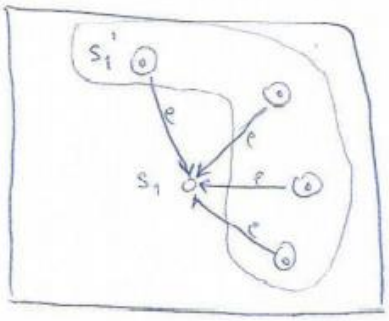
$(s_1, w_1) \vdash_{M_1}^* (p, e)$ și $(s_2, w_2) \vdash_{M_2}^* (f, e)$

c) Kleene Star

$M_1 = (K_1, \Sigma, \Delta_1, s_1, F_1)$

? M a.i. $L(M) = L(M_1)^*$





$M = (K, \Sigma, \Delta, s, F)$

$K = K_1 \cup \{s_1'\}$

$s = s_1'$

$F = F_1 \cup \{s_1'\}$

$\Delta = \Delta_1 \cup \{F_1 \times \{e\} \times \{s_1\} \cup \{(s_1', e, s_1)\}$

d) complementare

$M = (K, \Sigma, \delta, s, F)$ $\Rightarrow$ interschimb stările finale cu cele nefinale

? $\bar{M}$ a.i. $L(\bar{M}) = \Sigma^* - L(M)$

e) intersectie

M_1, M_2 A.F. , $L_1 = L(M_1)$, $L_2 = L(M_2)$

$L = L_1 \cap L_2 = \Sigma^* - ((\Sigma^* - L_1) \cup (\Sigma^* - L_2))$

AFD AFD

AFN

Convertesc AFN & AFD

sau

$M_1 = (K_1, \Sigma, \delta_1, s_1, F_1)$

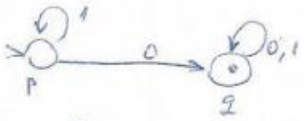
$M_2 = (K_2, \Sigma, \delta_2, s_2, F_2)$

?M a.i. $L(M) = L(M_1) \cap L(M_2)$

$M = (K_1 \times K_2, \Sigma, \delta, (s_1, s_2), F_1 \times F_2)$

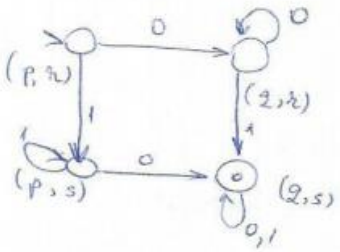
$\delta((p,q), a) = (\delta_1(p,a), \delta_2(q,a))$, $p \in K_1$, $q \in K_2$

ex : M_1 :



M_2 :





ECHIVALENTA LR = L(AF)

TEOREMA : Un limbaj este regulat daca si numai daca este accpetat de un automat finit.

DEM : LR / L AF / Clasa LR este clasa minima de limbaje / Ar ca L(M) este reuniunea unui nr finit de LR (R). / R este multimea sirurilor care duc automatul din starea qi in starea qj fara sa treaca printr-o stare intermediara.

Echivalenta L.R. ≡ L(A.F.)

$\{a, b\}^*$, $a^* b^*$

Th

Un limbaj este regulat (=> este ~~altceva~~ acceptat de un A.F.

demo

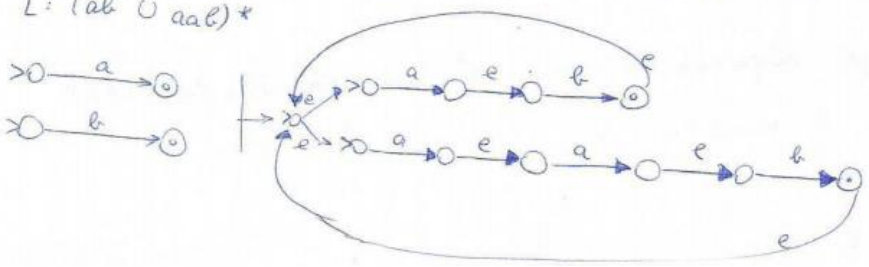
"=>" L.R. → L(A.F.)

Clasa L.R. este clasa minimă de limbaje care conține $\emptyset$, $\{\sigma\}$, $\forall \sigma \in \Sigma$ și este închisă în raport cu operațiile de $\cup$, $\cdot$, $*$.

=> A.F. pt. $\{\forall\}$, $\emptyset$

Aplicând operațiile (cf. Th. închidere a L(A.F.)) => $\cup$, $*$, $\cdot$

ex: $L = (ab \cup aab)^*$



"⇐" L(A.F.) ⇒ L.R.

Idee: Vreau să arăt că L(M) este reuniunea unui nr. finit de L.R. (R)

Fie $M = (K, \Sigma, \delta, s, F)$

$K = \{q_1, \dots, q_m\}$, $s = q_1$

$R(i, j, k)$ e mult. șirurilor care duc automatul din starea q_i în starea q_j fără să treacă printr-o stare interm. $\geq k$

$R(i, j, k) = \{x \in \Sigma^* \mid (q_i, x) \overset{*}{\underset{M}{\vdash}} (q_j, y)$ unde $\ell < k$ sau $\ell = j$ și $y = e$ sau $\ell = i$ și $x = y\}$

$R(i, j, x+1) = \{x \in \Sigma^* \mid (q, x) \overset{*}{\underset{M}{\vdash}} (q_j, e)\}$, $q_j \in F$

$L = \bigcup R(1, j, x+1)$, $q_j \in F$

⇓

R regulat

Inductie : finit => regulat

Dem. prin inducție

$$R(i, j, 1) = \begin{cases} \{x \mid (\delta(q_i, x) = (q_j, e), i \neq j\} \\ \{e\} \cup \{x \in \Sigma \mid \delta(q_i, x) = (q_j, e), i = j\} \end{cases}$$

finit ⇒ regulat

Ip. ind. → adev. pt. k

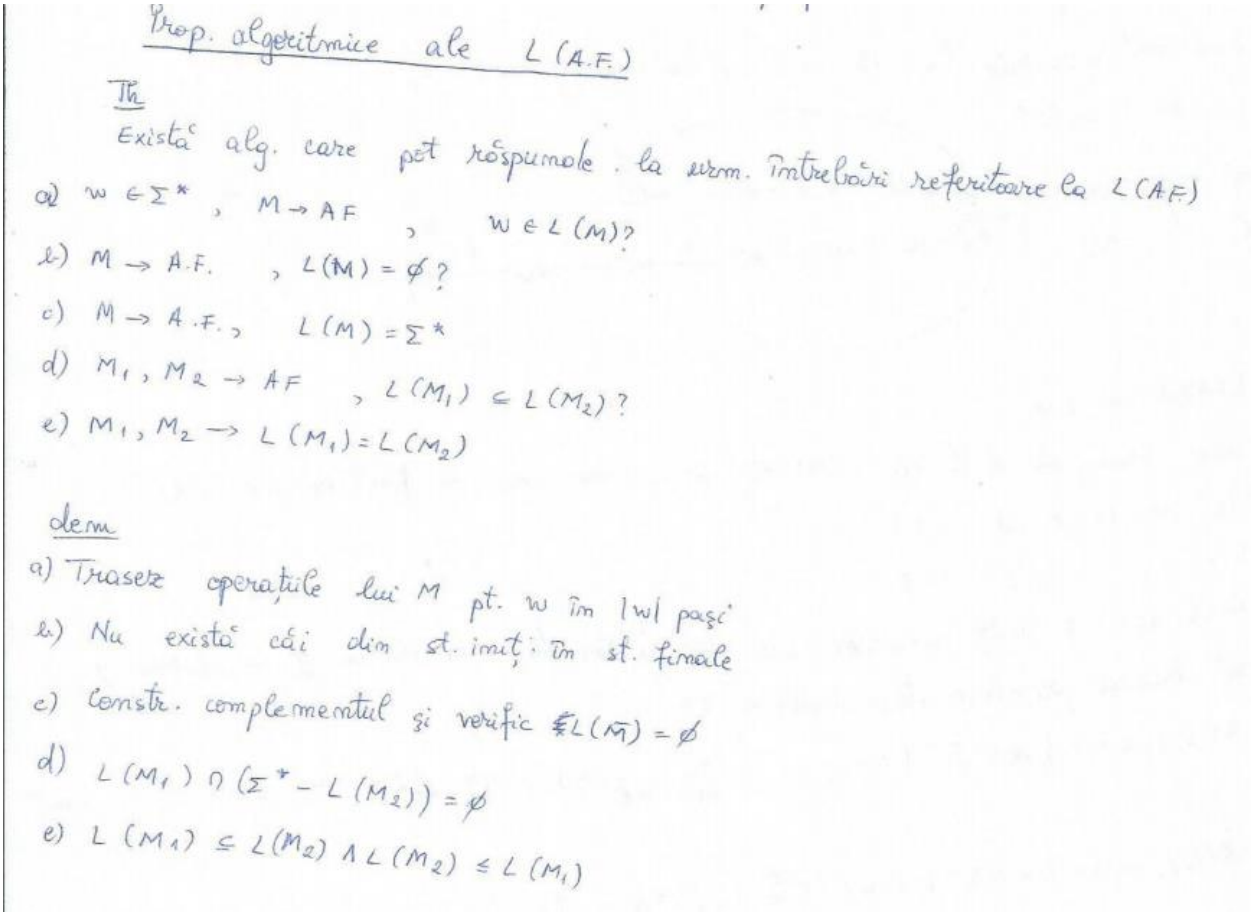
Pas de inducție → k+1

$R(i, j, k+1) = \underbrace{R(i, j, k)}_{\text{ip. ind.}} \cup \underbrace{R(i, k, k)}_{\text{ip ind.}} \underbrace{R(k, k, k)^*}_{\text{ip. ind.}} \underbrace{R(k, j, k)}_{\text{ip. ind.}}$ → pt. toate ține ipoteza inductivă ⇒

regulat (conf. prop. L.R)

Proprietati algoritmice ale L (AF)

Teorema : Exista algoritmi care pot raspunde la urmatoarele intreabr referitoare la L (AF)



Prop. algoritmice ale L (A.F.)

Th

Există alg. care pot răspunde la urm. întrebări referitoare la L (A.F.)

a) $w \in \Sigma^*$, $M \to AF$, $w \in L(M)$?

b) $M \to A.F.$, $L(M) = \emptyset$?

c) $M \to A.F.$, $L(M) = \Sigma^*$

d) $M_1, M_2 \to AF$, $L(M_1) \subseteq L(M_2)$?

e) $M_1, M_2 \to L(M_1) = L(M_2)$

dem

a) Trasez operațiile lui M pt. w în |w| pași

b) Nu există căi din st. inițiale în st. finale

c) Constr. complementul și verific $\in L(\overline{M}) = \emptyset$

d) $L(M_1) \cap (\Sigma^* - L(M_2)) = \emptyset$

e) $L(M_1) \subseteq L(M_2) \wedge L(M_2) \subseteq L(M_1)$

Tehnici de demonstrare a regularitatii limbajelor regulate :

Ex: Limbajul nr nat poz divizibil cu 2 sau 3 -> regulat

Tehnici de demonstrare a regularitatii limbajelor regulate

ex : Limbajul nr. nat. poz. divizibile cu 2 sau 3 → regulat

2, 6, 14 ∈ L

001, 5 ∉ L

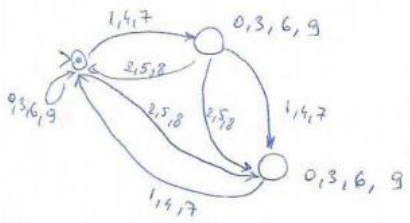
$\Sigma = \{0, 1, \dots, 9\}$

$L_1 = \{0\} \cup \{1, \dots, 9\}\ \Sigma^*$

$L_2 = L_1 \cap \Sigma^*$ $\{0, 2, 4, 6, 8\}$

$L_3 = L_1 \cap L(M)$

↓ nr. divizibile cu 3



ex $\Sigma = \{a, b\}$

$L = \{w \in \Sigma^* \mid |w|$ impară, w conține nr. par de $a\}$

$L_1 = \Sigma(\Sigma\Sigma)^*$

$L_2 = b^*(ab^*ab^*)^*$

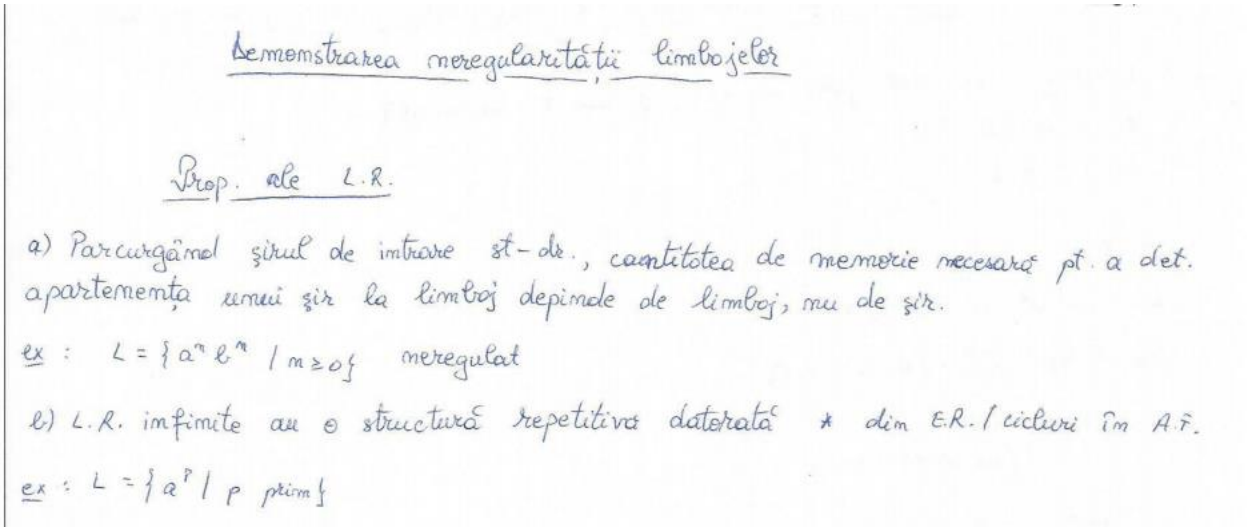
$L = L_1 \cap L_2$

Demonstrarea neregularitatii limbajelor :

Proprietati ale Limbajelor regulate :

-Parcurgand sirul st dr, cantitatea de memorie necesara pt a determia apartenenta unui sir la un limbaj depinde de limbaj, nu de sir.

-Limbajele regulate au o structura repetitiva datorata * din expresiile regulate / cicluri in automate finite



Demonstrarea neregularității limbajelor

Prop. ale L.R.

a) Parcurgând șirul de intrare st-dr., cantitatea de memorie necesară pt. a det. apartenența unui șir la limbaj depinde de limbaj, nu de șir.

ex: $L = \{a^n b^n \mid n \geq 0\}$ neregulat

b) L.R. infinite au o structură repetitivă datorată * din E.R. / cicluri în A.F.

ex: $L = \{a^p \mid p \text{ prim}\}$

Lema de pompare

Fie L un limaj regulat infinit. Atunci exista sirurile :

Th (Lema pompare)

Fie L un limbaj regulat infinit. Atunci $\exists$ şirurile $x, y \neq e, z$ a.î. $x y^m z \in L$, $\forall m \in \mathbb{N}$.

dem: L → limbaj regulat $\overset{\text{Th. echiv}}{\Rightarrow}$ $\exists M$, AFD a.î. $L = L(M)$

Pp. că M are n stări.

L → limbaj infinit $\Rightarrow$ $\exists w \in \Sigma^*$, $|w| = p \geq n$

Fie $w = \sigma_1 \sigma_2 \cdots \sigma_p$, $\sigma_1, \dots, \sigma_p \in \Sigma$

$w \in L$

$(q_0, \sigma_1 \dots \sigma_p) \vdash_M (q_1, \sigma_2 \dots \sigma_p) \vdash_M \cdots \vdash_M (q_p, e)$, $q_p \in F$

M are n stări $\Rightarrow$ $\exists\ 0 \leq i < j < n$ şi $\sigma_{i+1} \dots \sigma_j$ care duce M din q_i înapoi în q_i (Principiul lui Dirichlet / cuiburi)

$$\underbrace{\sigma_1 \sigma_2 \dots \sigma_i}_{x} \underbrace{(\sigma_{i+1} \dots \sigma_j)}_{y} \underbrace{\sigma_{j+1} \dots \sigma_p}_{z}$$

$$(q_0, x y^n z) \vdash_M^* (q_i, y^n z) \vdash_M^* (q_i, y^{n-1} z) \vdash_M^* \cdots \vdash_M^* (q_i, z) \vdash_M^* (q_p, e)$$

ex: $L = \{ a^n b^n \mid n \geq 0 \}$?

Pp. L → regulat, infinit $\Rightarrow$ aplic lema de pompare

$\exists\ x, y \neq e, z$ a.î. $x y^k z \in L$, $\forall k \geq 0$

I $\quad x = a^p$, $y = a^q$, $z = a^r b^s$ $\quad \Rightarrow x y^k z = a^p (a^q)^k a^r b^s = a^{p+qk+r} b^s$

Activați Windows

II $x = a^p$, $y = a^2 b^k$, $z = b^s$ $\Rightarrow xy^k z = a^p (a^2 b^k)^k b^s = a^p \underbrace{a^2 b^k a^2 b^k \dots a^2 b^k}_{k \text{ ori}} b^s \notin L$

III $x = a^p$, $y = b^k$, $z = b^s$ $\Rightarrow xy^k z = a^p b^2 (b^k)^k b^s = a^p b^{2+k \cdot k+s} \Rightarrow p = 2 + k \cdot k + s$, $\forall k \geq 0$ fals

Am ar. că nu-am cum să găsesc un pct. de pompare, iar lbj. să rămână regulat.

$\Rightarrow$ L - regulat $\rightarrow$ fals

Am aratat ca nu am cum sa gasesc un punct de pompare, iar limbajul sa ramana regulat.

Ex : Pp L limaj regulat infinit -> aplicam lema de pompare

ex: $L = \{a^p \mid p \text{ prim}\}$?

Pp $L \rightarrow$ regulat infinit

Aplic lema de pompare $\Rightarrow \exists x, y \neq e, z$ a.i. $x y^m z \in L$, $\forall m \geq 0$

$x = a^p$, $y = a^R$, $z = a^s$ $\Rightarrow a^{p + Rm + s} \Rightarrow a^p$, p prim, $\forall m \geq 0$, fals

O altă formă a lemei de pompare (LP):

Th

Fie $L \rightarrow$ lbj regulat, infinit, atunci $\exists$ n dependent de L și $x, y \neq e, z$ a.î. $|xy| \leq n$ și $xy^k z \in L$, $\forall k \geq 0$

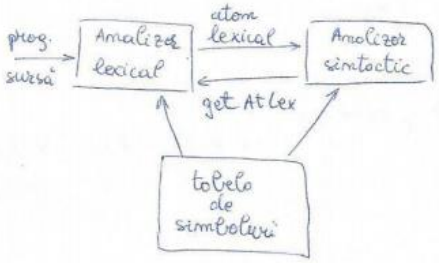
! EXAMEN : de demo. lema de pompare pt. lbj. regulate

ANALIZA LEXICALA :

Atom lexical / sablon / lexema

Limbaje pompare : atomi lexicali, cuvinte cheie, operator, constante, identificatori, siruri literali, simboluri de punctuatie

Analiza lexicală



Atom lexical, şablon, lexemă

Atom lexical → categoria asoc. unei mulţimi de şiruri

Lexemă → şir

Şablon → regula prin care verific clasificarea unui şir

ex: const pi =3.14;

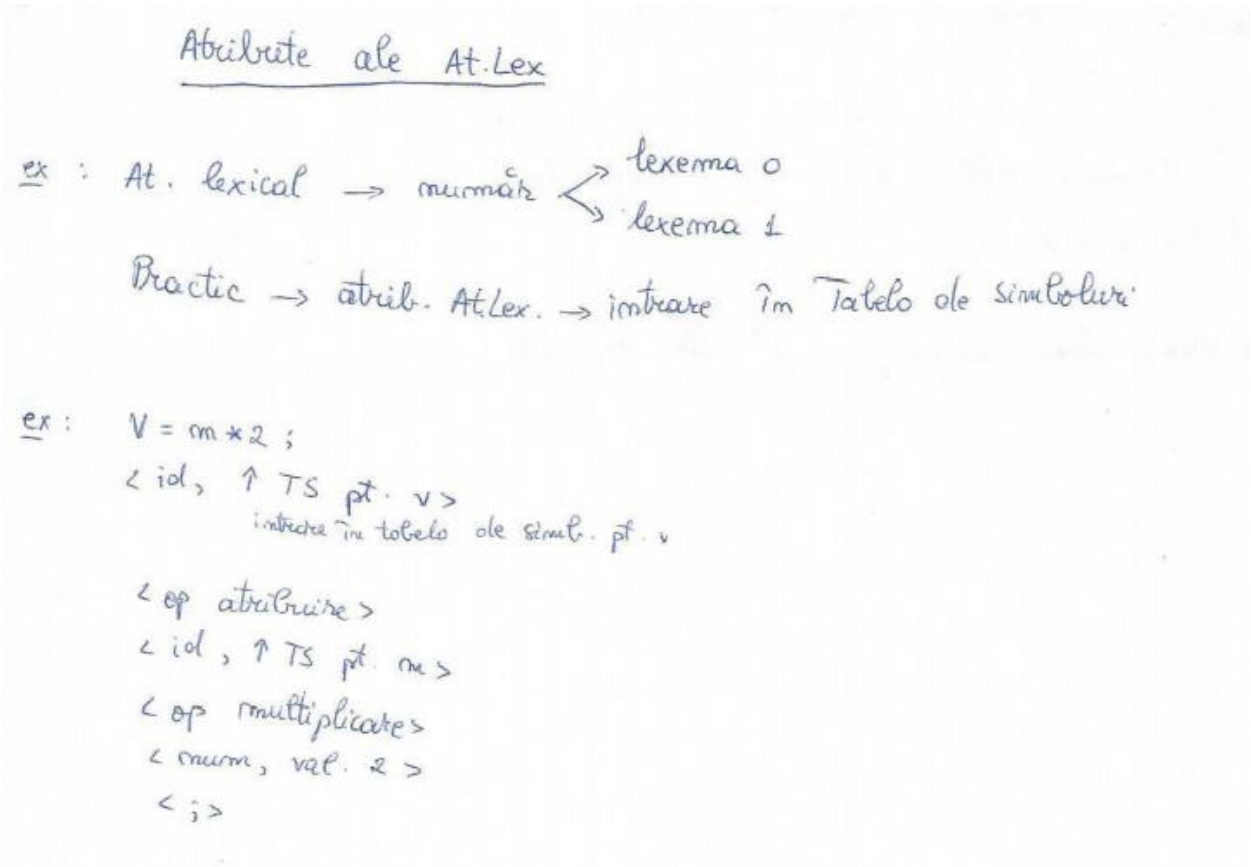
pi → lexema pt. atom lexical "identific"

Limbaje programare → at. lexicali, cuv. cheie, operatori, constante, identificatori, şiruri literali, simboluri de punctuaţie etc.

Activaţi Windows

ATRIBUTE ALE ATOMULUI LEXICAL :

Tabela de simboluri / lexema



Definitii Regulate :

Identificatorii sunt siruri de litere sau cifre care incep cu o litera

Constantele numerice fara semn

Definiții Regulate

Șablon → Expr. Reg.

$d_1 \to r_1$
$d_2 \to r_2$
.....
$d_m \to r_m$

peste Σ, r_i este def. peste $\Sigma \cup \{d_1, d_2, \ldots, d_{i-1}\}$

ex: identificatorii sunt șiruri de litere sau cifre care încep cu o literă

lit → A | ... | Z | a | ... | z

cifră → 0 | 1 | ... | 9

id → lit (lit | cif)*

ex: const. numerice fără semn: 2350, 4.99, 5.07 E-3

cif → 0 | 1 | ... | 9 cif1 → 1 | ... | 9

cifre → cif1 (cif)*

partea fracționară → . cifre | e

partea exponențială → E (+ | - | e) cifre | e

num. → cifre p-fract p-exp

Notatii :

Notații

$r+ \equiv rr^*$

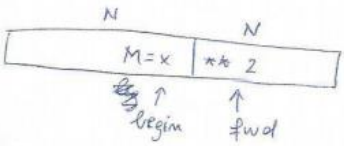
$r? \equiv r|e$

$[abc] \equiv a|b|c$

$[a-z] \equiv a|b|...|z$

Opțiuni implementare - bufferare intrare

buffer



N (1024, 4096)

Inițial → beg ≡ fwd

Deplasez fwd până la identific. unei lexeme ; fwd - primo poz. după lexemei

După prelucrarea lexemei : beg ← fwd

buffer → circular

ANALIZA STATICA :

- Delimitator
- Identificator
- Num
- Analizator lexical -> atom lexical + atribute

instr → if expr. there instr else instr

| if expr then instr

| e

op. rel - operator relațional

expr → term ~~op. rel~~ term

| term

analizor
~~analizator~~
sintactic

term → num | id

if → if

then → then

else → else

oprel → < | <= | > | >= | = | <>

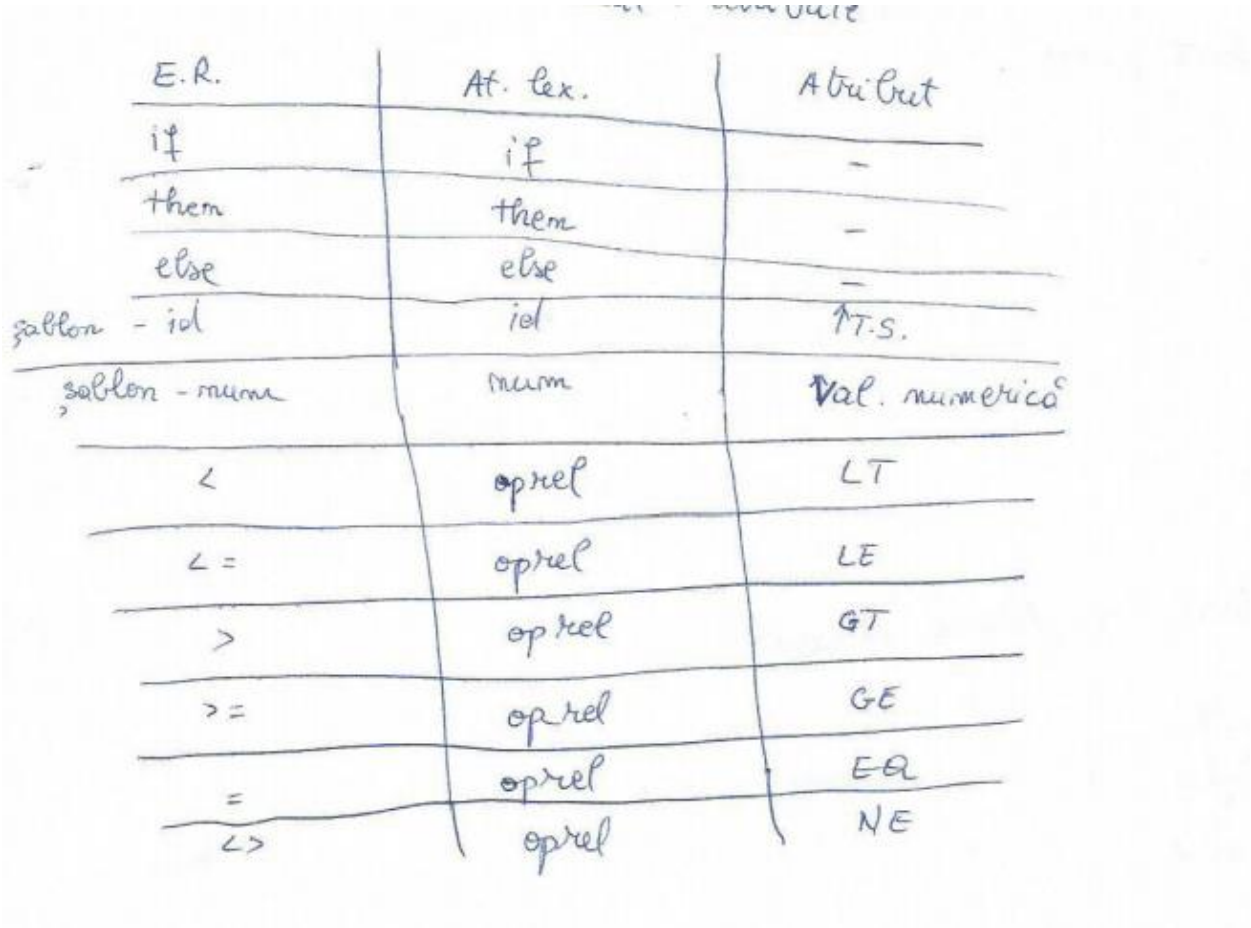
id → lit (lit | cif)*

num → cif | cif* (. cifre)? (E (+/-)? cifre)?

delim → blank | tab | new line

ws → delim+

Am. lexical → at. lexical + atribute



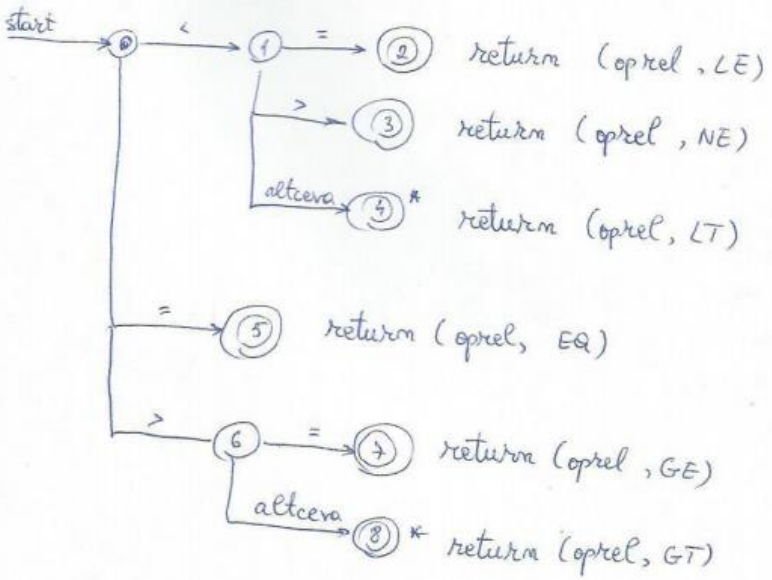
E.R.	At. lex.	Atribut
if	if	-
them	them	-
else	else	-
şablon - id	id	↑T.S.
şablon - numa	numa	Val. numerică
<	oprel	LT
< =	oprel	LE
>	oprel	GT
> =	oprel	GE
=	oprel	EQ
< >	oprel	NE

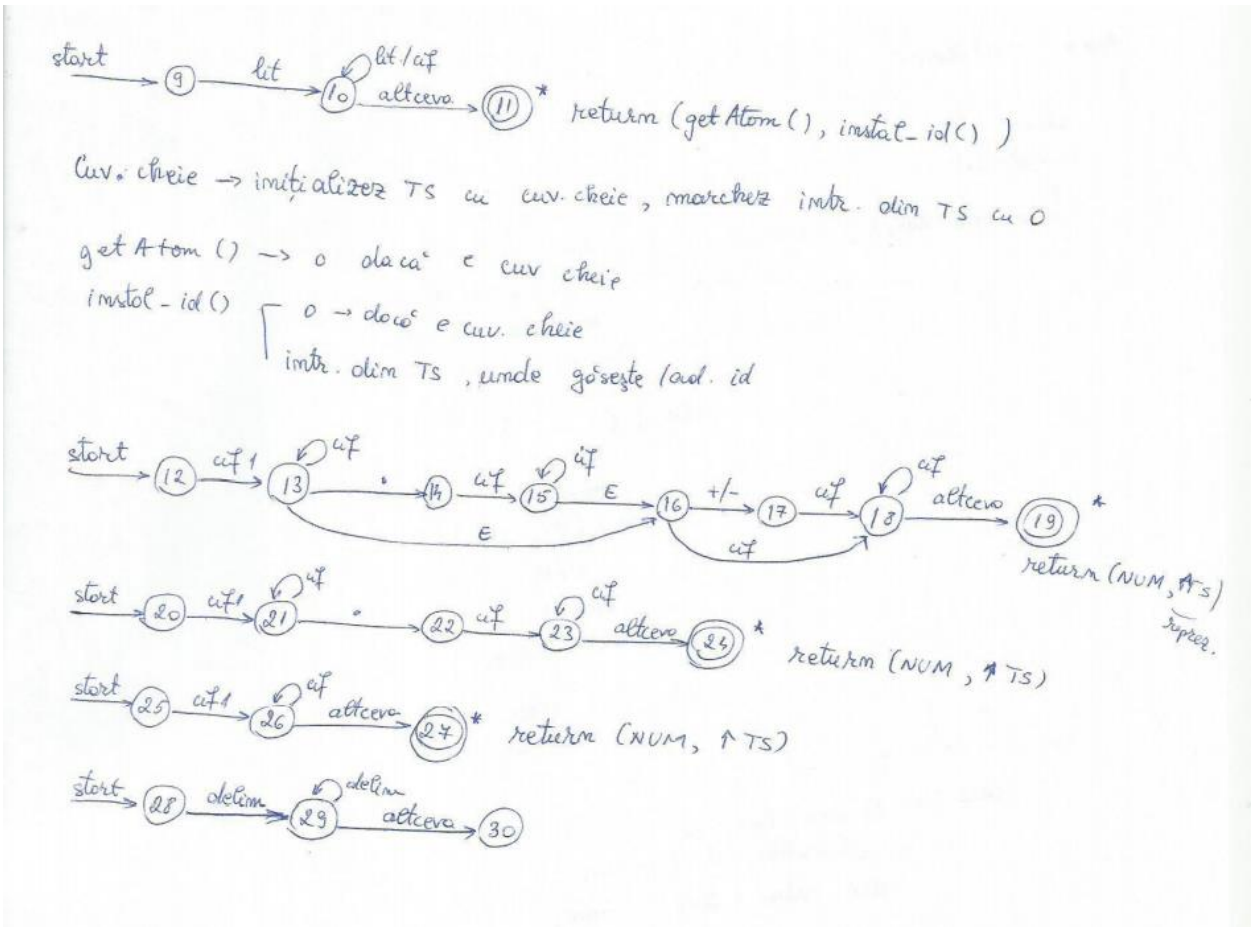
Definirea AF : Diagrame de tranzitii :

La recunoasterea unui Atom Lexical -> atom + atribut

Definirea AF → diagrame de tranziții

La recunoașterea unui AtLex → atom + atribut





IMPLEMENTARE :

Implementare

Am : nextchar() → citește câte un caracter din buffer, avans. ~~forward~~ fwd

```
int stare = 0, start = 0;
int fail()
{
   fwd = beg ;
   switch (start) {
      case 0 :  start = 9 ; break;
      case 9 :  start = 12 ; break;
      case 12 : start =20; break;
      case 20 : start = 25 ; break;
      case 25 : recover (); break;
      default;  // ERR comp
   } // end switch
   return start;
}
```

```
Aterm nextAterm ()
{
    char c;
    while (1)
    {
      switch (state) { case 0: c = next char();
                       if (c==' ' || c=='\t' || c=='\n'){
                           c = next char();
                           beg++;
                       }
                       else if (c == 'c')
                             state = 1;
                             break;
                          else if ( c== '='
                                state = 5;
                             else if (c=='s')
                                state 6;
                                else
                                 state = fail();
                                 break;
      case 9 : c= next char ();
               if (is letter (c)) state =10;
               else state = fail (); break;
```

```
case 10 :   c = nextchar ();
            if (isletter (c) || isdigit (c))   state = 10;
            else state = 11; break;

case 11 :  retract (); instal_id () ; return getAtom ();

. . .

case 25 :  c = nextchar ();
            if (isdigit (c) && (c - '0' > 0))] state = 26 ; break;
            else state = fail (); break;

case 26 :  c = nextchar ();
            if (isdigit (c))   state = 26 ;
            else state = 27; break;
case 27 :  retract () ; return NUM;
        }
    }
}
```